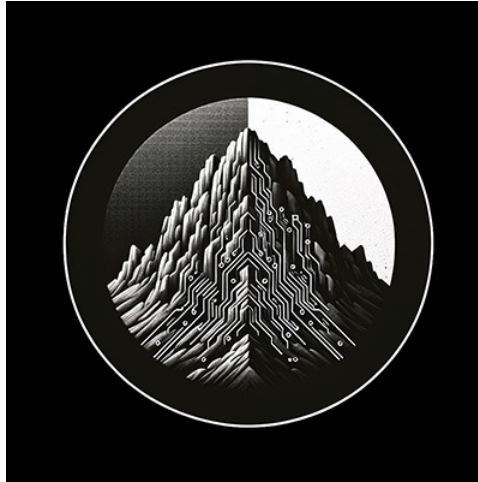




RTL Design Sherpa

RTL AMBA Shared Infrastructure Module Reference — Rev 0.90

July 2026

Table of Contents

1 GAXI Drop FIFO Sync.....	53
1.1 Overview.....	53
1.2 Module Parameters.....	53
1.3 Interface Signals.....	54
1.3.1 Clock and Reset.....	54
1.3.2 Write Interface (GAXI Master → FIFO).....	54
1.3.3 Read Interface (FIFO → GAXI Slave).....	54
1.3.4 Drop Interface.....	54
1.3.5 Status Signals.....	55
1.4 Drop Operation Behavior.....	55
1.4.1 Drop by Count.....	55
1.4.2 Drop All.....	55
1.4.3 Simultaneous Operations.....	56
1.5 Timing Diagrams.....	56
1.5.1 Fill FIFO.....	56
1.5.2 Drop by Count.....	56
1.5.3 Drop All.....	56
1.5.4 Comprehensive Scenario.....	56
1.6 Usage Examples.....	57
1.6.1 Basic Write/Read.....	57
1.6.2 Drop Operation.....	57
1.7 Test Coverage.....	58

1.7.1 Running Tests.....	59
1.8 Design Notes.....	59
1.8.1 FIFO Depth Requirements.....	59
1.8.2 Registered vs. Mux Mode.....	59
1.8.3 Drop Latency.....	60
1.8.4 Almost Full/Empty Margins.....	60
1.9 Related Modules.....	60
1.10 Revision History.....	60
2 GAXI Asynchronous FIFO.....	60
2.1 Overview.....	60
2.1.1 Key Features.....	61
2.2 Module Interface.....	61
2.3 Parameters.....	61
2.4 Functional Description.....	63
2.4.1 Why Gray Code?.....	64
2.4.2 Dependencies.....	64
2.5 Usage Examples.....	64
2.5.1 Example 1: Basic CDC FIFO.....	64
2.5.2 Example 2: High-Speed to Low-Speed CDC.....	65
2.6 Timing Characteristics.....	65
2.7 Design Considerations.....	66
2.7.1 Depth Sizing for Clock Ratio.....	66
2.7.2 Reset Synchronization.....	66
2.7.3 Metastability Protection.....	66

2.8 Error Checking.....	67
2.9 Testing.....	67
2.10 Common Issues.....	67
2.10.1 Issue 1: Metastability.....	67
2.10.2 Issue 2: Pointer Synchronization Failure.....	67
2.10.3 Issue 3: Underflow/Overflow.....	68
2.11 Related Modules.....	68
2.12 References.....	68
3 GAXI Synchronous FIFO.....	68
3.1 Overview.....	68
3.1.1 Key Features.....	68
3.2 Module Interface.....	69
3.3 Parameters.....	69
3.4 Functional Description.....	70
3.4.1 Read Modes.....	70
3.4.2 Dependencies.....	70
3.5 Usage Examples.....	71
3.5.1 Example 1: Mux Mode (Low Latency).....	71
3.5.2 Example 2: Flop Mode (Timing Closure).....	71
3.6 Timing Characteristics.....	71
3.7 Resource Utilization.....	72
3.8 Testing.....	72
3.9 Related Modules.....	72
4 GAXI Register Slice.....	73

4.1 Overview.....	73
4.1.1 Key Features.....	73
4.2 Module Interface.....	73
4.3 Parameters.....	74
4.4 Functional Description.....	74
4.4.1 Architecture.....	74
4.4.2 Ready/Valid Protocol.....	74
4.4.3 Transfer Scenarios.....	75
4.4.4 Storage Update Logic.....	76
4.5 Usage Examples.....	76
4.5.1 Example 1: Pipeline Timing Isolation.....	76
4.5.2 Example 2: AXI Channel Register Slice.....	76
4.5.3 Example 3: Multiple Register Slices for Deep Pipelining.....	77
4.6 Timing Characteristics.....	77
4.6.1 Latency Guarantee.....	78
4.7 Comparison with Related Modules.....	78
4.7.1 gaxi_regslice vs gaxi_skid_buffer.....	78
4.7.2 gaxi_regslice vs gaxi_fifo_sync.....	79
4.8 Resource Utilization.....	79
4.8.1 FPGA Resources (Typical).....	79
4.9 Testing.....	79
4.9.1 Test Coverage.....	79
4.9.2 Run Tests.....	80
4.10 Design Considerations.....	80

4.10.1 Port Compatibility.....	80
4.10.2 Status Signals.....	80
4.10.3 Assertions.....	80
4.11 Related Modules.....	81
4.12 Common Use Cases.....	81
4.12.1 1. Breaking Long Combinatorial Paths.....	81
4.12.2 2. AXI Interface Pipelining.....	81
4.12.3 3. CDC Preparation.....	81
5 GAXI Asynchronous Skid Buffer.....	82
5.1 Overview.....	82
5.1.1 Key Features.....	82
5.2 Module Interface.....	82
5.2.1 Internal Components.....	83
5.3 Parameters.....	84
5.4 Usage Examples.....	84
5.4.1 Example 1: Basic Async Skid Buffer.....	84
5.4.2 Example 2: Pipeline Stage with CDC.....	85
5.5 Timing Characteristics.....	85
5.5.1 Latency Components.....	85
5.5.2 Throughput.....	85
5.6 Design Considerations.....	86
5.6.1 When to Use This Module.....	86
5.6.2 Depth Selection.....	86
5.6.3 Reset Synchronization.....	86

5.7 Resource Utilization.....	86
5.8 Testing.....	87
5.9 Common Issues.....	87
5.9.1 Issue 1: CDC Violations.....	87
5.9.2 Issue 2: Write-Side Stalls.....	87
5.9.3 Issue 3: Unexpected Latency.....	87
5.10 Related Modules.....	87
5.11 References.....	87
6 GAXI Skid Buffer (Double-Drain).....	88
6.1 Overview.....	88
6.1.1 Key Features.....	88
6.2 Module Purpose.....	89
6.3 Parameters.....	89
6.4 Port Groups.....	90
6.4.1 Clock and Reset.....	90
6.4.2 Write Interface (input side).....	90
6.4.3 Read Interface (output side).....	90
6.4.4 Status.....	90
6.5 Functional Description.....	91
6.5.1 Transfer Detection.....	91
6.5.2 Storage Update.....	91
6.5.3 Ready/Valid Logic.....	92
6.5.4 Output Assignments.....	92
6.5.5 Legality Assertion.....	92

6.6 Usage Example.....	92
6.7 Design Notes.....	93
6.8 Related Modules.....	93
6.8.1 Used By.....	93
6.8.2 Uses.....	93
6.8.3 See Also.....	93
6.9 References.....	94
6.9.1 Source Code.....	94
6.9.2 Documentation.....	94
6.10 Navigation.....	94
7 GAXI Skid Buffer.....	94
7.1 Overview.....	94
7.1.1 Key Features.....	94
7.2 Module Interface.....	95
7.3 Parameters.....	95
7.4 Ports.....	96
7.4.1 Clock and Reset.....	96
7.4.2 Write Interface.....	96
7.4.3 Read Interface.....	96
7.4.4 Status.....	96
7.5 Functional Description.....	97
7.5.1 Operation Modes.....	97
7.5.2 Internal State Machine.....	99
7.5.3 Ready/Valid Logic.....	100

7.6 Timing Characteristics.....	100
7.6.1 Latency.....	100
7.6.2 Throughput.....	101
7.6.3 Critical Paths.....	101
7.7 Comprehensive Timing Examples.....	101
7.7.1 Scenario 1: Zero-Latency Bypass.....	101
7.7.2 Scenario 2: Burst Write Until Full.....	102
7.7.3 Scenario 3: Simultaneous Read/Write.....	103
7.7.4 Scenario 4: Burst Read Until Empty.....	104
7.7.5 Scenario 5: Fill Then Drain Pattern.....	105
7.7.6 Scenario 6: Alternating Read/Write.....	106
7.8 Usage Examples.....	107
7.8.1 Example 1: Basic Pipeline Stage.....	107
7.8.2 Example 2: Backpressure Absorption.....	108
7.8.3 Example 3: Monitoring Occupancy.....	108
7.9 Design Considerations.....	109
7.9.1 Depth Selection.....	109
7.9.2 Timing Closure.....	109
7.9.3 Area vs. Performance Trade-offs.....	110
7.10 Error Checking.....	110
7.11 Testing.....	111
7.11.1 Test File.....	111
7.11.2 WaveDrom Test.....	111
7.11.3 Running Tests.....	111

7.12 Common Issues.....	111
7.12.1 Issue 1: Timing Violation on Bypass Path.....	111
7.12.2 Issue 2: Buffer Overflow.....	111
7.12.3 Issue 3: Unexpected Latency.....	112
7.13 Related Modules.....	112
7.14 References.....	112
8 gaxi_skid_buffer_struct.....	112
8.1 Overview.....	112
8.1.1 Key Features.....	112
8.2 Module Declaration.....	113
8.3 Parameters.....	113
8.4 Ports.....	114
8.4.1 Clock and Reset.....	114
8.4.2 Write Interface.....	114
8.4.3 Read Interface.....	114
8.4.4 Status.....	115
8.5 Usage Examples.....	115
8.5.1 Example 1: AXI Read Address Channel.....	115
8.5.2 Example 2: Custom Packet with Mixed Fields.....	116
8.5.3 Example 3: Simple Data Array.....	116
8.6 Behavior.....	117
8.6.1 Identical to gaxi_skid_buffer.....	117
8.6.2 Key Difference: Type Safety.....	117
8.7 Advantages Over Base Skid Buffer.....	118

8.7.1 Advantage 1: Type Safety.....	118
8.7.2 Advantage 2: No Manual Width Calculation.....	118
8.7.3 Advantage 3: Cleaner Code.....	118
8.8 Design Patterns.....	118
8.8.1 Pattern 1: AXI Channel Buffering.....	118
8.8.2 Pattern 2: Pipeline Stages with Complex Data.....	119
8.9 Testing.....	119
8.10 Synthesis Notes.....	120
8.10.1 Resource Usage.....	120
8.10.2 Type Requirements.....	120
8.11 Related Modules.....	120
8.12 When to Use.....	120
8.13 Navigation.....	120
9 GAXI (Generic AXI) Modules.....	121
9.1 Overview.....	121
9.1.1 Key Features.....	121
9.2 Module Categories.....	121
9.2.1 Skid Buffers (Elastic Buffering).....	121
9.2.2 FIFOs (Deep Buffering).....	122
9.2.3 Test-Code Multi-Instance Helpers.....	122
9.3 Protocol Overview.....	122
9.3.1 Interface Signals.....	122
9.3.2 Handshake Rules.....	123
9.4 Quick Start.....	123

9.4.1 Basic Skid Buffer (Zero-Latency).....	123
9.4.2 Synchronous FIFO (Deep Buffering).....	123
9.4.3 Asynchronous FIFO (Clock Domain Crossing).....	124
9.4.4 Type-Safe Struct Buffer.....	124
9.4.5 Drop FIFO (Packet Processing).....	125
9.5 Testing.....	125
9.6 Design Patterns.....	126
9.6.1 Pattern 1: Timing Closure (Pipeline Stage).....	126
9.6.2 Pattern 2: Rate Adaptation.....	126
9.6.3 Pattern 3: Clock Domain Crossing.....	127
9.7 Performance Characteristics.....	127
9.7.1 Latency.....	127
9.7.2 Throughput.....	128
9.7.3 Resource Utilization (Typical).....	128
9.8 Common Use Cases.....	128
9.8.1 Video Processing (Skid Buffer).....	128
9.8.2 Network Packet Processing (Drop FIFO).....	128
9.8.3 Clock Domain Crossing (Async FIFO).....	129
9.9 Parameter Selection Guidelines.....	129
9.9.1 Buffer Depth (DEPTH).....	129
9.9.2 Output Mode (REGISTERED).....	129
9.9.3 CDC Synchronizer Stages (N_FLOP_CROSS).....	130
9.10 Related Documentation.....	130
9.10.1 Comprehensive Guide.....	130

9.10.2 Protocol Specifications.....	130
9.10.3 RTL Design Sherpa Documentation.....	130
9.10.4 Source Code.....	130
9.11 Design Notes.....	130
9.11.1 When to Use GAXI vs AXI4-Stream.....	130
9.11.2 Buffer vs FIFO Selection.....	131
9.12 Common Issues and Debugging.....	131
9.12.1 Issue 1: Data Loss.....	131
9.12.2 Issue 2: Timing Violation.....	131
9.12.3 Issue 3: CDC Metastability.....	131
9.13 Navigation.....	131
10 AMBA Clock Gate Controller.....	132
10.1 Overview.....	132
10.1.1 Key Features.....	132
10.2 Module Purpose.....	132
10.3 Parameters.....	133
10.4 Port Groups.....	133
10.4.1 Clock and Reset.....	133
10.4.2 Configuration Interface.....	133
10.4.3 Activity Monitoring.....	134
10.4.4 Clock Gating Outputs.....	134
10.5 Functional Description.....	134
10.5.1 Activity Detection Logic.....	134
10.5.2 Idle Signal Generation.....	135

10.5.3 Clock Gating Control Integration.....	135
10.5.4 Gating Threshold Behavior.....	135
10.5.5 Gating/Ungating Timing.....	136
10.6 Usage Example.....	136
10.6.1 Basic Integration with AXI Interface.....	136
10.6.2 Multi-Master Arbitration with Clock Gating.....	137
10.6.3 Dynamic Threshold Adjustment.....	138
10.7 Design Notes.....	138
10.7.1 Wakeup Signal Registration.....	138
10.7.2 Activity Signal Selection.....	139
10.7.3 Integration with ICG Cells.....	139
10.7.4 Power Measurement Methodology.....	139
10.7.5 Performance Considerations.....	140
10.8 Related Modules.....	140
10.8.1 Used By.....	140
10.8.2 Uses.....	140
10.8.3 See Also.....	140
10.9 References.....	141
10.9.1 Source Code.....	141
10.9.2 Documentation.....	141
10.9.3 Industry Standards.....	141
10.10 Navigation.....	141
11 AXI4 DMA Observer.....	141
11.1 Overview.....	142

11.2 What's inside.....	143
11.3 Parameters.....	143
11.4 Port surface.....	145
11.5 axi_bus_meter integration.....	145
11.5.1 Read-side channel attribution: runtime rid → channel map.....	146
11.5.2 Write-side channel attribution: optional sideband.....	146
11.5.3 Outputs.....	146
11.6 Pass-through correctness.....	147
11.7 Test.....	147
11.8 Related Modules.....	147
12 AXI4 DMA Slaves.....	148
12.1 Overview.....	148
12.1.1 Key Features.....	148
12.2 Module Purpose.....	149
12.3 Parameters.....	149
12.4 Port Groups.....	150
12.4.1 Clock and Reset.....	150
12.4.2 Read-Side Telemetry.....	151
12.4.3 Write-Side Telemetry.....	151
12.4.4 AXI4 Read Slave (AR + R).....	152
12.4.5 AXI4 Write Slave (AW + W + B).....	152
12.4.6 Status.....	153
12.5 Functional Description.....	153
12.5.1 Structural Composition.....	153

12.5.2 Read Side (LFSR Source).....	153
12.5.3 Write Side (CRC Sink).....	154
12.5.4 Shared CRC vs Independent Resets.....	154
12.6 Usage Example.....	154
12.7 Design Notes.....	155
12.8 Related Modules.....	155
12.8.1 Used By.....	155
12.8.2 Uses.....	156
12.8.3 See Also.....	156
12.9 References.....	156
12.9.1 Source Code.....	156
12.9.2 Documentation.....	156
12.10 Navigation.....	156
13 AXI4 Master Read CRC Checker.....	156
13.1 Overview.....	157
13.1.1 Key Features.....	157
13.2 Module Purpose.....	157
13.3 Parameters.....	158
13.4 Port Groups.....	159
13.4.1 Clock and Reset.....	159
13.4.2 Configuration (CSR) — same shape as the write generator.....	159
13.4.3 Telemetry.....	160
13.4.4 M-Side AXI4 Read (out to fabric/controller).....	161
13.4.5 Debug Observability (active when DBG_FIFO_DEPTH > 0).....	161

13.5 Functional Description.....	162
13.5.1 Workload FSM.....	162
13.5.2 Decoupled AR and R Address Generation.....	162
13.5.3 Expected Data: LFSR vs Address Hash.....	162
13.5.4 Pipelined Compare.....	163
13.5.5 CRC Accumulation.....	163
13.5.6 AR-ID Generation.....	163
13.5.7 Out-of-Order Completion — Known Limitation.....	163
13.5.8 Optional Debug FIFO.....	163
13.5.9 Standard Protocol Handler.....	164
13.6 Usage Example.....	164
13.7 Design Notes.....	165
13.8 Related Modules.....	165
13.8.1 Used By.....	165
13.8.2 Uses.....	166
13.8.3 See Also.....	166
13.9 References.....	166
13.9.1 Source Code.....	166
13.9.2 Documentation.....	166
13.10 Navigation.....	166
14 AXI4 Master Write Pattern Generator.....	167
14.1 Overview.....	167
14.1.1 Key Features.....	167
14.2 Module Purpose.....	167

14.3 Parameters.....	168
14.4 Port Groups.....	169
14.4.1 Clock and Reset.....	169
14.4.2 Configuration (CSR).....	170
14.4.3 Telemetry.....	171
14.4.4 M-Side AXI4 Write (out to fabric/controller).....	171
14.5 Functional Description.....	172
14.5.1 Workload FSM.....	172
14.5.2 Decoupled AW and W Address Generation.....	172
14.5.3 Data Path: LFSR vs Address Hash.....	172
14.5.4 Hash Pipeline and Staging FIFO.....	173
14.5.5 AW-ID Generation.....	173
14.5.6 CRC Accumulation and Completion.....	173
14.5.7 Standard Protocol Handler.....	173
14.6 Usage Example.....	174
14.7 Design Notes.....	175
14.8 Related Modules.....	175
14.8.1 Used By.....	175
14.8.2 Uses.....	175
14.8.3 See Also.....	176
14.9 References.....	176
14.9.1 Source Code.....	176
14.9.2 Documentation.....	176
14.10 Navigation.....	176

15 AXI4 Slave Read Pattern Generator.....	176
15.1 Overview.....	176
15.1.1 Key Features.....	177
15.2 Module Purpose.....	177
15.3 Parameters.....	178
15.4 Port Groups.....	179
15.4.1 Clock and Reset.....	179
15.4.2 Test Control.....	179
15.4.3 Per-Channel Telemetry.....	179
15.4.4 AXI4 Read Address Channel (AR).....	180
15.4.5 AXI4 Read Data Channel (R).....	181
15.4.6 Status.....	181
15.5 Functional Description.....	181
15.5.1 LFSR Pattern Generation.....	181
15.5.2 Data Replication to the Bus Width.....	181
15.5.3 Per-Channel CRC-32.....	182
15.5.4 Burst FSM and Gapless Back-to-Back Bursts.....	182
15.5.5 Standard Protocol Handler.....	182
15.6 Usage Example.....	182
15.7 Design Notes.....	183
15.8 Related Modules.....	184
15.8.1 Used By.....	184
15.8.2 Uses.....	184
15.8.3 See Also.....	184

15.9 References.....	184
15.9.1 Source Code.....	184
15.9.2 Documentation.....	184
15.10 Navigation.....	185
16 AXI4 Slave Write CRC Checker.....	185
16.1 Overview.....	185
16.1.1 Key Features.....	185
16.2 Module Purpose.....	186
16.3 Parameters.....	186
16.4 Port Groups.....	187
16.4.1 Clock and Reset.....	187
16.4.2 Test Control.....	187
16.4.3 Per-Channel Telemetry.....	188
16.4.4 AXI4 Write Address Channel (AW).....	188
16.4.5 AXI4 Write Data Channel (W).....	189
16.4.6 AXI4 Write Response Channel (B).....	189
16.4.7 Status.....	190
16.5 Functional Description.....	190
16.5.1 Data Slice Extraction.....	190
16.5.2 Per-Channel CRC-32.....	190
16.5.3 Write Burst FSM and Gapless Bursts.....	190
16.5.4 Back-to-Back B Response Id Latching.....	190
16.5.5 Standard Protocol Handler.....	191
16.6 Usage Example.....	191

16.7 Design Notes.....	192
16.8 Related Modules.....	192
16.8.1 Used By.....	192
16.8.2 Uses.....	192
16.8.3 See Also.....	193
16.9 References.....	193
16.9.1 Source Code.....	193
16.9.2 Documentation.....	193
16.10 Navigation.....	193
17 AXI Bus Meter.....	193
17.1 Overview.....	193
17.1.1 Key Features.....	194
17.2 Module Purpose.....	194
17.3 Parameters.....	195
17.4 Port Groups.....	195
17.4.1 Clock and Reset.....	195
17.4.2 Window Control.....	195
17.4.3 AXI Bus Snoop.....	195
17.4.4 Aggregate Counters.....	196
17.4.5 Per-Channel Counters.....	196
17.5 Functional Description.....	197
17.5.1 Bucket Classification.....	197
17.5.2 Aggregate Counters.....	197
17.5.3 Per-Channel Counters and Overflow Stickies.....	197

17.5.4 Recommended Wiring.....	198
17.5.5 Clear and Freeze Semantics.....	198
17.6 Usage Example.....	198
17.7 Design Notes.....	199
17.7.1 Passive Observer.....	199
17.7.2 Counter Width Rationale.....	199
17.7.3 Verilator Output Drivers.....	199
17.7.4 Window Alignment.....	200
17.8 Related Modules.....	200
17.8.1 Used By.....	200
17.8.2 Uses.....	200
17.8.3 See Also.....	200
17.9 References.....	200
17.9.1 Source Code.....	200
17.9.2 Documentation.....	200
17.10 Navigation.....	201
18 AXI Burst Address Generator.....	201
18.1 Overview.....	201
18.1.1 Key Features.....	201
18.2 Parameters.....	201
18.3 Ports.....	202
18.4 Functional Description.....	202
18.4.1 Increment Calculation.....	202
18.4.2 Burst Type Handling.....	202

18.4.3 Aligned Address Output.....	203
18.5 Usage Example.....	203
18.6 Design Notes.....	203
18.6.1 Data Width Conversion.....	203
18.6.2 WRAP Burst Alignment.....	204
18.6.3 Pure Combinational.....	204
18.7 Related Modules.....	204
18.7.1 Used By.....	204
18.7.2 Related Infrastructure.....	204
18.8 References.....	204
18.8.1 Specifications.....	204
18.8.2 Source Code.....	204
18.9 Navigation.....	204
19 AXI Master Read Splitter.....	205
19.1 Overview.....	205
19.1.1 Key Features.....	205
19.2 Module Purpose.....	205
19.3 Parameters.....	206
19.4 Port Groups.....	206
19.4.1 Clock and Reset.....	206
19.4.2 Configuration Interface.....	207
19.4.3 Master AXI Read Interface (Downstream).....	207
19.4.4 Slave AXI Read Interface (Upstream / FUB).....	208
19.4.5 Split Information Interface.....	210

19.5 Functional Description.....	210
19.5.1 Transaction Splitting Overview.....	210
19.5.2 State Machine Architecture.....	211
19.5.3 Boundary Crossing Detection.....	211
19.5.4 AR Channel Forwarding Logic.....	211
19.5.5 Ready Signal Management.....	211
19.5.6 R Channel Handling.....	212
19.5.7 Split Information Tracking.....	212
19.6 Usage Example.....	212
19.6.1 Basic Integration (4KB Boundary Alignment).....	212
19.6.2 Transaction Splitting Example Scenario.....	214
19.7 Design Notes.....	214
19.7.1 AXI Protocol Assumptions.....	214
19.7.2 Performance Characteristics.....	215
19.7.3 Error Handling.....	215
19.7.4 Integration Considerations.....	215
19.8 Related Modules.....	216
19.8.1 Used By.....	216
19.8.2 Uses.....	216
19.8.3 See Also.....	216
19.9 References.....	216
19.9.1 Source Code.....	216
19.9.2 Documentation.....	216
19.10 Navigation.....	217

20 AXI Master Write Splitter.....	217
20.1 Overview.....	217
20.1.1 Key Features.....	217
20.2 Module Purpose.....	217
20.3 Parameters.....	218
20.4 Port Groups.....	219
20.4.1 Clock and Reset.....	219
20.4.2 Configuration Interface.....	219
20.4.3 Master AXI Write Interface (Downstream).....	219
20.4.4 Slave AXI Write Interface (Upstream / FUB).....	221
20.4.5 Split Information Interface.....	223
20.5 Functional Description.....	223
20.5.1 Write Transaction Splitting Overview.....	223
20.5.2 State Machine Architecture.....	224
20.5.3 Write Data Channel Management.....	224
20.5.4 Response Consolidation Logic.....	224
20.5.5 Boundary Crossing Detection.....	225
20.5.6 AW Channel Forwarding Logic.....	225
20.5.7 Ready Signal Management.....	226
20.6 Usage Example.....	226
20.6.1 Basic Integration (4KB Boundary, Response Consolidation).....	226
20.6.2 Response Consolidation Example.....	228
20.7 Design Notes.....	228
20.7.1 AXI Protocol Assumptions.....	228

20.7.2 Response Consolidation Details.....	228
20.7.3 Performance Characteristics.....	229
20.7.4 Error Handling.....	229
20.7.5 Integration Considerations.....	229
20.8 Related Modules.....	230
20.8.1 Used By.....	230
20.8.2 Uses.....	230
20.8.3 See Also.....	230
20.9 References.....	230
20.9.1 Source Code.....	230
20.9.2 Documentation.....	230
20.10 Navigation.....	231
21 AXI Performance Latency Histogram.....	231
21.1 Overview.....	231
21.1.1 Key Features.....	231
21.2 Module Purpose.....	231
21.3 Parameters.....	232
21.4 Port Groups.....	233
21.4.1 Clock and Reset.....	233
21.4.2 Window Control.....	233
21.4.3 Command Channel (AR for reads, AW for writes).....	233
21.4.4 Read Data Channel (R) — used when IS_READ=1.....	233
21.4.5 Write Response Channel (B) — used when IS_READ=0.....	234
21.4.6 CSR-Indexed Readout.....	234

21.5 Functional Description.....	234
21.5.1 Metrics.....	234
21.5.2 Transaction Matching.....	234
21.5.3 Log2 Binning.....	235
21.5.4 Free-Running Timestamp.....	235
21.5.5 Four-Stage Update Pipeline.....	235
21.5.6 Window Control.....	235
21.5.7 Readout.....	236
21.6 Usage Example.....	236
21.7 Design Notes.....	237
21.7.1 Snoop-Only, Base Untouched.....	237
21.7.2 Timestamp Not Frozen.....	237
21.7.3 FIFO Depth vs Outstanding.....	237
21.7.4 Pipeline Drain Before Readback.....	237
21.8 Related Modules.....	237
21.8.1 Used By.....	237
21.8.2 Uses.....	237
21.8.3 See Also.....	238
21.9 References.....	238
21.9.1 Source Code.....	238
21.9.2 Documentation.....	238
21.10 Navigation.....	238
22 AXI-Stream Master Pattern Generator.....	238
22.1 Overview.....	238

22.1.1 Key Features.....	239
22.2 Module Purpose.....	239
22.3 Parameters.....	239
22.4 Port Groups.....	241
22.4.1 Clock and Reset.....	241
22.4.2 Configuration / Control.....	241
22.4.3 Per-Channel Telemetry.....	242
22.4.4 AXI-Stream Master (m_axis).....	242
22.5 Functional Description.....	243
22.5.1 Scheduling FSM.....	243
22.5.2 Sequential Per-Channel Scheduling.....	243
22.5.3 Per-Channel LFSR + CRC (Verbatim from the AXI4 blocks).....	243
22.5.4 Stream Outputs and tlast Cadence.....	243
22.5.5 Backpressure Insensitivity.....	243
22.6 Usage Example.....	244
22.7 Design Notes.....	244
22.8 Related Modules.....	245
22.8.1 Used By.....	245
22.8.2 Uses.....	245
22.8.3 See Also.....	245
22.9 References.....	245
22.9.1 Source Code.....	245
22.9.2 Documentation.....	246
22.10 Navigation.....	246

23 AXI-Stream Slave Pattern Checker.....	246
23.1 Overview.....	246
23.1.1 Key Features.....	246
23.2 Module Purpose.....	247
23.3 Parameters.....	247
23.4 Port Groups.....	249
23.4.1 Clock and Reset.....	249
23.4.2 Configuration / Control.....	249
23.4.3 Telemetry.....	249
23.4.4 AXI-Stream Slave (s_axis).....	250
23.5 Functional Description.....	250
23.5.1 Readiness and Beat Handshake.....	250
23.5.2 Per-Channel LFSR + CRC Regeneration (Verbatim from the Generator)	250
23.5.3 Per-Beat Compare and Sticky Error.....	251
23.5.4 Packet Counting.....	251
23.6 Usage Example.....	251
23.7 Design Notes.....	252
23.8 Related Modules.....	252
23.8.1 Used By.....	252
23.8.2 Uses.....	253
23.8.3 See Also.....	253
23.9 References.....	253
23.9.1 Source Code.....	253

23.9.2 Documentation.....	253
23.10 Navigation.....	253
24 AXIS Bus Meter.....	253
24.1 Overview.....	254
24.1.1 Key Features.....	254
24.2 Module Purpose.....	254
24.3 Parameters.....	255
24.4 Port Groups.....	255
24.4.1 Clock and Reset.....	255
24.4.2 Window Control.....	255
24.4.3 AXIS Bus Snoop.....	256
24.4.4 Aggregate Cycle Buckets.....	256
24.4.5 Aggregate AXIS-Native Throughput (productive beats only).....	256
24.4.6 Per-Channel Cycle Buckets (binned by tid).....	257
24.5 Functional Description.....	257
24.5.1 Bucket Classification.....	257
24.5.2 Payload-Byte Popcount.....	258
24.5.3 AXIS-Native Throughput Counters.....	258
24.5.4 Per-Channel Cycle Buckets.....	258
24.5.5 Clear and Freeze Semantics.....	258
24.6 Usage Example.....	258
24.7 Design Notes.....	259
24.7.1 Why Byte/Packet Counters Are Window-Independent.....	259
24.7.2 Per-Channel Idle Is Not Attributed.....	260

24.7.3 tstrb Handling.....	260
24.7.4 Counter Widths.....	260
24.8 Related Modules.....	260
24.8.1 Used By.....	260
24.8.2 Uses.....	260
24.8.3 See Also.....	260
24.9 References.....	260
24.9.1 Source Code.....	260
24.9.2 Documentation.....	260
24.10 Navigation.....	261
25 AXI Split Combinational Logic.....	261
25.1 Overview.....	261
25.1.1 Key Features.....	261
25.2 Module Purpose.....	261
25.3 Parameters.....	262
25.4 Port Groups.....	262
25.4.1 Clock and Reset (For Assertions Only).....	262
25.4.2 Input Signals.....	263
25.4.3 Output Signals.....	263
25.5 Functional Description.....	264
25.5.1 Boundary Crossing Detection Algorithm.....	264
25.5.2 Split Length Calculation.....	265
25.5.3 Remaining Length Calculation.....	265
25.5.4 State Machine Helper.....	265

25.6 Design Assumptions.....	265
25.6.1 Assumption 1: Address Alignment.....	265
25.6.2 Assumption 2: Fixed Transfer Size.....	266
25.6.3 Assumption 3: Incrementing Bursts Only.....	266
25.6.4 Assumption 4: No Address Wraparound.....	266
25.7 Usage Example.....	266
25.7.1 Integration with Read Splitter.....	266
25.7.2 Example Calculation Walkthrough.....	268
25.8 Design Notes.....	269
25.8.1 Synthesis Optimization.....	269
25.8.2 Assertion Coverage.....	269
25.8.3 Timing Considerations.....	269
25.8.4 Debug Support.....	270
25.9 Related Modules.....	270
25.9.1 Used By.....	270
25.9.2 Dependencies.....	270
25.9.3 See Also.....	270
25.10 References.....	271
25.10.1 Source Code.....	271
25.10.2 Documentation.....	271
25.11 Navigation.....	271
26 CDC 2-Phase Handshake.....	271
26.1 Overview.....	271
26.1.1 Key Features.....	271

26.1.2 2-Phase vs 4-Phase Comparison.....	272
26.2 Parameters.....	272
26.3 Port Groups.....	272
26.3.1 Source Clock Domain Interface.....	272
26.3.2 Destination Clock Domain Interface.....	273
26.4 Theory of Operation.....	273
26.4.1 Source Domain FSM.....	273
26.4.2 Destination Domain FSM.....	274
26.4.3 Edge Detection.....	274
26.5 Usage Example.....	274
26.5.1 Swapping with 4-Phase.....	274
26.6 Required SDC Constraints.....	275
26.7 Related Modules.....	275
26.8 Resources.....	276
26.9 Navigation.....	276
27 CDC 4-Phase Handshake.....	276
27.1 Overview.....	276
27.1.1 Key Features.....	276
27.2 Module Purpose.....	277
27.3 Parameters.....	277
27.4 Port Groups.....	278
27.4.1 Source Clock Domain Interface.....	278
27.4.2 Destination Clock Domain Interface.....	278
27.5 Functional Description.....	279

27.5.1 CDC Protocol Overview.....	279
27.5.2 Source Domain FSM.....	279
27.5.3 Destination Domain FSM.....	280
27.5.4 Synchronizer Architecture.....	281
27.5.5 Data Stability Guarantee.....	281
27.6 Usage Example.....	282
27.6.1 Basic CDC for Monitor Packets (64-bit).....	282
27.6.2 Configuration Register Update Across Domains.....	283
27.6.3 Burst Transfer with Backpressure.....	284
27.7 Design Notes.....	285
27.7.1 Handshake Protocol Timing.....	285
27.7.2 Reset Considerations.....	285
27.7.3 Data Width Considerations.....	286
27.7.4 Metastability Protection Validation.....	286
27.7.5 Performance Optimization.....	286
27.8 Related Modules.....	287
27.8.1 Used By.....	287
27.8.2 Dependencies.....	287
27.8.3 See Also.....	287
27.9 References.....	287
27.9.1 Source Code.....	287
27.9.2 Documentation.....	287
27.9.3 Industry Standards.....	287
27.10 Navigation.....	288

28 CDC Open-Loop.....	288
28.1 Overview.....	288
28.1.1 Key Features.....	288
28.1.2 When to Use.....	288
28.1.3 When NOT to Use.....	289
28.2 Module Interface.....	289
28.3 Parameters.....	290
28.3.1 Setting STRETCH_CYCLES manually (AUTO_STRETCH=0).....	290
28.3.2 Auto-computed STRETCH (AUTO_STRETCH=1).....	291
28.3.3 Back-to-back pulses.....	291
28.4 Ports.....	292
28.5 Architecture.....	292
28.5.1 Timing Diagram.....	293
28.6 Usage Example.....	293
28.6.1 Manual STRETCH (back-compat default).....	293
28.6.2 Auto STRETCH from clock frequencies.....	293
28.7 SDC Constraints.....	294
28.8 Resources.....	294
28.9 References.....	295
28.10 Navigation.....	295
29 CDC Primer: Clock Domain Crossing Techniques.....	295
29.1 The Three Vector CDC Categories.....	296
29.2 Category 1: Open-Loop CDC.....	296
29.2.1 Module: cdc_open_loop.....	296

29.2.2 Supporting Primitives.....	297
29.3 Category 2: Closed-Loop Handshake CDC.....	297
29.3.1 2-Phase (Toggle / NRZ).....	298
29.3.2 4-Phase (Level / RZ).....	298
29.3.3 Modules.....	298
29.3.4 Protocol-Level CDC Using Handshakes.....	299
29.4 Category 3: Gray-Coded Pointers / Async FIFO.....	299
29.4.1 How It Works.....	299
29.4.2 Standard Gray Code vs Johnson Counter.....	300
29.4.3 Sizing the Depth.....	300
29.4.4 Modules.....	301
29.4.5 Usage Example.....	302
29.4.6 Supporting Primitives.....	302
29.5 Decision Guide: Which CDC Technique?.....	302
29.5.1 Quick Reference.....	303
29.6 Common Mistakes.....	304
29.7 References.....	304
29.8 Navigation.....	305
30 CDC Synchronizer.....	305
30.1 Overview.....	305
30.1.1 Key Features.....	305
30.2 Module Interface.....	305
30.3 Parameters.....	306
30.4 Ports.....	306

30.5 Usage Guidelines.....	306
30.6 Instantiation Example.....	306
30.7 Resources.....	307
31 Clock-Gated Variants Guide.....	307
31.1 Overview.....	307
31.2 Key Principle.....	307
31.3 Common Clock Gating Parameters.....	308
31.4 Architecture.....	308
31.4.1 Clock Gating Domains.....	308
31.5 Usage Patterns.....	311
31.5.1 Pattern 1: Maximum Power Savings.....	311
31.5.2 Pattern 2: Balanced Performance and Power.....	312
31.5.3 Pattern 3: Clock Gating Disabled.....	312
31.6 When to Use Clock-Gated Variants.....	313
31.6.1 Use CG Variant When:.....	313
31.6.2 Use Base Module When:.....	313
31.7 Power Savings Analysis.....	313
31.7.1 Typical Power Reduction by Utilization.....	313
31.7.2 Wake-Up Latency vs Power Savings.....	314
31.8 Synthesis Considerations.....	314
31.8.1 FPGA Implementations.....	314
31.8.2 ASIC Implementations.....	314
31.9 Verification Considerations.....	315
31.9.1 Functional Verification.....	315

31.9.2 Power Verification.....	315
31.10 Available Clock-Gated Variants.....	315
31.10.1 AXI4 Protocol (8 modules).....	315
31.10.2 AXI4-Lite Protocol (8 modules).....	316
31.10.3 APB Protocol (3 modules).....	316
31.10.4 AXIS Protocol (2 modules).....	317
31.11 Detailed Examples.....	317
31.12 Configuration Guidelines.....	317
31.12.1 Step 1: Choose Base Configuration.....	317
31.12.2 Step 2: Enable/Disable Clock Gating.....	317
31.12.3 Step 3: Set Idle Threshold.....	317
31.12.4 Step 4: Configure Gating Domains.....	317
31.12.5 Step 5: Connect Ports.....	318
31.13 Performance Impact.....	318
31.14 Related Documentation.....	318
31.15 Navigation.....	318
32 Shared Infrastructure Documentation Status.....	319
32.1 Completion Status.....	319
32.1.1 Completed Documentation.....	319
32.1.2 New shared infrastructure (no dedicated page, covered above).....	322
32.1.3 Remaining Documentation (15 modules).....	322
32.2 Documentation Template.....	323
32.3 Design Notes.....	325
32.3.1 [Important design aspect 1].....	325

32.4 Related Modules.....	325
32.4.1 Used By.....	325
32.4.2 Uses.....	325
32.5 References.....	325
32.5.1 Specifications.....	325
32.5.2 Source Code.....	325
32.6 Navigation.....	325
32.6.1 axi_monitor_filtered.....	335
32.6.2 axi_monitor_trans_mgr.....	336
32.6.3 axi_monitor_reporter.....	337
32.6.4 axi_monitor_timeout.....	337
32.6.5 axi_monitor_timer.....	338
32.6.6 amba_clock_gate_ctrl.....	338
32.7 Arbitration.....	339
32.7.1 monbus_arbiter.....	339
32.7.2 arbiter_monbus_common.....	340
32.7.3 arbiter_rr_pwm_monbus.....	340
32.7.4 arbiter_wrr_pwm_monbus.....	340
32.8 AXI Utilities.....	341
32.8.1 axi_gen_addr.....	341
32.8.2 axi_master_rd_splitter.....	342
32.8.3 axi_master_wr_splitter.....	342
32.8.4 axi_split_combi.....	342
32.8.5 cdc_4_phase_handshake.....	343

32.9 Testing.....	344
32.10 Design Notes.....	344
32.10.1 Monitor Infrastructure.....	344
32.10.2 When to Use Each Module.....	345
32.11 Configuration Best Practices.....	345
32.11.1 Monitor Configuration.....	345
32.11.2 Timeout Configuration.....	346
32.11.3 Clock Gating Configuration.....	346
32.12 Performance Characteristics.....	346
32.12.1 Monitor Overhead.....	346
32.12.2 CDC Latency.....	346
32.12.3 Arbitration.....	346
32.13 Synthesis Notes.....	347
32.13.1 Clock Gating.....	347
32.13.2 CDC Constraints.....	347
32.14 Related Documentation.....	347
32.14.1 RTL Design Sherpa Documentation.....	347
32.14.2 Monitor Package.....	347
32.14.3 Source Code.....	347
32.15 Summary.....	348
32.16 Navigation.....	348
33 SDPRAM Core.....	348
33.1 Overview.....	348
33.1.1 Key Features.....	349

33.2 Module Purpose.....	349
33.3 Parameters.....	350
33.4 Port Groups.....	350
33.4.1 Clock and Reset.....	350
33.4.2 FUB Write Side (AW + W + B).....	350
33.4.3 FUB Read Side (AR + R).....	352
33.4.4 Bulk-Clear Control.....	352
33.4.5 Observation.....	353
33.5 Functional Description.....	353
33.5.1 FUB Interface Contract.....	353
33.5.2 Write Path — Burst Tracker.....	353
33.5.3 Read Path — Burst Tracker.....	354
33.5.4 Address Generation.....	354
33.5.5 BRAM Array.....	354
33.5.6 Bulk-Clear FSM.....	354
33.5.7 Observation Outputs.....	354
33.6 Usage Example.....	355
33.7 Design Notes.....	356
33.7.1 No Reset on the RAM Array.....	356
33.7.2 One Wire Format, Four Wrappers.....	356
33.7.3 WRAP Bursts.....	356
33.7.4 Single Outstanding Burst Per Side.....	356
33.8 Related Modules.....	356
33.8.1 Used By.....	356

33.8.2 Uses.....	357
33.8.3 See Also.....	357
33.9 References.....	357
33.9.1 Source Code.....	357
33.9.2 Documentation.....	357
33.10 Navigation.....	357
34 SDPRAM Slave — AXI4 Write / AXI4 Read.....	358
34.1 Overview.....	358
34.1.1 Key Features.....	358
34.2 Module Purpose.....	358
34.3 Parameters.....	359
34.4 Port Groups.....	359
34.4.1 Clock and Reset.....	359
34.4.2 AXI4 Slave Write (AW + W + B).....	360
34.4.3 AXI4 Slave Read (AR + R).....	361
34.4.4 Bulk-Clear Control and Debug.....	362
34.5 Functional Description.....	363
34.5.1 Structure.....	363
34.5.2 Field Pass-Through and Tie-Offs.....	363
34.5.3 Memory Behavior.....	364
34.5.4 Debug Taps.....	364
34.5.5 WRAP Assertion.....	364
34.6 Usage Example.....	364
34.7 Design Notes.....	365

34.7.1 Native AXI4, No Emulation.....	365
34.7.2 One Backend, Four Wrappers.....	365
34.7.3 WRAP Support Is Sim-Gated.....	365
34.8 Related Modules.....	365
34.8.1 Used By.....	365
34.8.2 Uses.....	365
34.8.3 See Also.....	366
34.9 References.....	366
34.9.1 Source Code.....	366
34.9.2 Documentation.....	366
34.10 Navigation.....	366
35 SDPRAM Slave — AXI4 Write / AXIL Read.....	366
35.1 Overview.....	366
35.1.1 Key Features.....	367
35.2 Module Purpose.....	367
35.3 Parameters.....	367
35.4 Port Groups.....	368
35.4.1 Clock and Reset.....	368
35.4.2 AXI4 Slave Write (AW + W + B).....	368
35.4.3 AXIL Slave Read (AR + R).....	370
35.4.4 Bulk-Clear Control and Debug.....	370
35.5 Functional Description.....	371
35.5.1 Structure.....	371
35.5.2 AXIL Read Bridge.....	371

35.5.3 Write Path and Tie-Offs.....	372
35.5.4 Memory Behavior.....	372
35.5.5 WRAP Assertion.....	372
35.6 Usage Example.....	372
35.7 Design Notes.....	373
35.7.1 Single-Beat Adaptation Lives in One Place.....	373
35.7.2 One Backend, Four Wrappers.....	373
35.7.3 Read Beat Size.....	373
35.8 Related Modules.....	373
35.8.1 Used By.....	373
35.8.2 Uses.....	373
35.8.3 See Also.....	374
35.9 References.....	374
35.9.1 Source Code.....	374
35.9.2 Documentation.....	374
35.10 Navigation.....	374
36 SDPRAM Slave — AXIL Write / AXI4 Read.....	374
36.1 Overview.....	374
36.1.1 Key Features.....	375
36.2 Module Purpose.....	375
36.3 Parameters.....	375
36.4 Port Groups.....	376
36.4.1 Clock and Reset.....	376
36.4.2 AXIL Slave Write (AW + W + B).....	376

36.4.3 AXI4 Slave Read (AR + R).....	377
36.4.4 Bulk-Clear Control and Debug.....	378
36.5 Functional Description.....	379
36.5.1 Structure.....	379
36.5.2 AXIL Write Bridge.....	379
36.5.3 Read Path and Tie-Offs.....	380
36.5.4 Memory Behavior.....	380
36.5.5 WRAP Assertion.....	380
36.6 Usage Example.....	380
36.7 Design Notes.....	381
36.7.1 Descriptor-RAM Fit.....	381
36.7.2 Single-Beat Adaptation Lives in One Place.....	381
36.7.3 One Backend, Four Wrappers.....	381
36.8 Related Modules.....	381
36.8.1 Used By.....	381
36.8.2 Uses.....	382
36.8.3 See Also.....	382
36.9 References.....	382
36.9.1 Source Code.....	382
36.9.2 Documentation.....	382
36.10 Navigation.....	382
37 SDPRAM Slave — AXIL Write / AXIL Read.....	382
37.1 Overview.....	383
37.1.1 Key Features.....	383

37.2 Module Purpose.....	383
37.3 Parameters.....	384
37.4 Port Groups.....	384
37.4.1 Clock and Reset.....	384
37.4.2 AXIL Slave Write (AW + W + B).....	385
37.4.3 AXIL Slave Read (AR + R).....	385
37.4.4 Bulk-Clear Control and Debug.....	386
37.5 Functional Description.....	387
37.5.1 Structure.....	387
37.5.2 Single-Beat Defaults on Both Sides.....	387
37.5.3 Memory Behavior.....	387
37.5.4 No WRAP Assertion.....	387
37.6 Usage Example.....	387
37.7 Design Notes.....	388
37.7.1 The Only “Fake” Fields, In One Place Per Side.....	388
37.7.2 Monitor-Bus Pairing.....	388
37.7.3 One Backend, Four Wrappers.....	389
37.8 Related Modules.....	389
37.8.1 Used By.....	389
37.8.2 Uses.....	389
37.8.3 See Also.....	389
37.9 References.....	389
37.9.1 Source Code.....	389
37.9.2 Documentation.....	389

37.10 Navigation.....	390
38 Simple Dual-Port BRAM Slave Family.....	390
38.1 Overview.....	390
38.2 Why four wrappers + a backend?.....	390
38.3 Common Parameters.....	391
38.4 Burst Support.....	392
38.5 Bulk Clear.....	392
38.6 Debug / Observation Outputs.....	392
38.7 Use in the Monitor System.....	393
38.8 Test.....	393
38.9 Migration.....	393
38.10 Related Modules.....	394
39 axi4_to_apb_convert.....	394
39.1 Overview.....	395
39.1.1 Key Features.....	395
39.2 Module Declaration.....	395
39.3 Parameters.....	396
39.4 Architecture.....	397
39.4.1 Component Structure.....	397
39.4.2 Data Flow.....	397
39.5 Behavior.....	398
39.5.1 Burst Decomposition.....	398
39.5.2 Address Generation.....	398
39.5.3 Width Adaptation.....	399

39.5.4 Error Handling.....	399
39.5.5 Side Information Tracking.....	399
39.6 Packet Formats.....	400
39.6.1 APB Command Packet.....	400
39.6.2 APB Response Packet.....	400
39.7 Usage Examples.....	400
39.7.1 Example 1: Standalone Conversion (Same Width).....	400
39.7.2 Example 2: Width Conversion (64b → 32b).....	401
39.8 State Machines.....	402
39.9 Performance.....	404
39.9.1 Latency (Cycles).....	404
39.9.2 Throughput.....	404
39.10 Testing.....	404
39.11 Constraints and Limitations.....	405
39.12 Synthesis Notes.....	405
39.12.1 Resource Usage.....	405
39.13 Related Modules.....	405
39.14 When to Use.....	406
39.15 Navigation.....	406
40 axi4_to_apb_shim.....	406
40.1 Overview.....	406
40.1.1 Key Features.....	406
40.2 Module Declaration.....	407
40.3 Parameters.....	409

40.3.1 Skid Buffer Depths.....	409
40.3.2 Internal Buffering.....	409
40.3.3 AXI Interface.....	410
40.3.4 APB Interface.....	410
40.4 Architecture.....	410
40.4.1 Component Hierarchy.....	410
40.4.2 Data Flow.....	411
40.5 Behavior.....	411
40.5.1 AXI to APB Conversion.....	411
40.5.2 Clock Domain Crossing.....	412
40.6 Usage Examples.....	412
40.6.1 Example 1: Same Clock, Same Width.....	412
40.6.2 Example 2: Dual-Clock with Width Conversion.....	412
40.6.3 Example 3: CPU-Peripheral Bridge.....	413
40.7 Performance Characteristics.....	414
40.7.1 Latency.....	414
40.7.2 Throughput.....	414
40.8 Testing.....	414
40.9 Constraints and Limitations.....	415
40.10 Synthesis Notes.....	415
40.10.1 Resource Usage (Typical).....	415
40.10.2 Timing Closure.....	415
40.11 Related Modules.....	416
40.12 When to Use.....	416

40.13 Navigation.....	416
41 peakrdl_to_cmdrsp.....	416
41.1 Overview.....	417
41.1.1 Key Features.....	417
41.2 Background: PeakRDL Integration.....	417
41.3 Module Declaration.....	417
41.4 Parameters.....	419
41.5 Ports.....	419
41.5.1 CMD/RSP Interface (RTL Design Sherpa Standard).....	419
41.5.2 PeakRDL Passthrough Interface.....	420
41.6 Behavior.....	421
41.6.1 Command State Machine.....	421
41.6.2 Response State Machine.....	421
41.6.3 Strobe to Bit Enable Conversion.....	422
41.6.4 Stall Handling.....	422
41.7 Usage Examples.....	422
41.7.1 Example 1: Basic Integration with PeakRDL Register Block.....	422
41.7.2 Example 2: APB to PeakRDL Complete Path.....	424
41.8 Timing Diagrams.....	425
41.8.1 Read Transaction (No Stall).....	425
41.8.2 Write Transaction with Stall.....	425
41.9 Testing.....	426
41.10 Assertions.....	426
41.11 Performance.....	426

41.11.1 Latency.....	426
41.11.2 Throughput.....	427
41.12 Synthesis Notes.....	427
41.12.1 Resource Usage.....	427
41.13 Related Modules.....	427
41.14 When to Use.....	427
41.15 PeakRDL Workflow.....	428
41.16 Navigation.....	428
42 AMBA Protocol Shims.....	428
42.1 Overview.....	428
42.1.1 Key Features.....	428
42.2 Available Shims.....	429
42.3 Quick Start.....	429
42.3.1 AXI4 to APB Bridge.....	429
42.3.2 PeakRDL Register Integration.....	430
42.4 Design Patterns.....	430
42.4.1 Pattern 1: CPU to Peripherals Bridge.....	430
42.4.2 Pattern 2: High-Speed to Low-Speed Bridge.....	431
42.4.3 Pattern 3: Register Block Integration.....	431
42.5 Performance Characteristics.....	431
42.5.1 AXI4 to APB Bridge.....	431
42.5.2 PeakRDL Adapter.....	432
42.6 Testing.....	432
42.7 Common Use Cases.....	433

42.7.1 Use Case 1: SoC Integration.....	433
42.7.2 Use Case 2: DMA to Peripherals.....	433
42.7.3 Use Case 3: Configuration Registers.....	433
42.8 Parameter Selection Guidelines.....	434
42.8.1 AXI to APB Bridge.....	434
42.8.2 PeakRDL Adapter.....	434
42.9 Design Tradeoffs.....	435
42.9.1 AXI to APB Bridge.....	435
42.9.2 PeakRDL Adapter.....	435
42.10 Synthesis Notes.....	435
42.10.1 Resource Usage.....	435
42.10.2 Timing Closure.....	435
42.11 Related Documentation.....	436
42.11.1 Protocol Specifications.....	436
42.11.2 RTL Design Sherpa Documentation.....	436
42.11.3 External Resources.....	436
42.11.4 Source Code.....	436
42.12 Design Notes.....	437
42.12.1 When to Use Shims.....	437
42.12.2 Clock Domain Crossing.....	437
42.13 Migration Guide.....	437
42.13.1 Upgrading from Legacy AXI-APB Bridges.....	437
42.14 Troubleshooting.....	438
42.14.1 Issue 1: High Latency.....	438

42.14.2 Issue 2: PeakRDL Stalls.....	438
42.14.3 Issue 3: Data Corruption.....	438
42.15 Navigation.....	438

1 GAXI Drop FIFO Sync

1.1 Overview

The `gaxi_drop_fifo_sync` module is a synchronous FIFO with dynamic drop capability. It provides standard GAXI (Generic AXI) valid/ready handshaking for both write and read operations, plus an additional drop interface for removing entries from the FIFO without reading them.

Key Features: - Standard GAXI valid/ready handshaking - Configurable data width and depth - Drop by count (remove N oldest entries) - Drop all (flush entire FIFO) - I/O blocking during drop operations (3-cycle latency) - Registered or mux-based output modes - FIFO count and almost-full/almost-empty flags

Location: `rtl/amba/gaxi/gaxi_drop_fifo_sync.sv`

Testbench: `val/amba/test_gaxi_drop_fifo_sync.py`

1.2 Module Parameters

Parameter	Type	Default	Description
DATA_WIDTH	int	32	Width of data bus in bits
DEPTH	int	16	FIFO depth (must be power of 2)
REGISTERED	int	0	0 = mux mode, 1 = registered output
ALMOST_WR_MARGIN	int	1	Threshold for almost_full flag
ALMOST_RD_MARGIN	int	1	Threshold for almost_empty flag

1.3 Interface Signals

1.3.1 Clock and Reset

Signal	Direction	Width	Description
axi_aclk	input	1	Clock signal
axi_aresetn	input	1	Active-low asynchronous reset

1.3.2 Write Interface (GAXI Master → FIFO)

Signal	Direction	Width	Description
wr_valid	input	1	Write data valid
wr_ready	output	1	FIFO ready to accept write
wr_data	input	DATA_WIDTH	Write data

Handshake: Write occurs when wr_valid && wr_ready on rising edge of clock.

1.3.3 Read Interface (FIFO → GAXI Slave)

Signal	Direction	Width	Description
rd_valid	output	1	Read data valid
rd_ready	input	1	Downstream ready to accept read
rd_data	output	DATA_WIDTH	Read data

Handshake: Read occurs when rd_valid && rd_ready on rising edge of clock.

1.3.4 Drop Interface

Signal	Direction	Width	Description
drop_valid	input	1	Drop request valid
drop_ready	output	1	Drop operation complete
drop_count	input	8	Number of

Signal	Direction	Width	Description
drop_all	input	1	entries to drop Drop all entries (ignore count)

Handshake: Drop completes when drop_valid && drop_ready on rising edge of clock.

Drop Latency: 3 clock cycles from drop_valid assertion to drop_ready assertion.

1.3.5 Status Signals

Signal	Direction	Width	Description
count	output	$\lceil \log_2(\text{DEPTH} + 1) \rceil$	Current number of entries in FIFO
almost_full	output	1	High when count $\geq$ (DEPTH - ALMOST_WR_MARGIN)
almost_empty	output	1	High when count $\leq$ ALMOST_RD_MARGIN

1.4 Drop Operation Behavior

1.4.1 Drop by Count

When drop_valid=1 and drop_all=0, the FIFO removes drop_count oldest entries:

- Cycle 0:** Assert drop_valid, set drop_count=N
- Cycle 1-2:** Drop operation in progress, drop_ready=0, I/O blocked
- Cycle 3:** Drop complete, drop_ready=1
- Result:** FIFO count decreases by min(N, current_count)

I/O Blocking: During drop operation (cycles 1-2): - wr_ready = 0 (writes blocked) - rd_valid = 0 (reads blocked)

1.4.2 Drop All

When drop_valid=1 and drop_all=1, the FIFO is completely flushed:

- Cycle 0:** Assert drop_valid and drop_all=1
- Cycle 1-2:** Drop operation in progress
- Cycle 3:** Drop complete, drop_ready=1

4. **Result:** FIFO count becomes 0

Note: drop_count is ignored when drop_all=1.

1.4.3 Simultaneous Operations

Write + Drop: Blocked - writes cannot proceed during drop operation.

Read + Drop: Blocked - reads cannot proceed during drop operation.

Drop + Drop: Not supported - wait for drop_ready before issuing next drop.

1.5 Timing Diagrams

1.5.1 Fill FIFO

Generated by test_gaxi_drop_fifo_wavedrom.py

This scenario shows writing 4 entries to an empty FIFO: - Write handshakes occur when wr_valid && wr_ready - FIFO count increments with each successful write - Data values: 0xA0, 0xA1, 0xA2, 0xA3

[WaveDrom diagram: gaxi_drop_fifo_fill.json]

1.5.2 Drop by Count

Generated by test_gaxi_drop_fifo_wavedrom.py

This scenario demonstrates dropping 2 oldest entries: - drop_valid asserted with drop_count=2 - 3-cycle latency until drop_ready assertion - FIFO count decreases by 2 - Normal I/O blocked during drop

[WaveDrom diagram: gaxi_drop_fifo_drop_by_count.json]

1.5.3 Drop All

Generated by test_gaxi_drop_fifo_wavedrom.py

This scenario shows flushing the entire FIFO: - drop_valid asserted with drop_all=1 - 3-cycle latency until drop_ready assertion - FIFO count becomes 0 - All entries discarded

[WaveDrom diagram: gaxi_drop_fifo_drop_all.json]

1.5.4 Comprehensive Scenario

Generated by test_gaxi_drop_fifo_wavedrom.py

This scenario demonstrates mixed operations: - Read 1 entry from FIFO - Drop remaining entries with drop_all=1 - Shows interaction between read and drop interfaces

[WaveDrom diagram: gaxi_drop_fifo_comprehensive.json]

1.6 Usage Examples

1.6.1 Basic Write/Read

```
// Instantiate FIFO
gaxi_drop_fifo_sync #(
    .DATA_WIDTH(32),
    .DEPTH(16),
    .REGISTERED(0)
) u_fifo (
    .axi_aclk      (clk),
    .axi_aresetn   (rst_n),
    .wr_valid       (wr_valid),
    .wr_ready       (wr_ready),
    .wr_data        (wr_data),
    .rd_valid       (rd_valid),
    .rd_ready       (rd_ready),
    .rd_data        (rd_data),
    .drop_valid     (1'b0),           // No drop
    .drop_ready     (),
    .drop_count     (8'h0),
    .drop_all       (1'b0),
    .count          (fifo_count),
    .almost_full    (almost_full),
    .almost_empty   (almost_empty)
);

// Write logic
always_ff @(posedge clk) begin
    if (!rst_n) begin
        wr_valid <= 1'b0;
    end else if (wr_valid && wr_ready) begin
        wr_valid <= 1'b0; // Deassert after handshake
    end else if (have_data_to_write && !almost_full) begin
        wr_valid <= 1'b1;
        wr_data  <= next_data;
    end
end
```

1.6.2 Drop Operation

```
// Drop state machine
typedef enum logic [1:0] {
    IDLE,
    DROP_WAIT,
    DROP_DONE
} drop_state_t;
```

```

drop_state_t state;

always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        state <= IDLE;
        drop_valid <= 1'b0;
    end else begin
        case (state)
            IDLE: begin
                if (trigger_drop_all) begin
                    drop_valid <= 1'b1;
                    drop_all <= 1'b1;
                    drop_count <= 8'h0; // Ignored when drop_all=1
                    state <= DROP_WAIT;
                end else if (trigger_drop_count) begin
                    drop_valid <= 1'b1;
                    drop_all <= 1'b0;
                    drop_count <= num_to_drop;
                    state <= DROP_WAIT;
                end
            end
            DROP_WAIT: begin
                if (drop_ready) begin
                    drop_valid <= 1'b0;
                    state <= DROP_DONE;
                end
            end
            DROP_DONE: begin
                // Single-cycle state
                state <= IDLE;
            end
        endcase
    end
end

```

1.7 Test Coverage

Comprehensive verification is provided in `val/amba/test_gaxi_drop_fifo_sync.py`:

Test Scenario	Coverage
Basic FIFO Operation	Standard write/read without drops
Drop by Count	Partial FIFO flush with specific

Test Scenario	Coverage
	count
Drop All	Complete FIFO flush
Drop During I/O	Verify I/O blocking during drop
Wraparound with Drop	Drop across pointer wraparound boundary

Test Configurations: - Data widths: 8, 32, 64 bits - Depths: 8, 16, 64, 256 entries - Modes: Mux (REGISTERED=0) and Flop (REGISTERED=1)

Results: All 8 parameterized test cases pass ✓

1.7.1 Running Tests

Run all drop FIFO tests

```
pytest val/amba/test_gaxi_drop_fifo_sync.py -v
```

Run specific test configuration

```
pytest
"val/amba/test_gaxi_drop_fifo_sync.py::test_gaxi_drop_fifo_sync[8-8-0-
minimal-mux]" -v
```

Run smoke test (quick verification)

```
pytest val/amba/test_gaxi_drop_fifo_sync.py::test_gaxi_drop_fifo_smoke
-v
```

Generate waveforms for documentation

```
env ENABLE_WAVEDROM=1 pytest val/amba/test_gaxi_drop_fifo_wavedrom.py
-v
```

1.8 Design Notes

1.8.1 FIFO Depth Requirements

MUST be a power of 2 for efficient address pointer implementation.

Valid: 8, 16, 32, 64, 128, 256, ...

Invalid: 10, 15, 20, 100, ...

1.8.2 Registered vs. Mux Mode

Mux Mode (REGISTERED=0): - Lower latency (combinatorial path from FIFO RAM to rd_data) - Better for timing-critical paths where output is registered downstream - Simpler implementation

Registered Mode (REGISTERED=1): - Extra register stage on rd_data output - Better timing closure for long data paths - Adds 1 cycle of read latency

1.8.3 Drop Latency

The 3-cycle drop latency is inherent to the implementation: 1. **Cycle 1:** Latch drop request 2. **Cycle 2:** Update read pointer 3. **Cycle 3:** Assert drop_ready

This latency ensures clean separation between normal FIFO operations and drop operations.

1.8.4 Almost Full/Empty Margins

Configure ALMOST_WR_MARGIN and ALMOST_RD_MARGIN based on: - Pipeline depth of upstream producer - Pipeline depth of downstream consumer - Latency tolerance requirements

Example: If upstream has 4-cycle latency, set ALMOST_WR_MARGIN $\geq$ 4.

1.9 Related Modules

- `fifo_control.sv` - Core FIFO control logic
- `counter_bin.sv` - Binary counter for address generation
- `counter_bin_load.sv` - Loadable binary counter for drop pointer updates
- `gaxi_drop_fifo_async.sv` - Asynchronous clock domain version (future)

1.10 Revision History

Version	Date	Description
1.0	2025-10-17	Initial documentation with WaveDrom examples

2 GAXI Asynchronous FIFO

Module: `gaxi_fifo_async.sv` **Location:** `rtl/amba/gaxi/` **Status:** ✓ Production Ready

2.1 Overview

The GAXI asynchronous FIFO enables safe clock domain crossing (CDC) between independent clock domains. It uses Gray code pointers and multi-flop synchronizers to prevent metastability and ensure data integrity.

2.1.1 Key Features

- ✓ **Clock Domain Crossing:** Safe transfer between independent clocks
 - ✓ **Gray Code Pointers:** Prevents multi-bit synchronization issues
 - ✓ **Configurable CDC Stages:** 2-3 flop synchronizers (3 recommended)
 - ✓ **Arbitrary Clock Ratios:** Works with any write:read clock ratio
 - ✓ **Two Read Modes:** Mux or Flop mode
-

2.2 Module Interface

```
module gaxi_fifo_async #(
    parameter int REGISTERED = 0,           // 0=mux mode, 1=flop mode
    parameter int DATA_WIDTH = 8,
    parameter int DEPTH = 10,
    parameter int N_FLOP_CROSS = 2,        // CDC synchronizer stages
    parameter int ALMOST_WR_MARGIN = 1,
    parameter int ALMOST_RD_MARGIN = 1,
    parameter      INSTANCE_NAME = "DEADF1F0"
) (
    // Write Domain
    input  logic      axi_wr_aclk,
    input  logic      axi_wr_aresetn,
    input  logic      wr_valid,
    output logic      wr_ready,           // not full
    input  logic [DW-1:0] wr_data,

    // Read Domain
    input  logic      axi_rd_aclk,
    input  logic      axi_rd_aresetn,
    input  logic      rd_ready,
    output logic      rd_valid,         // not empty
    output logic [DW-1:0] rd_data
);
```

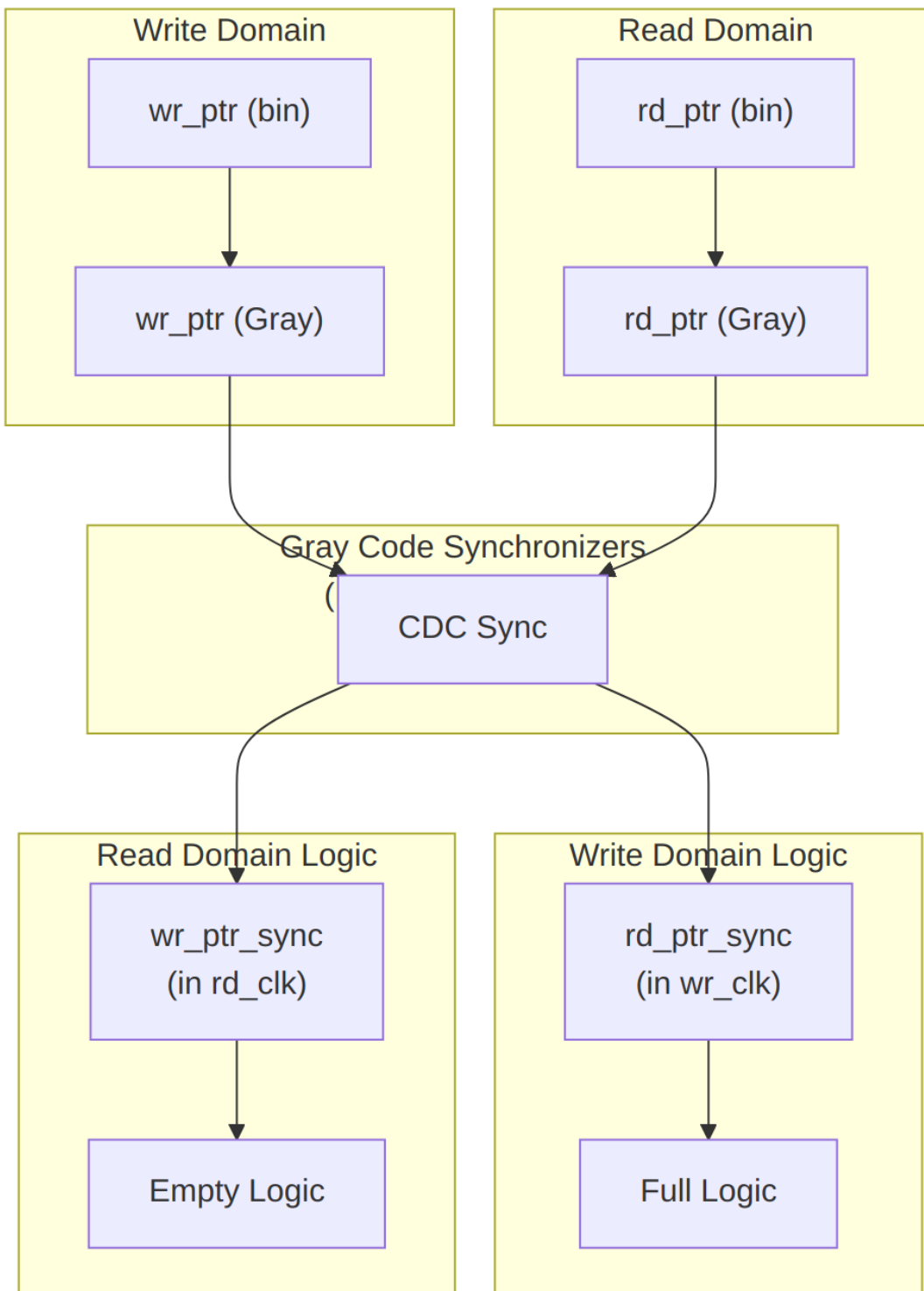
2.3 Parameters

Parameter	Default	Description
REGISTERED	0	0=mux mode, 1=flop mode (read path)
DATA_WIDTH	8	Data bus width
DEPTH	10	FIFO depth (even

Parameter	Default	Description
N_FLOP_CROSS	2	values recommended) Synchronizer stages (3 recommended for safety)
ALMOST_WR_MARGIN	1	Almost full threshold
ALMOST_RD_MARGIN	1	Almost empty threshold

 **Important:** Set N_FLOP_CROSS=3 for production designs to ensure metastability protection.

2.4 Functional Description



Clock Domain Crossing Architecture

2.4.1 Why Gray Code?

Gray code ensures only one bit changes at a time during pointer updates:

Binary: 011 → 100 (2 bits change - hazard!)

Gray: 010 → 110 (1 bit changes - safe!)

This prevents glitches during synchronization across clock domains.

2.4.2 Dependencies

- counter_bin.sv - Binary counters
 - counter_johnson.sv - Gray code (Johnson) counters
 - glitch_free_n_dff_arn.sv - Multi-flop synchronizers
 - grayj2bin.sv - Gray-to-binary conversion
 - fifo_control.sv - Full/empty flag generation
-

2.5 Usage Examples

2.5.1 Example 1: Basic CDC FIFO

```
gaxi_fifo_async #(
    .DATA_WIDTH(32),
    .DEPTH(16),
    .N_FLOP_CROSS(3),      // 3-stage synchronizer
    .REGISTERED(0)
) u_cdc_fifo (
    // Write domain @ 100 MHz
    .axi_wr_aclk    (clk_100mhz),
    .axi_wr_aresetn (rst_100_n),
    .wr_valid       (domain_a_valid),
    .wr_ready       (domain_a_ready),
    .wr_data        (domain_a_data),

    // Read domain @ 50 MHz
    .axi_rd_aclk    (clk_50mhz),
    .axi_rd_aresetn (rst_50_n),
    .rd_ready       (domain_b_ready),
    .rd_valid       (domain_b_valid),
    .rd_data        (domain_b_data)
);
```

2.5.2 Example 2: High-Speed to Low-Speed CDC

```
// Fast writer (250 MHz) → Slow reader (62.5 MHz)
// Needs deeper FIFO to handle burst traffic
gaxi_fifo_async #(
    .DATA_WIDTH(64),
    .DEPTH(32),           // Deeper for rate mismatch
    .N_FLOP_CROSS(3),
    .REGISTERED(1)       // Flop mode for timing
) u_rate_converter (
    .axi_wr_aclk   (clk_250mhz),
    .axi_wr_aresetn (rst_fast_n),
    .wr_valid      (fast_valid),
    .wr_ready      (fast_ready),
    .wr_data       (fast_data),

    .axi_rd_aclk   (clk_62p5mhz),
    .axi_rd_aresetn (rst_slow_n),
    .rd_ready      (slow_ready),
    .rd_valid      (slow_valid),
    .rd_data       (slow_data)
);
```

2.6 Timing Characteristics

Clock Ratio (wr:rd)	Latency	Notes
1:1 (same freq)	3-5 cycles	CDC synchronizat ion overhead
2:1 (fast→slow)	4-6 cycles	Additional write-side delay
1:2 (slow→fast)	3-5 cycles	Read-side samples faster
Any ratio	3-7 cycles	Depends on synchronizer stages + clock relationship

Latency Formula: $\sim (2 \times N_FLOP_CROSS) + 1$ in slower clock domain cycles

2.7 Design Considerations

2.7.1 Depth Sizing for Clock Ratio

When write clock >> read clock, size FIFO to handle burst accumulation:

Required Depth $\geq$ Burst Size $\times$ (Write Freq / Read Freq) $\times$ Safety Margin

Example:

- Write: 100 MHz, Read: 25 MHz
 - Burst: 16 transfers
 - Safety: 1.5x
- Depth $\geq 16 \times (100/25) \times 1.5 = 96$ entries

2.7.2 Reset Synchronization

Critical: Both clock domains must have properly synchronized resets!

// Separate reset synchronizers for each domain

```
reset_sync u_wr_rst_sync (  
    .clk(axi_wr_aclk),  
    .async_rst_n(global_rst_n),  
    .sync_rst_n(axi_wr_aresetn)  
);
```

```
reset_sync u_rd_rst_sync (  
    .clk(axi_rd_aclk),  
    .async_rst_n(global_rst_n),  
    .sync_rst_n(axi_rd_aresetn)  
);
```

2.7.3 Metastability Protection

N_FLOP_CROSS determines MTBF (Mean Time Between Failures):

Stages	MTBF (typical @ 100 MHz)	Use Case
2	~years	Short-term prototyping only
3	~centuries	Production standard
4	~millennia	Ultra-critical systems

Recommendation: Always use N_FLOP_CROSS=3 in production.

2.8 Error Checking

Simulation-only assertions catch protocol violations:

```
// Write domain checks
always @(posedge axi_wr_aclk) begin
    if (!axi_wr_aresetn && wr_valid && !wr_ready) begin
        $display("Error: %s write while FIFO full", INSTANCE_NAME);
    end
end

// Read domain checks
always @(posedge axi_rd_aclk) begin
    if (!axi_rd_aresetn && rd_ready && !rd_valid) begin
        $display("Error: %s read while FIFO empty", INSTANCE_NAME);
    end
end
```

2.9 Testing

```
# Async FIFO tests with various clock ratios
pytest val/amba/test_gaxi_buffer_async.py -v

# Test specific clock ratio (wr:rd)
pytest val/amba/test_gaxi_buffer_async.py -k "wr10_rd12" -v # 10ns : 12ns
```

Test matrix includes: - Same clocks (1:1) - 1.25x ratio (10ns : 12ns) - 2x ratio (10ns : 20ns) - 2.5x ratio (8ns : 20ns)

2.10 Common Issues

2.10.1 Issue 1: Metastability

Symptom: Random data corruption, simulation/hardware mismatch

Solution: Increase N_FLOP_CROSS to 3 or 4

2.10.2 Issue 2: Pointer Synchronization Failure

Symptom: FIFO full/empty signals incorrect

Debug: 1. Verify clocks are truly independent (no PLL relationship violations) 2. Check reset synchronization - both domains must reset properly 3. Verify Gray code conversion logic

2.10.3 Issue 3: Underflow/Overflow

Symptom: Data loss or corruption

Debug: 1. Check DEPTH is sufficient for clock ratio 2. Verify flow control respected in both domains 3. Monitor pointer values in both domains

2.11 Related Modules

- [gaxi_fifo_sync](#) - Single clock domain version
 - [gaxi_skid_buffer_async](#) - Async skid buffer
 - [GAXI Index](#) - Overview
-

2.12 References

- **Clifford Cummings:** “Simulation and Synthesis Techniques for Asynchronous FIFO Design” (Sunburst Design)
 - **Source:** rtl/amba/gaxi/gaxi_fifo_async.sv
 - **Tests:** val/amba/test_gaxi_buffer_async.py
-

Version: 1.0 **Last Updated:** 2025-10-06

3 GAXI Synchronous FIFO

Module: gaxi_fifo_sync.sv **Location:** rtl/amba/gaxi/ **Status:** ✓ Production Ready

3.1 Overview

The GAXI synchronous FIFO provides elastic buffering for any depth (power of 2 recommended). Unlike the skid buffer, this module uses a true FIFO architecture with read/write pointers, supporting larger depths efficiently.

3.1.1 Key Features

- ✓ **Arbitrary Depth:** Any depth supported (power of 2 optimal)

- ✓ **Two Read Modes:** Mux mode (combinatorial) or Flop mode (registered)
- ✓ **Counter-Based:** Binary counters with wrapping
- ✓ **Almost Full/Empty:** Configurable thresholds
- ✓ **Single Clock Domain:** Synchronous design

3.2 Module Interface

```

module gaxi_fifo_sync #(
    parameter int REGISTERED = 0,           // 0=mux mode, 1=flop mode
    parameter int DATA_WIDTH = 4,
    parameter int DEPTH = 4,
    parameter int ALMOST_WR_MARGIN = 1,
    parameter int ALMOST_RD_MARGIN = 1,
    parameter      INSTANCE_NAME = "DEADF1F0"
) (
    input  logic      axi_aclk,
    input  logic      axi_aresetn,
    input  logic      wr_valid,
    output logic      wr_ready,           // not full
    input  logic [DW-1:0] wr_data,
    input  logic      rd_ready,
    output logic [AW:0] count,
    output logic      rd_valid,         // not empty
    output logic [DW-1:0] rd_data
);

```

3.3 Parameters

Parameter	Default	Description
REGISTERED	0	0=mux mode (comb read), 1=flop mode (reg read)
DATA_WIDTH	4	Data bus width
DEPTH	4	FIFO depth (any value, power-of-2 optimal)
ALMOST_WR_MARGIN	1	Almost full threshold
ALMOST_RD_MARGIN	1	Almost empty threshold

Parameter	Default	Description
INSTANCE_NAME	“DEADF1F0”	Debug instance name

3.4 Functional Description

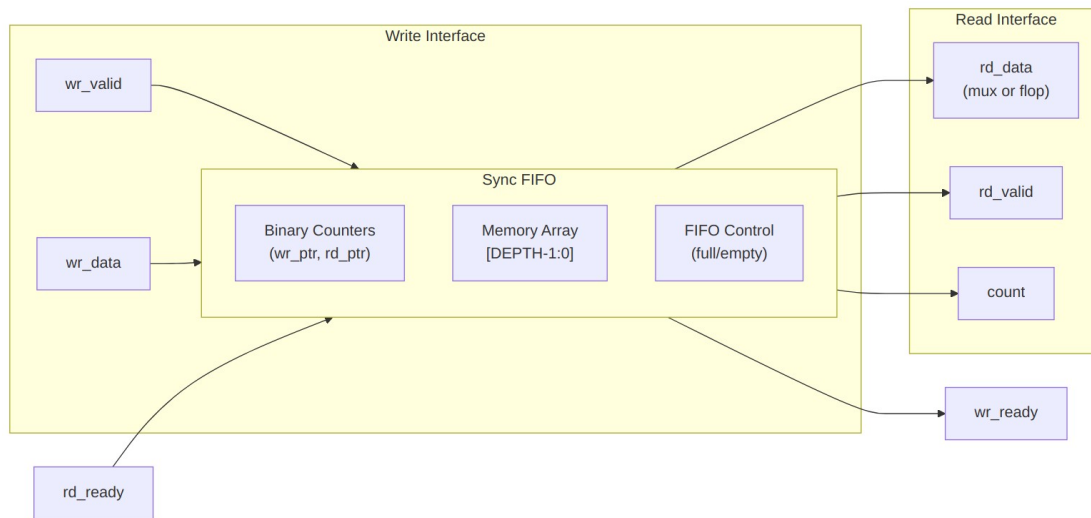
3.4.1 Read Modes

3.4.1.1 Mux Mode (REGISTERED=0)

- **Latency:** 1 cycle write → read
- **Read Path:** Combinatorial from memory
- **Use Case:** Low latency applications
- **Timing:** May create combinatorial path

3.4.1.2 Flop Mode (REGISTERED=1)

- **Latency:** 2 cycles write → read
- **Read Path:** Registered output
- **Use Case:** Timing closure, deep pipelines
- **Timing:** No combinatorial paths



Architecture

3.4.2 Dependencies

- `counter_bin.sv` - Binary counters for read/write pointers
- `fifo_control.sv` - Full/empty flag generation

3.5 Usage Examples

3.5.1 Example 1: Mux Mode (Low Latency)

```
gaxi_fifo_sync #(
    .DATA_WIDTH(64),
    .DEPTH(32),
    .REGISTERED(0)           // Mux mode: 1-cycle latency
) u_low_latency_fifo (
    .axi_aclk      (clk),
    .axi_aresetn   (rst_n),
    .wr_valid      (wr_valid),
    .wr_ready      (wr_ready),
    .wr_data       (wr_data),
    .rd_ready      (rd_ready),
    .rd_valid      (rd_valid),
    .rd_data       (rd_data), // Combinatorial read
    .count         (fifo_level)
);
```

3.5.2 Example 2: Flop Mode (Timing Closure)

```
gaxi_fifo_sync #(
    .DATA_WIDTH(128),
    .DEPTH(64),
    .REGISTERED(1)           // Flop mode: 2-cycle latency
) u_registered_fifo (
    .axi_aclk      (clk),
    .axi_aresetn   (rst_n),
    .wr_valid      (wr_valid),
    .wr_ready      (wr_ready),
    .wr_data       (wr_data),
    .rd_ready      (rd_ready),
    .rd_valid      (rd_valid),
    .rd_data       (rd_data), // Registered output
    .count         (fifo_level)
);
```

3.6 Timing Characteristics

Mode	Write→Read Latency	Max Throughput	Read Path
Mux	1 cycle	1/cycle	Combinatorial

Mode	Write→Read Latency	Max Throughput	Read Path
(REGISTERED=0)			
Flop	2 cycles	1/cycle	Registered
(REGISTERED=1)			

3.7 Resource Utilization

DEPTH	Mode	Flops	LUTs	Memory Bits
16	Mux	16×DW + ~20	~80	16×DW
16	Flop	16×DW + DW + ~20	~80	16×DW
64	Mux	64×DW + ~30	~120	64×DW
64	Flop	64×DW + DW + ~30	~120	64×DW

3.8 Testing

```
# Test FIFO modes
pytest val/amba/test_gaxi_buffer_sync.py -k "fifo_mux" -v # Mux mode
pytest val/amba/test_gaxi_buffer_sync.py -k "fifo_flop" -v # Flop mode
```

3.9 Related Modules

- [gaxi_skid_buffer](#) - Zero-latency bypass, smaller depths
- [gaxi_fifo_async](#) - Clock domain crossing version
- [GAXI Index](#) - Overview of all GAXI modules

Version: 1.0 **Last Updated:** 2025-10-06

4 GAXI Register Slice

Module: gaxi_regslice.sv **Location:** rtl/amba/gaxi/ **Status:** ✓ Production Ready

4.1 Overview

The GAXI register slice provides a timing-friendly 1-deep elastic buffer for pipeline timing isolation. It implements the classic register slice pattern with always-registered data transfer, guaranteeing predictable 1-cycle latency and consistent throughput.

4.1.1 Key Features

- ✓ **Fixed 1-Cycle Latency:** Always registered, predictable timing
 - ✓ **Full Throughput:** 1 beat/cycle in steady state
 - ✓ **Elastic Buffering:** Absorbs single-cycle backpressure
 - ✓ **Port Compatible:** Intentionally aligned with gaxi_skid_buffer
 - ✓ **Ready/Valid Handshake:** Industry-standard AXI-like protocol
 - ✓ **Minimal Resources:** Single register stage
-

4.2 Module Interface

```
module gaxi_regslice #(
    parameter int DATA_WIDTH    = 32,
    parameter    INSTANCE_NAME = "REGSL1D",
    parameter int DW             = DATA_WIDTH    // Derived
) (
    // Global Clock and Reset
    input logic    axi_aclk,
    input logic    axi_aresetn,

    // Write Interface (Input Side)
    input logic    wr_valid,
    output logic    wr_ready,
    input logic [DW-1:0] wr_data,

    // Read Interface (Output Side)
    output logic    rd_valid,
    input logic    rd_ready,
    output logic [DW-1:0] rd_data,

    // Status/Monitoring
```

```

    output logic [3:0] count,      // 0 or 1
    output logic [3:0] rd_count    // mirror of count
);

```

4.3 Parameters

Parameter	Default	Description
DATA_WIDTH	32	Data bus width (arbitrary)
INSTANCE_NAME	“REGSL1D”	Debug instance name
DW	DATA_WIDTH	Derived parameter (internal use)

4.4 Functional Description

4.4.1 Architecture

The register slice implements a single-entry elastic buffer with simultaneous push/pop capability:

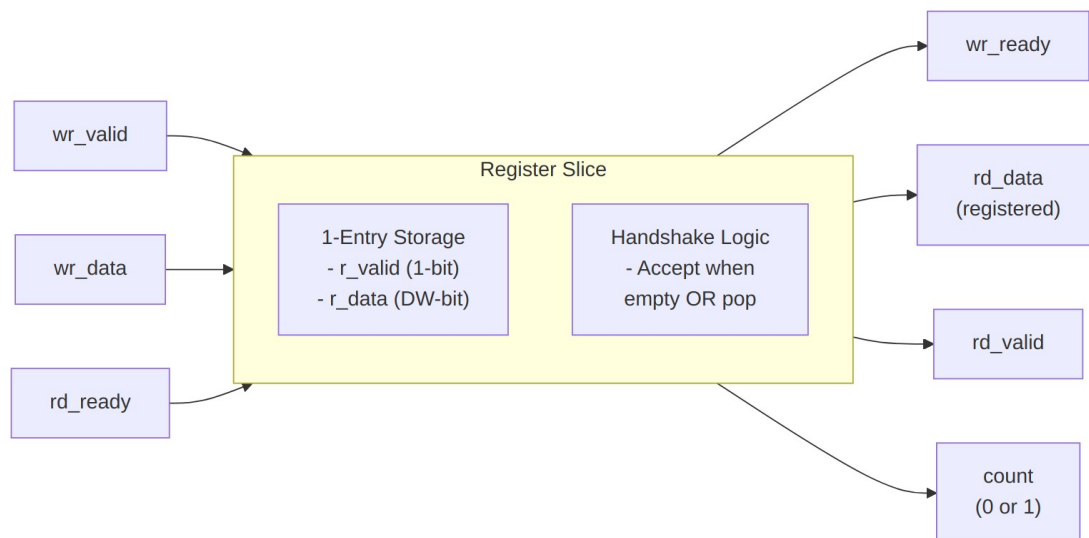


Diagram 3

4.4.2 Ready/Valid Protocol

Write Ready Conditions:

```
wr_ready = (!r_valid) || (r_valid && rd_ready)
```

- Accept when storage is **empty** (r_valid=0)
- Accept when storage is **full** AND downstream **consuming** (simultaneous transfer)

Read Valid:

```
rd_valid = r_valid
```

- Valid when storage is occupied

4.4.3 Transfer Scenarios

Scenario	wr_valid	wr_ready	rd_valid	rd_ready	Action
Idle	0	1 (empty)	0	X	Storage empty, ready to accept
Fill	1	1	0	X	Write data → storage, r_valid=1
Full	0	0	1	0	Storage occupied, downstream not consuming
Drain	0	1 (will empty)	1	1	Read consumes data, r_valid=0
Pass-through	1	1	1	1	Simultaneous write+read, data passes through

4.4.4 Storage Update Logic

```
unique case ({w_wxfer, w_rxfer})
  2'b10: begin // Write only: fill
    r_valid <= 1'b1;
    r_data  <= wr_data;
  end
  2'b01: begin // Read only: drain
    r_valid <= 1'b0;
  end
  2'b11: begin // Simultaneous: pass through with register
    r_valid <= 1'b1;
    r_data  <= wr_data;
  end
  default: begin // Idle: hold state
    r_valid <= r_valid;
    r_data  <= r_data;
  end
endcase
```

4.5 Usage Examples

4.5.1 Example 1: Pipeline Timing Isolation

```
// Insert register slice to break long combinatorial path
gaxi_regslice #(
  .DATA_WIDTH(64)
) u_pipeline_break (
  .axi_aclk      (clk),
  .axi_aresetn   (rst_n),
  .wr_valid      (stage1_valid),
  .wr_ready      (stage1_ready),
  .wr_data       (stage1_data),
  .rd_valid      (stage2_valid),
  .rd_ready      (stage2_ready),
  .rd_data       (stage2_data),
  .count         () // Optional monitoring
);
```

4.5.2 Example 2: AXI Channel Register Slice

```
// Register slice for AXI AR channel
gaxi_regslice #(
  .DATA_WIDTH(AXI_ID_WIDTH + AXI_ADDR_WIDTH + 8 + 3 + 2),
  .INSTANCE_NAME("AR_REGSL")
) u_ar_regslice (
  .axi_aclk      (aclk),
```



```

        .axi_aresetn    (aresetn),
        .wr_valid      (m_axi_arvalid),
        .wr_ready      (m_axi_arready),
        .wr_data       ({m_axi_arid, m_axi_araddr, m_axi_arlen,
                        m_axi_arsize, m_axi_arburst}),
        .rd_valid      (s_axi_arvalid),
        .rd_ready      (s_axi_arready),
        .rd_data       ({s_axi_arid, s_axi_araddr, s_axi_arlen,
                        s_axi_arsize, s_axi_arburst}),
        .count         (ar_occupancy)
    );

```

4.5.3 Example 3: Multiple Register Slices for Deep Pipelining

// Chain multiple register slices for multi-cycle pipeline

```

gaxi_regslice #(.DATA_WIDTH(32)) u_stage1 (
    .axi_aclk(clk), .axi_aresetn(rst_n),
    .wr_valid(in_valid), .wr_ready(in_ready), .wr_data(in_data),
    .rd_valid(pipel_valid), .rd_ready(pipel_ready), .rd_data(pipel_dat
a)
);

```

```

gaxi_regslice #(.DATA_WIDTH(32)) u_stage2 (
    .axi_aclk(clk), .axi_aresetn(rst_n),
    .wr_valid(pipel_valid), .wr_ready(pipel_ready), .wr_data(pipel_dat
a),
    .rd_valid(pipe2_valid), .rd_ready(pipe2_ready), .rd_data(pipe2_dat
a)
);

```

```

gaxi_regslice #(.DATA_WIDTH(32)) u_stage3 (
    .axi_aclk(clk), .axi_aresetn(rst_n),
    .wr_valid(pipe2_valid), .wr_ready(pipe2_ready), .wr_data(pipe2_dat
a),
    .rd_valid(out_valid), .rd_ready(out_ready), .rd_data(out_data)
);

```

// Total latency: 3 cycles, full throughput maintained

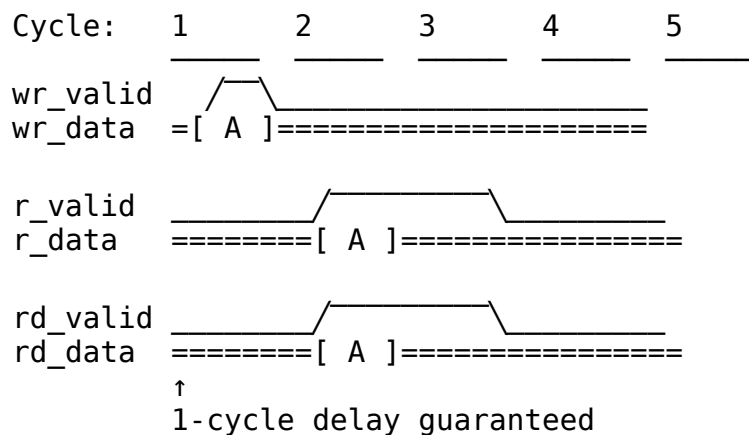
4.6 Timing Characteristics

Characteristic	Value	Notes
Write→Read Latency	1 cycle	Always registered, fixed latency
Max Throughput	1 beat/cycle	Sustained in steady

Characteristic	Value	Notes
Backpressure Absorption	1 beat	state Single-entry elastic buffer
Read Path	Registered	No combinatorial paths
Write Path	Combinatorial	wr_ready depends on rd_ready

4.6.1 Latency Guarantee

Unlike skid buffers which can have 0-cycle bypass, the register slice **always** introduces 1-cycle latency:



4.7 Comparison with Related Modules

4.7.1 gaxi_regslice vs gaxi_skid_buffer

Feature	gaxi_regslice	gaxi_skid_buffer
Depth	1 entry	1 entry (+ skid slot)
Latency	1 cycle (fixed)	0-1 cycle (variable)
Bypass Path	✗ No	✓ Yes (when empty)
Throughput	1 beat/cycle	1 beat/cycle
Use Case	Timing isolation	Low latency
Registered Output	✓ Always	⚠ Only when filled

Feature	<code>gaxi_regslice</code>	<code>gaxi_skid_buffer</code>
Timing Closure	✓ Better (no bypass)	⚠ Has combinatorial bypass

When to Choose: - `gaxi_regslice`: Timing closure priority, need predictable latency - `gaxi_skid_buffer`: Latency-sensitive paths, tolerate variable timing

4.7.2 `gaxi_regslice` vs `gaxi_fifo_sync`

Feature	<code>gaxi_regslice</code>	<code>gaxi_fifo_sync</code>
Depth	1 entry (fixed)	Parameterized (N entries)
Latency	1 cycle	1-2 cycles (mode-dependent)
Resources	Minimal (1 register)	Scales with depth
Use Case	Pipeline breaks	Buffering, rate matching

4.8 Resource Utilization

4.8.1 FPGA Resources (Typical)

DATA_WIDTH	Flops	LUTs	Slice Registers
8	9	~6	9
32	33	~12	33
64	65	~18	65
128	129	~24	129

Scaling: Approximately $\text{DATA_WIDTH} + 1$ flops (data + valid flag)

4.9 Testing

4.9.1 Test Coverage

Test File: `val/amba/test_gaxi_regslice.py`

Test Methods: - Simple incremental loops (fill/drain cycles) - Back-to-back transfers (sustained throughput) - Comprehensive randomizer sweep (varied timing patterns) - Stress test with random patterns

Test Levels: - **basic:** Quick smoke test (~30s, 4 loops) - **medium:** Moderate coverage (~2min, expanded patterns) - **full:** Comprehensive validation (~5min, 100+ loops)

4.9.2 Run Tests

Basic test (quick validation)

```
TEST_LEVEL=basic pytest val/amba/test_gaxi_regslice.py -v
```

Medium test (normal CI)

```
TEST_LEVEL=medium pytest val/amba/test_gaxi_regslice.py -v
```

Full test (pre-release validation)

```
TEST_LEVEL=full pytest val/amba/test_gaxi_regslice.py -v
```

4.10 Design Considerations

4.10.1 Port Compatibility

Intentional alignment with **gaxi_skid_buffer** allows drop-in replacement:

```
// Both modules share identical port signatures
gaxi_regslice #(.DATA_WIDTH(64)) u_option1 (...);
gaxi_skid_buffer #(.DATA_WIDTH(64)) u_option2 (...);
```

This enables easy experimentation: - Start with **gaxi_skid_buffer** for low latency - Upgrade to **gaxi_regslice** if timing closure issues arise

4.10.2 Status Signals

count and **rd_count** are 4-bit for interface compatibility: - Value 0: Storage empty - Value 1: Storage occupied - Both signals always equal (redundant, but matches skid buffer interface)

4.10.3 Assertions

Built-in simulation checks (disabled in synthesis):

```
// Detect backpressure hot spots
if (wr_valid && !wr_ready) ...
```

```
// Detect invalid reads
if (rd_ready && !rd_valid) ...
```

```
// Sanity check
if (count > 4'd1) $error(...)
```

4.11 Related Modules

- [gaxi_skid_buffer](#) - Zero-latency bypass alternative
 - [gaxi_fifo_sync](#) - Multi-entry FIFO version
 - [gaxi_fifo_async](#) - Clock domain crossing version
 - [GAXI Index](#) - Overview of all GAXI modules
-

4.12 Common Use Cases

4.12.1 1. Breaking Long Combinatorial Paths

Problem: Critical path through multiple modules

```
// Before: Long combinatorial path
assign module_b_in = complex_function(module_a_out);
```

Solution: Insert register slice

```
// After: Broken with 1-cycle register
gaxi_regslice #(.DATA_WIDTH(32)) u_break (
    .axi_aclk(clk), .axi_aresetn(rst_n),
    .wr_valid(module_a_valid), .wr_data(module_a_out), .wr_ready(modul
e_a_ready),
    .rd_valid(module_b_valid), .rd_data(module_b_in), .rd_ready(modu
le_b_ready)
);
```

4.12.2 2. AXI Interface Pipelining

Improve timing across AXI channels:

```
// Pipeline all 5 AXI4 channels
gaxi_regslice #(...) u_ar_slice (...); // Address Read
gaxi_regslice #(...) u_r_slice (...); // Read Data
gaxi_regslice #(...) u_aw_slice (...); // Address Write
gaxi_regslice #(...) u_w_slice (...); // Write Data
gaxi_regslice #(...) u_b_slice (...); // Write Response
```

4.12.3 3. CDC Preparation

Use before async FIFO to ensure registered inputs:

```
// Register slice before CDC FIFO
gaxi_regslice #(.DATA_WIDTH(32)) u_pre_cdc (
    .axi_aclk(clk_a), .axi_aresetn(rst_a_n),
    .wr_valid(src_valid), .wr_data(src_data), .wr_ready(src_ready),
    .rd_valid(fifo_wr_valid), .rd_data(fifo_wr_data), .rd_ready(fifo_w
r_ready)
);

gaxi_fifo_async #(.DATA_WIDTH(32), .DEPTH(16)) u_cdc_fifo (
    .wr_clk(clk_a), .rd_clk(clk_b), ...
    .wr_valid(fifo_wr_valid), .wr_data(fifo_wr_data), .wr_ready(fifo_w
r_ready),
    ...
);
```

Version: 1.0 Last Updated: 2025-10-23

5 GAXI Asynchronous Skid Buffer

Module: gaxi_skid_buffer_async.sv **Location:** rtl/amba/gaxi/ **Status:** ✓ Production Ready

5.1 Overview

The GAXI asynchronous skid buffer combines the zero-latency bypass feature of the skid buffer with clock domain crossing capability. It's a wrapper that instantiates a synchronous skid buffer on the write side followed by an asynchronous FIFO.

5.1.1 Key Features

- ✓ **CDC Capability:** Safe clock domain crossing
 - ✓ **Elastic Buffering:** Combines skid buffer + async FIFO
 - ✓ **Write-Side Buffering:** Skid buffer in write domain prevents stalls
 - ✓ **Configurable CDC:** Adjustable synchronizer stages
 - ✓ **Dual Clock Domains:** Independent write/read clocks
-

5.2 Module Interface

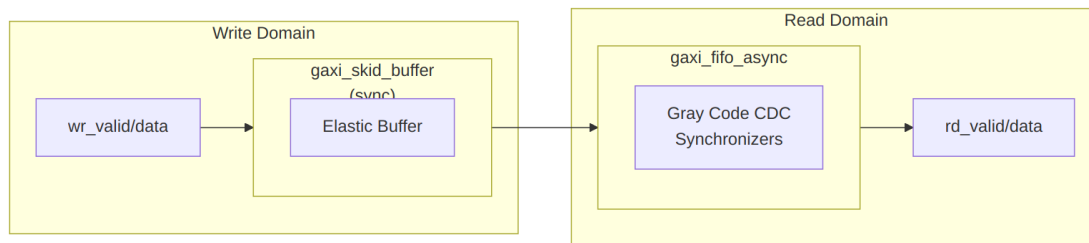
```
module gaxi_skid_buffer_async #(
    parameter int REGISTERED = 0,           // FIFO read mode
    parameter int DATA_WIDTH = 32,
```

```

parameter int DEPTH = 2,           // Async FIFO depth
parameter int N_FLOP_CROSS = 2,   // CDC stages
parameter     INSTANCE_NAME = "DEADF1F0"
) (
    // Write Domain
    input  logic      axi_wr_aclk,
    input  logic      axi_wr_aresetn,
    input  logic      wr_valid,
    output logic      wr_ready,
    input  logic [DW-1:0] wr_data,

    // Read Domain
    input  logic      axi_rd_aclk,
    input  logic      axi_rd_aresetn,
    output logic      rd_valid,
    input  logic      rd_ready,
    output logic [DW-1:0] rd_data
);

```



Architecture

5.2.1 Internal Components

1. **gaxi_skid_buffer (Write Domain):**
 - Provides elastic buffering
 - Prevents write-side stalls
 - Fixed depth (typically 2-4 entries)
 2. **gaxi_fifo_async (CDC):**
 - Handles clock domain crossing
 - Configurable depth (DEPTH parameter)
 - Gray code pointer synchronization
-

5.3 Parameters

Parameter	Default	Description
REGISTERED	0	Async FIFO read mode (0=mux, 1=flop)
DATA_WIDTH	32	Data bus width
DEPTH	2	Async FIFO depth (CDC stage)
N_FLOP_CROSS	2	Synchronizer stages (3 recommended)
INSTANCE_NAME	“DEADF1F0”	Debug instance name

Note: Skid buffer depth is fixed (determined by gaxi_skid_buffer default).

5.4 Usage Examples

5.4.1 Example 1: Basic Async Skid Buffer

```
gaxi_skid_buffer_async #(
    .DATA_WIDTH(64),
    .DEPTH(4),           // Async FIFO depth
    .N_FLOP_CROSS(3),    // CDC stages
    .REGISTERED(0)
) u_async_skid (
    // Write domain @ 100 MHz
    .axi_wr_aclk      (clk_100),
    .axi_wr_aresetn   (rst_100_n),
    .wr_valid         (src_valid),
    .wr_ready         (src_ready),
    .wr_data          (src_data),

    // Read domain @ 75 MHz
    .axi_rd_aclk      (clk_75),
    .axi_rd_aresetn   (rst_75_n),
    .rd_valid         (dst_valid),
    .rd_ready         (dst_ready),
    .rd_data          (dst_data)
);
```


5.4.2 Example 2: Pipeline Stage with CDC

```
// Break long path AND cross clock domains
gaxi_skid_buffer_async #(
    .DATA_WIDTH(128),
    .DEPTH(8),           // Larger for rate buffering
    .N_FLOP_CROSS(3),
    .REGISTERED(1)       // Registered output for timing
) u_cdc_pipeline (
    .axi_wr_aclk    (source_clk),
    .axi_wr_aresetn (source_rst_n),
    .wr_valid       (stage_in_valid),
    .wr_ready       (stage_in_ready),
    .wr_data        (stage_in_data),

    .axi_rd_aclk    (dest_clk),
    .axi_rd_aresetn (dest_rst_n),
    .rd_valid       (stage_out_valid),
    .rd_ready       (stage_out_ready),
    .rd_data        (stage_out_data)
);
```

5.5 Timing Characteristics

5.5.1 Latency Components

Component	Latency	Domain
Skid buffer (empty)	0 cycles	Write
Skid buffer (buffered)	1 cycle	Write
Async FIFO CDC	3-5 cycles	Both
Total (empty path)	3-5 cycles	End-to-end
Total (buffered)	4-6 cycles	End-to-end

5.5.2 Throughput

- **Write Side:** 1 transfer/cycle (when not full)
 - **Read Side:** 1 transfer/cycle (when not empty)
 - **Sustained:** Limited by slower clock domain
-

5.6 Design Considerations

5.6.1 When to Use This Module

Use async skid buffer when: - Need CDC between clock domains - Want elastic buffering on write side - Write-side needs low-latency response

Use plain async FIFO instead when: - Only need CDC, no special write-side buffering - Want deeper buffering (larger DEPTH) - Don't need skid buffer complexity

5.6.2 Depth Selection

Total buffering = Skid buffer entries + DEPTH

Recommendations: - **DEPTH=4:** Minimal CDC buffering, light traffic - **DEPTH=8:** Moderate buffering, balanced - **DEPTH=16:** Heavy buffering, burst traffic

5.6.3 Reset Synchronization

Critical: Both domains need synchronized resets!

```
// Example reset synchronization
reset_sync #(.STAGES(3)) u_wr_sync (
    .clk(axi_wr_aclk),
    .async_rst_n(global_rst_n),
    .sync_rst_n(axi_wr_aresetn)
);

reset_sync #(.STAGES(3)) u_rd_sync (
    .clk(axi_rd_aclk),
    .async_rst_n(global_rst_n),
    .sync_rst_n(axi_rd_aresetn)
);
```

5.7 Resource Utilization

Combines resources of both sub-modules:

Component	Flops	LUTs
Skid buffer (typical)	~4×DW	~50
Async FIFO (DEPTH=8)	8×DW + ~40	~120
Total (DEPTH=8)	~12×DW + 40	~170

5.8 Testing

Async skid buffer tests

```
pytest val/amba/test_gaxi_buffer_async.py -k "skid" -v
```

Test specific clock ratio

```
pytest val/amba/test_gaxi_buffer_async.py -k "skid" -k "wr10_rd20" -v
```

5.9 Common Issues

5.9.1 Issue 1: CDC Violations

Symptom: Metastability, data corruption

Solution: - Set N_FLOP_CROSS=3 (minimum for production) - Verify clocks are truly independent - Check reset synchronization

5.9.2 Issue 2: Write-Side Stalls

Symptom: wr_ready deasserts unexpectedly

Debug: 1. Check async FIFO depth (DEPTH parameter) 2. Verify read-side is consuming data 3. Monitor clock ratio - may need deeper FIFO

5.9.3 Issue 3: Unexpected Latency

Symptom: Data takes longer than expected

Explanation: - Total latency = skid buffer + async FIFO CDC - Minimum ~3-5 cycles even when empty - This is inherent to CDC design

5.10 Related Modules

- [gaxi_skid_buffer](#) - Synchronous version
 - [gaxi_fifo_async](#) - Async FIFO without skid buffer
 - [GAXI Index](#) - Overview
-

5.11 References

- **Source:** rtl/amba/gaxi/gaxi_skid_buffer_async.sv
- **Dependencies:**

- gaxi_skid_buffer.sv
 - gaxi_fifo_async.sv
 - **Tests:** val/amba/test_gaxi_buffer_async.py
-

Version: 1.0 **Last Updated:** 2025-10-06

[← Back to GAXI Index](#) | [← Back to RTLAmba Index](#)

6 GAXI Skid Buffer (Double-Drain)

Module: gaxi_skid_buffer_dbldrn.sv **Location:** rtl/amba/gaxi/ **Status:** Production Ready

6.1 Overview

gaxi_skid_buffer_dbldrn is a “double-drain” variant of the gaxi_skid_buffer. It keeps the same elastic write side and single-read output, but adds a **second read output** (rd_data2) and a **second drain request** (rd_ready2) so the consumer can pop **two entries in one clock** when at least two are buffered. This is useful for consumers that can retire two items per cycle (for example a 2-wide unpack or a burst aligner) and would otherwise be throughput-limited by a one-per-cycle drain.

Like the base skid buffer, storage is a shift register of DEPTH entries and the read/write handshakes use the GAXI valid/ready convention.

6.1.1 Key Features

- **Double-drain:** Assert rd_ready2 (legal only when rd_count >= 2) to pop two entries in a single cycle
- **Two read data outputs:** rd_data (lowest entry) and rd_data2 (next entry) presented simultaneously
- **Single-drain compatible:** With rd_ready2 low it behaves as an ordinary single-drain skid buffer
- **Priority-encoded update:** Write / single-read / double-read combinations handled by a one-hot case, double-drain prioritized over single-drain
- **Backpressure-aware ready/valid:** wr_ready and rd_valid account for both drain widths
- **Legality assertion:** Simulation \$error fires if rd_ready2 is asserted with fewer than two entries buffered

6.2 Module Purpose

The base skid buffer drains at most one entry per cycle, which caps a fast consumer at one item per clock. This variant lets a consumer that can accept two items per cycle drain both the lowest and next-lowest buffered entries together, doubling drain throughput while still honoring GAXI backpressure and preserving in-order (lowest-position-first) delivery.

Use Cases:

- Feeding a consumer that retires two entries per clock (2-wide unpack, dual-issue, burst aligner)
- Draining a buffer faster than one-per-cycle to relieve upstream backpressure
- Any place a `gaxi_skid_buffer` is used but the sink can opportunistically take a pair

Key Benefit: Doubles peak drain throughput (two entries per clock) when two or more are buffered, while degrading gracefully to ordinary single-drain skid-buffer behavior when only one entry is available.

6.3 Parameters

Parameter	Type	Default	Description
DATA_WIDTH	int	32	Data bus width in bits
DEPTH	int	4	Buffer depth in entries (must be one of {2, 4, 6, 8})
DW	int	DATA_WIDTH	Derived alias for DATA_WIDTH (do not override)
BUF_WIDTH	int	DATA_WIDTH * DEPTH	Derived total shift-register width (do not override)
BW	int	BUF_WIDTH	Derived alias for BUF_WIDTH (do not override)

6.4 Port Groups

6.4.1 Clock and Reset

Port	Direction	Width	Description
axi_aclk	Input	1	Clock
axi_aresetn	Input	1	Active-low asynchronous reset

6.4.2 Write Interface (input side)

Port	Direction	Width	Description
wr_valid	Input	1	Write data valid
wr_ready	Output	1	Ready to accept a write (registered)
wr_data	Input	DATA_WIDTH	Write data

6.4.3 Read Interface (output side)

Port	Direction	Width	Description
rd_valid	Output	1	Read data valid (registered)
rd_ready	Input	1	Consumer ready to accept a read
rd_ready2	Input	1	Double-drain request; legal only when rd_count >= 2
rd_data	Output	DATA_WIDT H	First (lowest-position) entry
rd_data2	Output	DATA_WIDT H	Second (next-position) entry, valid when double-draining

6.4.4 Status

Port	Direction	Width	Description
count	Output	4	Current buffer

Port	Direction	Width	Description
rd_count	Output	4	occupancy Same as count (read-side occupancy)

6.5 Functional Description

6.5.1 Transfer Detection

Three transfer conditions are decoded combinationally. Note that a single read is only recognized when rd_ready2 is **not** asserted, and a double read additionally requires two entries present:

```
assign w_wr_xfer      = wr_valid & wr_ready;
assign w_rd_xfer      = rd_valid & rd_ready & ~rd_ready2;
// single drain
assign w_rd_dbl_xfer = rd_valid & rd_ready & rd_ready2 & (r_data_count
>= 2); // double drain
```

6.5.2 Storage Update

The shift register r_data and occupancy r_data_count are updated from a one-hot case over {w_wr_xfer, w_rd_dbl_xfer, w_rd_xfer}, with double-drain prioritized above single-drain:

Case	Meaning	r_data action	count change
3'b100	Write only	Load wr_data at the top of the occupied region	+1
3'b001	Single read only	Shift down by one entry	-1
3'b010	Double read only	Shift down by two entries	-2
3'b101	Write + single read	Shift down one, then insert wr_data at the new top	0
3'b110	Write + double read	Shift down two, then insert wr_data at the new top	-1
default	Idle / illegal (000, 011, 111)	No change	0

The 011 and 111 combinations are illegal (a single- and double-drain cannot both fire) and fall through to the no-change default. Reads always drain from the low positions (rd_data = entry 0, rd_data2 = entry 1), preserving in-order delivery.

6.5.3 Ready/Valid Logic

wr_ready and rd_valid are registered and computed one cycle ahead, accounting for both drain widths so that space freed by a double-drain is reflected in wr_ready:

```
wr_ready <= (r_data_count <= DEPTH-2) ||  
            (r_data_count == DEPTH-1 && (~w_wr_xfer || w_rd_xfer ||  
w_rd_dbl_xfer)) ||  
            (r_data_count == DEPTH && (w_rd_xfer || w_rd_dbl_xfer));  
  
rd_valid <= (r_data_count >= 2) ||  
            (r_data_count == 1 && (~w_rd_xfer || w_wr_xfer)) ||  
            (r_data_count == 0 && w_wr_xfer);
```

6.5.4 Output Assignments

The two lowest shift-register slots are presented on the two data outputs, and the occupancy drives both status ports:

```
assign rd_data  = r_data[DW-1:0];      // first item (lowest position)  
assign rd_data2 = r_data[2*DW-1:DW];  // second item (next position)  
assign rd_count = r_data_count;  
assign count    = r_data_count;
```

6.5.5 Legality Assertion

A simulation-only check enforces the double-drain contract: asserting rd_ready2 (with rd_ready) when fewer than two entries are buffered raises a \$error. This catches consumers that request a double-drain the buffer cannot service.

6.6 Usage Example

```
// Skid buffer feeding a consumer that can retire two entries per  
clock.  
gaxi_skid_buffer_dbldrn #(  
    .DATA_WIDTH(32),  
    .DEPTH(4)  
) u_dbldrn (  
    .axi_aclk      (clk),  
    .axi_aresetn   (rst_n),  
    // write side  
    .wr_valid      (src_valid),  
    .wr_ready      (src_ready),  
    .wr_data       (src_data),  
    // read side  
    .rd_valid      (dst_valid),  
    .rd_ready      (dst_ready),  
    .rd_ready2     (dst_take_two & (rd_occupancy >= 2)), // only when  
    >= 2 buffered
```



```

        .rd_data      (dst_data0),
        .rd_data2     (dst_data1),
        // status
        .count        (rd_occupancy),
        .rd_count      ()
    );

```

6.7 Design Notes

- **rd_ready2 legality.** Only assert rd_ready2 (together with rd_ready) when rd_count >= 2. The RTL guards the transfer with the same condition, and the simulation assertion flags misuse.
- **Priority.** When both a single- and double-drain could be decoded, double-drain wins; the illegal simultaneous cases (011, 111) are ignored by the default branch.
- **In-order delivery.** Entry 0 is always the oldest; rd_data/rd_data2 expose the two oldest entries, and drains shift the register down, so ordering is preserved.
- **Graceful degradation.** With rd_ready2 tied low the module is functionally an ordinary single-drain skid buffer.
- **Reset macros.** The module uses the project reset_defs.svh ALWAYS_FF_RST / RST_ASSERTED macros for its registered state.

6.8 Related Modules

6.8.1 Used By

- Consumers that retire two GAXI entries per clock (2-wide unpack / aligner front-ends)

6.8.2 Uses

- None (self-contained; relies only on the reset_defs.svh reset macros)

6.8.3 See Also

- [gaxi_skid_buffer](#) - The single-drain base skid buffer this variant extends
- [gaxi_skid_buffer_struct](#) - Struct-typed skid buffer variant
- [gaxi_fifo_sync](#) - Larger-depth synchronous FIFO with optional registered output

6.9 References

6.9.1 Source Code

- rtl/amba/gaxi/gaxi_skid_buffer_dbldrn.sv
- rtl/amba/gaxi/gaxi_skid_buffer.sv (base variant)

6.9.2 Documentation

- rtl/amba/PRD.md
-

Last Updated: 2026-07-15

6.10 Navigation

- [← Back to GAXI Index](#)
- [← Back to RTLAmba Index](#)

7 GAXI Skid Buffer

Module: gaxi_skid_buffer.sv **Location:** rtl/amba/gaxi/ **Status:** ✓ Production Ready

7.1 Overview

The GAXI skid buffer is an elastic buffer that provides **zero-latency bypass** when empty while offering elastic buffering to prevent pipeline stalls. It combines the best of both worlds: low latency for the common case (empty) and elastic storage for handling backpressure.

7.1.1 Key Features

- ✓ **Zero-Latency Bypass:** When empty, data appears at output immediately (0 cycles)
 - ✓ **Elastic Buffering:** Depth 2-8 entries prevent pipeline stalls
 - ✓ **Single Clock Domain:** Synchronous design for simplicity
 - ✓ **Parameterized:** Configurable data width and depth
 - ✓ **Efficient:** Shift register implementation, minimal logic
-

7.2 Module Interface

```
module gaxi_skid_buffer #(
    parameter int DATA_WIDTH = 32,
    parameter int DEPTH = 2,           // Must be one of {2, 4, 6, 8}
    parameter    INSTANCE_NAME = "DEADF1F0",
    parameter int DW = DATA_WIDTH,
    parameter int BUF_WIDTH = DATA_WIDTH * DEPTH,
    parameter int BW = BUF_WIDTH
) (
    // Global Clock and Reset
    input logic    axi_aclk,
    input logic    axi_aresetn,

    // Write Interface (Input Side)
    input logic    wr_valid,
    output logic   wr_ready,
    input logic [DW-1:0] wr_data,

    // Read Interface (Output Side)
    output logic   rd_valid,
    input logic    rd_ready,
    output logic [DW-1:0] rd_data,

    // Status/Monitoring
    output logic [3:0] count,    // Current buffer occupancy
    output logic [3:0] rd_count  // Same as count (for
compatibility)
);
```

7.3 Parameters

Parameter	Type	Default	Description
DATA_WIDTH	int	32	Data bus width in bits
DEPTH	int	2	Buffer depth (must be 2, 4, 6, or 8)
INSTANCE_NAME	string	"DEADF1F0"	Instance name for debug messages

Derived Parameters: - DW = DATA_WIDTH - BUF_WIDTH = DATA_WIDTH × DEPTH (total buffer storage) - BW = BUF_WIDTH

7.4 Ports

7.4.1 Clock and Reset

Port	Direction	Width	Description
axi_aclk	input	1	Clock
axi_aresetn	input	1	Active-low asynchronous reset

7.4.2 Write Interface

Port	Direction	Width	Description
wr_valid	input	1	Write data valid
wr_ready	output	1	Ready to accept write
wr_data	input	DATA_WIDTH	Write data

7.4.3 Read Interface

Port	Direction	Width	Description
rd_valid	output	1	Read data valid
rd_ready	input	1	Ready to accept read
rd_data	output	DATA_WIDTH	Read data

7.4.4 Status

Port	Direction	Width	Description
count	output	4	Current buffer occupancy (0 to DEPTH)
rd_count	output	4	Same as count (for compatibility)

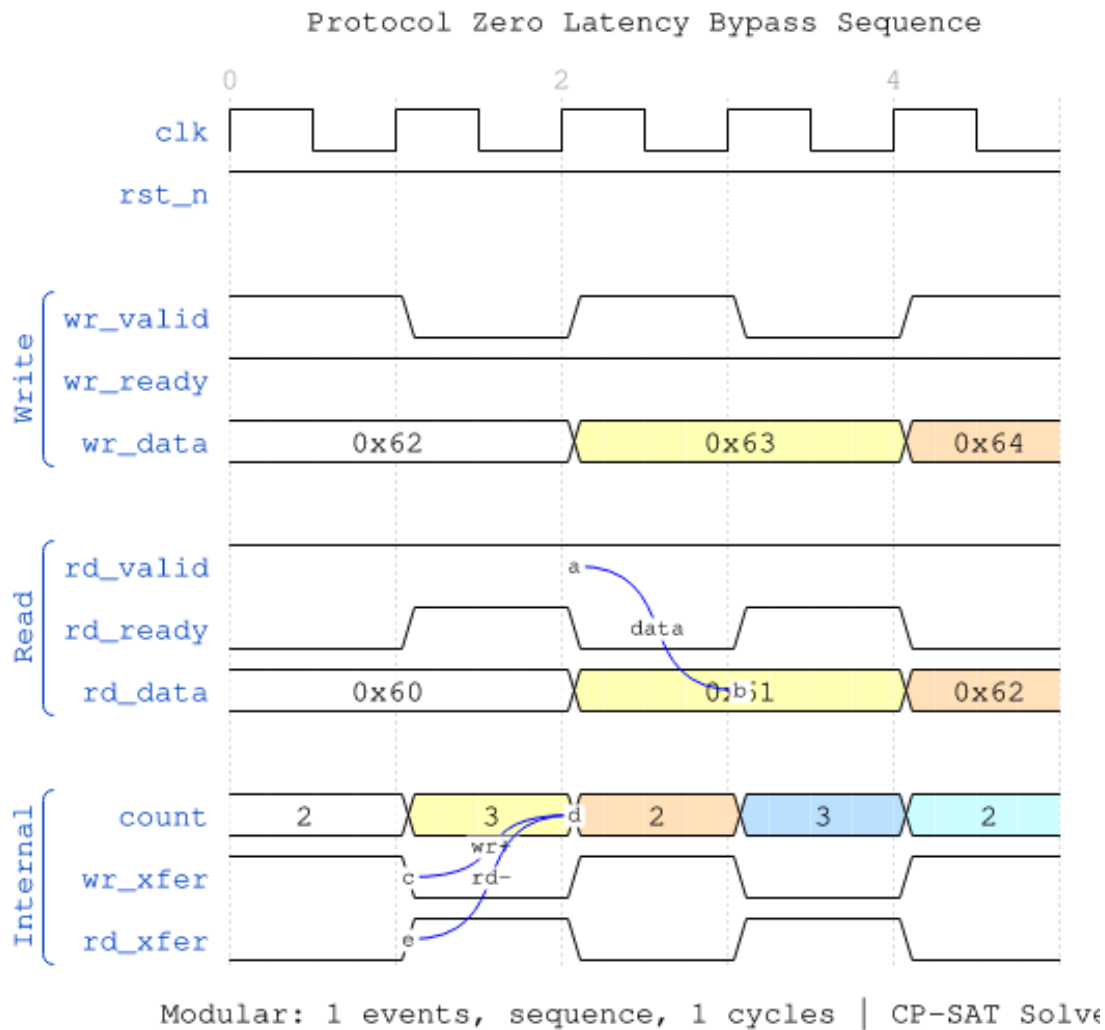
7.5 Functional Description

7.5.1 Operation Modes

The skid buffer operates in different modes based on occupancy:

7.5.1.1 1. Empty Mode (*count* = 0)

Zero-Latency Bypass: - *wr_data* → *rd_data* **immediately** (combinatorial path) - *rd_valid* asserts same cycle as *wr_valid* - No buffering delay



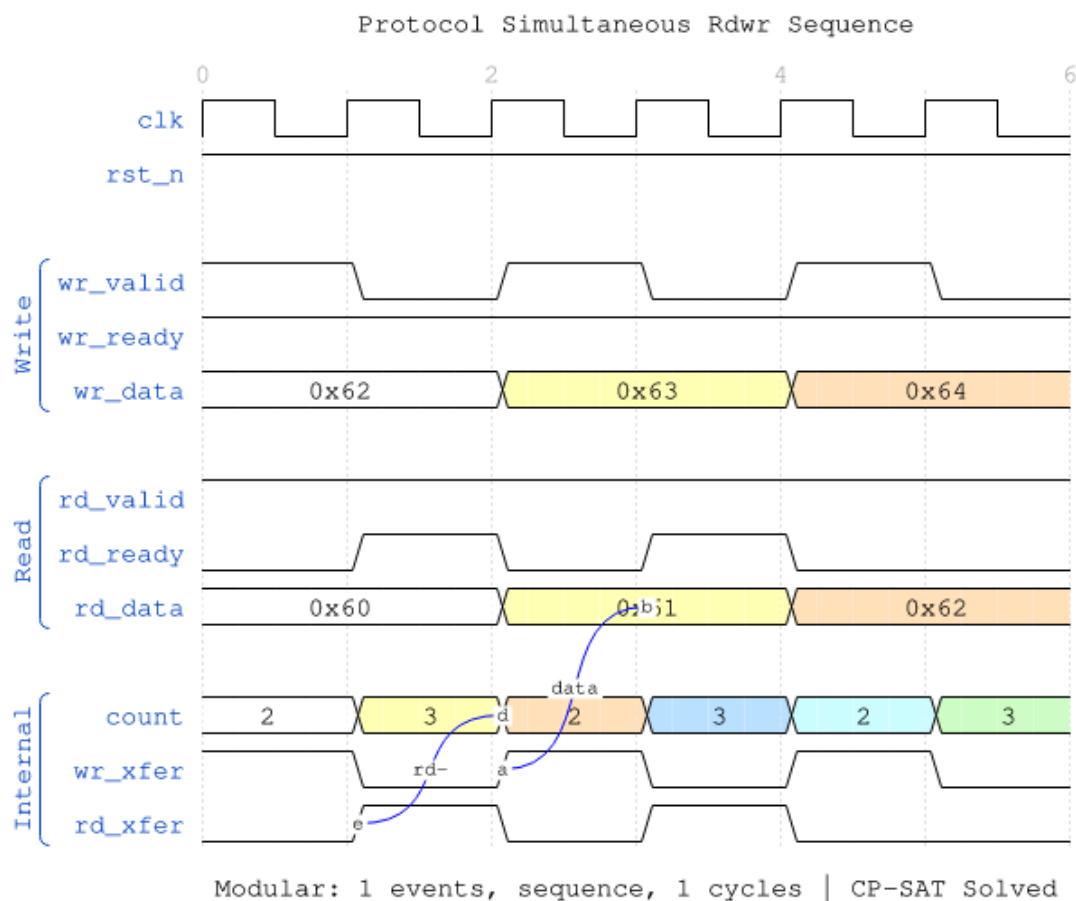
Zero-Latency Bypass

WaveJSON: [zero_latency_bypass_001.json](#)

Key Observations: - Write at cycle N → Read valid at cycle N - Data flows through with 0-cycle latency - count increments when write occurs without simultaneous read

7.5.1.2.2. Partially Full Mode ($0 < \text{count} < \text{DEPTH}$)

Elastic Storage: - Data buffered in shift register - Both write and read can occur simultaneously - count tracks occupancy



Simultaneous Read/Write

WaveJSON: [simultaneous_rdwr_001.json](#)

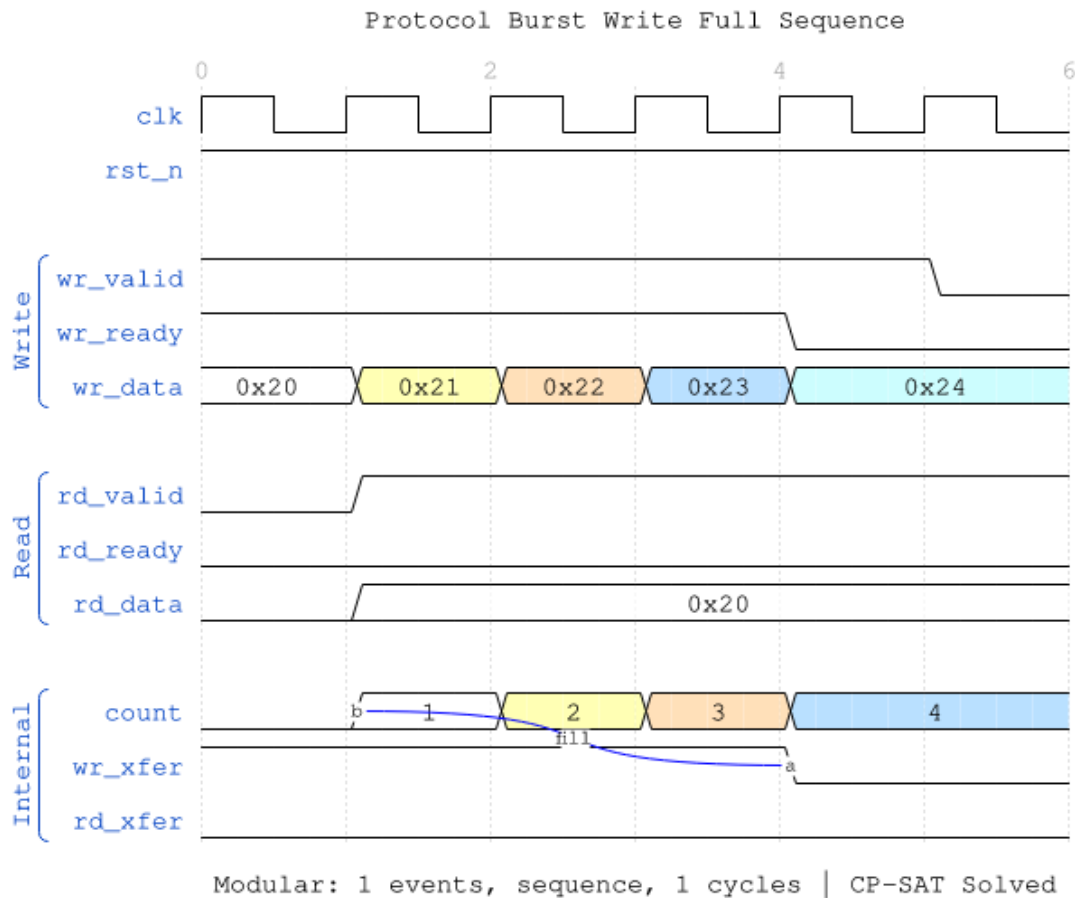
Key Observations: - Simultaneous operations: count stays constant - Write-only: count increments - Read-only: count decrements

7.5.1.3.3. Nearly Full Mode ($\text{count} = \text{DEPTH} - 1$)

Smart Backpressure: - wr_ready stays high if read occurring - Prevents false stalls

7.5.1.4.4. Full Mode (count = DEPTH)

Backpressure Applied: - wr_ready = 0 (cannot accept more writes) - Must read to make space



Burst Write Until Full

WaveJSON: [burst_write_full_001.json](#)

Key Observations: - wr_ready deasserts when buffer full - Subsequent writes blocked until space available - Reading creates space, wr_ready reasserts

7.5.2 Internal State Machine

The module implements a shift register for data storage:

```
// Data storage: shift register
logic [BUF_WIDTH-1:0] r_data;           // [DEPTH * DATA_WIDTH - 1 : 0]
logic [3:0] r_data_count;               // Current occupancy

// Transfer detection
```

```

logic w_wr_xfer = wr_valid & wr_ready;
logic w_rd_xfer = rd_valid & rd_ready;

```

State Transitions:

Condition	wr_xfer	rd_xfer	Action	count Change
Write only	1	0	Shift data in	+1
Read only	0	1	Shift data out	-1
Simultaneous	1	1	Shift data in/out	0
Idle	0	0	No change	0

7.5.3 Ready/Valid Logic

```

// Write ready conditions
wr_ready <= (count <= DEPTH-2) ||
            (count == DEPTH-1 && (~wr_xfer || rd_xfer)) ||
            (count == DEPTH && rd_xfer);

// Read valid conditions
rd_valid <= (count >= 2) ||
            (count == 1 && (~rd_xfer || wr_xfer)) ||
            (count == 0 && wr_xfer);

```

7.6 Timing Characteristics

7.6.1 Latency

Condition	Latency	Cycles
Empty → Read	0	Immediate (combinatorial)
Write → Read (buffered)	1	One clock cycle
Backpressure response	1	One clock after full

7.6.2 Throughput

- **Maximum:** 1 transfer per clock cycle
- **Sustained:** 1 transfer per clock (with proper flow control)

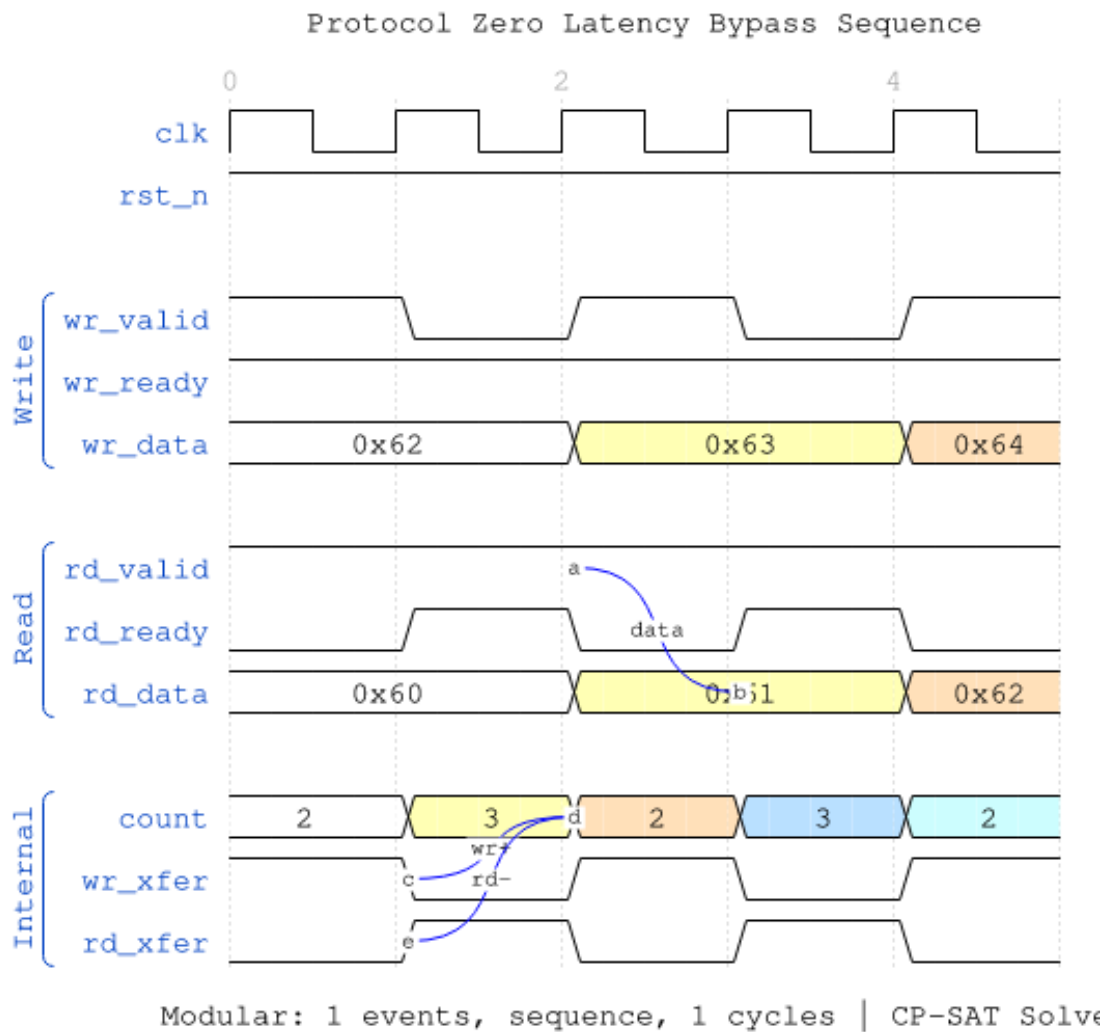
7.6.3 Critical Paths

- **Write → Read (empty):** Combinatorial path (may need timing closure in deep pipelines)
 - **Ready/Valid Logic:** Single-cycle registered
-

7.7 Comprehensive Timing Examples

7.7.1 Scenario 1: Zero-Latency Bypass

Write to empty buffer shows immediate `rd_valid`:

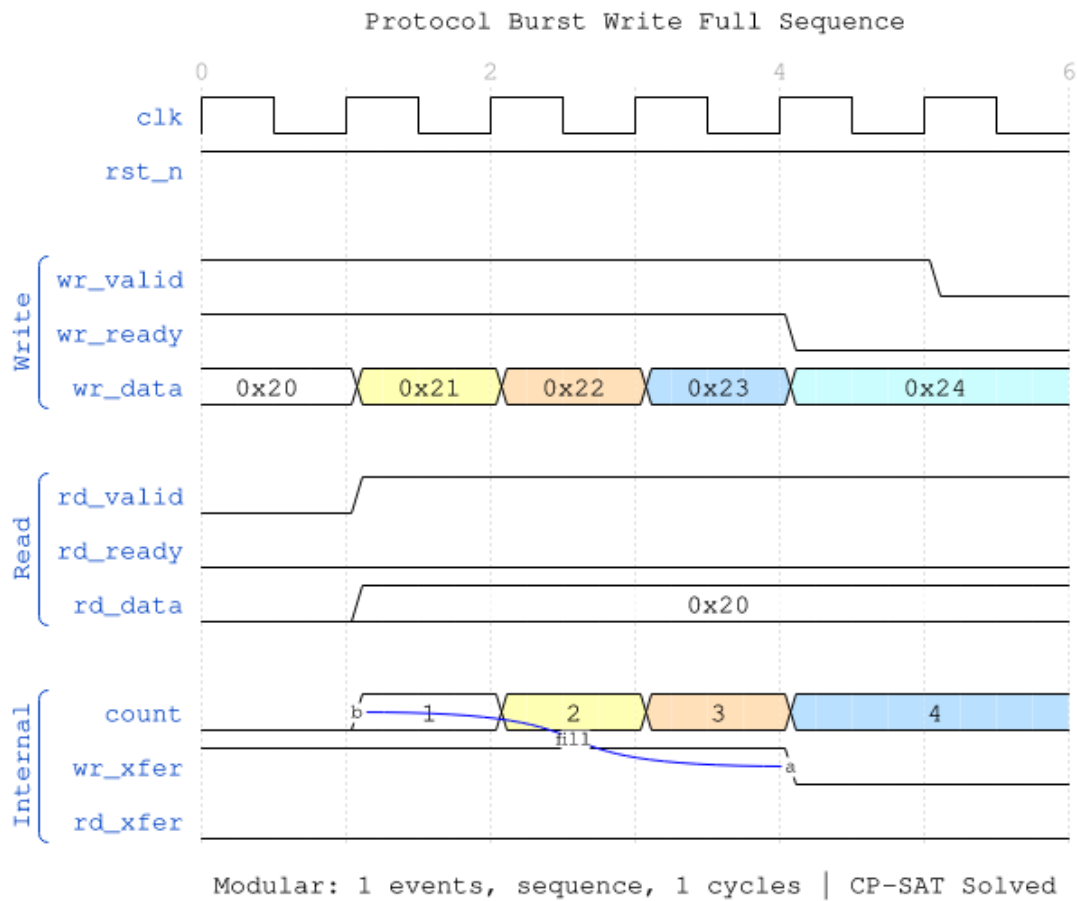


Zero-Latency Bypass

WaveJSON: [zero_latency_bypass_001.json](#)

7.7.2 Scenario 2: Burst Write Until Full

Demonstrates backpressure behavior:

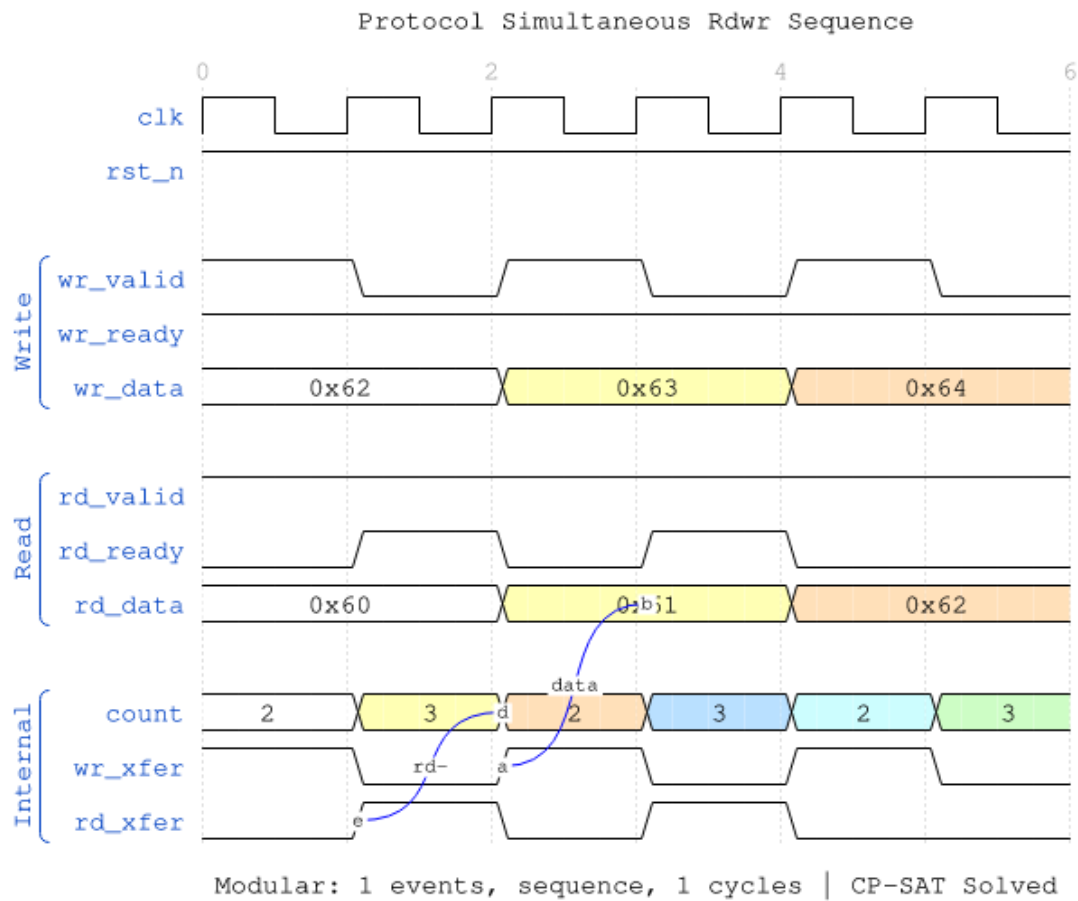


Burst Write Full

WaveJSON: [burst_write_full_001.json](#)

7.7.3 Scenario 3: Simultaneous Read/Write

Pass-through operation with constant occupancy:

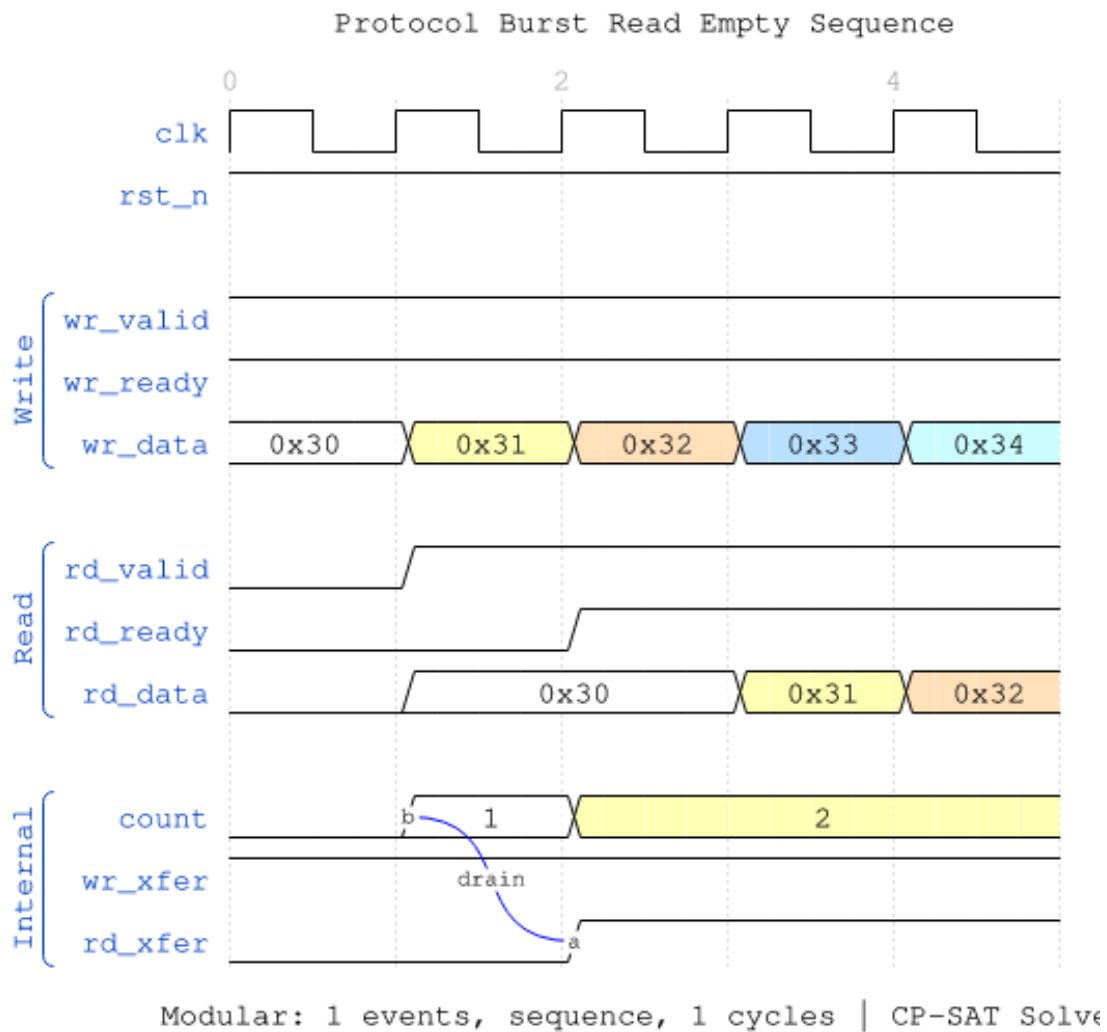


Simultaneous R/W

WaveJSON: [simultaneous_rdwr_001.json](#)

7.7.4 Scenario 4: Burst Read Until Empty

Draining the buffer:

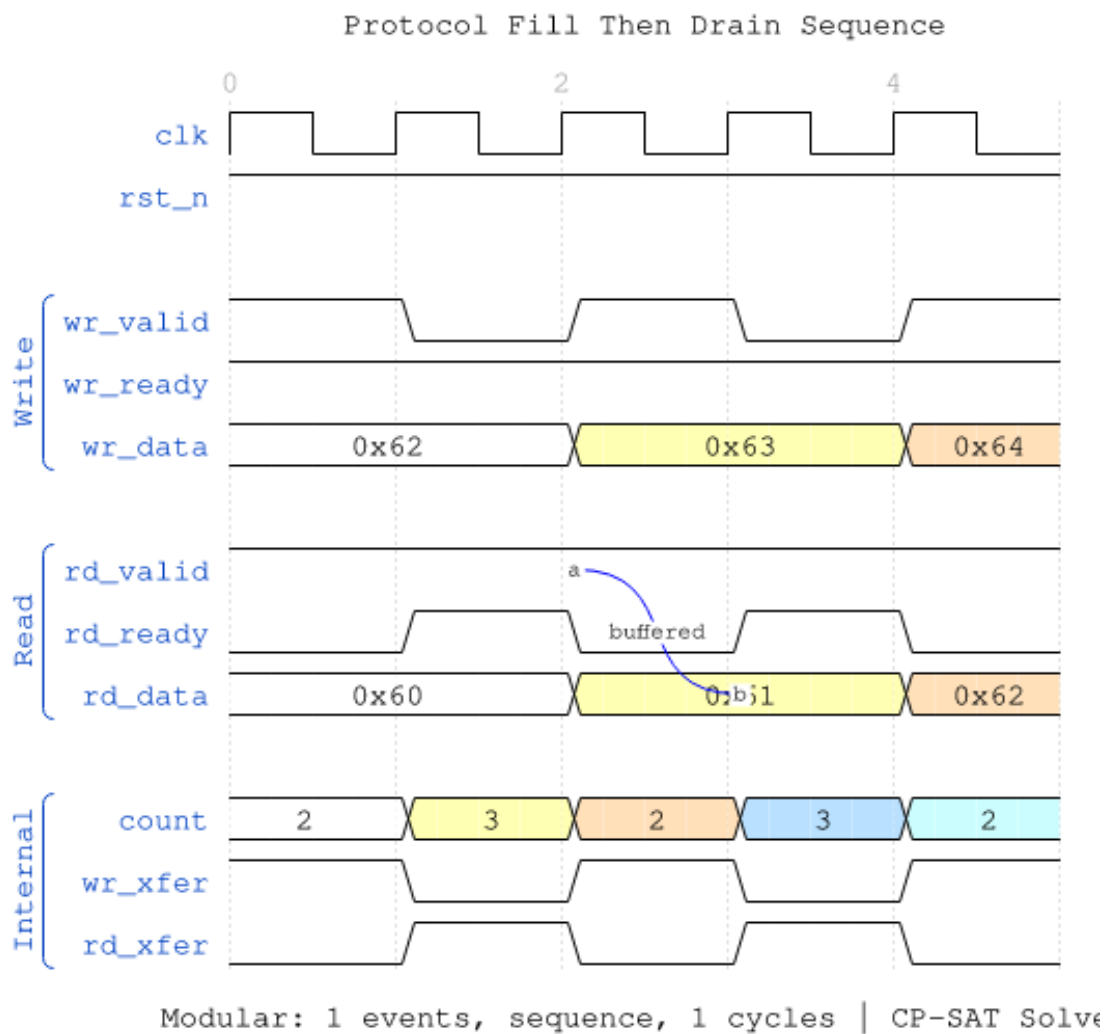


Burst Read Empty

WaveJSON: [burst_read_empty_001.json](#)

7.7.5 Scenario 5: Fill Then Drain Pattern

Complete fill phase followed by drain:

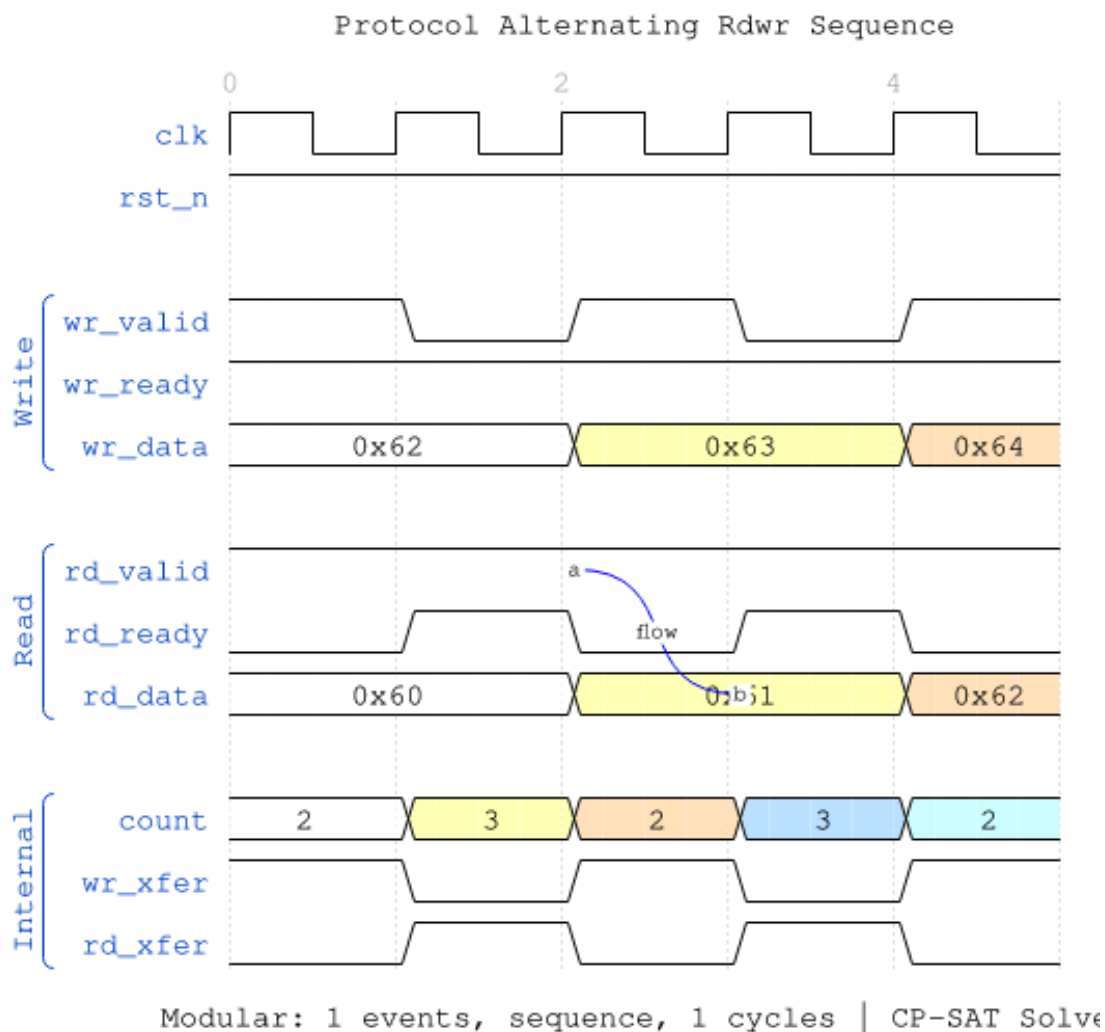


Fill Then Drain

WaveJSON: [fill_then_drain_001.json](#)

7.7.6 Scenario 6: Alternating Read/Write

Continuous interleaved operations:



Alternating R/W

WaveJSON: [alternating_rdwr_001.json](#)

7.8 Usage Examples

7.8.1 Example 1: Basic Pipeline Stage

```
// Break long combinatorial path
gaxi_skid_buffer #(
    .DATA_WIDTH(64),
    .DEPTH(4)
) u_pipeline_stage (
```

```

        .axi_aclk      (clk),
        .axi_aresetn  (rst_n),
        .wr_valid     (stage1_valid),
        .wr_ready     (stage1_ready),
        .wr_data      (stage1_data),
        .rd_valid     (stage2_valid),
        .rd_ready     (stage2_ready),
        .rd_data      (stage2_data),
        .count        (stage_occupancy),
        .rd_count     ()
    );

```

7.8.2 Example 2: Backpressure Absorption

// Absorb temporary backpressure from downstream

```

gaxi_skid_buffer #(
    .DATA_WIDTH(128),
    .DEPTH(8),           // Larger depth for more buffering
    .INSTANCE_NAME("BP_ABSORB")
) u_backpressure_buffer (
    .axi_aclk      (clk),
    .axi_aresetn  (rst_n),
    .wr_valid     (continuous_valid), // Always producing
    .wr_ready     (continuous_ready),
    .wr_data      (continuous_data),
    .rd_valid     (bursty_valid),     // Intermittent consumer
    .rd_ready     (bursty_ready),
    .rd_data      (bursty_data),
    .count        (buffer_level),
    .rd_count     ()
);

```

// Monitor buffer level for flow control

assign almost_full = (buffer_level >= 6); *// Threshold*

7.8.3 Example 3: Monitoring Occupancy

```

gaxi_skid_buffer #(
    .DATA_WIDTH(32),
    .DEPTH(4)
) u_monitored (
    .axi_aclk      (clk),
    .axi_aresetn  (rst_n),
    .wr_valid     (wr_valid),
    .wr_ready     (wr_ready),
    .wr_data      (wr_data),
    .rd_valid     (rd_valid),
    .rd_ready     (rd_ready),
    .rd_data      (rd_data),

```



```

        .count      (fifo_count),
        .rd_count   ()
    );

    // Performance monitoring
    always_ff @(posedge clk) begin
        if (fifo_count > max_count_seen)
            max_count_seen <= fifo_count;

        if (fifo_count == 4'h0 && wr_valid && wr_ready)
            bypass_events <= bypass_events + 1; // Track bypass usage
    end

```

7.9 Design Considerations

7.9.1 Depth Selection

DEPTH	Use Case	Latency	Buffering
2	Minimal buffering, tight timing	0-1 cycles	Limited backpressure tolerance
4	Standard pipeline stage	0-1 cycles	Moderate backpressure tolerance
6	Heavier buffering needs	0-1 cycles	Good backpressure tolerance
8	Maximum buffering	0-1 cycles	Excellent backpressure tolerance

Recommendation: Use DEPTH=4 for most cases - good balance of buffering and resource usage.

7.9.2 Timing Closure

If zero-latency bypass creates timing violations:

Option 1: Add pipeline stage after skid buffer

```

// Skid buffer (combinatorial read)
gaxi_skid_buffer #(.DATA_WIDTH(32), .DEPTH(4)) u_skid (...);

// Additional register stage
always_ff @(posedge clk) begin

```

```

    if (skid_rd_valid && skid_rd_ready) begin
        final_data <= skid_rd_data;
        final_valid <= 1'b1;
    end else if (final_ready) begin
        final_valid <= 1'b0;
    end
end

```

Option 2: Use FIFO with REGISTERED=1 instead (see [gaxi_fifo_sync](#))

7.9.3 Area vs. Performance Trade-offs

Metric	DEPTH=2	DEPTH=4	DEPTH=8
Flops	2×DW	4×DW	8×DW
LUTs	~40	~50	~70
Bypass Latency	0 cycles	0 cycles	0 cycles
Backpressure Tolerance	Low	Moderate	High

7.10 Error Checking

The module includes assertion checks (simulation only):

```

// synopsys translate_off
always @(posedge axi_aclk) begin
    if ((wr_valid && !wr_ready) && (wr_xfer)) begin
        $display("Error: %s write while buffer full, %t",
INSTANCE_NAME, $time);
    end
end

always @(posedge axi_aclk) begin
    if ((rd_ready && !rd_valid) && (rd_xfer)) begin
        $display("Error: %s read while buffer empty, %t",
INSTANCE_NAME, $time);
    end
end
// synopsys translate_on

```

These checks help catch: - Protocol violations (writes when not ready) - Invalid reads (reading when empty)

7.11 Testing

7.11.1 Test File

Location: val/amba/test_gaxi_buffer_sync.py

Test Modes: - mode='skid' → Tests this module specifically - mode='fifo_mux' → Tests gaxi_fifo_sync with REGISTERED=0 - mode='fifo_flop' → Tests gaxi_fifo_sync with REGISTERED=1

7.11.2 WaveDrom Test

Location: val/amba/test_gaxi_wavedrom_example.py

Generates comprehensive timing diagrams showing all 6 scenarios.

7.11.3 Running Tests

```
# Functional test (skid buffer mode)
pytest val/amba/test_gaxi_buffer_sync.py -k "skid" -v
```

```
# Generate waveforms
pytest val/amba/test_gaxi_wavedrom_example.py -v
cd val/amba && bash wd_cmd.sh
```

7.12 Common Issues

7.12.1 Issue 1: Timing Violation on Bypass Path

Symptom: Setup time violation from wr_data to rd_data

Solution:

```
// Add register stage after skid buffer
gaxi_skid_buffer #(...) u_skid (...);
```

```
always_ff @(posedge clk) begin
    registered_data <= skid_rd_data;
    registered_valid <= skid_rd_valid;
end
```

7.12.2 Issue 2: Buffer Overflow

Symptom: Data loss, count exceeds DEPTH

Debug: 1. Check write logic respects wr_ready 2. Verify DEPTH parameter is sufficient 3. Monitor count signal - should never exceed DEPTH

7.12.3 Issue 3: Unexpected Latency

Symptom: Data takes multiple cycles to appear at output

Debug: 1. Check buffer was empty when write occurred 2. Verify `rd_ready` is asserted (prevents data from appearing) 3. Look at count - if non-zero, data is buffered

7.13 Related Modules

- [gaxi_fifo_sync](#) - Larger depth, optional registered output
 - [gaxi_fifo_async](#) - Clock domain crossing
 - [gaxi_skid_buffer_async](#) - Async version
-

7.14 References

- **Tutorial:** [GAXI WaveDrom Tutorial](#)
 - **Overview:** [GAXI Index](#)
 - **Source:** `rtl/amba/gaxi/gaxi_skid_buffer.sv`
 - **Tests:** `val/amba/test_gaxi_buffer_sync.py`,
`val/amba/test_gaxi_wavedrom_example.py`
-

Version: 1.0 **Last Updated:** 2025-10-06 **Maintained By:** RTL Design Sherpa Project

8 gaxi_skid_buffer_struct

Module: `gaxi_skid_buffer_struct.sv` **Location:** `rtl/amba/gaxi/` **Status:** ✓ Production Ready

8.1 Overview

Struct-aware variant of [gaxi_skid_buffer](#) that accepts SystemVerilog type parameters instead of explicit data width. This enables clean handling of complex data structures (e.g., AXI channels, custom packets) without manual packing/unpacking.

8.1.1 Key Features

- ✓ **Type-Parameterized:** Accepts any SystemVerilog type (struct, union, enum, packed array)

- ✓ **Automatic Width Calculation:** Uses \$bits() to determine buffer width
 - ✓ **Zero-Latency Bypass:** Identical performance to base skid buffer
 - ✓ **Same Architecture:** Shift register with valid/ready flow control
 - ✓ **Debug Support:** Instance naming and transaction logging
-

8.2 Module Declaration

```

module gaxi_skid_buffer_struct #(
    parameter type STRUCT_TYPE = logic [31:0], // Any SystemVerilog
type
    parameter int DEPTH = 2, // Must be {2, 4, 6,
8}
    parameter INSTANCE_NAME = "DEADF1F0", // Debug identifier

    // Automatically derived
    localparam int STRUCT_WIDTH = $bits(STRUCT_TYPE),
    localparam int BUF_WIDTH = STRUCT_WIDTH * DEPTH
) (
    // Global Clock and Reset
    input logic axi_aclk,
    input logic axi_aresetn,

    // Input side
    input logic wr_valid,
    output logic wr_ready,
    input STRUCT_TYPE wr_data, // Type-safe input

    // Output side
    output logic [3:0] count,
    output logic rd_valid,
    input logic rd_ready,
    output logic [3:0] rd_count,
    output STRUCT_TYPE rd_data // Type-safe output
);

```

8.3 Parameters

Parameter	Type	Default	Description
STRUCT_TYPE	type	logic [31:0]	Any SystemVerilog type (struct, union, enum, array)

Parameter	Type	Default	Description
DEPTH	int	2	Buffer depth (must be 2, 4, 6, or 8)
INSTANCE_NAME	string	“DEADF1F0”	Debug identifier for simulation messages

8.4 Ports

8.4.1 Clock and Reset

Port	Width	Direction	Description
axi_aclk	1	Input	Clock
axi_aresetn	1	Input	Active-low asynchronous reset

8.4.2 Write Interface

Port	Width	Direction	Description
wr_valid	1	Input	Write data valid
wr_ready	1	Output	Ready to accept write
wr_data	STRUCT_TYPE	Input	Input data (type-safe)

8.4.3 Read Interface

Port	Width	Direction	Description
rd_valid	1	Output	Read data valid
rd_ready	1	Input	Ready to consume read
rd_data	STRUCT_TYPE	Output	Output data (type-safe)

8.4.4 Status

Port	Width	Direction	Description
count	4	Output	Current buffer occupancy (0 to DEPTH)
rd_count	4	Output	Same as count (for compatibility)

8.5 Usage Examples

8.5.1 Example 1: AXI Read Address Channel

```
// Define AXI AR channel struct
typedef struct packed {
    logic [7:0] arid;
    logic [31:0] araddr;
    logic [7:0] arlen;
    logic [2:0] arsize;
    logic [1:0] arburst;
    logic arlock;
    logic [3:0] arcache;
    logic [2:0] arprot;
    logic [3:0] arqos;
} axi_ar_t;

// Instantiate struct-aware buffer
gaxi_skid_buffer_struct #(
    .STRUCT_TYPE(axi_ar_t),
    .DEPTH(4),
    .INSTANCE_NAME("AXI_AR_BUF")
) u_ar_buffer (
    .axi_aclk (axi_clk),
    .axi_aresetn (axi_resetn),

    // Input: Directly assign struct
    .wr_valid (master_arvalid),
    .wr_ready (master_arready),
    .wr_data ({master_arid, master_araddr, master_arlen,
               master_arsize, master_arburst, master_arlock,
               master_arcache, master_arprot, master_arqos}),

    // Output: Unpack struct
```

```

        .rd_valid      (slave_arvalid),
        .rd_ready      (slave_arready),
        .rd_data       ({slave_arid, slave_araddr, slave_arlen,
                        slave_arsize, slave_arburst, slave_arlock,
                        slave_arcache, slave_arprot, slave_arqos}),

        .count         (ar_buf_count),
        .rd_count      ()
    );

```

8.5.2 Example 2: Custom Packet with Mixed Fields

```

// Define custom packet structure
typedef struct packed {
    logic [15:0] packet_id;
    logic [7:0]  packet_type;
    logic [31:0] timestamp;
    logic [63:0] payload;
    logic [3:0]  priority;
    logic        last;
} custom_pkt_t;

// Buffer for packet pipeline
gaxi_skid_buffer_struct #(
    .STRUCT_TYPE(custom_pkt_t),
    .DEPTH(8),
    .INSTANCE_NAME("PKT_PIPELINE")
) u_pkt_buffer (
    .axi_aclk      (pkt_clk),
    .axi_aresetn   (pkt_resetn),

    .wr_valid      (ingress_pkt_valid),
    .wr_ready      (ingress_pkt_ready),
    .wr_data       (ingress_pkt),      // Direct struct assignment

    .rd_valid      (egress_pkt_valid),
    .rd_ready      (egress_pkt_ready),
    .rd_data       (egress_pkt),      // Direct struct output

    .count         (pkt_buf_depth),
    .rd_count      ()
);

```

8.5.3 Example 3: Simple Data Array

```

// Array of samples
typedef logic [15:0] sample_array_t [4];

```



```

gaxi_skid_buffer_struct #(
    .STRUCT_TYPE(sample_array_t),
    .DEPTH(4),
    .INSTANCE_NAME("SAMPLE_BUF")
) u_sample_buffer (
    .axi_aclk      (dsp_clk),
    .axi_aresetn   (dsp_resetn),

    .wr_valid      (sample_valid),
    .wr_ready      (sample_ready),
    .wr_data       (sample_array),      // 4 x 16-bit samples

    .rd_valid      (proc_valid),
    .rd_ready      (proc_ready),
    .rd_data       (proc_array),

    .count         (sample_count),
    .rd_count      ()
);

```

8.6 Behavior

8.6.1 Identical to gaxi_skid_buffer

The internal behavior is **identical** to [gaxi_skid_buffer](#):

1. **Shift Register:** Data stored in array, shifts on read
2. **Zero-Latency Bypass:** Empty buffer passes data combinatorially
3. **Valid/Ready Handshake:** Transfer occurs when valid && ready
4. **Backpressure:** wr_ready deasserts when full

8.6.2 Key Difference: Type Safety

gaxi_skid_buffer:

```

.DATA_WIDTH(96),      // Manual width calculation
.wr_data(packed_data), // Manual packing required
.rd_data(packed_out)  // Manual unpacking required

```

gaxi_skid_buffer_struct:

```

.STRUCT_TYPE(axi_ar_t), // Type parameter
.wr_data(ar_struct),    // Direct struct assignment
.rd_data(ar_out_struct) // Direct struct output

```

8.7 Advantages Over Base Skid Buffer

8.7.1 Advantage 1: Type Safety

```
// Base skid buffer - error-prone
gaxi_skid_buffer #(.DATA_WIDTH(96)) u_buf (
    .wr_data({arid, araddr, arlen, ...}) // Easy to get field order
    wrong
);
```

```
// Struct buffer - compiler-checked
gaxi_skid_buffer_struct #(.STRUCT_TYPE(axi_ar_t)) u_buf (
    .wr_data(ar_channel) // Struct ensures correct fields
);
```

8.7.2 Advantage 2: No Manual Width Calculation

```
// Base: Must calculate width manually
localparam AR_WIDTH = 8 + 32 + 8 + 3 + 2 + 1 + 4 + 3 + 4; // Error-prone
```

```
// Struct: Automatic using $bits()
localparam AR_WIDTH = $bits(axi_ar_t); // Always correct
```

8.7.3 Advantage 3: Cleaner Code

```
// Before: Manual packing/unpacking
wire [95:0] packed_ar;
assign packed_ar = {arid, araddr, arlen, arsize, ...};
assign {slave_arid, slave_araddr, ...} = unpacked_ar;
```

```
// After: Direct struct assignment
axi_ar_t master_ar, slave_ar;
assign slave_ar = master_ar; // Clean and readable
```

8.8 Design Patterns

8.8.1 Pattern 1: AXI Channel Buffering

```
// Define all AXI channel structs
typedef struct packed {
    logic [ID_W-1:0] awid;
    logic [ADDR_W-1:0] awaddr;
    // ... all AW signals
};
```

```

} axi_aw_t;

// Buffer each channel with appropriate type
gaxi_skid_buffer_struct #(.STRUCT_TYPE(axi_aw_t)) u_aw_buf (...);
gaxi_skid_buffer_struct #(.STRUCT_TYPE(axi_w_t)) u_w_buf (...);
gaxi_skid_buffer_struct #(.STRUCT_TYPE(axi_b_t)) u_b_buf (...);
gaxi_skid_buffer_struct #(.STRUCT_TYPE(axi_ar_t)) u_ar_buf (...);
gaxi_skid_buffer_struct #(.STRUCT_TYPE(axi_r_t)) u_r_buf (...);

```

8.8.2 Pattern 2: Pipeline Stages with Complex Data

```

// Define pipeline stage struct
typedef struct packed {
    logic [31:0] instruction;
    logic [4:0]  rs1, rs2, rd;
    logic [31:0] imm;
    logic [6:0]  opcode;
    logic        valid;
} pipeline_stage_t;

// Each pipeline stage buffered
pipeline_stage_t fetch_stage, decode_stage, execute_stage;

gaxi_skid_buffer_struct #(
    .STRUCT_TYPE(pipeline_stage_t),
    .INSTANCE_NAME("FETCH_DECODE")
) u_fd_buf (
    .wr_data(fetch_stage),
    .rd_data(decode_stage),
    ...
);

```

8.9 Testing

```

# Tests use parameterized structs
pytest val/amba/test_gaxi_skid_buffer_struct.py -v

# Test with different struct types
pytest val/amba/test_gaxi_skid_buffer_struct.py::test_axi_channel -v
pytest val/amba/test_gaxi_skid_buffer_struct.py::test_custom_struct -v

```

8.10 Synthesis Notes

8.10.1 Resource Usage

Same as **gaxi_skid_buffer** with equivalent DATA_WIDTH: - Logic: ~50 LUTs (for DEPTH=4) - Storage: DEPTH × \$bits(STRUCT_TYPE) flip-flops - No additional overhead from type parameter

8.10.2 Type Requirements

SystemVerilog types that work: - ✓ struct packed (recommended) - ✓ Packed arrays: logic [N:0][M:0] - ✓ Simple types: logic [N:0] - ⚠ struct (unpacked) - will be automatically packed by synthesis

Not supported: - ✗ Dynamic types - ✗ Classes - ✗ Queues/associative arrays

8.11 Related Modules

- [gaxi_skid_buffer](#) - Base implementation (DATA_WIDTH parameter)
 - [gaxi_fifo_sync](#) - Synchronous FIFO alternative
 - [gaxi_skid_buffer_async](#) - Clock domain crossing variant
-

8.12 When to Use

✓ **Use gaxi_skid_buffer_struct When:** - Working with complex data structures (AXI channels, custom packets) - Want type safety and compiler checks - Need cleaner, more readable code - Struct definition already exists

✓ **Use gaxi_skid_buffer When:** - Simple data widths (no complex structure) - Need absolute minimum resource usage - Working with legacy code that uses explicit widths

Last Updated: 2025-10-20

8.13 Navigation

- [← Back to GAXI Index](#)
- [← Back to RTLamba Index](#)

9 GAXI (Generic AXI) Modules

Location: rtl/amba/gaxi/ **Test Location:** val/amba/ **Status:** ✓ Production Ready

9.1 Overview

The GAXI subsystem provides a lightweight valid/ready handshake protocol for streaming data between components. Simpler than full AXI4-Stream while maintaining robust flow control, GAXI is used throughout the RTL Design Sherpa infrastructure for elastic buffering, clock domain crossing, and timing closure.

9.1.1 Key Features

- ✓ **Simple Handshake:** Valid/ready only - minimal overhead
- ✓ **Elastic Buffering:** Skid buffers for zero-latency bypass
- ✓ **Clock Domain Crossing:** Async variants with proper CDC
- ✓ **Type-Safe Structs:** Struct-aware buffers for complex data
- ✓ **Drop Capability:** Special FIFO variant with drop-by-count/drop-all
- ✓ **Flexible Modes:** Mux/flop output modes for area/speed trade-offs

9.2 Module Categories

9.2.1 Skid Buffers (Elastic Buffering)

Module	Description	Documentation	Status
<code>gaxi_skid_buffer</code>	Synchronous elastic buffer with zero-latency bypass	gaxi_skid_buffer.md	✓ Documented
<code>gaxi_skid_buffer_struct</code>	Type-parameterized variant for complex structs	gaxi_skid_buffer_struct.md	✓ Documented
<code>gaxi_skid_buffer_async</code>	Asynchronous elastic buffer for CDC	gaxi_skid_buffer_async.md	✓ Documented
<code>gaxi_skid_buffer_dbldr_n</code>	Double-drain skid buffer variant	gaxi_skid_buffer_dbldr_n.md	✓ Documented

9.2.2 FIFOs (Deep Buffering)

Module	Description	Documentation	Status
gaxi_fifo_sync	Synchronous FIFO with mux/flop modes	gaxi_fifo_sync.md	✓ Documented
gaxi_fifo_async	Asynchronous FIFO for clock domain crossing	gaxi_fifo_async.md	✓ Documented
gaxi_drop_fifo_sync	FIFO with drop-by-count and drop-all capability	gaxi_drop_fifo_sync.md	✓ Documented

9.2.3 Test-Code Multi-Instance Helpers

rtl/amba/testcode/ holds thin multi-instance wrappers used by the DV harnesses — they instantiate a bank of the base GAXI primitive (and, for the _sigmap variant, present a flattened array-signal port style). They are not documented individually; see the base module doc for the full behavior:

Test-code wrapper	Base module	Full documentation
gaxi_fifo_sync_multi	gaxi_fifo_sync	gaxi_fifo_sync.md
gaxi_fifo_async_multi	gaxi_fifo_async	gaxi_fifo_async.md
gaxi_skid_buffer_multi	gaxi_skid_buffer	gaxi_skid_buffer.md
gaxi_skid_buffer_multi_sigmap	gaxi_skid_buffer	gaxi_skid_buffer.md
gaxi_skid_buffer_async_multi	gaxi_skid_buffer_async	gaxi_skid_buffer_async.md

9.3 Protocol Overview

9.3.1 Interface Signals

Write Interface (Producer): - wr_valid - Data valid from source - wr_ready - Ready to accept data (backpressure) - wr_data[N:0] - Data from source

Read Interface (Consumer): - rd_valid - Data valid to sink - rd_ready - Ready to accept data from sink - rd_data[N:0] - Data to sink

Optional Monitoring: - count[3:0] - Current FIFO/buffer occupancy

9.3.2 Handshake Rules

1. **Transfer:** Occurs when valid && ready on rising clock edge
 2. **Data Stability:** data must be stable when valid is high
 3. **Backpressure:** ready may deassert to apply backpressure
 4. **Valid Persistence:** valid must remain high until transfer completes
-

9.4 Quick Start

9.4.1 Basic Skid Buffer (Zero-Latency)

```
gaxi_skid_buffer #(
    .DATA_WIDTH(32),
    .DEPTH(4)           // 4 entries
) u_skid (
    .axi_aclk      (clk),
    .axi_aresetn   (rst_n),

    // Write port (from producer)
    .wr_valid      (src_valid),
    .wr_ready      (src_ready),
    .wr_data       (src_data),

    // Read port (to consumer)
    .rd_valid      (dst_valid),
    .rd_ready      (dst_ready),
    .rd_data       (dst_data),

    // Status
    .count         (buf_count),
    .rd_count      ()
);
```

9.4.2 Synchronous FIFO (Deep Buffering)

```
gaxi_fifo_sync #(
    .DATA_WIDTH(64),
    .DEPTH(64),         // 64 entries
    .REGISTERED(0)      // 0=mux mode (fast), 1=flop mode (more area)
) u_fifo (
    .axi_aclk      (clk),
    .axi_aresetn   (rst_n),

    .wr_valid      (wr_valid),
    .wr_ready      (wr_ready),
```

```

        .wr_data      (wr_data),

        .rd_valid     (rd_valid),
        .rd_ready     (rd_ready),
        .rd_data      (rd_data),

        .count        (fifo_level)
    );

```

9.4.3 Asynchronous FIFO (Clock Domain Crossing)

```

gaxi_fifo_async #(
    .DATA_WIDTH(32),
    .DEPTH(16),           // Must be power of 2
    .N_FLOP_CROSS(3),    // CDC synchronizer stages
    .REGISTERED(0)
) u_async_fifo (
    // Write domain
    .axi_wr_aclk      (wr_clk),
    .axi_wr_aresetn   (wr_rst_n),
    .wr_valid         (wr_valid),
    .wr_ready         (wr_ready),
    .wr_data          (wr_data),

    // Read domain
    .axi_rd_aclk      (rd_clk),
    .axi_rd_aresetn   (rd_rst_n),
    .rd_valid         (rd_valid),
    .rd_ready         (rd_ready),
    .rd_data          (rd_data)
);

```

9.4.4 Type-Safe Struct Buffer

```

// Define AXI AR channel struct
typedef struct packed {
    logic [7:0]  arid;
    logic [31:0] araddr;
    logic [7:0]  arlen;
    logic [2:0]  arsize;
    logic [1:0]  arburst;
} axi_ar_t;

// Struct-aware buffer
gaxi_skid_buffer_struct #(
    .STRUCT_TYPE(axi_ar_t),
    .DEPTH(4)
) u_ar_buf (
    .axi_aclk      (clk),

```



```

        .axi_aresetn (rst_n),

        .wr_valid      (master_arvalid),
        .wr_ready      (master_arready),
        .wr_data       (master_ar),      // Type-safe struct

        .rd_valid      (slave_arvalid),
        .rd_ready      (slave_arready),
        .rd_data       (slave_ar),      // Type-safe struct

        .count         (ar_count)
    );

```

9.4.5 Drop FIFO (Packet Processing)

```

gaxi_drop_fifo_sync #(
    .DATA_WIDTH(512),
    .DEPTH(64),
    .REGISTERED(1)      // Flop mode for 512-bit width
) u_drop_fifo (
    .axi_aclk            (clk),
    .axi_aresetn        (rst_n),

    .wr_valid            (pkt_valid),
    .wr_ready            (pkt_ready),
    .wr_data             (pkt_data),

    .rd_valid            (proc_valid),
    .rd_ready            (proc_ready),
    .rd_data             (proc_data),

    // Drop control
    .drop_count          (drop_n_packets), // Drop N oldest entries
    .drop_count_valid    (drop_trigger),
    .drop_all            (flush_fifo),     // Flush entire FIFO

    .count               (pkt_count)
);

```

9.5 Testing

All GAXI modules are verified using CocoTB-based testbenches:

```

# Run all GAXI tests
pytest val/amba/test_gaxi*.py -v

```

```

# Run specific module tests
pytest val/amba/test_gaxi_buffer_sync.py -v      # Skid buffers + sync
FIFOs
pytest val/amba/test_gaxi_buffer_async.py -v     # Async FIFO
pytest val/amba/test_gaxi_drop_fifo_sync.py -v   # Drop FIFO

# Run with waveforms
pytest val/amba/test_gaxi_buffer_sync.py --vcd=waves.vcd -v

# Generate WaveDrom timing diagrams
pytest val/amba/test_gaxi_wavedrom_example.py -v
cd val/amba && bash wd_cmd.sh # Generate PNG/SVG

```

9.6 Design Patterns

9.6.1 Pattern 1: Timing Closure (Pipeline Stage)

Use skid buffer to break combinatorial paths:

```

// Without buffer: long combinatorial path
assign downstream_data = upstream_data; // May not meet timing

// With skid buffer: registered boundary
gaxi_skid_buffer #(.DATA_WIDTH(32), .DEPTH(2)) u_pipeline (
    .wr_valid(upstream_valid),
    .wr_ready(upstream_ready),
    .wr_data(upstream_data),
    .rd_valid(downstream_valid),
    .rd_ready(downstream_ready),
    .rd_data(downstream_data),
    ...
);

```

9.6.2 Pattern 2: Rate Adaptation

Use FIFO to handle burst traffic:

```

// Bursty writer, continuous reader
gaxi_fifo_sync #(.DATA_WIDTH(64), .DEPTH(64)) u_rate_adapter (
    .wr_valid(bursty_valid),      // Intermittent bursts
    .wr_ready(bursty_ready),
    .wr_data(bursty_data),
    .rd_valid(continuous_valid), // Always reading
    .rd_ready(continuous_ready),

```

```

        .rd_data(continuous_data),
        ...
    );

```

9.6.3 Pattern 3: Clock Domain Crossing

Use async FIFO for safe CDC:

```

// Domain A → Domain B
gaxi_fifo_async #(
    .DATA_WIDTH(32),
    .DEPTH(16),
    .N_FLOP_CROSS(3)      // 3-stage synchronizer
) u_cdc (
    .axi_wr_aclk(clk_a),
    .axi_wr_aresetn(rst_a_n),
    .wr_valid(domain_a_valid),
    .wr_ready(domain_a_ready),
    .wr_data(domain_a_data),
    .axi_rd_aclk(clk_b),
    .axi_rd_aresetn(rst_b_n),
    .rd_valid(domain_b_valid),
    .rd_ready(domain_b_ready),
    .rd_data(domain_b_data)
);

```

9.7 Performance Characteristics

9.7.1 Latency

Module	Empty Latency	Full Latency	Notes
gaxi_skid_buffer	0 cycles	1 cycle	Zero-latency bypass when empty
gaxi_fifo_sync (mux)	1 cycle	1 cycle	Combinatorial read
gaxi_fifo_sync (flop)	2 cycles	2 cycles	Registered read
gaxi_fifo_async	3-5 cycles	3-5 cycles	CDC synchroni

Module	Empty Latency	Full Latency	Notes
gaxi_drop_fifo_sync	1 cycle	1 cycle	zation overhead Plus 3 cycles for drop operation

9.7.2 Throughput

All modules support **1 transfer per clock cycle** when not applying backpressure.

9.7.3 Resource Utilization (Typical)

Module	Logic	Memory	Notes
gaxi_skid_buffer (D=4, DW=32)	~50 LUTs	128 FFs	Shift register
gaxi_fifo_sync (D=16, DW=32)	~100 LUTs	512 FFs	+ counters
gaxi_fifo_async (D=16, DW=32)	~150 LUTs	512 FFs	+ CDC logic
gaxi_drop_fifo_sync (D=64, DW=32)	~200 LUTs	2048 FFs	+ drop control

9.8 Common Use Cases

9.8.1 Video Processing (Skid Buffer)

Characteristics: - High throughput (pixel streams) - Minimal latency required - Regular flow control

Recommended:

```
gaxi_skid_buffer #(
    .DATA_WIDTH(96),    // 4 pixels x 24-bit RGB
    .DEPTH(4)           // Shallow buffer
)
```

9.8.2 Network Packet Processing (Drop FIFO)

Characteristics: - Variable packet sizes - Need to drop oldest on overflow - High data rate

Recommended:

```
gaxi_drop_fifo_sync #(
    .DATA_WIDTH(512),    // Wide data bus
    .DEPTH(64),          // Deep buffer
    .REGISTERED(1)       // Flop mode for timing
)
```

9.8.3 Clock Domain Crossing (Async FIFO)

Characteristics: - Independent clock domains - Metastability protection required - Variable clock ratios

Recommended:

```
gaxi_fifo_async #(
    .DATA_WIDTH(32),
    .DEPTH(16),          // Must be power of 2
    .N_FLOP_CROSS(3)     // 3-stage synchronizer
)
```

9.9 Parameter Selection Guidelines

9.9.1 Buffer Depth (DEPTH)

Application	Recommended Depth	Rationale
Pipeline stage	2-4	Minimal latency, timing closure
Video line buffer	16-32	Line-sized bursts
Network packet buffer	32-64	Variable packet sizes
CDC buffer	8-16	Handle clock ratio variations

9.9.2 Output Mode (REGISTERED)

Mux Mode (REGISTERED=0): - ✓ Lower latency (1 cycle) - ✓ Smaller area - ✗ May not meet timing with wide data - **Use for:** Narrow data (≤ 64 bits), non-critical paths

Flop Mode (REGISTERED=1): - ✓ Better timing (registered output) - ✓ Supports wide data - ✗ Higher latency (2 cycles) - ✗ More area - **Use for:** Wide data (≥ 128 bits), critical paths

9.9.3 CDC Synchronizer Stages (N_FLOP_CROSS)

Stages	Use Case
2	Slow clocks (<100 MHz), low risk
3	Recommended default
4+	High-speed (>400 MHz) or high-reliability

9.10 Related Documentation

9.10.1 Comprehensive Guide

- [GAXI Index \(Detailed\)](#) - Comprehensive guide with timing diagrams

9.10.2 Protocol Specifications

- ARM AMBA AXI4-Stream Protocol Specification (similar handshake)

9.10.3 RTL Design Sherpa Documentation

- [RTLamba Overview](#) - Complete AMBA subsystem architecture
- [AXI4 Modules](#) - Full AXI4 protocol
- [AXI4-Stream Modules](#) - AXI4-Stream protocol
- [GAXI WaveDrom Tutorial](#) - Timing diagrams

9.10.4 Source Code

- RTL: rtl/amba/gaxi/
 - Tests: val/amba/test_gaxi*.py
 - Framework: bin/TBClasses/components/gaxi/
-

9.11 Design Notes

9.11.1 When to Use GAXI vs AXI4-Stream

✓ **Use GAXI For:** - Internal buffering and timing closure - Simple data streaming - CDC with basic handshake - Infrastructure/glue logic

✓ **Use AXI4-Stream For:** - External interfaces - Need TID/TDEST/TUSER signals - Complex routing requirements - Standards compliance

9.11.2 Buffer vs FIFO Selection

Use Skid Buffer (gaxi_skid_buffer) When: - ✓ Need zero-latency bypass - ✓ Shallow depth (2-8 entries) - ✓ Timing closure only - ✓ Minimal area

Use FIFO (gaxi_fifo_sync) When: - ✓ Need deeper buffering (16+ entries) - ✓ Rate adaptation required - ✓ Burst traffic handling - ✓ More predictable latency

Use Async FIFO (gaxi_fifo_async) When: - ✓ Clock domain crossing required - ✓ Independent clock domains - ✓ Need CDC-safe handshake

Use Drop FIFO (gaxi_drop_fifo_sync) When: - ✓ Need to discard oldest data on overflow - ✓ Packet-based processing - ✓ Flush/reset functionality required

9.12 Common Issues and Debugging

9.12.1 Issue 1: Data Loss

Symptom: Missing data at output

Debug: 1. Check count signal - should not overflow (reach DEPTH) 2. Verify `wr_valid` stays high until `wr_ready` asserts 3. Increase DEPTH if necessary 4. Add backpressure handling upstream

9.12.2 Issue 2: Timing Violation

Symptom: Setup/hold violations in synthesis

Solutions: 1. Use flop mode: `.REGISTERED(1)` 2. Increase DEPTH for better buffering 3. Add pipeline stages with multiple skid buffers

9.12.3 Issue 3: CDC Metastability

Symptom: Incorrect data, simulation/silicon mismatch

Solutions: 1. Increase `N_FLOP_CROSS` (minimum 3 recommended) 2. Verify independent clock domains (no relationship) 3. Check reset synchronization in both domains 4. Use constraints for CDC paths

Last Updated: 2025-10-20

9.13 Navigation

- [← Back to RTLAmba Index](#)

- [← Back to Main Documentation Index](#)

10 AMBA Clock Gate Controller

Module: `amba_clock_gate_ctrl.sv` **Location:** `rtl/amba/shared/` **Status:** Production Ready

10.1 Overview

The AMBA Clock Gate Controller provides dynamic clock gating capability tailored for AMBA protocol interfaces. It monitors transaction activity on both user-side and AXI-side interfaces to automatically gate the clock during idle periods, reducing dynamic power consumption while maintaining protocol correctness.

10.1.1 Key Features

- Automatic activity detection from user and AXI valid signals
 - Configurable idle threshold before gating activation
 - Integration with standard ICG (Integrated Clock Gate) cells
 - Registered wakeup signal for metastability protection
 - Global enable/disable control
 - Idle status monitoring
 - Zero added latency when clock is ungated
-

10.2 Module Purpose

AMBA protocol interfaces often experience periods of inactivity during system operation. Traditional always-on clocking wastes power during these idle periods. The AMBA Clock Gate Controller solves this by:

1. Monitoring transaction activity on both interfaces
2. Detecting extended idle periods
3. Automatically gating the clock to save power
4. Instantly ungating when new activity arrives

Use Cases: - Power-critical AMBA interface implementations - Battery-powered systems with intermittent bus activity - Multi-master systems where individual masters idle frequently - ASIC designs requiring aggressive dynamic power reduction

Key Benefit: Transparent power saving with no protocol impact (activity detection ensures clock availability when needed).

10.3 Parameters

Parameter	Type	Default	Description
CG_IDLE_COUNT_WIDTH	int	4	Width of idle counter (determines maximum idle threshold)
ICW	int	CG_IDLE_COUNT_WIDTH	Shorthand alias for idle count width

Idle Count Range: - Minimum: 0 (immediate gating when idle) - Maximum: $2^{\text{CG_IDLE_COUNT_WIDTH}} - 1$ - Default (4-bit): 0 to 15 cycles

10.4 Port Groups

10.4.1 Clock and Reset

Port	Direction	Width	Description
clk_in	input	1	Input clock (always running, feeds ICG cell)
aresetn	input	1	Active-low asynchronous reset

10.4.2 Configuration Interface

Port	Direction	Width	Description
cfg_cg_enable	input	1	Global clock gating enable (1=allow gating, 0=always ungated)
cfg_cg_idle_count	input	ICW	Idle cycle threshold before gating activates

10.4.3 Activity Monitoring

Port	Direction	Width	Description
user_valid	input	1	User-side activity indicator (any valid signal from user interface)
axi_valid	input	1	AXI-side activity indicator (any valid signal from AXI interface)

10.4.4 Clock Gating Outputs

Port	Direction	Width	Description
clk_out	output	1	Gated output clock (drives clocked logic)
gating	output	1	Gating active indicator (1=clock currently gated)
idle	output	1	Idle status (1=no activity detected, 0=activity present)

10.5 Functional Description

10.5.1 Activity Detection Logic

The module combines activity signals from both user and AXI interfaces to form a unified wakeup signal:

wakeup = user_valid OR axi_valid

Why Both Interfaces? - User interface: Detects upstream master activity - AXI interface: Detects downstream slave activity - Combined: Ensures clock active for bidirectional traffic

Registered Wakeup:

```
always_ff @(posedge clk_in or negedge aresetn) begin
    if (!aresetn)
        r_wakeup <= 1'b1; // Default: Active (clock ungated at reset)
    else
        r_wakeup <= user_valid || axi_valid;
end
```

Key Property: Wakeup signal is registered to: 1. Prevent metastability (if activity signals cross clock domains) 2. Provide stable input to downstream clock gate control 3. Default to active state during reset (ensures clock available)

10.5.2 Idle Signal Generation

Idle status is the inverse of wakeup:

`idle = ~r_wakeup`

Interpretation: - idle = 1: No activity for at least 1 cycle (eligible for gating) - idle = 0: Recent activity detected (clock must remain ungated)

Usage: Exported for monitoring/debug (system can observe interface idle status).

10.5.3 Clock Gating Control Integration

The module instantiates a base clock gate controller:

```
clock_gate_ctrl #(
    .IDLE_CNTR_WIDTH    (ICW)
) u_clock_gate_ctrl (
    .clk_in              (clk_in),
    .aresetn             (aresetn),
    .cfg_cg_enable       (cfg_cg_enable),
    .cfg_cg_idle_count   (cfg_cg_idle_count),
    .wakeup              (r_wakeup),
    .clk_out             (clk_out),
    .gating              (gating)
);
```

Base Controller Operation: 1. Monitors r_wakeup signal 2. Increments idle counter when wakeup=0 (idle) 3. Gates clock when counter >= cfg_cg_idle_count 4. Ungates immediately when wakeup=1 (activity)

10.5.4 Gating Threshold Behavior

cfg_cg_idle_count Examples (4-bit counter):

Value	Behavior
0	Aggressive: Gate clock immediately when idle detected (1 cycle delay)
1	Gate after 1 idle cycle
4	Conservative: Gate after 4 consecutive idle cycles
15	Maximum delay: Gate after 15 consecutive

Value	Behavior
	idle cycles

Trade-off: - Low threshold: Maximum power saving, higher gating/ungating frequency - High threshold: Reduced gating overhead, less power saving

Recommendation: Start with moderate value (4-8) and tune based on measured activity patterns.

10.5.5 Gating/Ungating Timing

Gating Sequence (Activity → Idle): 1. Cycle N: user_valid=1, axi_valid=0 → r_wakeup=1 (active) 2. Cycle N+1: user_valid=0, axi_valid=0 → r_wakeup=0 (idle) 3. Idle counter starts incrementing 4. Cycle N+1+cfg_cg_idle_count: Counter threshold reached 5. Clock gates on next cycle edge (gating=1)

Ungating Sequence (Idle → Activity): 1. Clock currently gated (gating=1) 2. Cycle M: user_valid=1 (new activity arrives) 3. Cycle M+1: r_wakeup=1 (registered) 4. Clock ungates immediately (gating=0) 5. Normal operation resumes with no lost cycles

Critical Property: Ungating is immediate (no counter delay) to ensure zero protocol impact.

10.6 Usage Example

10.6.1 Basic Integration with AXI Interface

```
// Instantiate AMBA clock gate controller
amba_clock_gate_ctrl #(
    .CG_IDLE_COUNT_WIDTH      (4) // 0-15 cycle threshold
) u_clock_gate (
    // Input clock (always running)
    .clk_in                    (axi_aclk_raw),
    .aresetn                   (axi_aresetn),

    // Configuration
    .cfg_cg_enable             (power_save_enable), // From system
    .cfg_cg_idle_count         (4'd4),              // Gate after 4
idle cycles

    // Activity monitoring (OR together all valid signals)
    .user_valid                (s_axi_awvalid | s_axi_wvalid |
s_axi_arvalid),
    .axi_valid                  (m_axi_awvalid | m_axi_wvalid |
m_axi_arvalid |
                                m_axi_rvalid | m_axi_bvalid),
```

```

        // Gated clock output
        .clk_out          (axi_aclk_gated),
        .gating           (clock_gated_status),
        .idle             (interface_idle)
    );

    // Use gated clock for all clocked logic
    always_ff @(posedge axi_aclk_gated or negedge axi_aresetn) begin
        if (!axi_aresetn) begin
            // Reset logic
        end else begin
            // Normal operation with gated clock
        end
    end
end

```

```

// Monitor gating status for debug/power measurement
assign power_gating_active = clock_gated_status;
assign bus_idle_indicator = interface_idle;

```

10.6.2 Multi-Master Arbitration with Clock Gating

```

// Individual clock gating per master interface
genvar i;
generate
    for (i = 0; i < NUM_MASTERS; i++) begin : gen_master_cg
        amba_clock_gate_ctrl #(
            .CG_IDLE_COUNT_WIDTH    (4)
        ) u_master_cg (
            .clk_in                  (system_clk),
            .aresetn                 (aresetn),
            .cfg_cg_enable           (cg_enable[i]),
            .cfg_cg_idle_count       (cg_threshold[i]),

            // Monitor master i activity
            .user_valid              (master_valid[i]),
            .axi_valid               (1'b0), // No AXI side for master

            .clk_out                 (master_clk_gated[i]),
            .gating                  (master_gating[i]),
            .idle                    (master_idle[i])
        );
    end
endgenerate

```

```

// Power monitoring: Count active masters
assign num_masters_active = NUM_MASTERS - $countones(master_idle);

10.6.3 Dynamic Threshold Adjustment
// Adjust gating threshold based on system load
logic [3:0] dynamic_threshold;

always_comb begin
    case (system_load_level)
        2'b00: dynamic_threshold = 4'd2;    // Light load: Aggressive
gating
        2'b01: dynamic_threshold = 4'd4;    // Medium load: Moderate
gating
        2'b10: dynamic_threshold = 4'd8;    // High load: Conservative
gating
        2'b11: dynamic_threshold = 4'd15;   // Critical load: Minimal
gating
    endcase
end

amba_clock_gate_ctrl #(
    .CG_IDLE_COUNT_WIDTH    (4)
) u_adaptive_cg (
    .clk_in                  (clk),
    .aresetn                 (rst_n),
    .cfg_cg_enable           (1'b1),
    .cfg_cg_idle_count       (dynamic_threshold), // Runtime
adjustable
    .user_valid              (activity),
    .axi_valid               (1'b0),
    .clk_out                 (clk_gated),
    .gating                  (gating),
    .idle                    (idle)
);

```

10.7 Design Notes

10.7.1 Wakeup Signal Registration

Design Decision: The wakeup signal is registered before passing to clock gate controller.

Rationale: 1. **Metastability Protection:** Activity signals may originate from different clock domains 2. **Stable Input:** Clock gate controller receives glitch-free wakeup signal 3. **Reset Safety:** Defaults to active (1'b1) ensuring clock availability during reset

Trade-off: Adds 1 cycle latency to activity detection, but ensures robust operation.

10.7.2 Activity Signal Selection

Best Practices:

For User Interface (Upstream):

```
// AXI Master Interface
user_valid = awvalid | wvalid | arvalid

// AXI Slave Interface
user_valid = awready | wready | arready | rvalid | bvalid
```

For AXI Interface (Downstream):

```
// AXI Master Interface
axi_valid = awready | wready | arready | rvalid | bvalid

// AXI Slave Interface
axi_valid = awvalid | wvalid | arvalid
```

Why Not Ready Signals? - Valid signals indicate actual transaction progress - Ready signals may assert speculatively - Valid-based detection prevents premature gating

10.7.3 Integration with ICG Cells

The base clock_gate_ctrl module expects integration with standard ICG cell:

Typical ICG Cell Interface:

```
// Standard ICG cell instantiation (inside clock_gate_ctrl)
ICG u_icg (
    .CLK      (clk_in),
    .EN       (gate_enable), // From controller FSM
    .CLK_OUT  (clk_out)
);
```

ASIC Integration: - Use library-specific ICG cell (vendor-provided) - Ensure glitch-free enable signal (provided by controller) - Consider test mode bypass for scan insertion

FPGA Integration: - Use global clock mux primitives (BUFGMUX, BUFGCE) - Some FPGAs have dedicated clock gating resources - Alternative: Fine-grained clock enable on flip-flops

10.7.4 Power Measurement Methodology

Estimating Power Savings:

1. Measure Idle Time:

```

logic [31:0] idle_cycle_count;
always_ff @(posedge clk_in) begin
    if (idle)
        idle_cycle_count <= idle_cycle_count + 1;
end

```

2. Calculate Gating Efficiency:

Gating Efficiency = (gating_active_cycles / total_cycles) × 100%

3. Estimate Power Reduction:

Dynamic Power Saving ≈ Gating Efficiency × Clock Tree Power

Typical Results: - Low-activity interfaces: 60-80% gating efficiency - Burst-heavy traffic: 20-40% gating efficiency - Clock tree power: 20-30% of total dynamic power - Overall savings: 5-20% total chip dynamic power

10.7.5 Performance Considerations

Latency Impact: - No added latency when clock ungated (activity present) - 1 cycle wakeup registration latency - Ungating is immediate (no idle counter delay)

Throughput Impact: - Zero impact: Clock always available for valid transactions - Activity detection ensures ungating before transaction arrival

Area Overhead: - Minimal: Single register + base controller - Base controller: Small FSM + counter - Typical: <100 gates total

10.8 Related Modules

10.8.1 Used By

- AXI interface wrappers requiring power optimization
- AMBA protocol bridges with intermittent activity
- Multi-master interconnect fabrics
- Power-critical peripheral interfaces

10.8.2 Uses

- **clock_gate_ctrl.sv** - Base clock gating controller (idle counter, FSM)
- **reset_defs.svh** - Standard reset macro definitions

10.8.3 See Also

- **clock_gate_ctrl.sv** - Generic clock gating controller (rtl/common/)
- **icg.sv** - Integrated clock gate cell wrapper

10.9 References

10.9.1 Source Code

- RTL: rtl/amba/shared/amba_clock_gate_ctrl.sv
- Base Controller: rtl/common/clock_gate_ctrl.sv
- Tests: val/common/test_clock_gate_ctrl.py

10.9.2 Documentation

- Architecture: docs/markdown/RTLAmba/shared/README.md
- Common Module: docs/markdown/RTLCommon/clock_gate_ctrl.md
- Design Guide: rtl/amba/PRD.md

10.9.3 Industry Standards

- IEEE 1801 (UPF) - Unified Power Format for power-aware design
 - ARM AMBA specifications - Clock gating recommendations
-

Last Updated: 2025-10-24

10.10 Navigation

- [Back to Shared Infrastructure Index](#)
- [Back to RTLAmba Index](#)

11 AXI4 DMA Observer

Modules: - axi4_dma_observer.sv — standalone, DMA-agnostic observability wrapper - axi_bus_meter.sv — per-cycle valid/ready bucket counter (used inside the observer; also instantiable standalone)

Location: rtl/amba/shared/ **Category:** Observability / Characterization **Status:** Production Ready

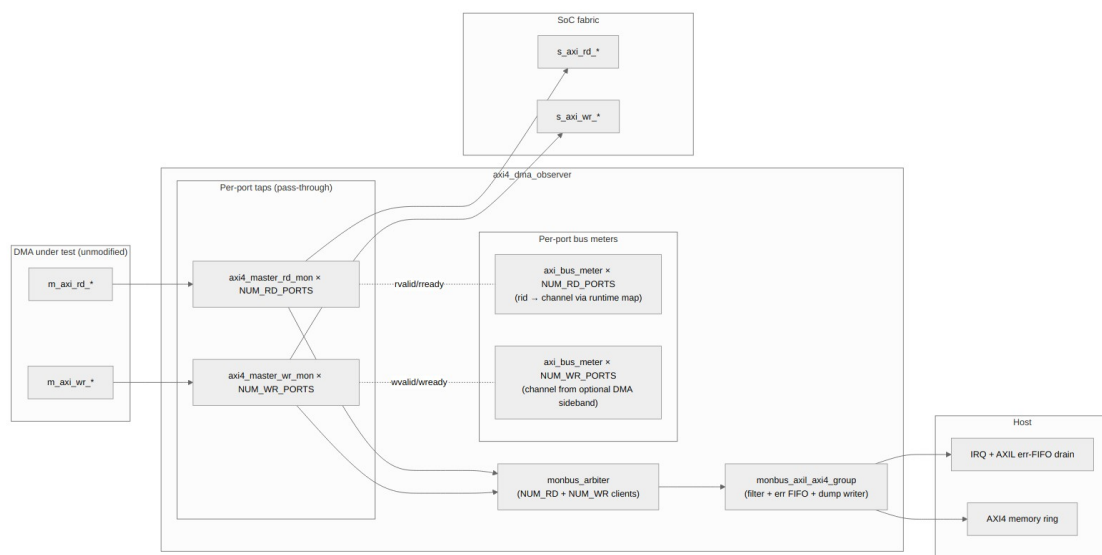
11.1 Overview

axi4_dma_observer is a **drop-in, DMA-agnostic** observability harness that snaps onto any AXI4-master DMA's external pins and produces:

- an **error / interrupt FIFO** drained over an AXI-Lite slave-read port
- a **bulk-trace memory dump** over an AXI4-burst master-write port (raw or compressed via monbus_compressor)
- per-port **valid/ready bucket counters** (productive / backpressure / starvation / idle) via axi_bus_meter, with optional per-channel attribution

The DMA under test is **not modified**. The observer sits inline between the DMA's AXI master pins and the fabric; from the DMA's perspective it's a transparent skid stage. From the fabric's perspective it's the DMA's master port, one register stage later.

This is the **external / non-intrusive** counterpart to the axi_monitor_* family — those wrap a specific DMA's interfaces from inside the DMA. The observer wraps from outside, so the same shape works for STREAM, RAPIDS, third-party DMAs, or anything else with an AXI4 master port.



Architecture

Pass-through latency = the existing _mon wrapper's skid stage. The observer adds **no new combinational depth** in the AXI path.

11.2 What's inside

Sub-module	Role
<code>axi4_master_rd_mon × NUM_RD_PORTS</code>	Read-side tap: pass-through skid + transaction monitor that emits a monbus packet on each event
<code>axi4_master_wr_mon × NUM_WR_PORTS</code>	Write-side tap (same pattern)
<code>monbus_arbiter (CLIENTS = NUM_RD + NUM_WR)</code>	Merges the per-port monbus streams into one
<code>monbus_axil_axi4_group</code>	Central filter + err FIFO (AXIL slave-read) + bulk-trace dump (AXI4 burst master-write) — see <code>monbus_group.md</code>
<code>axi_bus_meter × NUM_RD_PORTS + NUM_WR_PORTS</code>	Per-cycle valid/ready bucket counters (productive / backpressure / starvation / idle) — gated by <code>ENABLE_BUS_METER</code>
<code>axi_perf_latency_hist × NUM_RD_PORTS + NUM_WR_PORTS</code>	Per-transaction latency histograms (read: AR→first-R + AR→RLAST; write: AW→B) into <code>HIST_NUM_BINS</code> log2 bins — gated by <code>ENABLE_LATENCY_HIST</code> (RFC Stage E.3)

11.3 Parameters

Parameter	Default	Notes
<code>NUM_RD_PORTS</code>	1	Number of AXI4 read master ports to tap
<code>NUM_WR_PORTS</code>	1	Number of AXI4 write master ports to tap
<code>ADDR_WIDTH</code>	32	Address width on all tap ports + the observer's own dump port
<code>DATA_WIDTH</code>	128	DMA data-bus width
<code>AXI_ID_WIDTH</code>	8	DMA AXI ID width
<code>AXI_USER_WIDTH</code>	1	DMA AXI USER width
<code>OBS_AXI_ID_WIDTH</code>	4	Observer's own dump-

Parameter	Default	Notes
		port (m_axi_*) AXI ID width
MAX_BURST_BEATS	64	Maximum beats per dump-port AW (AXI4 max is 256)
FIFO_DEPTH_ERR	64	Err FIFO depth (records)
FIFO_DEPTH_WRITE	96	Write FIFO depth (beats)
FLUSH_TIMEOUT_CYCLES	1024	Cycles before timeout-driven flush
USE_COMPRESSION	0	0 omits the compressor; 1 elaborates it for runtime selection via <code>cfg_compress_en</code>
MAX_TRANSACTIONS	16	Per-tap monitor transaction-table depth
UNIT_ID	8'h10	UNIT_ID stamped into emitted monbus packets
ENABLE_BUS_METER	1	0 ties all bus-meter outputs to 0
WR_CH_FROM_AWID	0	1 = derive write per-channel attribution from awid via an internal AW→W order tracker (no <code>dma_wr_active_ch_*</code> sideband needed); 0 = use the sideband. Reads always attribute via <code>rid</code>
NUM_CHANNELS	1	1 = aggregate-only buckets; > 1 = per-channel attribution
ENABLE_LATENCY_HIST	1	0 ties all latency-histogram outputs to 0 (RFC Stage E.3)
HIST_NUM_BINS	16	log2 latency bins: bin b

Parameter	Default	Notes
HIST_MAX_OUTSTANDING	8	counts $[2^b, 2^{(b+1)})$ cycles Per-channel timestamp-FIFO depth in each histogram

Latency-histogram readout (indexed, shared across ports): drive `i_hist_metric` (0 = AR→first-R, 1 = AR→RLAST for reads; ignored for writes) and `i_hist_bin`, then read each port's `{rd,wr}_hist_count` (selected bin) and `{rd,wr}_hist_total` (per-metric transaction count = burst count). Frozen and cleared in lockstep with the meters via `i_meter_freeze / i_meter_clear`.

11.4 Port surface

The observer's port list splits into four groups:

1. **Per-port read taps** (`dma_rd_*` from DMA, `fab_rd_*` to fabric). Sized `[NUM_RD_PORTS-1:0]` on every signal so a single instance can tap multiple AXI4 read masters.
2. **Per-port write taps** (`dma_wr_*` / `fab_wr_*`), shape mirrored.
3. **Observer outputs** (`s_axil_*` for the IRQ-drain port, `m_axi_*` for the bulk-dump port, `irq_out`).
4. **Configuration** (address window, flush watermark, per-protocol filter masks — see `monbus_group.md` for the full filter description).
5. **Bus-meter controls and outputs** (clear/freeze inputs, `rid` → channel map, per-port bucket counter outputs).

The leaf-level `cfg_axi_*_mask` inputs on each `axi4_master_*_mon` are tied to 0 inside the observer — leaves do not pre-filter; all filtering happens centrally in `monbus_axil_axi4_group`. The 25 `cfg_<proto>_*` inputs at the observer top are the single point of control.

11.5 axi_bus_meter integration

Each tapped port instantiates an `axi_bus_meter` that classifies each cycle on the data channel (R for reads, W for writes) into one of four buckets:

valid	ready	bucket	meaning
1	1	productive	beat delivered
1	0	backpressure	master wants to send, slave stalls
0	1	starvation	slave ready, master not producing
0	0	idle	both quiet

Counters: 32-bit aggregate per port (4 buckets), plus 16-bit per-channel $\times$ NUM_CHANNELS $\times$ 4 buckets, with per-channel overflow stickies.

11.5.1 Read-side channel attribution: runtime rid $\rightarrow$ channel map

AXI4 R beats carry rid. For each rd port, the observer takes a runtime signal-list mapping:

```
input cfg_rd_rid_per_channel      [NUM_RD_PORTS][NUM_CHANNELS] //
AXI_ID_WIDTH each
input cfg_rd_rid_per_channel_valid [NUM_RD_PORTS][NUM_CHANNELS] // 1
bit each
```

For channel index N on port P, the user writes the expected rid into `cfg_rd_rid_per_channel[P][N]` and asserts the matching `_valid[P][N]`. When an R beat arrives with `rvalid=1`, the observer priority-encodes the lookup (first match wins) and feeds the result to the bus_meter as `i_channel_id + i_channel_valid`. No match $\rightarrow$ the cycle still counts in aggregate but not in any per-channel bucket.

11.5.2 Write-side channel attribution: optional sideband

AXI4 W beats **carry no AXI ID**. Per-channel attribution requires a sideband from the DMA's W-phase FSM. The observer takes optional inputs:

```
input dma_wr_active_ch_id      [NUM_WR_PORTS][CW] // channel index
input dma_wr_active_ch_valid  [NUM_WR_PORTS]      // sideband valid
```

DMA's that expose this (e.g. STREAM's `axi_write_engine` drives `o_active_channel_id / o_active_channel_valid` from its W-phase FSM) wire directly. DMA's without it: tie both to 0; only the aggregate counters tick.

11.5.3 Outputs

Per port (rd or wr): - `*_meter_agg_productive / backpressure / starvation / idle` — 32-bit aggregate counters - `*_meter_ch_productive / backpressure / starvation / idle` — 16-bit per-channel arrays of size NUM_CHANNELS - `*_meter_ch_overflow` — NUM_CHANNELS $\times$ 4 sticky bits (any per-channel counter wrapped)

The caller exposes these to software either by direct probe or via a custom register block — the observer doesn't include its own register-map decoder.

11.6 Pass-through correctness

For each tapped port pair, the observer's job on the AXI path is to forward every channel signal end-to-end with one register stage of skid latency. The test `val/amba/test_axi4_dma_observer.py` covers this directly in Phases 1 and 2:

- Phase 1 issues a fixed sequence of single-beat reads from the DMA side; the fabric-side responder captures every AR and the DMA-side receiver captures every R. Addresses, IDs, and data round-trip byte-exact.
- Phase 2 mirrors the check for the write path: $AW + W(\text{data} + \text{strb} + \text{last}) + B$ all match end-to-end.

Phase 3 then asserts that monbus dump beats actually appear on the observer's own `m_axi_w*` port after real DMA traffic has been observed (i.e. the filter + arbiter + group + writer chain is actually plumbed correctly).

Phase 4 asserts the bus-meter aggregate counters are non-zero after the smoke-test traffic.

11.7 Test

```
pytest val/amba/test_axi4_dma_observer.py -v
```

Default smoke-test parameters: `NUM_RD_PORTS=1`, `NUM_WR_PORTS=1`, `AXI_ID_WIDTH=4`, `NUM_CHANNELS=8`, `MAX_BURST_BEATS=64`, `USE_COMPRESSION=0`. Bus-meter inputs are exercised (`cfg_rd_rid_per_channel` set up as identity for the first 8 channels; write-side sideband tied to 0 — aggregate-only on the W side).

Multi-port and protocol-variant coverage is future work.

11.8 Related Modules

Module	Role
<code>axi_monitor_base</code>	Per-transaction monitor used inside each <code>axi4_master*_mon</code> tap
<code>axi_monitor_reporter</code>	The dispatcher + 6 packet-type sub-blocks that emit monbus packets

Module	Role
monbus_arbiter	Merges per-tap monbus streams
monbus_group family	Central filter + dump writer (this observer uses the AXIL/AXI4 variant)
monbus_compressor	Bulk-trace encoder (instantiated when USE_COMPRESSION=1)
axi4_master_rd_mon / axi4_master_wr_mon	The per-port skid + monitor wrappers the observer instantiates

12 AXI4 DMA Slaves

Module: axi4_dma_slaves.sv **Location:** rtl/amba/shared/ **Status:** Production Ready

12.1 Overview

axi4_dma_slaves bundles a synthetic AXI4 read *source* and write *sink* into one block that a DMA or streaming engine can plug into for source/sink characterization without a real memory backend. The read side (axi4_slave_rd_pattern_gen) answers AR bursts with LFSR-generated data; the write side (axi4_slave_wr_crc_check) accepts AW/W bursts and CRCs the data. Both compute CRC-32 with the same configuration, so a master that reads the pattern and writes it straight back produces matching read/write CRCs when the datapath is clean.

12.1.1 Key Features

- One-instance source + sink slave pair for DMA / streaming-engine characterization
- Read side: LFSR-driven synthetic data source with per-channel CRC-32
- Write side: CRC-32-checking sink with per-channel accounting
- Shared CRC configuration across both sides — read and write CRCs are directly comparable
- LFSR configuration affects only the read (pattern-generation) side
- Independent read-LFSR and write-CRC resets so either side can be re-armed alone
- Per-channel and aggregate CRC / beat-count telemetry from both sides
- Per-side busy_rd / busy_wr status for harness-level stop triggers

12.2 Module Purpose

Characterizing a DMA needs a memory model on both its read and write ports. Rather than instantiate a real RAM (and its checking logic) twice, this block wraps the two purpose-built synthetic slaves and exposes their combined port surface as a single AXI4 read+write slave. The read side manufactures a deterministic pattern; the write side verifies one. Because the CRC polynomial, init, xorout, and reflection settings are threaded identically into both children, the two CRCs are on the same footing — the master writes back the same LFSR data it read, and both sides compute against the same CRC.

Use Cases: - `stream_char_harness` attaches this to STREAM's `m_axi_rd / m_axi_wr` to characterize DMA throughput, response-delay sweeps, and end-to-end CRC integrity - RAPIDS characterization: source/sink termination for an engine's AXI4 master ports - Any AXI4 master needing a fast, checkable, memory-free source/sink pair

Key Benefit: A drop-in memory replacement that is both non-blocking (never the bottleneck) and self-checking (read and write CRCs must agree), all from a single instantiation.

12.3 Parameters

Parameter	Type	Default	Description
NUM_CHANNELS	int	1	Independent LFSR/CRC contexts on each side (channel = id low bits)
AXI_ID_WIDTH	int	8	AXI ID width
AXI_ADDR_WIDTH	int	32	AXI address width
AXI_DATA_WIDTH	int	64	AXI data width
AXI_USER_WIDTH	int	1	AXI user signal width
SKID_DEPTH_AR	int	2	Read AR skid depth
SKID_DEPTH_R	int	4	Read R skid depth
SKID_DEPTH_AW	int	2	Write AW skid depth
SKID_DEPTH_W	int	4	Write W skid depth
SKID_DEPTH_B	int	2	Write B skid depth
LFSR_WIDTH	int	32	LFSR width (read side only)
LFSR_SEED	logic [31:0]	32'hDEADBEEF	Base LFSR seed (read side only)

Parameter	Type	Default	Description
LFSR_TAPS	logic [47:0]	{12'd32, 12'd22, 12'd2, 12'd1}	Maximal-length Fibonacci taps (read side only)
CRC_WIDTH	int	32	CRC width (shared)
CRC_DATA_WIDTH	int	32	Bits per CRC update (shared)
CRC_POLY	logic [31:0]	32'h04C11DB7	CRC-32/Ethernet polynomial (shared)
CRC_INIT	logic [31:0]	32'hFFFFFFFF	CRC initial value (shared)
CRC_XOROUT	logic [31:0]	32'hFFFFFFFF	CRC final XOR (shared)
CRC_REFIN	int	1	Reflect input (shared)
CRC_REFOUT	int	1	Reflect output (shared)
REPLICATION_FACTOR	int	(AXI_DATA_WIDTH+31)/32	Read-side pattern replication factor
CRC_SLICE_OFFSET	int	0	Write-side 32-bit CRC slice offset
CIW	int	(NUM_CHANNELS LS>1) ? \$clog2(NUM_CHANNELS) : 1	Derived: channel-index width

Note: the CRC parameters are passed unchanged to both children on purpose — that shared configuration is what makes read and write CRCs comparable.

12.4 Port Groups

12.4.1 Clock and Reset

Port	Direction	Width	Description
aclk	input	1	AXI clock
aresetn	input	1	Active-low asynchronous reset

Port	Direction	Width	Description
read_lfsr_r reset	input	1	Re-arm the read-side LFSR + CRC (independent of write side)
write_crc_r reset	input	1	Re-arm the write-side CRC (independent of read side)

12.4.2 Read-Side Telemetry

Port	Direction	Width	Description
read_crc_v alue	output	NUM_CHAN NELS × 32	Per-channel read CRC-32
read_crc_v alid	output	NUM_CHAN NELS	Per-channel read CRC valid
read_beat_ count	output	NUM_CHAN NELS × 32	Per-channel read beats
read_beat_ count_total	output	32	Aggregate read beat count

12.4.3 Write-Side Telemetry

Port	Direction	Width	Description
write_crc_ value	output	NUM_CHAN NELS × 32	Per-channel write CRC-32
write_crc_ valid	output	NUM_CHAN NELS	Per-channel write CRC valid
write_beat_ _count	output	NUM_CHAN NELS × 32	Per-channel write beats
write_beat_ _count_tot al	output	32	Aggregate write beat count

12.4.4 AXI4 Read Slave (AR + R)

Port	Direction	Width	Description
s_axi_arid ...	input	—	AR channel (id, addr, len, size, burst, lock, cache, prot, qos, region, user, valid)
s_axi_arval id			
s_axi_arre ady	output	1	AR ready
s_axi_rid, s_axi_rdata , s_axi_rresp , s_axi_rlast, s_axi_ruser , s_axi_rvali d	output	—	R channel (LFSR pattern data, OKAY resp)
s_axi_rrea dy	input	1	R ready

12.4.5 AXI4 Write Slave (AW + W + B)

Port	Direction	Width	Description
s_axi_awid ...	input	—	AW channel (id, addr, len, size, burst, lock, cache, prot, qos, region, user, valid)
s_axi_awval lid			
s_axi_awre ady	output	1	AW ready
s_axi_wdat a, s_axi_wstr b, s_axi_wlast , s_axi_wuse	input	—	W channel

Port	Direction	Width	Description
r, s_axi_wvalid			
s_axi_wready	output	1	W ready
s_axi_bid, s_axi_bresp	output	—	B channel (OKAY resp)
p, s_axi_buser			
r, s_axi_bvalid			
s_axi_bready	input	1	B ready

12.4.6 Status

Port	Direction	Width	Description
busy_rd	output	1	Read-side busy (from the read pattern generator)
busy_wr	output	1	Write-side busy (from the write CRC checker)

12.5 Functional Description

12.5.1 Structural Composition

The module is a thin structural wrapper: it instantiates one `axi4_slave_rd_pattern_gen` (`u_rd_pattern_gen`) and one `axi4_slave_wr_crc_check` (`u_wr_crc_check`) and fans the top-level ports out to them. There is no additional logic — all behavior lives in the two children, documented separately.

12.5.2 Read Side (LFSR Source)

`u_rd_pattern_gen` handles the AR/R channels. It returns per-channel LFSR-generated data (32-bit LFSR replicated to the bus width) and accumulates a per-channel CRC-32 over the emitted stream. Its `crc_lfsr_reset` is driven from the top-level `read_lfsr_reset`. LFSR configuration (`LFSR_WIDTH`, `LFSR_SEED`, `LFSR_TAPS`, `REPLICATION_FACTOR`) reaches only this side.

12.5.3 Write Side (CRC Sink)

`u_wr_crc_check` handles the AW/W/B channels. It CRCs the selected 32-bit slice of each accepted W beat per channel and returns OKAY B responses. Its `crc_reset` is driven from the top-level `write_crc_reset`, and `CRC_SLICE_OFFSET` selects the CRC'd lane.

12.5.4 Shared CRC vs Independent Resets

The CRC configuration (`CRC_POLY`, `CRC_INIT`, `CRC_XOROUT`, `CRC_REFIN`, `CRC_REFOUT`, widths) is threaded identically into both children so the read and write CRCs compute over the same algorithm — the intended usage is that the master writes back the LFSR data it read, and the two CRCs must then agree. The two reset inputs are kept separate so the harness can re-arm one side without disturbing the other; in practice both are usually driven from the same CSR clear pulse.

12.6 Usage Example

```
// Source/sink pair on a DMA's read and write master ports.
axi4_dma_slaves #(
    .NUM_CHANNELS    (4),
    .AXI_ID_WIDTH    (8),
    .AXI_ADDR_WIDTH  (32),
    .AXI_DATA_WIDTH  (64)
) u_dma_slaves (
    .aclk             (aclk),
    .aresetn          (aresetn),
    .read_lfsr_reset  (csr_clear_pulse),
    .write_crc_reset  (csr_clear_pulse),

    // Read telemetry
    .read_crc_value   (rd_crc),    .read_crc_valid
(rd_crc_valid),
    .read_beat_count   (rd_beats), .read_beat_count_total
(rd_beats_total),
    // Write telemetry
    .write_crc_value   (wr_crc),    .write_crc_valid
(wr_crc_valid),
    .write_beat_count
(wr_beats), .write_beat_count_total(wr_beats_total),

    // Read slave <- DUT m_axi_rd
    .s_axi_arid        (m_rd_arid), /* ... AR ...
*/ .s_axi_arready(m_rd_arready),
    .s_axi_rid         (m_rd_rid),  /* ... R ... */ .s_axi_rready
(m_rd_rready),
```

```

    // Write slave <- DUT m_axi_wr
    .s_axi_awid (m_wr_awid), /* ... AW ...
*/ .s_axi_awready(m_wr_awready),
    .s_axi_wdata(m_wr_wdata), /* ... W ... */ .s_axi_wready
(m_wr_wready),
    .s_axi_bid (m_wr_bid), /* ... B ... */ .s_axi_bready
(m_wr_bready),

    .busy_rd (rd_busy),
    .busy_wr (wr_busy)
);

// End-to-end integrity: after a loopback run, compare per channel.
assign integrity_ok = (rd_crc[ch] == wr_crc[ch]);

```

12.7 Design Notes

- **Pure composition:** the block adds no logic of its own — it exists to give the harness one instance and one port list instead of two, and to enforce the shared-CRC contract at the parameter level.
 - **Shared CRC config is intentional:** identical CRC parameters on both children are what let a loopback test compare read vs write CRCs directly.
 - **LFSR is read-only:** the write side has no pattern generator; it only checks. LFSR parameters therefore reach only the read child.
 - **Independent re-arm:** separate read_lfsr_reset / write_crc_reset allow re-seeding one side mid-experiment; typical harnesses drive both from one CSR pulse.
 - **No memory backend:** addresses are ignored for data — the read side derives data from the LFSR phase (or, in the master-side blocks, from an address hash), not from stored contents.
-

12.8 Related Modules

12.8.1 Used By

- projects/NexysA7/stream_characterization/flows-stream-bridge/rtl/stream_char_harness.sv — attaches to STREAM's m_axi_rd / m_axi_wr
- projects/NexysA7/rapids_characterization/flows-rapids-beats/rtl/rapids_char_harness.sv — source/sink termination

12.8.2 Uses

- **axi4_slave_rd_pattern_gen.sv** — the read-side LFSR data source + CRC
- **axi4_slave_wr_crc_check.sv** — the write-side CRC-checking sink

12.8.3 See Also

- **axi4_master_wr_pattern_gen.sv** / **axi4_master_rd_crc_check.sv** — the master-side counterparts (drive traffic instead of terminating it)
 - **axi4_dma_observer.sv** — non-intrusive observability wrapper for a DMA's master ports
-

12.9 References

12.9.1 Source Code

- RTL: rtl/amba/shared/axi4_dma_slaves.sv

12.9.2 Documentation

- Architecture: docs/markdown/RTLAmba/shared/README.md
 - Index: docs/markdown/RTLAmba/index.md
 - Read child:
docs/markdown/RTLAmba/shared/axi4_slave_rd_pattern_gen.md
 - Write child:
docs/markdown/RTLAmba/shared/axi4_slave_wr_crc_check.md
-

Last Updated: 2026-07-15

12.10 Navigation

- [Back to Shared Infrastructure Index](#)
- [Back to RTLAmba Index](#)

13 AXI4 Master Read CRC Checker

Module: axi4_master_rd_crc_check.sv **Location:** rtl/amba/shared/ **Status:** Production Ready

13.1 Overview

`axi4_master_rd_crc_check` is a CSR-programmed AXI4 read *master* and integrity checker for memory-controller characterization. It walks the *same* algorithmic address mix (via `dma_address_gen`) and the *same* LFSR / hash schedule as `axi4_master_wr_pattern_gen`, so each returned R beat can be compared bit-for-bit against a locally regenerated pattern. It also accumulates a CRC-32 over the regenerated stream, so the harness can compare `o_actual_crc` against the writer's `o_expected_crc`.

13.1.1 Key Features

- CSR-programmed read workload with the *same descriptor shape* as the write pattern generator (one CSR word drives both)
 - Fully decoupled AR and R paths — two independent `dma_address_gen` instances walk the same descriptor in parallel
 - Two expected-data sources: 32-bit Fibonacci LFSR (phase counter) or address-derived Murmur3-fmix hash (out-of-order-safe)
 - Pipelined compare (two isolated 32-bit multiplies) so the mode-1 hash closes 100 MHz; returned data rides the same stages to align the compare
 - Three AR-ID generation modes: FIXED, COUNTER, LFSR
 - Sticky `o_data_error` on any per-beat data mismatch, mismatch counter, and sticky `o_rresp_error` on non-OKAY R beats
 - CRC-32 over the regenerated LFSR stream (`o_actual_crc`) comparable to the writer's expected CRC
 - Optional debug FIFO capturing (`actual`, `expected`, `mismatch`) per R beat for ground-truth disagreement logging
-

13.2 Module Purpose

The read half of a memory-controller integrity loop must know what it *should* receive. This block regenerates the writer's exact data locally — either from the same LFSR phase counter or from the same address hash — and compares every returned R beat against it, latching any mismatch. It also rolls the regenerated data into a CRC-32 that should equal the writer's expected CRC when the wire is clean. Together these give both a per-beat pinpoint (which beat disagreed) and a whole-run summary (CRC compare).

Use Cases: - Reading back a DDR / memory controller and verifying the data written by `axi4_master_wr_pattern_gen` - Address-pattern integrity sweeps sharing one descriptor with the write generator - Multi-id / out-of-order read validation using hash (address-derived) expected data - On-chip (FPGA) read checker in the DDR2 characterization harness

Key Benefit: End-to-end integrity in one block — per-beat compare localizes corruption while the CRC-32 gives a single comparable summary, both derived from exactly the writer’s algorithm.

13.3 Parameters

Parameter	Type	Default	Description
SKID_DEPTH_AR	int	2	AR skid depth
SKID_DEPTH_R	int	4	R skid depth
AXI_ID_WIDTH	int	8	AXI ID width
AXI_ADDR_WIDTH	int	32	AXI address width
AXI_DATA_WIDTH	int	64	AXI data width
AXI_USER_WIDTH	int	1	AXI user width
LFSR_WIDTH	int	32	LFSR width (must match writer)
LFSR_SEED	logic [31:0]	32’hDEADBEEF	LFSR seed default (must match writer)
LFSR_TAPS	logic [47:0]	{12’d32, 12’d22, 12’d2, 12’d1}	Maximal-length Fibonacci taps
CRC_WIDTH	int	32	CRC width (must match writer)
CRC_DATA_WIDTH	int	32	Bits per CRC update
CRC_POLY	logic [CRC_WIDTH-1:0]	32’h04C11DB7	CRC polynomial
CRC_POLY_INIT	logic [CRC_WIDTH-1:0]	’1	CRC initial value
CRC_XOROUT	logic [CRC_WIDTH-1:0]	’1	CRC final XOR
TXN_COUNT_WIDTH	int	16	Width of cfg_txn_count
INDEX_WIDTH	int	16	dma_address_gen index width

Parameter	Type	Default	Description
STRIDE_WIDTH	int	24	dma_address_gen signed stride width
DBG_FIFO_DEPTH	int	0	>0 elaborates a debug FIFO capturing per-beat records; 0 ties dbg_* off
IW / AW / DW / UW	int	—	Aliases for id/addr/data/user widths

Note: LFSR + CRC parameters must match axi4_master_wr_pattern_gen or the compare and CRC roll-up are meaningless. Internally $REPLICATION_FACTOR = (DW+31)/32$ and $HSTAGES = 4$ compare-pipeline stages.

13.4 Port Groups

13.4.1 Clock and Reset

Port	Direction	Width	Description
aclk	input	1	AXI clock
aresetn	input	1	Active-low asynchronous reset

13.4.2 Configuration (CSR) — same shape as the write generator

Port	Direction	Width	Description
cfg_start_a ddr	input	AW	Base address for dma_address_gen
cfg_addr_st ride_0	input	signed STRIDE_WID TH	Address stride, dimension 0
cfg_addr_st ride_1	input	signed STRIDE_WID TH	Address stride, dimension 1
cfg_addr_ wrap_mas k_0	input	AW	Address wrap mask, dimension 0

Port	Direction	Width	Description
cfg_addr_wrap_mask_1	input	AW	Address wrap mask, dimension 1
cfg_burst_length	input	8	Beats per burst (1..256); arlen = len - 1
cfg_txn_count	input	TXN_COUNT_WIDTH	Total bursts to issue
cfg_axi_id	input	IW	FIXED-mode id / seed for COUNTER+LFSR modes
cfg_id_mode	input	2	AR-ID mode: 0=FIXED, 1=COUNTER, 2=LFSR
cfg_axi_size	input	3	arsize
cfg_axi_burst	input	2	arburst
cfg_lfsr_seed	input	LFSR_WIDTH	Seed override (0 → use param)
cfg_data_mode	input	1	0 = phase-counter LFSR; 1 = address hash (OOO-safe)
cfg_hash_seed0/1/2	input	32 each	Murmur3-fmix mixer seeds
cfg_rd_gap	input	4	Inter-burst idle cycles (0..15), independent of the writer's gap
cfg_start	input	1	Pulse to begin the workload
cfg_done	output	1	High when all bursts complete and the compare pipeline drains

13.4.3 Telemetry

Port	Direction	Width	Description
o_actual_crc	output	CRC_WIDTH	Running CRC-32 over the regenerated LFSR stream

Port	Direction	Width	Description
o_actual_crc_valid	output	1	High with cfg_done (LFSR mode)
o_data_error	output	1	Sticky on any per-beat data mismatch
o_rresp_error	output	1	Sticky on any non-OKAY R beat
o_beats_mismatched	output	TXN_COUNT_WIDTH	Count of mismatching R beats

13.4.4 M-Side AXI4 Read (out to fabric/controller)

Port	Direction	Width	Description
m_axi_arid ...	output	—	AR channel (id, addr, len, size, burst, lock, cache, prot, qos, region, user, valid)
m_axi_arvalid	input	1	AR ready
m_axi_arready	input	—	R channel
m_axi_rid, m_axi_rdata, m_axi_rresp, m_axi_rlast, m_axi_ruser, m_axi_rvalid	output	1	R ready

13.4.5 Debug Observability (active when DBG_FIFO_DEPTH > 0)

Port	Direction	Width	Description
dbg_valid	output	1	A captured record is

Port	Direction	Width	Description
			available
dbg_ready	input	1	Bench pops a record
dbg_actual	output	DW	Returned data for this beat
dbg_expected	output	DW	Locally regenerated expected data
dbg_mismatch	output	1	This beat disagreed

13.5 Functional Description

13.5.1 Workload FSM

A four-state FSM (S_IDLE, S_RUN, S_GAP, S_DONE) mirrors the write generator. `cfg_start` latches the CSR program and enters S_RUN (or S_DONE if `cfg_txn_count == 0`). S_RUN runs the AR and R paths concurrently; S_GAP inserts `cfg_rd_gap` idle cycles between bursts (pausing both AR and R together); S_DONE holds until re-armed. Because the config shape is identical to the writer, one harness CSR word drives both blocks.

13.5.2 Decoupled AR and R Address Generation

Two independent `dma_address_gen` instances (`u_addr_gen_ar`, `u_addr_gen_r`) walk the same descriptor. The AR path issues one AR per index as fast as `arready` and the address pipeline allow; `arvalid` stays asserted from its first cycle to the last AR handshake at `cfg_rd_gap = 0`. The R path produces the current burst's base address (popped on `rlast`) so the hash-mode expected-data regen has the right anchor for the following beat. R beats are absorbed only after the AR for that burst is on the wire and the R address generator has produced the base.

13.5.3 Expected Data: LFSR vs Address Hash

Muxed by `cfg_data_mode`:

- **Mode 0 (LFSR):** a 32-bit Fibonacci LFSR advances on every accepted R beat — the same logic as the writer, so with the same total beat count the two streams stay phase-aligned and match bit-for-bit. Replicated across the bus for the compare.

- **Mode 1 (address hash):** each 32-bit slice is a Murmur3-fmix function of the beat's byte address and the three hash seeds — a pure function of the address, so out-of-order returns still validate.

13.5.4 Pipelined Compare

The mode-1 hash chains two 32-bit multiplies — the same ~25 ns / 4-DSP cone as the writer, too slow to feed the per-beat compare combinationally at 100 MHz. Each multiply is isolated in its own register stage (HSTAGES = 4); the returned `rdata` rides through the same stages so the compare happens at the pipeline output, aligned with the delayed expected value. Beat/burst accounting stays keyed on the R-beat handshake — only the data compare is delayed. A per-beat mismatch at the pipeline output sets `o_data_error` and increments `o_beats_mismatched`; `rresp` is checked immediately at the R beat (no hash dependency) into `o_rresp_error`. `cfg_done` waits for the compare pipeline to drain (! (| `r_cp_valid`)) so no trailing mismatch is missed.

13.5.5 CRC Accumulation

A `dataint_crc` accumulates over the regenerated LFSR stream (not the returned `rdata`) — matching the writer's accounting so `o_actual_crc` equals the writer's `o_expected_crc` on a clean wire. `o_actual_crc_valid` sets with `cfg_done` in LFSR mode; hash mode relies on the per-beat compare (`o_data_error`).

13.5.6 AR-ID Generation

`arid` is muxed by `cfg_id_mode` exactly like the writer's `awid`: FIXED, an 8-bit COUNTER, or an 8-bit Fibonacci LFSR (taps {8,6,5,4}, seeded `cfg_axi_id[7:0] | 1`).

13.5.7 Out-of-Order Completion — Known Limitation

The v1 LFSR mirror advances on *arrival* index, so it is only correct with a single outstanding AR, or when all ARs share one ID (AXI4 mandates in-order R per id). With multiple outstanding ARs at distinct IDs the controller may return bursts interleaved / fully OOO, which reorders R beats versus the writer's W phase and breaks both the per-beat compare and the CRC roll-up. The header documents the v2 plan (per-address hash expected function + commutative CRC roll-up). Until then, the harness CSR must keep the read block single-outstanding (or all-same-id) when the controller has OOO enabled — or use hash data mode, which is inherently OOO-safe for the per-beat compare.

13.5.8 Optional Debug FIFO

When `DBG_FIFO_DEPTH > 0` a `gaxi_fifo_sync` captures (`actual`, `expected`, `mismatch`) at each compare-pipeline output so the bench can walk the exact disagreements instead of inferring from `o_data_error`. When depth is 0 the generate else arm ties the `dbg_*` outputs off and the FIFO is not built.

13.5.9 Standard Protocol Handler

AR/R skid buffering and AXI compliance are delegated to an `axi4_master_rd` instance; the FSM/LFSR/hash/compare logic drives its FUB side and the `m_axi_*` ports pass straight out to the fabric.

13.6 Usage Example

```
// Read checker paired with the write generator on a DDR2 sweep.
axi4_master_rd_crc_check #(
    .AXI_ID_WIDTH      (8),
    .AXI_ADDR_WIDTH    (32),
    .AXI_DATA_WIDTH    (64),
    .DBG_FIFO_DEPTH    (0)
) u_rd_chk (
    .aclk      (aclk),
    .aresetn   (aresetn),

    // SAME descriptor word as the write generator
    .cfg_start_addr      (csr_base_addr),
    .cfg_addr_stride_0    (csr_stride_0),
    .cfg_addr_stride_1    (24'sd0),
    .cfg_addr_wrap_mask_0 (csr_wrap_0),
    .cfg_addr_wrap_mask_1 ('0),
    .cfg_burst_len        (8'd16),
    .cfg_txn_count        (16'd1024),
    .cfg_axi_id           (8'd0),
    .cfg_id_mode          (2'd0),          // FIXED / single-outstanding-
safe
    .cfg_axi_size         (3'd3),
    .cfg_axi_burst        (2'd1),
    .cfg_lfsr_seed        (32'd0),
    .cfg_data_mode        (1'b0),          // LFSR (CRC comparable to
writer)
    .cfg_hash_seed0       (32'd0),
    .cfg_hash_seed1       (32'd0),
    .cfg_hash_seed2       (32'd0),
    .cfg_rd_gap           (4'd0),
    .cfg_start            (csr_start_pulse),
    .cfg_done             (rd_done),

    .o_actual_crc         (rd_actual_crc),
    .o_actual_crc_valid   (rd_actual_crc_valid),
    .o_data_error         (rd_data_error),
    .o_rresp_error        (rd_rresp_error),
    .o_beats_mismatched   (rd_beats_bad),
```



```

// M-side AXI from the controller
.m_axi_arid (arid), /* ... AR ... */ .m_axi_arready(arready),
.m_axi_rid  (rid),  /* ... R ... */ .m_axi_rready (rready),

// Debug FIFO tied off (DBG_FIFO_DEPTH == 0)
.dbg_valid (), .dbg_ready (1'b0),
.dbg_actual(), .dbg_expected(), .dbg_mismatch()
);

// Clean-wire check: CRCs agree and no per-beat mismatch.
assign integrity_ok = (rd_actual_crc == wr_expected_crc) && !
rd_data_error;

```

13.7 Design Notes

- **Mirror the writer exactly:** LFSR seed schedule, CRC config, address descriptor, and ID modes are all mirror images of `axi4_master_wr_pattern_gen` — that symmetry is what makes the CRCs and per-beat expected values comparable.
 - **Per-beat compare + CRC are complementary:** the compare localizes the failing beat; the CRC gives a single-register whole-run summary. Both are provided so a failure can be both detected and pinpointed.
 - **Compare pipeline aligns delayed expected with delayed rdata:** the returned data is intentionally delayed the same HSTAGES as the hash so the compare is apples-to-apples, and `cfg_done` waits for the pipeline to drain.
 - **OOO is a real v1 limitation:** with multi-id OOO completion, use hash data mode (`cfg_data_mode = 1`) or hold the read block single-outstanding — the header spells out the v2 per-address-hash fix.
 - **Debug FIFO is opt-in:** `DBG_FIFO_DEPTH > 0` adds ground-truth per-beat capture at essentially no cost when disabled.
-

13.8 Related Modules

13.8.1 Used By

- `projects/NexysA7/ddr2-characterization/flows-ours-uart/rtl/ddr2_char_harness.sv` — on-chip read checker

- DDR2 characterization macro / harness CSR blocks under `projects/NexysA7/ddr2-characterization/`

13.8.2 Uses

- **`axi4_master_rd.sv`** — standard AXI4 read master protocol handler (AR/R skid + compliance)
- **`dma_address_gen.sv`** — algorithmic address sequence generator (×2, decoupled AR/R)
- **`shifter_lfsr_fibonacci.sv`** — expected-data LFSR and 8-bit AR-ID LFSR
- **`dataint_crc.sv`** — CRC-32 accumulator
- **`gaxi_fifo_sync.sv`** — optional debug capture FIFO

13.8.3 See Also

- **`axi4_master_wr_pattern_gen.sv`** — the matching write-side driver (same LFSR/CRC/hash/descriptor config)
 - **`axi4_slave_rd_pattern_gen.sv`** — the slave-side read pattern source
-

13.9 References

13.9.1 Source Code

- RTL: `rtl/amba/shared/axi4_master_rd_crc_check.sv`
- Protocol Handler: `rtl/amba/axi4/axi4_master_rd.sv`

13.9.2 Documentation

- Architecture: `docs/markdown/RTLAmba/shared/README.md`
 - Index: `docs/markdown/RTLAmba/index.md`
 - Harness: `projects/NexysA7/ddr2-characterization/README.md`
-

Last Updated: 2026-07-15

13.10 Navigation

- [Back to Shared Infrastructure Index](#)
- [Back to RTLAmba Index](#)

14 AXI4 Master Write Pattern Generator

Module: `axi4_master_wr_pattern_gen.sv` **Location:** `rtl/amba/shared/` **Status:** Production Ready

14.1 Overview

`axi4_master_wr_pattern_gen` is a CSR-programmed AXI4 write *master* for memory-controller characterization. On a start pulse it walks an algorithmic address mix (via `dma_address_gen`) and streams LFSR-pattern data through `axi4_master_wr`, accumulating a CRC-32 over the data it writes. It pairs with `axi4_master_rd_crc_check`, which regenerates the same pattern on the read side so the two CRCs (and per-beat compares) validate end-to-end data integrity through a real DRAM controller.

14.1.1 Key Features

- CSR-programmed write workload: start address, address-generator strides/wrap masks, burst length, transaction count, AXI id/size/burst attributes
 - Fully decoupled AW and W paths — two independent `dma_address_gen` instances walk the same descriptor in parallel
 - Two data sources: a 32-bit Fibonacci LFSR (phase-counter) or an address-derived Murmur3-fmix hash (out-of-order-safe)
 - Pipelined hash datapath (two isolated 32-bit multiplies) to close 100 MHz, decoupled from the AXI W handshake by a staging FIFO
 - Three AW-ID generation modes: FIXED, COUNTER, LFSR
 - CRC-32 accumulator over the written LFSR stream (`o_expected_crc`) for the read side to compare against
 - Configurable inter-burst idle gap (`cfg_wr_gap`) for throughput-stress sweeps
 - Sticky `o_bresp_error` on any non-OKAY write response; direct re-arm from the done state
-

14.2 Module Purpose

Bringing up and characterizing a memory controller means driving it with realistic, deterministic write traffic and proving the data survives the round trip. This block drives the writes: it walks a programmable address pattern and emits reproducible data, folding that data into a CRC-32 that

becomes the “expected” value for the read-side checker. The write data can be a simple LFSR phase counter (fast, but order-sensitive) or an address-derived hash (each beat’s data is a pure function of its byte address, so multi-id / out-of-order completion still validates).

Use Cases: - Driving a DDR / memory-controller’s AXI4 write port during characterization sweeps
 - Generating the “golden” CRC / data that axi4_master_rd_crc_check compares against - Address-pattern stress (incremental, row-major, column-major page attacks) via the dma_address_gen descriptor - On-chip (FPGA) write stimulus in the DDR2 characterization harness

Key Benefit: A CSR-driven, deterministic, self-documenting write generator whose data can be regenerated bit-for-bit anywhere — turning a memory-controller bring-up into a single expected-vs-actual CRC comparison.

14.3 Parameters

Parameter	Type	Default	Description
SKID_DEPTH_AW	int	2	AW skid depth
SKID_DEPTH_W	int	4	W skid depth
SKID_DEPTH_B	int	2	B skid depth
AXI_ID_WIDTH	int	8	AXI ID width
AXI_ADDR_WIDTH	int	32	AXI address width
AXI_DATA_WIDTH	int	64	AXI data width
AXI_USER_WIDTH	int	1	AXI user width
AXI_WSTRB_WIDTH	int	AXI_DATA_WIDTH/8	Write strobe width (derived)
LFSR_WIDTH	int	32	LFSR width; matches slave side so CRCs interchange
LFSR_SEED	logic [31:0]	32’hDEADBEEF	LFSR seed default
LFSR_TAPS	logic [47:0]	{12’d32, 12’d22, 12’d2, 12’d1}	Maximal-length Fibonacci taps
CRC_WIDTH	int	32	CRC width
CRC_DATA_WIDTH	int	32	Bits per CRC update (fixed 32 to match slave side)

Parameter	Type	Default	Description
CRC_POLY	logic [CRC_WID TH-1:0]	32'h04C11DB7	CRC polynomial
CRC_POLY_INIT	logic [CRC_WID TH-1:0]	'1	CRC initial value
CRC_XOROUT	logic [CRC_WID TH-1:0]	'1	CRC final XOR
TXN_COUNT_WIDTH	int	16	Width of cfg_txn_count (up to 64K bursts)
INDEX_WIDTH	int	16	dma_address_gen index width
STRIDE_WIDTH	int	24	dma_address_gen signed stride width
IW / AW / DW / UW / SW	int	—	Aliases for id/addr/data/user/strobe widths

Note: LFSR + CRC parameters mirror axi4_master_rd_crc_check exactly so the two blocks' CRC values are interchangeable. Internally $REP = (DW+31)/32$ 32-bit slices per beat and $HSTAGES = 4$ hash-pipeline stages.

14.4 Port Groups

14.4.1 Clock and Reset

Port	Direction	Width	Description
aclk	input	1	AXI clock
aresetn	input	1	Active-low asynchronous reset

14.4.2 Configuration (CSR)

Port	Direction	Width	Description
cfg_start_addr	input	AW	Base address handed to dma_address_gen
cfg_addr_stride_0	input	signed STRIDE_WIDTH	Address stride, dimension 0
cfg_addr_stride_1	input	signed STRIDE_WIDTH	Address stride, dimension 1 (held at index 0 here)
cfg_addr_wrap_mask_0	input	AW	Address wrap mask, dimension 0
cfg_addr_wrap_mask_1	input	AW	Address wrap mask, dimension 1
cfg_burst_length	input	8	Beats per burst (1..256); awlen = len - 1
cfg_txn_count	input	TXN_COUNT_WIDTH	Total bursts to issue
cfg_axi_id	input	IW	FIXED-mode id / seed for COUNTER+LFSR modes
cfg_id_mode	input	2	AW-ID mode: 0=FIXED, 1=COUNTER, 2=LFSR
cfg_axi_size	input	3	awsize
cfg_axi_burst	input	2	awburst (INCR=1)
cfg_lfsr_seed	input	LFSR_WIDTH	Seed override (0 → use LFSR_SEED param)
cfg_data_mode	input	1	0 = phase-counter LFSR; 1 = address-derived hash (OOO-safe)
cfg_hash_s	input	32 each	Murmur3-fmix mixer

Port	Direction	Width	Description
eed0/1/2			seeds (hash mode)
cfg_wr_gap	input	4	Inter-burst idle cycles (0..15)
cfg_start	input	1	Pulse to begin the workload
cfg_done	output	1	High once all B responses received

14.4.3 Telemetry

Port	Direction	Width	Description
o_expected_crc	output	CRC_WIDTH	Running CRC-32 over the written LFSR stream
o_expected_crc_valid	output	1	High with cfg_done (LFSR mode only)
o_bresp_err	output	1	Sticky on any non-OKAY BRESP

14.4.4 M-Side AXI4 Write (out to fabric/controller)

Port	Direction	Width	Description
m_axi_awi d ... m_axi_awv alid	output	—	AW channel (id, addr, len, size, burst, lock, cache, prot, qos, region, user, valid)
m_axi_awr eady	input	1	AW ready
m_axi_wda ta, m_axi_wst rb, m_axi_wla st, m_axi_wus er, m_axi_wva lid	output	—	W channel (strokes tied full-beat)

Port	Direction	Width	Description
m_axi_wready	input	1	W ready
m_axi_bid, m_axi_bresp, m_axi_buser, m_axi_bvalid	input	—	B channel
m_axi_bready	output	1	B ready

14.5 Functional Description

14.5.1 Workload FSM

A four-state FSM (S_IDLE, S_RUN, S_GAP, S_DONE) sequences a run. On `cfg_start` the CSR program is latched (so software can change the `cfg_*` inputs on the next cycle without disturbing the run) and the block enters S_RUN (or S_DONE immediately if `cfg_txn_count == 0`). In S_RUN the AW and W paths run concurrently; S_GAP inserts `cfg_wr_gap` idle cycles between bursts; S_DONE awaits the final B responses. S_DONE also accepts a fresh `cfg_start` for a direct re-arm without passing back through S_IDLE.

14.5.2 Decoupled AW and W Address Generation

Two independent `dma_address_gen` instances (`u_addr_gen_aw`, `u_addr_gen_w`) walk the *same* descriptor with the same indices, so both produce the identical address sequence. The AW path issues one AW per index at `awready` rate; the W path produces the current burst's base address for the hash datapath, popped on WLAST. `awvalid` stays continuously asserted from its first cycle to the last AW handshake when `cfg_wr_gap = 0` — the address generator produces one result per cycle once warmed. Outstanding depth is bounded by the slave's `awready` throttling, not by this block. Only `index_0` is walked (`index_1` held at 0); a second instance with a different `cfg_addr_stride_1` would exercise the 2D path.

14.5.3 Data Path: LFSR vs Address Hash

Two data sources are muxed by `cfg_data_mode`:

- **Mode 0 (LFSR):** a 32-bit Fibonacci LFSR (`shifter_lfsr_fibonacci`, taps {32, 22, 2, 1}) advances on every accepted W beat, replicated across the

data bus (REP copies). The data stream is a deterministic function of (seed, total_beats_so_far). This is order-sensitive and breaks under multi-id / out-of-order completion.

- **Mode 1 (address hash):** each 32-bit slice is a Murmur3-fmix-style function of its byte address and the three cfg_hash_seed values (xor-shift + odd multiplies). Because each beat's data depends only on its address, reorder does not perturb the per-beat compare on the read side.

The per-beat byte address is anchored on the W address-generator output and stepped by 2**awsz bytes per beat. Full-beat writes only — wstrb is tied all-ones.

14.5.4 Hash Pipeline and Staging FIFO

The mode-1 hash chains two 32-bit multiplies — combinationally a ~25 ns / 4-DSP cone that misses 100 MHz. Each multiply is isolated in its own register stage (HSTAGES = 4), with the WLAST/mode/LFSR-data riding alongside so the output stays beat-aligned. The pipeline is latency-insensitive: per-beat data values and order are unchanged, so the CRC / per-beat-compare contract still holds. A show-ahead staging FIFO (gaxi_fifo_sync, depth 16) between the pipeline output and the AXI W handshake lets W stream back-to-back independent of pipeline fill or master backpressure; a beat is admitted only when the generator has one and the FIFO has room for it plus the HSTAGES beats already in flight.

14.5.5 AW-ID Generation

awid is muxed by cfg_id_mode: FIXED passes cfg_axi_id through; COUNTER is an 8-bit counter seeded at cfg_axi_id[7:0], +1 per AW; LFSR is an 8-bit maximal-length Fibonacci LFSR (taps {8,6,5,4}) seeded with cfg_axi_id[7:0] | 1 to avoid the all-zero seed. Internal counter/LFSR are 8-bit and zero-extended/truncated to IW.

14.5.6 CRC Accumulation and Completion

A dataint_crc instance accumulates over the same LFSR stream sent on W (not over the hash), latched into o_expected_crc at the last W beat of the last burst — meaningful only in LFSR mode, where o_expected_crc_valid sets. In hash mode the CRC is not load-bearing and validity is gated low; the harness uses the read side's per-beat compare instead. B responses are counted independently of FSM phase (except never accepted pre-start); o_bresp_error sticks on any non-OKAY BRESP. cfg_done asserts once in S_DONE with all cfg_txn_count B's received.

14.5.7 Standard Protocol Handler

AW/W/B skid buffering and AXI compliance are delegated to an axi4_master_wr instance; the FSM/LFSR/hash logic drives its FUB side and the m_axi_* ports pass straight out to the fabric.

14.6 Usage Example

```
// Write driver for a DDR2 characterization sweep (LFSR data mode).
axi4_master_wr_pattern_gen #(
    .AXI_ID_WIDTH    (8),
    .AXI_ADDR_WIDTH  (32),
    .AXI_DATA_WIDTH  (64)
) u_wr_gen (
    .aclk      (aclk),
    .aresetn   (aresetn),

    // CSR program (from the harness register block)
    .cfg_start_addr      (csr_base_addr),
    .cfg_addr_stride_0    (csr_stride_0),
    .cfg_addr_stride_1    (24'sd0),
    .cfg_addr_wrap_mask_0 (csr_wrap_0),
    .cfg_addr_wrap_mask_1 ('0),
    .cfg_burst_len        (8'd16),
    .cfg_txn_count        (16'd1024),
    .cfg_axi_id            (8'd0),
    .cfg_id_mode           (2'd0),          // FIXED
    .cfg_axi_size          (3'd3),          // 8 bytes/beat
    .cfg_axi_burst         (2'd1),          // INCR
    .cfg_lfsr_seed         (32'd0),         // use param default
    .cfg_data_mode         (1'b0),          // LFSR (golden CRC valid)
    .cfg_hash_seed0        (32'd0),
    .cfg_hash_seed1        (32'd0),
    .cfg_hash_seed2        (32'd0),
    .cfg_wr_gap            (4'd0),
    .cfg_start             (csr_start_pulse),
    .cfg_done              (wr_done),

    // Golden CRC for the read side to compare against
    .o_expected_crc        (wr_expected_crc),
    .o_expected_crc_valid  (wr_expected_crc_valid),
    .o_bresp_error         (wr_bresp_error),

    // M-side AXI to the controller
    .m_axi_awid (awid), /* ... AW ... */ .m_axi_awready(awready),
    .m_axi_wdata(wdata), /* ... W ... */ .m_axi_wready (wready),
    .m_axi_bid  (bid),  /* ... B ... */ .m_axi_bready (bready)
);
```

14.7 Design Notes

- **Two data modes for two threat models:** LFSR mode is the fast default with a single golden CRC; hash mode trades the CRC for OOO-safe per-beat verification when multi-id traffic reorders completions.
 - **Hash pipelining was a timing fix:** the two-multiply Murmur cone was isolated into DSP-register stages specifically to close 100 MHz on FPGA; the staging FIFO then hides the added latency from the AXI handshake.
 - **CRC follows the LFSR, not the wire:** the CRC absorbs the LFSR words (32-bit), keeping it interchangeable with the read checker and independent of bus width; it is only latched valid in LFSR mode.
 - **Decoupled AW/W:** separate address generators mean AW can race ahead at a ready rate while W streams at a ready rate, exposing controller reorder/backpressure behavior a lock-stepped generator would mask.
 - **Direct re-arm:** S_DONE re-latches on cfg_start so back-to-back sweeps don't require a return to idle.
-

14.8 Related Modules

14.8.1 Used By

- projects/NexysA7/ddr2-characterization/flows-ours-uart/rtl/ddr2_char_harness.sv — on-chip write driver
- DDR2 characterization macro / harness CSR blocks under projects/NexysA7/ddr2-characterization/

14.8.2 Uses

- **axi4_master_wr.sv** — standard AXI4 write master protocol handler (AW/W/B skid + compliance)
- **dma_address_gen.sv** — algorithmic address sequence generator (×2, decoupled AW/W)
- **shifter_lfsr_fibonacci.sv** — LFSR data source and 8-bit AW-ID LFSR
- **dataint_crc.sv** — CRC-32 accumulator
- **gaxi_fifo_sync.sv** — show-ahead staging FIFO decoupling the hash pipeline from W

14.8.3 See Also

- **axi4_master_rd_crc_check.sv** — the matching read-side driver + checker (same LFSR/CRC/hash config)
 - **axi4_slave_wr_crc_check.sv** — the slave-side write CRC sink
-

14.9 References

14.9.1 Source Code

- RTL: rtl/amba/shared/axi4_master_wr_pattern_gen.sv
- Protocol Handler: rtl/amba/axi4/axi4_master_wr.sv

14.9.2 Documentation

- Architecture: docs/markdown/RTLAmba/shared/README.md
 - Index: docs/markdown/RTLAmba/index.md
 - Harness: projects/NexysA7/ddr2-characterization/README.md
-

Last Updated: 2026-07-15

14.10 Navigation

- [Back to Shared Infrastructure Index](#)
- [Back to RTLAmba Index](#)

15 AXI4 Slave Read Pattern Generator

Module: axi4_slave_rd_pattern_gen.sv **Location:** rtl/amba/shared/ **Status:** Production Ready

15.1 Overview

axi4_slave_rd_pattern_gen is a read-only AXI4 slave that answers AR bursts with deterministic, LFSR-generated data and accumulates a running CRC-32 over the stream it emits. It exists to serve as a synthetic data *source* for DMA / streaming-engine characterization: a master under test issues read bursts, the slave returns a reproducible pseudo-random pattern, and the

same pattern (and CRC) can be regenerated anywhere else in the system to prove end-to-end data integrity.

15.1.1 Key Features

- Read-only AXI4 slave (AR + R channels) built on the standard `axi4_slave_rd` protocol handler
 - Per-channel independent LFSR + CRC-32 state, demuxed off the low bits of `arid`
 - 32-bit maximal-length Fibonacci LFSR (seed `0xDEADBEEF`, taps `{32, 22, 2, 1}`) replicated to fill the data bus
 - CRC-32/Ethernet accumulator per channel (poly `0x04C11DB7`, reflected in/out) matching the write-side checker bit-for-bit
 - Gapless back-to-back bursts (AR accepted on the last beat) so `rvalid` never drops mid-stream
 - Per-channel CRC / beat-count telemetry plus an aggregate beat-count total for harness stop triggers
 - Single `crc_lfsr_reset` pulse re-arms all channel LFSRs and CRCs together
-

15.2 Module Purpose

Characterizing a DMA read path needs a slave that is both *fast* (never the bottleneck) and *checkable* (the returned data is a known function of the request). A real memory backend gives neither cheaply. This module instead generates its read data from a Fibonacci LFSR, so every beat is a deterministic function of (`seed`, `beat_index`), and folds that same data into a CRC-32 the harness can compare against an independently computed golden value.

Use Cases: - Feeding a DMA / streaming engine's read (`m_axi_rd`) port during throughput characterization - End-to-end integrity checks where read data is looped back to a write-CRC sink - Per-channel multi-context validation, where each channel's data must be independent of another channel's interleave on the shared AXI port - On-chip (FPGA) stimulus in the STREAM / RAPIDS characterization harnesses

Key Benefit: A memory-free, deterministic read slave whose per-channel CRC-32 is bit-identical to the write-side checker — integrity can be proven with a single register compare, and the slave never starves the master.

15.3 Parameters

Parameter	Type	Default	Description
NUM_CHANNELS	int	1	Number of independent LFSR/CRC contexts; channel index is <code>arid[CIW-1:0]</code>
SKID_DEPTH_AR	int	2	AR channel skid buffer depth
SKID_DEPTH_R	int	4	R channel skid buffer depth
AXI_ID_WIDTH	int	8	AXI ID width
AXI_ADDR_WIDTH	int	32	AXI address width
AXI_DATA_WIDTH	int	64	AXI data width (may be wider; must be handled by replication)
AXI_USER_WIDTH	int	1	AXI user signal width
LFSR_WIDTH	int	32	LFSR width (fixed at 32 for timing/simplicity)
LFSR_SEED	logic [31:0]	32'hDEADBEEF	Base LFSR seed; channel N uses $\text{LFSR_SEED} \wedge N$
LFSR_TAPS	logic [47:0]	{12'd32, 12'd22, 12'd2, 12'd1}	Maximal-length Fibonacci tap indices
CRC_WIDTH	int	32	CRC width
CRC_DATA_WIDTH	int	32	Bits processed per CRC update (the 32-bit LFSR output)
CRC_POLY	logic [31:0]	32'h04C11DB7	CRC-32/Ethernet polynomial
CRC_INIT	logic [31:0]	32'hFFFFFFFF	CRC initial value
CRC_XOROUT	logic [31:0]	32'hFFFFFFFF	CRC final XOR
CRC_REFIN	int	1	Reflect input bytes
CRC_REFOUT	int	1	Reflect output

Parameter	Type	Default	Description
REPLICATION_FACTOR	int	(AXI_DATA_WIDTH+31)/32	Derived: number of 32-bit LFSR copies per beat
CIW	int	(NUM_CHANNELS > 1) ? \$clog2(NUM_CHANNELS) : 1	Derived: channel-index width

Note: CRC_* must match axi4_slave_wr_crc_check (and the AXIS pattern blocks) exactly, or cross-block CRC comparison is meaningless.

15.4 Port Groups

15.4.1 Clock and Reset

Port	Direction	Width	Description
aclk	input	1	AXI clock
aresetn	input	1	Active-low asynchronous reset

15.4.2 Test Control

Port	Direction	Width	Description
crc_lfsr_rese	input	1	Pulse to reload all channel LFSR seeds and clear all CRCs / beat counts

15.4.3 Per-Channel Telemetry

Port	Direction	Width	Description
read_crc_value	output	NUM_CHANNELS × 32	Running CRC-32 per channel
read_crc_valid	output	NUM_CHANNELS	Per-channel CRC valid (set after first accepted beat)
read_beat_count	output	NUM_CHANNELS × 32	Per-channel beats emitted
read_beat_sum	output	32	Sum of per-channel beat

Port	Direction	Width	Description
count_total			counts (harness stop trigger)

15.4.4 AXI4 Read Address Channel (AR)

Port	Direction	Width	Description
s_axi_arid	input	AXI_ID_WIDTH	Read address ID (low bits select the channel context)
s_axi_araddr	input	AXI_ADDR_WIDTH	Read address (unused for data generation)
s_axi_arlen	input	8	Burst length minus 1 (beats – 1)
s_axi_arsize	input	3	Burst size
s_axi_arburst	input	2	Burst type
s_axi_arlock	input	1	Lock type
s_axi_arcache	input	4	Cache attributes
s_axi_arprot	input	3	Protection attributes
s_axi_arqos	input	4	Quality of service
s_axi_arregion	input	4	Region identifier
s_axi_aruser	input	AXI_USER_WIDTH	User sideband (echoed on R)
s_axi_arvalid	input	1	AR valid
s_axi_arready	output	1	AR ready

15.4.5 AXI4 Read Data Channel (R)

Port	Direction	Width	Description
s_axi_rid	output	AXI_ID_WIDTH	Read data ID (echoes captured arid)
s_axi_rdata	output	AXI_DATA_WIDTH	LFSR pattern data (32-bit LFSR replicated to fill the bus)
s_axi_rresp	output	2	Response, always OKAY
s_axi_rlast	output	1	Last beat of burst
s_axi_ruser	output	AXI_USER_WIDTH	User sideband (echoes captured aruser)
s_axi_rvalid	output	1	R valid
s_axi_rready	input	1	R ready

15.4.6 Status

Port	Direction	Width	Description
busy	output	1	Protocol-handler busy indicator (from axi4_slave_rd)

15.5 Functional Description

15.5.1 LFSR Pattern Generation

Each channel owns a `shifter_lfsr_fibonacci` instance (32-bit, taps {32, 22, 2, 1}) seeded with `LFSR_SEED ^ channel_index`. XOR-ing the channel index into the seed guarantees each channel produces a distinct stream, so channel interleave at the shared AXI port cannot corrupt any one channel's sequence. Only the channel currently being served advances on a given R-beat handshake; all other channel LFSRs hold their state. On the global `crc_lfsr_reset` pulse every channel reloads its seed simultaneously.

15.5.2 Data Replication to the Bus Width

The 32-bit LFSR output is expanded to `AXI_DATA_WIDTH` by replication. A generate block picks the right path: a direct assignment when the bus is exactly 32 bits, a whole-number replication when the width is a multiple of 32, or a replicate-and-truncate for non-aligned widths

(`REPLICATION_FACTOR = (AXI_DATA_WIDTH+31)/32` copies, sliced down). The active channel's replicated word drives `s_axi_rdata`.

15.5.3 Per-Channel CRC-32

Each channel has its own `dataint_crc` instance (CRC-32/Ethernet, `REFIN/REFOUT = 1`). The CRC absorbs one 32-bit LFSR word per accepted beat for that channel (`load_from_cascade` gated by “R beat AND active channel == this channel”, `cascade_sel = 4'b1000`). A per-channel valid flag sets on the first accepted beat and a per-channel beat counter increments alongside. `crc_lfsr_reset` clears the CRC (via `load_crc_start`), the valid flag, and the counter. The `read_beat_count_total` output is a combinational sum across all channels, used by the harness as a workload-completion trigger.

15.5.4 Burst FSM and Gapless Back-to-Back Bursts

A small two-state FSM (`RD_IDLE`, `RD_BURST`) drives the FUB-side AR/R handshake. AR is accepted when idle *or* on the last beat of the current burst — the latter (`w_rd_last_beat`) lets the next burst reload `r_rd_id / r_rd_beats_remaining` on the same cycle the current one finishes, keeping `rvalid` continuously asserted. The original idle-only accept forced a one-cycle dead gap per burst, which surfaced as ~1 starvation cycle per burst on the master's R channel (a slave-model artifact, not a DUT limitation); accepting the AR on `rlast` removes it. `rresp` is hardwired to `OKAY` and `rlast` asserts when `r_rd_beats_remaining` reaches 0.

15.5.5 Standard Protocol Handler

All AR/R skid buffering and AXI protocol compliance are delegated to a `axi4_slave_rd` instance. The pattern-gen FSM and LFSR/CRC logic ride on that module's FUB-side (`fub_axi_ar* / fub_axi_r*`) interface, so the external `s_axi_*` ports are fully AXI4-compliant. (A header TODO notes that hand-rolled FUB logic should eventually wrap `axi4_slave_rd_mon` instead — tracked as task #78 — but the current handler is the standard `axi4_slave_rd`.)

15.6 Usage Example

```
// Synthetic read source for a 4-channel DMA characterization run.
axi4_slave_rd_pattern_gen #(
    .NUM_CHANNELS    (4),
    .AXI_ID_WIDTH    (8),
    .AXI_ADDR_WIDTH  (32),
    .AXI_DATA_WIDTH  (64),
    .LFSR_SEED       (32'hDEADBEEF)
) u_rd_src (
    .aclk              (aclk),
    .aresetn           (aresetn),
    .crc_lfsr_reset    (csr_clear_pulse), // re-arm all channels
```

```

// Telemetry to the harness CSR block
.read_crc_value      (rd_crc_value),
.read_crc_valid      (rd_crc_valid),
.read_beat_count     (rd_beat_count),
.read_beat_count_total (rd_beats_total),

// AR/R wired to the DUT's read master port
.s_axi_arid      (dut_arid),   .s_axi_araddr (dut_araddr),
.s_axi_arlen     (dut_arlen),   .s_axi_arsize (dut_arsize),
.s_axi_arburst   (dut_arburst), .s_axi_arlock (dut_arlock),
.s_axi_arcache   (dut_arcache), .s_axi_arprot (dut_arprot),
.s_axi_arqos     (dut_arqos),   .s_axi_arregion(dut_arregion),
.s_axi_aruser    (dut_aruser),  .s_axi_arvalid (dut_arvalid),
.s_axi_arready   (dut_arready),

.s_axi_rid      (dut_rid),   .s_axi_rdata (dut_rdata),
.s_axi_rresp    (dut_rresp), .s_axi_rlast  (dut_rlast),
.s_axi_ruser    (dut_ruser), .s_axi_rvalid (dut_rvalid),
.s_axi_rready   (dut_rready),

.busy (rd_busy)
);

```

15.7 Design Notes

- **Seed-per-channel independence:** LFSR_SEED ^ N is deliberately simple but sufficient — a maximal-length LFSR seeded at distinct points produces streams that stay decorrelated over the characterization window, so per-channel CRCs are independently checkable.
- **CRC over the LFSR word, not the replicated bus:** only the 32-bit LFSR output feeds the CRC (cascade_sel = 4'b1000), so the CRC value is bus-width independent and interchangeable with the 32-bit-wide write checker and AXIS blocks.
- **Gapless accept is a measurement fix:** the back-to-back AR accept was added specifically so the read model wouldn't inject a false ~6% starvation artifact into the master's utilization numbers.
- **Reset separation:** crc_lfsr_reset is distinct from aresetn so the harness can re-arm the pattern between runs without a full block reset.

- **Standards note:** the AR/R glue is hand-rolled on top of `axi4_slave_rd`; the header flags a future refactor to the monitored `axi4_slave_rd_mon` wrapper (task #78).
-

15.8 Related Modules

15.8.1 Used By

- `axi4_dma_slaves` — bundles this read source with `axi4_slave_wr_crc_check` into a single source/sink slave pair
- `projects/NexysA7/stream_characterization/flows-stream-bridge/rtl/stream_char_harness.sv` — on-chip read stimulus
- `projects/NexysA7/rapids_characterization/flows-rapids-beats/rtl/rapids_char_harness.sv` — on-chip read stimulus

15.8.2 Uses

- `axi4_slave_rd.sv` — standard AXI4 read slave protocol handler (AR/R skid + compliance)
- `shifter_lfsr_fibonacci.sv` — per-channel maximal-length Fibonacci LFSR
- `dataint_crc.sv` — per-channel CRC-32 accumulator

15.8.3 See Also

- `axi4_slave_wr_crc_check.sv` — the matching write-side CRC sink (same CRC config)
 - `axi4_dma_slaves.sv` — the source/sink bundle
 - `axis4_master_pattern_gen.sv` — AXIS equivalent of this generator
-

15.9 References

15.9.1 Source Code

- RTL: `rtl/amba/shared/axi4_slave_rd_pattern_gen.sv`
- Protocol Handler: `rtl/amba/axi4/axi4_slave_rd.sv`

15.9.2 Documentation

- Architecture: `docs/markdown/RTLAmba/shared/README.md`
- Index: `docs/markdown/RTLAmba/index.md`

- [Harness: projects/NexysA7/ddr2-characterization/README.md](#)
-

Last Updated: 2026-07-15

15.10 Navigation

- [Back to Shared Infrastructure Index](#)
- [Back to RTLAmba Index](#)

16 AXI4 Slave Write CRC Checker

Module: `axi4_slave_wr_crc_check.sv` **Location:** `rtl/amba/shared/` **Status:** Production Ready

16.1 Overview

`axi4_slave_wr_crc_check` is a write-only AXI4 slave that accepts AW/W bursts, folds the written data into a per-channel CRC-32, and returns B responses. It is the *sink* counterpart to `axi4_slave_rd_pattern_gen`: a master under test writes data, this slave CRCs it, and the resulting CRC can be compared against an independently computed golden value to prove the write path carried the data intact.

16.1.1 Key Features

- Write-only AXI4 slave (AW + W + B channels) built on the standard `axi4_slave_wr` protocol handler
- Per-channel independent CRC-32 state, demuxed off the low bits of the captured `awid`
- CRC-32/Ethernet (poly 0x04C11DB7, reflected in/out) — bit-identical to `axi4_slave_rd_pattern_gen`
- Configurable 32-bit data slice (`CRC_SLICE_OFFSET`) selects which 32-bit lane of a wide bus is CRC'd
- Gapless back-to-back bursts (AW accepted on the last W beat) so `wready` never drops mid-stream
- Separately-latched B `id/user` so back-to-back bursts return the correct response `id`

- Per-channel CRC / beat-count telemetry plus an aggregate beat-count total for harness stop triggers
-

16.2 Module Purpose

The write half of a DMA characterization loop needs a slave that accepts write traffic at full rate and reports whether the data arrived correctly. This module CRCs the incoming write data per channel using exactly the same CRC-32 configuration as the read pattern generator, so if a master reads the LFSR pattern and writes it straight back, the write-side CRC must equal the read-side CRC. Any mismatch localizes a data-path corruption to the write path.

Use Cases: - Terminating a DMA / streaming engine's write (`m_axi_wr`) port during characterization - End-to-end integrity checks where read data is looped back and re-verified on write - Per-channel multi-context validation on a shared write port - On-chip (FPGA) write sink in the STREAM / RAPIDS characterization harnesses

Key Benefit: A memory-free write slave that produces a per-channel CRC-32 directly comparable to the read generator — a single register compare confirms the write path is clean, and the sink never backpressures the master unnecessarily.

16.3 Parameters

Parameter	Type	Default	Description
NUM_CHANNELS	int	1	Number of independent CRC contexts; channel index is <code>awid[CIW-1:0]</code>
SKID_DEPTH_AW	int	2	AW channel skid buffer depth
SKID_DEPTH_W	int	4	W channel skid buffer depth
SKID_DEPTH_B	int	2	B channel skid buffer depth
AXI_ID_WIDTH	int	8	AXI ID width
AXI_ADDR_WIDTH	int	32	AXI address width
AXI_DATA_WIDTH	int	64	AXI data width
AXI_USER_WIDTH	int	1	AXI user signal width
CRC_WIDTH	int	32	CRC width

Parameter	Type	Default	Description
CRC_DATA_WIDTH	int	32	Bits processed per CRC update
CRC_POLY	logic [31:0]	32'h04C11DB7	CRC-32/Ethernet polynomial
CRC_INIT	logic [31:0]	32'hFFFFFFFF	CRC initial value
CRC_XOROUT	logic [31:0]	32'hFFFFFFFF	CRC final XOR
CRC_REFIN	int	1	Reflect input bytes
CRC_REFOUT	int	1	Reflect output
CRC_SLICE_OFFSET	int	0	Which 32-bit slice of wdata to CRC (in 32-bit units)
CIW	int	(NUM_CHANNE LS>1) ? \$clog2(NUM_CH ANNELS) : 1	Derived: channel-index width

Note: CRC_* must match axi4_slave_rd_pattern_gen exactly. A compile-time \$error fires if CRC_SLICE_OFFSET selects a slice beyond AXI_DATA_WIDTH.

16.4 Port Groups

16.4.1 Clock and Reset

Port	Direction	Width	Description
aclk	input	1	AXI clock
aresetn	input	1	Active-low asynchronous reset

16.4.2 Test Control

Port	Direction	Width	Description
crc_reset	input	1	Pulse to clear all channel CRCs, valid flags, and beat

Port	Direction	Width	Description
			counts

16.4.3 Per-Channel Telemetry

Port	Direction	Width	Description
write_crc_value	output	NUM_CHAN NELS × 32	Running CRC-32 per channel
write_crc_valid	output	NUM_CHAN NELS	Per-channel CRC valid (set after first accepted W beat)
write_beat_count	output	NUM_CHAN NELS × 32	Per-channel W beats absorbed
write_beat_count_total	output	32	Sum of per-channel beat counts (harness stop trigger)

16.4.4 AXI4 Write Address Channel (AW)

Port	Direction	Width	Description
s_axi_awid	input	AXI_ID_WIDTH	Write address ID (low bits select the channel context)
s_axi_awa_ddr	input	AXI_ADDR_WIDTH	Write address (unused for CRC)
s_axi_awlen	input	8	Burst length minus 1
s_axi_awsize	input	3	Burst size
s_axi_awburst	input	2	Burst type
s_axi_awlock	input	1	Lock type
s_axi_awcache	input	4	Cache attributes
s_axi_awprot	input	3	Protection attributes
s_axi_awqos	input	4	Quality of service

Port	Direction	Width	Description
os			
s_axi_awregion	input	4	Region identifier
s_axi_awuser	input	AXI_USER_WIDTH	User sideband (echoed on B)
s_axi_awvalid	input	1	AW valid
s_axi_awready	output	1	AW ready

16.4.5 AXI4 Write Data Channel (W)

Port	Direction	Width	Description
s_axi_wdata	input	AXI_DATA_WIDTH	Write data (the selected 32-bit slice is CRC'd)
s_axi_wstrb	input	AXI_DATA_WIDTH/8	Write byte strobes (not used to gate the CRC)
s_axi_wlast	input	1	Last beat of burst
s_axi_wuser	input	AXI_USER_WIDTH	User sideband
s_axi_wvalid	input	1	W valid
s_axi_wready	output	1	W ready

16.4.6 AXI4 Write Response Channel (B)

Port	Direction	Width	Description
s_axi_bid	output	AXI_ID_WIDTH	Response ID (separately latched at WLAST)
s_axi_bresp	output	2	Response, always OKAY
s_axi_buser	output	AXI_USER_WIDTH	User sideband
s_axi_bvalid	output	1	B valid

Port	Direction	Width	Description
s_axi_bready	input	1	B ready

16.4.7 Status

Port	Direction	Width	Description
busy	output	1	Protocol-handler busy indicator (from axi4_slave_wr)

16.5 Functional Description

16.5.1 Data Slice Extraction

A wide AXI bus carries several 32-bit lanes; only one is CRC'd so the CRC stays interchangeable with the 32-bit read generator. A generate block assigns `data_slice` directly when the bus is 32 bits wide, and otherwise selects `wdata[CRC_SLICE_OFFSET*32 +: 32]`. A compile-time assertion rejects a `CRC_SLICE_OFFSET` that would read past the end of the bus.

16.5.2 Per-Channel CRC-32

Each channel owns a `dataint_crc` instance (CRC-32/Ethernet, `REFIN/REFOUT = 1`, `cascade_sel = 4'b1000`) fed the 32-bit `data_slice`. The channel selector `w_active_ch` is the low CIW bits of the captured `awid` (`r_wr_id`) — because the AW FSM accepts one burst at a time and W is in-order with AW, the active burst's id identifies the channel during its W beats. A CRC absorbs one word per accepted W beat for its channel (`ch_load_from_cascade = "accepted W beat AND active channel == this channel"`). Per-channel valid flags and beat counters track alongside; `crc_reset` clears them all. `write_beat_count_total` is a combinational sum across channels for the harness stop trigger.

16.5.3 Write Burst FSM and Gapless Bursts

A compact FSM tracks a single active burst via `r_wr_active`. `awready` asserts when not active *or* on the last W beat of the current burst (`w_wr_last_beat`), letting the next burst's AW be accepted back-to-back so `wready` (`= r_wr_active`) never drops between bursts. This mirrors the read-slave gapless fix — without it, a one-cycle !active gap per burst would become the write-side throughput limiter once the read slave feeds data gaplessly.

16.5.4 Back-to-Back B Response Id Latching

When bursts run back-to-back, `r_wr_id` is reloaded with the *next* burst's id on the same cycle the current burst's WLAST lands. The completing burst's B response would then carry the wrong id. To avoid that, the id/user are separately latched into `r_b_id` / `r_b_user` at WLAST and those

drive the B channel. B is single-outstanding (r_b_pending), which is safe because the STREAM master drains B within the burst period; a higher-rate multi-channel sink would need a B FIFO (noted in the RTL).

16.5.5 Standard Protocol Handler

AW/W/B skid buffering and AXI compliance are delegated to a `axi4_slave_wr` instance; the CRC and FSM logic ride on its FUB-side (`fub_axi_aw*` / `fub_axi_w*` / `fub_axi_b*`) interface. (A header TODO tracks a future refactor to wrap the monitored `axi4_slave_wr_mon` — task #79.)

16.6 Usage Example

// Synthetic write sink for a 4-channel DMA characterization run.

```
axi4_slave_wr_crc_check #(
    .NUM_CHANNELS    (4),
    .AXI_ID_WIDTH    (8),
    .AXI_ADDR_WIDTH  (32),
    .AXI_DATA_WIDTH  (64),
    .CRC_SLICE_OFFSET(0)
) u_wr_sink (
    .aclk              (aclk),
    .aresetn           (aresetn),
    .crc_reset         (csr_clear_pulse),
```

// Telemetry to the harness CSR block

```
.write_crc_value      (wr_crc_value),
.write_crc_valid      (wr_crc_valid),
.write_beat_count     (wr_beat_count),
.write_beat_count_total (wr_beats_total),
```

// AW/W/B wired to the DUT's write master port

```
.s_axi_awid    (dut_awid),    .s_axi_awaddr    (dut_awaddr),
.s_axi_awlen   (dut_awlen),   .s_axi_awsz     (dut_awsz),
.s_axi_awburst (dut_awburst), .s_axi_awlock   (dut_awlock),
.s_axi_awcache (dut_awcache), .s_axi_awprot   (dut_awprot),
.s_axi_awqos   (dut_awqos),   .s_axi_awregion (dut_awregion),
.s_axi_awuser  (dut_awuser),  .s_axi_awvalid  (dut_awvalid),
.s_axi_awready (dut_awready),
```

```
.s_axi_wdata (dut_wdata), .s_axi_wstrb (dut_wstrb),
.s_axi_wlast (dut_wlast), .s_axi_wuser (dut_wuser),
.s_axi_wvalid (dut_wvalid), .s_axi_wready (dut_wready),
```

```
.s_axi_bid    (dut_bid),    .s_axi_bresp    (dut_bresp),
.s_axi_buser  (dut_buser),  .s_axi_bvalid   (dut_bvalid),
```

```
.s_axi_bready(dut_bready),  
  
.busy (wr_busy)  
);
```

16.7 Design Notes

- **CRC config must match the source:** the whole point is a comparable value. CRC_POLY, CRC_INIT, CRC_XOROUT, CRC_REFIN, CRC_REFOUT, and the 32-bit slice width are all fixed to mirror axi4_slave_rd_pattern_gen.
 - **Channel demux by AW id, not W sideband:** because W is in-order behind AW and only one burst is active at a time, the captured awid unambiguously names the channel for that burst's W beats.
 - **Single-outstanding B is deliberate but bounded:** it assumes the master drains B within a burst period. The RTL explicitly flags that a faster multi-channel sink needs a B FIFO.
 - **Gapless accept is a measurement fix:** the back-to-back AW accept prevents the sink from injecting a false ~1-cycle-per-burst starvation into write-side utilization numbers.
 - **Standards note:** the AW/W/B glue is hand-rolled on top of axi4_slave_wr; a future refactor to axi4_slave_wr_mon is tracked as task #79.
-

16.8 Related Modules

16.8.1 Used By

- axi4_dma_slaves — bundles this write sink with axi4_slave_rd_pattern_gen into a single source/sink slave pair
- projects/NexysA7/stream_characterization/flows-stream-bridge/rtl/stream_char_harness.sv — on-chip write sink
- projects/NexysA7/rapids_characterization/flows-rapids-beats/rtl/rapids_char_harness.sv — on-chip write sink

16.8.2 Uses

- **axi4_slave_wr.sv** — standard AXI4 write slave protocol handler (AW/W/B skid + compliance)
- **dataint_crc.sv** — per-channel CRC-32 accumulator

16.8.3 See Also

- **axi4_slave_rd_pattern_gen.sv** — the matching read-side pattern source (same CRC config)
 - **axi4_dma_slaves.sv** — the source/sink bundle
 - **axis4_slave_pattern_check.sv** — AXIS equivalent checker
-

16.9 References

16.9.1 Source Code

- RTL: rtl/amba/shared/axi4_slave_wr_crc_check.sv
- Protocol Handler: rtl/amba/axi4/axi4_slave_wr.sv

16.9.2 Documentation

- Architecture: docs/markdown/RTLAmba/shared/README.md
 - Index: docs/markdown/RTLAmba/index.md
 - Harness: projects/NexysA7/ddr2-characterization/README.md
-

Last Updated: 2026-07-15

16.10 Navigation

- [Back to Shared Infrastructure Index](#)
- [Back to RTLAmba Index](#)

17 AXI Bus Meter

Module: axi_bus_meter.sv **Location:** rtl/amba/shared/ **Status:** Production Ready

17.1 Overview

The AXI Bus Meter is a per-cycle valid/ready classifier for a single AXI data channel. Every clock cycle it inspects the valid and ready handshake pair and attributes that cycle to one of four buckets — productive, backpressure, starvation, or idle — accumulating both aggregate (32-bit)

and per-channel (16-bit) counts. It is a pure passive observer that drives nothing back onto the bus, making it safe to drop onto any live AXI R or W channel to characterize datapath utilization.

17.1.1 Key Features

- Four-bucket per-cycle cycle classification (productive / backpressure / starvation / idle)
 - Aggregate 32-bit counters (~42.9 s at 100 MHz before wrap) — always attributed based on bus state
 - Per-channel 16-bit counters, NUM_CHANNELS deep, binned by a caller-supplied channel id
 - Per-channel 4-bit sticky overflow mask flags counters that wrapped
 - Synchronous one-cycle `i_clear` for atomic reset with the surrounding measurement substrate
 - `i_freeze` window control to close the measurement window the instant the workload finishes
 - Passive snoop — no protocol interaction, zero backpressure onto the metered bus
-

17.2 Module Purpose

AXI datapath performance is dominated by how often data actually moves versus how often a handshake stalls. A raw throughput number cannot distinguish a slow producer (master not offering data) from a congested consumer (slave withholding ready). The AXI Bus Meter separates these root causes by classifying every cycle into one of four mutually-exclusive buckets, so a post-run analysis can compute utilization and, more importantly, attribute lost cycles to the correct side of the interface.

The block is instantiated one per bus to be measured. On a read engine it drops onto the R channel (aggregate plus per-channel via `rid`); on a write engine it drops onto the W channel (aggregate plus per-channel via an engine-side sideband, since AXI4 W beats carry no id).

Use Cases: - Metering read R-bus and write W-bus utilization on the stream/rapids characterization engines - Root-causing throughput shortfalls (backpressure-bound vs starvation-bound datapaths) - Per-channel utilization breakdown in multi-channel DMA engines - On-silicon performance characterization runs driven from a host CSR interface

Key Benefit: Turns a single throughput figure into an attributable four-way cycle budget, so lost bandwidth can be pinned to producer starvation or consumer backpressure per channel.

17.3 Parameters

Parameter	Type	Default	Description
NUM_CHANNELS	int	8	Number of per-channel bins (depth of the per-channel counter arrays)
CW	int	$(\text{NUM_CHANNELS} > 1) ? \lceil \log_2(\text{NUM_CHANNELS}) \rceil : 1$	Derived channel-id width; do not override

17.4 Port Groups

17.4.1 Clock and Reset

Port	Direction	Width	Description
aclk	input	1	Clock
aresetn	input	1	Active-low asynchronous reset

17.4.2 Window Control

Port	Direction	Width	Description
i_clear	input	1	Synchronous one-cycle clear pulse; zeroes all counters and overflow stickies
i_freeze	input	1	When high, every counter and overflow sticky holds its value (window closed)

17.4.3 AXI Bus Snoop

Port	Direction	Width	Description
i_valid	input	1	Snooped channel valid (e.g. m_axi_rvalid / m_axi_wvalid)
i_ready	input	1	Snooped channel ready

Port	Direction	Width	Description
i_channel_id	input	CW	(e.g. m_axi_rready / m_axi_wready) Channel index selecting the per-channel bin to increment this cycle
i_channel_valid	input	1	Gates per-channel accumulation; high when a channel is on the hook this cycle

17.4.4 Aggregate Counters

Port	Direction	Width	Description
o_agg_productive	output	32	Cycles with valid && ready (data delivered)
o_agg_backpressure	output	32	Cycles with valid && !ready (master offering, slave stalling)
o_agg_starvation	output	32	Cycles with !valid && ready (slave ready, master not producing)
o_agg_idle	output	32	Cycles with !valid && !ready (both sides quiet)

17.4.5 Per-Channel Counters

Port	Direction	Width	Description
o_ch_productive	output	16 × NUM_CHANNELS	Per-channel productive-cycle counts
o_ch_backpressure	output	16 × NUM_CHANNELS	Per-channel backpressure-cycle counts
o_ch_starvation	output	16 × NUM_CHANNELS	Per-channel starvation-cycle counts

Port	Direction	Width	Description
o_ch_idle	output	16 × NUM_CHAN NELS	Per-channel idle-cycle counts
o_ch_overflow	output	NUM_CHAN NELS*4	Sticky overflow mask, packed {prod, bp, starv, idle} per channel

17.5 Functional Description

17.5.1 Bucket Classification

The four buckets are decoded combinationally from the handshake pair and are mutually exclusive — exactly one is asserted every cycle:

```
w_prod = i_valid && i_ready // productive  – data delivered
w_bp   = i_valid && !i_ready // backpressure – master wants to
send, slave stalls
w_starv = !i_valid && i_ready // starvation  – slave ready, master
not producing
w_idle  = !i_valid && !i_ready // idle       – both sides quiet
```

This is the reference methodology documented in `DMA_UTILIZATION_MEASUREMENT.md` §3.

17.5.2 Aggregate Counters

The four 32-bit aggregate counters always increment based on the raw bus state (subject only to `i_clear` and `i_freeze`). At 100 MHz a 32-bit counter lasts ~42.9 s before overflow — ample for a single characterization run. These provide the whole-bus utilization figure regardless of which channel was active.

17.5.3 Per-Channel Counters and Overflow Stickies

The per-channel arrays are 16 bits wide and `NUM_CHANNELS` deep. Only the bin selected by `i_channel_id` is incremented, and only when `i_channel_valid` is high. The caller asserts `i_channel_valid` whenever some channel is on the hook for the current cycle (for example mid-burst); for cycles where the engine has no work on any channel, `i_channel_valid` is left low and only the aggregate idle/starvation counts are attributed.

Because 16 bits wraps after 65 536 cycles (~655 μs at 100 MHz), each per-channel counter has a companion sticky overflow bit. When a counter is at 16'hFFFF and would increment again, its overflow bit latches high. The four bits per channel are packed {prod, bp, starv, idle} into `o_ch_overflow`. Software checks this mask after stopping the meter; any set bit means the burst

outran 16-bit per-channel resolution and the per-channel numbers for that channel should be discarded (the aggregate 32-bit figures remain valid).

17.5.4 Recommended Wiring

- **Read engine:** `i_valid = m_axi_rvalid, i_ready = m_axi_rready, i_channel_id = m_axi_rid[CW-1:0], i_channel_valid = m_axi_rvalid` (rid is only meaningful when rvalid is high).
- **Write engine:** `i_valid = m_axi_wvalid, i_ready = m_axi_wready, i_channel_id = the engine's internal write-channel-id sideband, i_channel_valid = a "W beats in flight on this channel" indicator (W beats carry no id in AXI4).`

17.5.5 Clear and Freeze Semantics

`i_clear` is a synchronous one-cycle pulse: on the cycle it is high, all counters and all overflow stickies reset to zero. It is intended to be wired to the same control bit that clears the surrounding measurement state (debug SRAM, harness CRC) so a single CSR write atomically resets the entire measurement substrate.

`i_freeze` holds every counter and sticky frozen while high — no bucket increments, no overflow flips. It is driven from the characterization timer's done so the window closes the moment the workload finishes. Without it, the lifetime starvation count would drift upward at one bit per cycle during post-burst host polling, contaminating the in-window utilization math. Hold `i_freeze` low for unbounded free-running measurement.

17.6 Usage Example

```
// Meter the read engine's R channel: aggregate + per-channel by rid.
axi_bus_meter #(
    .NUM_CHANNELS    (8)
) u_r_meter (
    .aclk             (aclk),
    .aresetn          (aresetn),

    // Window control (share with the char timer + CRC clear)
    .i_clear          (perf_run_rising),    // one-cycle pulse on RUN
    rising edge
    .i_freeze         (~perf_run),          // freeze when the window
    is closed

    // R channel snoop
    .i_valid          (m_axi_rvalid),
    .i_ready          (m_axi_rready),
```

```

        .i_channel_id      (m_axi_rid[2:0]),
        .i_channel_valid   (m_axi_rvalid),      // rid valid only when
rvalid

        // Aggregate readback (to CSR / host)
        .o_agg_productive   (r_agg_prod),
        .o_agg_backpressure (r_agg_bp),
        .o_agg_starvation   (r_agg_starv),
        .o_agg_idle         (r_agg_idle),

        // Per-channel readback
        .o_ch_productive    (r_ch_prod),
        .o_ch_backpressure  (r_ch_bp),
        .o_ch_starvation    (r_ch_starv),
        .o_ch_idle          (r_ch_idle),
        .o_ch_overflow       (r_ch_overflow)
    );

    // Utilization = productive / (productive + backpressure + starvation
+ idle)
    // Backpressure-heavy => consumer bound; starvation-heavy => producer
bound.

```

17.7 Design Notes

17.7.1 Passive Observer

The meter drives nothing back onto the AXI bus — it only reads `valid/ready`. It can be added to or removed from a design without any protocol impact, and multiple meters can snoop different channels independently.

17.7.2 Counter Width Rationale

Aggregate counters are 32-bit because they must survive a full-length run (~42.9 s at 100 MHz). Per-channel counters are deliberately kept at 16 bits to bound area across `NUM_CHANNELS` bins; the sticky overflow mask makes the resulting ~655 μ s resolution limit self-diagnosing rather than silently wrong.

17.7.3 Verilator Output Drivers

The per-channel register arrays are copied to the unpacked output arrays through explicit `always_comb` loops, and the overflow stickies are packed into `o_ch_overflow` with a `c*4 +: 4` slice, because Verilator cannot infer unpacked-array output assignments directly from a register array.

17.7.4 Window Alignment

For correct in-window math, drive `i_clear` and `i_freeze` from the same window controller used by the companion `axi_perf_latency_hist` block so all meters open and close on the same cycle boundaries.

17.8 Related Modules

17.8.1 Used By

- `stream_char` datapath utilization metering (read R bus, write W bus)
- `rapids_char` AXI datapath meters (see `projects/NexysA7/.../rapids_char_harness.sv`)
- Host-driven on-silicon performance characterization harnesses

17.8.2 Uses

- `reset_defs.svh` - `ALWAYS_FF_RST` / `RST_ASSERTED` reset macros

17.8.3 See Also

- `axis_bus_meter.sv` - AXIS analogue with byte/packet throughput counters
 - `axi_perf_latency_hist.sv` - Per-channel latency histogram, same window-control convention
-

17.9 References

17.9.1 Source Code

- RTL: `rtl/amba/shared/axi_bus_meter.sv`

17.9.2 Documentation

- Methodology: `docs/.../DMA_UTILIZATION_MEASUREMENT.md` (window semantics, four-bucket definition)
 - Architecture: `docs/markdown/RTLAmba/shared/README.md`
 - Design Guide: `rtl/amba/PRD.md`
-

Last Updated: 2026-07-15

17.10 Navigation

- [Back to Shared Infrastructure Index](#)
- [Back to RTLAmba Index](#)

18 AXI Burst Address Generator

Module: `axi_gen_addr.sv` **Location:** `rtl/amba/shared/` **Status:** Production Ready

18.1 Overview

The AXI Burst Address Generator calculates the next address for AXI burst transactions, supporting FIXED, INCR, and WRAP burst types. It handles data width conversions and provides both next address and aligned address outputs for boundary-aware transaction processing.

18.1.1 Key Features

- Support for all AXI burst types (FIXED, INCR, WRAP)
- Data width conversion handling (DW != ODW)
- Next address calculation with proper increment
- Aligned address generation for boundary checking
- Pure combinational logic for zero-latency operation

18.2 Parameters

Parameter	Type	Default	Description
AW	int	32	Address bus width
DW	int	32	Input data width (internal)
ODW	int	32	Output data width (external bus)
LEN	int	8	Burst length field width

18.3 Ports

Port	Direction	Width	Description
curr_addr	input	AW	Current address
size	input	3	AXI size encoding (2 ^{size} bytes per beat)
burst	input	2	AXI burst type (00=FIXED, 01=INCR, 10=WRAP)
len	input	LEN	AXI burst length (beats - 1)
next_addr	output	AW	Next transaction address
next_addr_align	output	AW	Next address aligned to ODW

18.4 Functional Description

18.4.1 Increment Calculation

The address increment is determined by the size field:

```
increment_pre = (1 << size) // 2size bytes per beat
```

When data widths differ (DW != ODW), the increment is clamped:

```
if (increment_pre > ODWBYTES)
    increment = ODWBYTES;
```

18.4.2 Burst Type Handling

FIXED Burst (burst = 2'b00): - Address remains constant for all beats - next_addr = curr_addr

INCR Burst (burst = 2'b01): - Address increments by size each beat - $\text{next_addr} = \text{curr_addr} + \text{increment}$

WRAP Burst (burst = 2'b10): - Address wraps at burst boundary - Wrap mask = $(1 \ll (\text{size} + \log_2(\text{len}+1))) - 1$ - Aligned address = $(\text{curr_addr} + \text{increment}) \& \sim(\text{increment} - 1)$ - Wrap address = $(\text{curr_addr} \& \sim\text{wrap_mask}) | (\text{aligned_addr} \& \text{wrap_mask})$

18.4.3 Aligned Address Output

The aligned address output aligns the next address to the output data width:

```
alignment_mask = ODWBYTES - 1
next_addr_align = next_addr & ~alignment_mask
```

This is useful for boundary crossing detection in transaction splitters.

18.5 Usage Example

```
// Generate next address for AXI4 64-bit bus
axi_gen_addr #(
    .AW    (32),
    .DW    (64),    // Internal 64-bit
    .ODW   (64),    // External 64-bit
    .LEN   (8)
) u_addr_gen (
    .curr_addr    (32'h1000),
    .size         (3'b011),    // 8 bytes per beat
    .burst        (2'b01),    // INCR
    .len          (8'd3),    // 4 beats total
    .next_addr    (addr_next),
    .next_addr_align (addr_aligned)
);

// addr_next = 0x1000 + 8 = 0x1008
// addr_aligned = 0x1008 (already aligned to 8-byte boundary)
```

18.6 Design Notes

18.6.1 Data Width Conversion

When $\text{DW} \neq \text{ODW}$, the module handles width mismatches: - If $\text{increment} > \text{output bus bytes}$, clamp to output bus width - This prevents address increments larger than the physical bus

18.6.2 WRAP Burst Alignment

WRAP bursts require careful address calculation: - Wrap boundary determined by size and length
- Wrapped address stays within the wrap region - Example: 4-beat wrap at 0xFF8 wraps back to 0xFF8 (not 0x1000)

18.6.3 Pure Combinational

This module is purely combinational for zero latency: - No clock or reset - Instant address generation - Use in address calculation pipelines

18.7 Related Modules

18.7.1 Used By

- `axi_master_rd_splitter.sv` (boundary crossing detection)
- `axi_master_wr_splitter.sv` (boundary crossing detection)
- Address generation pipelines

18.7.2 Related Infrastructure

- `axi_split_combi.sv` (uses aligned addresses for split decisions)
-

18.8 References

18.8.1 Specifications

- ARM IHI 0022E: AMBA AXI4 Protocol Specification (Section A3.4 - Address Structure)
- Internal: `rtl/amba/PRD.md`

18.8.2 Source Code

- RTL: `rtl/amba/shared/axi_gen_addr.sv`
-

Last Updated: 2025-10-24

18.9 Navigation

- [Back to Shared Infrastructure Index](#)

- [Back to RTLAmba Index](#)

19 AXI Master Read Splitter

Module: `axi_master_rd_splitter.sv` **Location:** `rtl/amba/shared/` **Status:** Production Ready

19.1 Overview

The AXI Master Read Splitter is a critical infrastructure module that splits single upstream AXI read transactions into multiple downstream transactions when boundary crossings occur. It enables address range interleaving, memory-mapped I/O access, and heterogeneous memory system integration while maintaining full AXI4 protocol compliance.

19.1.1 Key Features

- Automatic boundary crossing detection and transaction splitting
 - Configurable alignment boundary (4KB, 64KB, etc.)
 - AR channel forwarding with address/length calculation
 - R channel pass-through with transparent ID handling
 - Split transaction tracking via dedicated FIFO interface
 - Zero added latency for non-split transactions
 - Full AXI4 protocol compliance (burst, ID, user fields)
-

19.2 Module Purpose

In modern SoC designs, memory systems are often partitioned across multiple address ranges, memory controllers, or protection domains. The AXI Master Read Splitter solves the fundamental problem of handling transactions that cross these architectural boundaries.

Problem Statement: - Upstream master issues single AXI read (ADDR=0x0FC0, LEN=7, 8 beats total) - Transaction crosses 4KB boundary at 0x1000 - Downstream slaves require separate transactions per boundary region

Solution: - Module detects boundary crossing using combinational split logic - Automatically generates N split transactions (where $N \geq 1$) - First split: ADDR=0x0FC0, LEN=0 (1 beat to boundary) - Second split: ADDR=0x1000, LEN=6 (7 beats remaining) - All response data aggregated transparently

Key Invariants: - Upstream sees exactly ONE transaction request/acceptance - Downstream sees N split transactions (boundary-aligned) - Total response beats equals original transaction beat count - All AXI signaling (ID, USER, PROT, QOS) preserved across splits

19.3 Parameters

Parameter	Type	Default	Description
AXI_ID_WIDTH	int	8	Width of AXI ID signals (ARID, RID)
AXI_ADDR_WIDTH	int	32	Width of AXI address bus (ARADDR)
AXI_DATA_WIDTH	int	32	Width of AXI data bus (RDATA), must be power of 2 (32-1024 bits)
AXI_USER_WIDTH	int	1	Width of AXI user extension fields (ARUSER, RUSER)
SPLIT_FIFO_DEPTH	int	4	Depth of split information tracking FIFO
IW	int	AXI_ID_WIDTH	Shorthand alias for ID width
AW	int	AXI_ADDR_WIDTH	Shorthand alias for address width
DW	int	AXI_DATA_WIDTH	Shorthand alias for data width
UW	int	AXI_USER_WIDTH	Shorthand alias for user width

19.4 Port Groups

19.4.1 Clock and Reset

Port	Direction	Width	Description
acclk	input	1	Global clock for all AXI interfaces
aresetn	input	1	Active-low

Port	Direction	Width	Description
			asynchronous reset

19.4.2 Configuration Interface

Port	Direction	Width	Description
alignment_mask	input	12	12-bit boundary alignment mask (e.g., 0xFFF for 4KB boundaries)
block_read_y	input	1	Backpressure from error monitor (suppresses fub_arready when asserted)

19.4.3 Master AXI Read Interface (Downstream)

AR Channel (Address Read):

Port	Direction	Width	Description
m_axi_arid	output	IW	Transaction ID for split read transactions
m_axi_ara_ddr	output	AW	Calculated address for current split (aligned to boundaries)
m_axi_arlen	output	8	Calculated burst length for current split (AXI encoding: beats-1)
m_axi_arsize	output	3	Transfer size (preserved from original transaction)
m_axi_arburst	output	2	Burst type (preserved from original, typically INCR)
m_axi_arlock	output	1	Lock type (preserved from original)
m_axi_arcache	output	4	Cache attributes (preserved from original)
m_axi_arprot	output	3	Protection attributes

Port	Direction	Width	Description
rot			(preserved from original)
m_axi_arqos	output	4	Quality of Service (preserved from original)
m_axi_arregion	output	4	Region identifier (preserved from original)
m_axi_aruser	output	UW	User-defined extension (preserved from original)
m_axi_arvalid	output	1	Valid signal for current split transaction
m_axi_arready	input	1	Ready signal from downstream slave

R Channel (Read Data):

Port	Direction	Width	Description
m_axi_rid	input	IW	Response transaction ID (matches ARID)
m_axi_rdata	input	DW	Read data payload
m_axi_rresp	input	2	Response status (OKAY, EXOKAY, SLVERR, DECERR)
m_axi_rlast	input	1	Last beat indicator for current split
m_axi_ruser	input	UW	User-defined extension for response
m_axi_rvalid	input	1	Valid signal for response data
m_axi_rready	output	1	Ready signal to downstream slave (pass-through from fub)

19.4.4 Slave AXI Read Interface (Upstream / FUB)

AR Channel (Address Read):

Port	Direction	Width	Description
fub_arid	input	IW	Original transaction ID from upstream master
fub_araddr	input	AW	Original starting address (may cross boundaries)
fub_arlen	input	8	Original burst length (may span multiple boundaries)
fub_arsize	input	3	Transfer size per beat
fub_arburst	input	2	Burst type (FIXED, INCR, WRAP)
fub_arlock	input	1	Lock type for atomic operations
fub_arcache	input	4	Cache attributes
fub_arprot	input	3	Protection attributes
fub_arqos	input	4	Quality of Service priority
fub_arregion	input	4	Region identifier
fub_aruser	input	UW	User-defined extension
fub_arvalid	input	1	Valid signal for original transaction
fub_arready	output	1	Ready signal to upstream master (suppressed until all splits complete)

R Channel (Read Data):

Port	Direction	Width	Description
fub_rid	output	IW	Response transaction ID (pass-through from m_axi)
fub_rdata	output	DW	Read data payload (pass-through from m_axi)
fub_rresp	output	2	Response status (pass-through from m_axi)
fub_rlast	output	1	Last beat indicator (pass-

Port	Direction	Width	Description
fub_ruser	output	UW	through from m_axi) User-defined extension (pass-through from m_axi)
fub_rvalid	output	1	Valid signal for response (pass-through from m_axi)
fub_rready	input	1	Ready signal from upstream master

19.4.5 Split Information Interface

Port	Direction	Width	Description
fub_split_a ddr	output	AW	Original transaction starting address
fub_split_i d	output	IW	Original transaction ID
fub_split_c nt	output	8	Number of split transactions generated (2+ if split occurred)
fub_split_v alid	output	1	Valid signal for split information
fub_split_r eady	input	1	Ready signal from split information consumer

19.5 Functional Description

19.5.1 Transaction Splitting Overview

The module operates as a transparent pass-through for non-split transactions and as an intelligent splitter for boundary-crossing transactions.

Non-Split Path (Fast): - Transaction fully contained within single boundary region - AR signals pass directly from fub to m_axi - fub_arready = m_axi_arready (immediate acceptance) - R channel completely transparent - Split count = 1

Split Path (Multi-Transaction): - Transaction crosses one or more boundary regions - AR signals buffered and regenerated for each split - fub_arready suppressed until final split accepted - R channel remains transparent (ID-based aggregation) - Split count = N (number of boundary crossings + 1)

19.5.2 State Machine Architecture

The module implements a simple two-state FSM:

IDLE State: - Awaits new transactions on fub_ar interface - Combinational split logic evaluates boundary crossing - If no split: immediate pass-through with fub_arready = m_axi_arready - If split: buffer transaction, transition to SPLITTING, suppress fub_arready

SPLITTING State: - Issue current split using buffered address/length - Update next_addr and remaining_len for subsequent split - If more splits needed: stay in SPLITTING, increment split_count - If final split: assert fub_arready when accepted, return to IDLE

19.5.3 Boundary Crossing Detection

Combinational logic module (axi_split_combi) provides real-time splitting decisions:

Inputs: - current_addr: Transaction start address - current_len: Total beats remaining (AXI encoding) - ax_size: Bytes per beat - alignment_mask: Boundary size (e.g., 0xFFF for 4KB)

Outputs: - split_required: Transaction crosses boundary - split_len: Beats to consume in current split (AXI encoding) - next_boundary_addr: Address at next boundary alignment - remaining_len_after_split: Beats remaining after current split

Key Calculation:

```
next_boundary_addr = (current_addr | alignment_mask) + 1
bytes_to_boundary = next_boundary_addr - current_addr
beats_to_boundary = bytes_to_boundary >> ax_size
split_len = beats_to_boundary - 1 // AXI encoding
```

19.5.4 AR Channel Forwarding Logic

Address/Length Calculation: - IDLE state: Use live fub_araddr and calculated split_len - SPLITTING state: Use buffered r_current_addr and split_len - Always output minimum of (split_len, current_len) for safety

Control Signals (ID, PROT, QOS, etc.): - IDLE state: Pass through from fub_ar* signals - SPLITTING state: Use buffered r_orig_ar* signals (captured at acceptance) - Ensures all split transactions carry identical attributes

Valid Signal Generation: - IDLE: m_axi_arvalid = fub_arvalid - SPLITTING: m_axi_arvalid = 1 (always asserting next split)

19.5.5 Ready Signal Management

Critical Protocol Constraint: - fub_arready must NOT assert until entire original transaction accepted - Upstream master expects single handshake for its request

Ready Logic: - IDLE + no split: fub_arready = m_axi_arready (immediate) - IDLE + split needed: fub_arready = 0 (suppress until done) - SPLITTING + intermediate: fub_arready = 0 - SPLITTING + final split: fub_arready = m_axi_arready (complete handshake)

19.5.6 R Channel Handling

The R channel is completely transparent:

Pass-Through Signals: - fub_rid = m_axi_rid - fub_rdata = m_axi_rdata - fub_rresp = m_axi_rresp - fub_rlast = m_axi_rlast - fub_ruser = m_axi_ruser - fub_rvalid = m_axi_rvalid - m_axi_rready = fub_rready

Why This Works: - Split transactions use same ID as original - Downstream slaves respond with matching ID - Upstream master sees aggregated response stream - Total beats = sum of all split lengths - Final RLAST indicates completion

19.5.7 Split Information Tracking

A dedicated FIFO captures metadata for each completed transaction:

FIFO Write Conditions: - Written when fub_arvalid && fub_arready (transaction accepted) - Contains: {original_addr, transaction_id, split_count}

Split Count Values: - No split: split_cnt = 1 - Split occurred: split_cnt = N (number of splits generated)

Use Cases: - Performance monitoring (detect excessive splitting) - Error correlation (map responses to original requests) - Debug visibility (trace transaction decomposition)

19.6 Usage Example

19.6.1 Basic Integration (4KB Boundary Alignment)

```
axi_master_rd_splitter #(
    .AXI_ID_WIDTH      (8),
    .AXI_ADDR_WIDTH    (32),
    .AXI_DATA_WIDTH    (64),
    .AXI_USER_WIDTH    (1),
    .SPLIT_FIFO_DEPTH  (16)
) u_rd_splitter (
    .aclk               (axi_clk),
    .aresetn            (axi_rst_n),

    // 4KB boundary alignment
    .alignment_mask     (12'hFFF),

    // Error monitor backpressure
```



```

.block_ready      (error_detected),

// Upstream AXI read interface (from master)
.fub_arid         (master_arid),
.fub_araddr       (master_araddr),
.fub_arlen        (master_arlen),
.fub_arsize       (master_arsize),
.fub_arburst      (master_arburst),
.fub_arlock       (master_arlock),
.fub_arcache      (master_arcache),
.fub_arprot       (master_arprot),
.fub_arqos        (master_arqos),
.fub_arregion     (master_arregion),
.fub_aruser       (master_aruser),
.fub_arvalid      (master_arvalid),
.fub_arready      (master_arready),

.fub_rid         (master_rid),
.fub_rdata        (master_rdata),
.fub_rresp        (master_rresp),
.fub_rlast        (master_rlast),
.fub_ruser        (master_ruser),
.fub_rvalid       (master_rvalid),
.fub_rready       (master_rready),

// Downstream AXI read interface (to memory system)
.m_axi_arid       (mem_arid),
.m_axi_araddr     (mem_araddr),
.m_axi_arlen      (mem_arlen),
.m_axi_arsize     (mem_arsize),
.m_axi_arburst    (mem_arburst),
.m_axi_arlock     (mem_arlock),
.m_axi_arcache    (mem_arcache),
.m_axi_arprot     (mem_arprot),
.m_axi_arqos      (mem_arqos),
.m_axi_arregion   (mem_arregion),
.m_axi_aruser     (mem_aruser),
.m_axi_arvalid    (mem_arvalid),
.m_axi_arready    (mem_arready),

.m_axi_rid        (mem_rid),
.m_axi_rdata      (mem_rdata),
.m_axi_rresp      (mem_rresp),
.m_axi_rlast      (mem_rlast),
.m_axi_ruser      (mem_ruser),
.m_axi_rvalid     (mem_rvalid),
.m_axi_rready     (mem_rready),

```

```

    // Split information for monitoring
    .fub_split_addr      (split_addr),
    .fub_split_id        (split_id),
    .fub_split_cnt        (split_count),
    .fub_split_valid      (split_valid),
    .fub_split_ready      (split_ready)
);

// Connect split information to monitoring infrastructure
gaxi_fifo_sync #(
    .DATA_WIDTH          (32 + 8 + 8), // addr + id + count
    .DEPTH                (64)
) u_split_fifo (
    .axi_aclk             (axi_clk),
    .axi_aresetn          (axi_rst_n),
    .wr_valid              (split_valid),
    .wr_data               ({split_addr, split_id, split_count}),
    .wr_ready              (split_ready),
    .rd_valid              (monitor_split_valid),
    .rd_data               (monitor_split_data),
    .rd_ready              (monitor_split_ready),
    .count                 (split_fifo_level)
);

```

19.6.2 Transaction Splitting Example Scenario

Original Transaction: - Address: 0x0FC0 (4032 bytes into 4KB region) - Length: 7 (8 beats total in AXI encoding) - Size: 3'b011 (8 bytes per beat = 64 bits) - Total bytes: 8 beats × 8 bytes = 64 bytes - End address: 0x0FC0 + 64 - 1 = 0x0FFF

Boundary Analysis: - 4KB boundary at 0x1000 - Bytes to boundary: 0x1000 - 0x0FC0 = 64 bytes - Beats to boundary: 64 / 8 = 8 beats - Transaction crosses boundary: NO (ends exactly at boundary) - Split required: NO

Modified Scenario (Crosses Boundary): - Address: 0x0FC0 - Length: 8 (9 beats) - Total bytes: 9 × 8 = 72 bytes - End address: 0x0FC0 + 72 - 1 = 0x1007

Split Sequence: 1. First split: ADDR=0x0FC0, LEN=7 (8 beats to reach 0x1000) 2. Second split: ADDR=0x1000, LEN=0 (1 remaining beat) 3. Split count = 2

19.7 Design Notes

19.7.1 AXI Protocol Assumptions

The module enforces several critical assumptions for correct operation:

Assumption 1: Address Alignment - All AXI addresses aligned to data bus width - If DATA_WIDTH = 512 bits, ARADDR always 64-byte aligned - Verified by assertions in axi_split_combi

Assumption 2: Fixed Transfer Size - All transfers use maximum size equal to bus width - ARSIZE always matches $\log_2(\text{DATA_WIDTH}/8)$ - Example: 64-bit bus → ARSIZE = 3'b011 (8 bytes)

Assumption 3: Incrementing Bursts Only - Only INCR burst type supported (ARBURST = 2'b01) - No FIXED or WRAP burst handling - Simplifies address calculation logic

Assumption 4: No Address Wraparound - Transactions never wrap 0xFFFFFFFF → 0x00000000 - Guaranteed by system software/memory allocators - Enables simplified boundary crossing logic

19.7.2 Performance Characteristics

Latency: - No split: Zero added cycles (combinational pass-through) - Split required: 1 cycle per split transaction - Example: 3-way split adds 2 cycles total latency

Throughput: - Full AXI bandwidth for non-split traffic - Sustained split traffic: Limited by split calculation (1 cycle/split) - R channel: Full bandwidth (pass-through)

Area: - Minimal overhead: State machine + transaction buffer - Major component: Split information FIFO - Configurable via SPLIT_FIFO_DEPTH parameter

19.7.3 Error Handling

Transaction Table Exhaustion: - Split FIFO can fill if consumer stalls - Module continues operation (FIFO write stalls fub_arready) - Recommendation: Size FIFO $\geq$ max outstanding transactions

Protocol Violations: - Module assumes well-formed AXI transactions - Assertions validate assumptions (alignment, size, burst type) - Synthesis-time checks enforce parameter constraints

Backpressure Propagation: - block_ready input suppresses fub_arready - Enables error monitor integration - Prevents new transactions during error conditions

19.7.4 Integration Considerations

Alignment Mask Configuration: - 12-bit mask supports boundaries: 4KB to 8MB - Common values: 0xFFF (4KB), 0xFFFF (64KB) - Must match downstream memory region alignment

FIFO Sizing: - SPLIT_FIFO_DEPTH should accommodate burst traffic - Typical value: 16 entries for general-purpose use - High-performance: 32-64 entries

Clock Domain Constraints: - Single clock domain (aclk) for all interfaces - No CDC required within module - Ensure timing closure on split calculation path

19.8 Related Modules

19.8.1 Used By

- Memory controller wrappers requiring boundary-aligned transactions
- Multi-bank DRAM controllers with interleaving requirements
- Bridge modules interfacing heterogeneous memory regions
- DMA engines with address range constraints

19.8.2 Uses

- **axi_split_combi.sv** - Combinational splitting logic (boundary detection, length calculation)
- **gaxi_fifo_sync.sv** - Split information tracking FIFO
- **reset_defs.svh** - Standard reset macro definitions

19.8.3 See Also

- **axi_master_wr_splitter.sv** - Write channel splitting (AW + W channels)
 - **axi_split_combi.sv** - Combined read/write splitting wrapper
-

19.9 References

19.9.1 Source Code

- RTL: rtl/amba/shared/axi_master_rd_splitter.sv
- Tests: val/amba/test_axi_master_rd_splitter.py
- Testbench: bin/TBClasses/amba/axi_master_rd_splitter_tb.py

19.9.2 Documentation

- Architecture: docs/markdown/RTLAmba/shared/README.md
 - AXI Specification: AMBA AXI4 Protocol Specification (ARM IHI 0022E)
 - Design Guide: rtl/amba/PRD.md
-

Last Updated: 2025-10-24

19.10 Navigation

- [Back to Shared Infrastructure Index](#)
- [Back to RTLamba Index](#)

20 AXI Master Write Splitter

Module: `axi_master_wr_splitter.sv` **Location:** `rtl/amba/shared/` **Status:** Production Ready

20.1 Overview

The AXI Master Write Splitter handles the complex task of splitting boundary-crossing write transactions while maintaining data integrity and response consolidation. Unlike read splitting which simply passes through responses, write splitting must aggregate multiple split responses into a single consolidated response with proper error priority handling.

20.1.1 Key Features

- Automatic boundary crossing detection for write transactions
 - AW + W channel forwarding with synchronized address/data flow
 - B channel response consolidation (N responses → 1 consolidated response)
 - Error priority handling (DECERR > SLVERR > EXOKAY > OKAY)
 - WLAST regeneration for split boundaries
 - Zero added latency for non-split transactions
 - Full AXI4 protocol compliance with user extensions
-

20.2 Module Purpose

Write transaction splitting presents unique challenges compared to read splitting due to the tight coupling between address (AW), data (W), and response (B) channels. The AXI Master Write Splitter solves these challenges while maintaining protocol correctness.

Problem Statement: - Upstream master issues write (ADDR=0x0FC0, LEN=7, 8 data beats) - Transaction crosses 4KB boundary at 0x1000 - Downstream requires separate transactions per boundary - Each split generates independent response - Upstream expects single consolidated response

Solution: - Module detects boundary crossing using combinational split logic - AW channel: Generates N split address transactions - W channel: Regenerates WLAST at split boundaries - B channel: Consolidates N responses into 1 (error priority) - Example split: - Split 1: ADDR=0x0FC0,

LEN=0 (1 beat) → Response: OKAY - Split 2: ADDR=0x1000, LEN=6 (7 beats) → Response: OKAY - Consolidated: OKAY (worst error across all splits)

Key Invariants: - Upstream sees exactly ONE write transaction with ONE response - Downstream sees N split transactions with N responses - Consolidated response reflects worst error status - Write data flows through unchanged (WLAST regenerated) - All AXI signaling preserved across splits

20.3 Parameters

Parameter	Type	Default	Description
AXI_ID_WIDTH	int	8	Width of AXI ID signals (AWID, BID)
AXI_ADDR_WIDTH	int	32	Width of AXI address bus (AWADDR)
AXI_DATA_WIDTH	int	32	Width of AXI data bus (WDATA), must be power of 2 (32-1024 bits)
AXI_USER_WIDTH	int	1	Width of AXI user extension fields (AWUSER, WUSER, BUSER)
SPLIT_FIFO_DEPTH	int	4	Depth of split information tracking FIFO
IW	int	AXI_ID_WIDTH	Shorthand alias for ID width
AW	int	AXI_ADDR_WIDTH	Shorthand alias for address width
DW	int	AXI_DATA_WIDTH	Shorthand alias for data width
UW	int	AXI_USER_WIDTH	Shorthand alias for user width

20.4 Port Groups

20.4.1 Clock and Reset

Port	Direction	Width	Description
aclk	input	1	Global clock for all AXI interfaces
aresetn	input	1	Active-low asynchronous reset

20.4.2 Configuration Interface

Port	Direction	Width	Description
alignment_mask	input	12	12-bit boundary alignment mask (e.g., 0xFFF for 4KB boundaries)
block_read_y	input	1	Backpressure from error monitor (suppresses fub_awready when asserted)

20.4.3 Master AXI Write Interface (Downstream)

AW Channel (Address Write):

Port	Direction	Width	Description
m_axi_awi_d	output	IW	Transaction ID for split write transactions
m_axi_awa_ddr	output	AW	Calculated address for current split (boundary-aligned)
m_axi_awl_en	output	8	Calculated burst length for current split (AXI encoding: beats-1)
m_axi_aws_ize	output	3	Transfer size (preserved from original transaction)
m_axi_awb	output	2	Burst type (preserved

Port	Direction	Width	Description
urst			from original, typically INCR)
m_axi_awlock	output	1	Lock type (preserved from original)
m_axi_awcache	output	4	Cache attributes (preserved from original)
m_axi_awprot	output	3	Protection attributes (preserved from original)
m_axi_awqos	output	4	Quality of Service (preserved from original)
m_axi_awregion	output	4	Region identifier (preserved from original)
m_axi_awuser	output	UW	User-defined extension (preserved from original)
m_axi_awvalid	output	1	Valid signal for current split transaction
m_axi_awready	input	1	Ready signal from downstream slave

W Channel (Write Data):

Port	Direction	Width	Description
m_axi_wdata	output	DW	Write data payload (pass-through from fub)
m_axi_wstrobe	output	DW/8	Write strobes (pass-through from fub)
m_axi_wlast	output	1	Last beat indicator (regenerated at split boundaries)
m_axi_wuser	output	UW	User-defined extension (pass-through from fub)
m_axi_wvalid	output	1	Valid signal for write data (pass-through from fub)
m_axi_wready	input	1	Ready signal from

Port	Direction	Width	Description
ady			downstream slave

B Channel (Write Response):

Port	Direction	Width	Description
m_axi_bid	input	IW	Response transaction ID (matches AWID)
m_axi_bresp	input	2	Response status (OKAY=00, EXOKAY=01, SLVERR=10, DECERR=11)
m_axi_buser	input	UW	User-defined extension for response
m_axi_bvalid	input	1	Valid signal for response
m_axi_bready	output	1	Ready signal to downstream slave (controlled during consolidation)

20.4.4 Slave AXI Write Interface (Upstream / FUB)

AW Channel (Address Write):

Port	Direction	Width	Description
fub_awid	input	IW	Original transaction ID from upstream master
fub_awaddr	input	AW	Original starting address (may cross boundaries)
fub_awlen	input	8	Original burst length (may span multiple boundaries)
fub_awsize	input	3	Transfer size per beat
fub_awburst	input	2	Burst type (FIXED, INCR, WRAP)
fub_awlock	input	1	Lock type for atomic operations
fub_awcac	input	4	Cache attributes

Port	Direction	Width	Description
he			
fub_awprot	input	3	Protection attributes
fub_awqos	input	4	Quality of Service priority
fub_awregion	input	4	Region identifier
fub_awuser	input	UW	User-defined extension
fub_awvalid	input	1	Valid signal for original transaction
fub_awready	output	1	Ready signal to upstream master (suppressed until all splits complete)

W Channel (Write Data):

Port	Direction	Width	Description
fub_wdata	input	DW	Write data payload from upstream master
fub_wstrb	input	DW/8	Write strobes from upstream master
fub_wlast	input	1	Last beat indicator from upstream master
fub_wuser	input	UW	User-defined extension
fub_wvalid	input	1	Valid signal for write data
fub_wready	output	1	Ready signal to upstream master (pass-through from m_axi)

B Channel (Write Response):

Port	Direction	Width	Description
fub_bid	output	IW	Consolidated response transaction ID (original AWID)

Port	Direction	Width	Description
fub_bresp	output	2	Consolidated response status (worst error across all splits)
fub_buser	output	UW	User-defined extension (from original transaction)
fub_bvalid	output	1	Valid signal for consolidated response (only after all splits received)
fub_bready	input	1	Ready signal from upstream master

20.4.5 Split Information Interface

Port	Direction	Width	Description
fub_split_addr	output	AW	Original transaction starting address
fub_split_id	output	IW	Original transaction ID
fub_split_count	output	8	Number of split transactions generated (2+ if split occurred)
fub_split_valid	output	1	Valid signal for split information
fub_split_ready	input	1	Ready signal from split information consumer

20.5 Functional Description

20.5.1 Write Transaction Splitting Overview

Write splitting requires coordinated management of three independent AXI channels with different timing characteristics.

Non-Split Path (Fast): - Transaction fully contained within single boundary - AW/W signals pass directly from fub to m_axi - fub_awready = m_axi_awready (immediate acceptance) - B channel passes through directly - Split count = 1

Split Path (Multi-Transaction + Response Consolidation): - Transaction crosses one or more boundaries - AW signals buffered and regenerated for each split - W channel WLAST regenerated at split boundaries - B channel: Collect N responses, consolidate into 1 - fub_awready suppressed until final split accepted - Split count = N (number of splits)

20.5.2 State Machine Architecture

Similar to read splitter but with additional W channel tracking:

IDLE State: - Awaits new transactions on fub_aw interface - Combinational split logic evaluates boundary crossing - If no split: pass-through mode (single response) - If split: buffer transaction, setup response consolidation, transition to SPLITTING

SPLITTING State: - Issue current split using buffered address/length - Track W channel beats to regenerate WLAST correctly - Collect B channel responses (consolidate errors) - Update next_addr and remaining_len for next split - Return to IDLE when final split accepted

20.5.3 Write Data Channel Management

WLAST Regeneration:

The critical challenge is regenerating WLAST at split boundaries:

Beat Counter Tracking: - r_expected_beats: Beats required for current split (from split_len + 1) - r_beat_counter: Current beat being transferred - w_split_boundary_reached: Counter matches expected beats

WLAST Logic:

```
If splitting:
    WLAST = (beat_counter + 1 >= expected_beats)
Else:
    WLAST = fub_wlast (pass-through)
```

Example (8 beats split into 1 + 7): - Original: 8 beats with WLAST on beat 8 - Split 1: Beat 1 → Generate WLAST (boundary reached) - Split 2: Beats 2-8 → Original WLAST on beat 8

20.5.4 Response Consolidation Logic

CRITICAL FEATURE: Multiple split responses must aggregate into single upstream response.

Consolidation State: - r_expected_response_count: Total splits generated - r_received_response_count: Responses received so far - r_waiting_for_responses: Consolidation active flag - r_consolidated_resp_status: Aggregated error status - r_original_txn_id: Transaction ID for consolidated response

Response Priority (Worst Error Wins):

Priority: DECERR (11) > SLVERR (10) > EXOKAY (01) > OKAY (00)

```
if (m_axi_bresp > r_consolidated_resp_status):
    r_consolidated_resp_status = m_axi_bresp
```

Consolidation Flow:

1. **Setup (at transaction start):**
 - Capture original AWID
 - Initialize consolidated_resp_status = OKAY
 - Set expected_response_count = estimated splits
 - Assert waiting_for_responses flag
2. **Collection (each B response):**
 - Accept response even if fub_bready not asserted
 - OR error status into consolidated_resp_status
 - Increment received_response_count
 - Keep fub_bvalid suppressed
3. **Completion (final response):**
 - Check: received_response_count >= expected_response_count
 - Assert fub_bvalid with consolidated status
 - Wait for fub_bready handshake
 - Clear waiting_for_responses flag

Pass-Through Mode (No Split): - waiting_for_responses = 0 - Direct connection: fub_b* = m_axi_b* - Single response, no aggregation needed

20.5.5 Boundary Crossing Detection

Reuses same combinational logic as read splitter (axi_split_combi):

Inputs: - current_addr, current_len, ax_size, alignment_mask - Same calculation as read splitter

Outputs: - split_required, split_len, next_boundary_addr, remaining_len_after_split - Used for both AW channel splitting and W channel beat tracking

20.5.6 AW Channel Forwarding Logic

Address/Length Calculation: - IDLE state: Use live fub_awaddr and calculated split_len - SPLITTING state: Use buffered r_current_addr and split_len

Control Signals: - IDLE state: Pass through from fub_aw* signals - SPLITTING state: Use buffered r_orig_aw* signals

Valid Signal: - IDLE: m_axi_awvalid = fub_awvalid - SPLITTING: m_axi_awvalid = 1 (asserting next split)

20.5.7 Ready Signal Management

fub_awready Logic: - IDLE + no split: fub_awready = m_axi_awready - IDLE + split needed: fub_awready = 0 (suppress) - SPLITTING + intermediate: fub_awready = 0 - SPLITTING + final split: fub_awready = m_axi_awready

W Channel Ready: - Always pass-through: fub_wready = m_axi_wready - Backpressure handled naturally by AXI protocol

B Channel Ready: - Consolidation mode: m_axi_bready = 1 (accept all responses) - Pass-through mode: m_axi_bready = fub_bready - Critical: Must accept all split responses even if upstream not ready

20.6 Usage Example

20.6.1 Basic Integration (4KB Boundary, Response Consolidation)

```
axi_master_wr_splitter #(
    .AXI_ID_WIDTH      (8),
    .AXI_ADDR_WIDTH    (32),
    .AXI_DATA_WIDTH    (64),
    .AXI_USER_WIDTH    (1),
    .SPLIT_FIFO_DEPTH  (16)
) u_wr_splitter (
    .aclk               (axi_clk),
    .aresetn            (axi_rst_n),

    // 4KB boundary alignment
    .alignment_mask     (12'hFFF),

    // Error monitor backpressure
    .block_ready        (error_detected),

    // Upstream AXI write interface (from master)
    .fub_awid           (master_awid),
    .fub_awaddr         (master_awaddr),
    .fub_awlen          (master_awlen),
    .fub_awsz           (master_awsz),
    .fub_awburst        (master_awburst),
    .fub_awlock         (master_awlock),
    .fub_awcache        (master_awcache),
    .fub_awprot         (master_awprot),
    .fub_awqos          (master_awqos),
    .fub_awregion       (master_awregion),
    .fub_awuser         (master_awuser),
    .fub_awvalid        (master_awvalid),
```

```

.fub_awready      (master_awready),

.fub_wdata        (master_wdata),
.fub_wstrb        (master_wstrb),
.fub_wlast        (master_wlast),
.fub_wuser        (master_wuser),
.fub_wvalid       (master_wvalid),
.fub_wready       (master_wready),

.fub_bid          (master_bid),
.fub_bresp        (master_bresp),
.fub_buser        (master_buser),
.fub_bvalid       (master_bvalid),
.fub_bready       (master_bready),

// Downstream AXI write interface (to memory)
.m_axi_awid       (mem_awid),
.m_axi_awaddr     (mem_awaddr),
.m_axi_awlen      (mem_awlen),
.m_axi_awsz       (mem_awsz),
.m_axi_awburst    (mem_awburst),
.m_axi_awlock     (mem_awlock),
.m_axi_awcache    (mem_awcache),
.m_axi_awprot     (mem_awprot),
.m_axi_awqos      (mem_awqos),
.m_axi_awregion   (mem_awregion),
.m_axi_awuser     (mem_awuser),
.m_axi_awvalid    (mem_awvalid),
.m_axi_awready    (mem_awready),

.m_axi_wdata      (mem_wdata),
.m_axi_wstrb      (mem_wstrb),
.m_axi_wlast      (mem_wlast),
.m_axi_wuser      (mem_wuser),
.m_axi_wvalid     (mem_wvalid),
.m_axi_wready     (mem_wready),

.m_axi_bid        (mem_bid),
.m_axi_bresp      (mem_bresp),
.m_axi_buser      (mem_buser),
.m_axi_bvalid     (mem_bvalid),
.m_axi_bready     (mem_bready),

// Split information tracking
.fub_split_addr   (split_addr),
.fub_split_id     (split_id),
.fub_split_cnt    (split_count),

```

```

        .fub_split_valid      (split_valid),
        .fub_split_ready     (split_ready)
    );

```

20.6.2 Response Consolidation Example

Scenario: 2-way split with error in second split

Original Transaction: - ADDR=0x0FC0, LEN=7 (8 beats), ID=0x42

Split Transactions: 1. Split 1: ADDR=0x0FC0, LEN=0 (1 beat) → Response: OKAY (00) 2. Split 2: ADDR=0x1000, LEN=6 (7 beats) → Response: SLVERR (10)

Consolidation Logic:

```

Initial: consolidated_resp = OKAY (00)
Split 1 response: OKAY (00)
    → 00 > 00? No → Keep OKAY
Split 2 response: SLVERR (10)
    → 10 > 00? Yes → Update to SLVERR

```

Final consolidated response: SLVERR (10)

Upstream View: - Master issues 1 write transaction (ADDR=0x0FC0, LEN=7, ID=0x42) - Receives 1 response (ID=0x42, BRESP=SLVERR) - Error correctly propagated despite partial success

20.7 Design Notes

20.7.1 AXI Protocol Assumptions

Same assumptions as read splitter:

Assumption 1: Address Alignment - All addresses aligned to data bus width

Assumption 2: Fixed Transfer Size - AWSIZE matches data width

Assumption 3: Incrementing Bursts Only - Only INCR burst type supported

Assumption 4: No Address Wraparound - Transactions never wrap address space

20.7.2 Response Consolidation Details

Why Consolidation is Critical: - AXI protocol: 1 address transaction → 1 response - Splitting creates N address transactions → N responses - Upstream expects exactly 1 response - Must aggregate N responses into 1 consolidated response

Error Priority Rationale:

DECERR (Decode Error):	Transaction to invalid address
SLVERR (Slave Error):	Valid address, processing error
EXOKAY (Exclusive Okay):	Exclusive access succeeded
OKAY:	Normal successful completion

Worst error wins → System sees most severe failure

Consolidation Timing: - Must wait for ALL split responses before asserting fub_bvalid - Prevents partial success reports - Ensures atomicity from upstream perspective

20.7.3 Performance Characteristics

Latency: - No split: Zero added cycles (pass-through) - Split required: 1 cycle per split + response consolidation - Response consolidation adds latency equal to slowest split

Throughput: - AW channel: Limited by split calculation (1 cycle/split) - W channel: Full bandwidth (pass-through with WLAST regeneration) - B channel: Responses accumulated (no upstream throughput impact)

Area: - Overhead: State machine + transaction buffer + response consolidation logic - Major components: Split FIFO + response tracking registers

20.7.4 Error Handling

Split Response Timeout: - Module waits indefinitely for all split responses - No timeout mechanism (relies on downstream protocol compliance) - Recommendation: Implement system-level timeout monitor

FIFO Overflow: - Split FIFO can fill if consumer stalls - Backpressure propagates to fub_awready - Size FIFO $\geq$ max outstanding writes

Data Channel Synchronization: - W channel must deliver exact beat count per split - WLAST regeneration critical for correctness - Assertions validate beat counter operation

20.7.5 Integration Considerations

Response Consolidation Requirements: - Downstream must return exactly 1 response per split - Response IDs must match split transaction IDs - All responses must eventually arrive

WLAST Timing: - Generated combinationally based on beat counter - Timing-critical path in high-speed designs - Consider registering if timing closure issues

Backpressure Handling: - B channel: Module accepts all split responses (m_axi_bready=1 during consolidation) - Upstream backpressure (fub_bready=0) only delays final consolidated response - Internal buffering for responses: Single register (r_consolidated_resp_status)

20.8 Related Modules

20.8.1 Used By

- Memory controller wrappers with boundary constraints
- Multi-bank DRAM controllers requiring aligned writes
- Bridge modules interfacing heterogeneous memory systems
- DMA engines with address range restrictions

20.8.2 Uses

- **axi_split_combi.sv** - Combinational splitting logic (shared with read splitter)
- **gaxi_fifo_sync.sv** - Split information tracking FIFO
- **reset_defs.svh** - Standard reset macro definitions

20.8.3 See Also

- **axi_master_rd_splitter.sv** - Read channel splitting (AR + R channels)
 - **axi_split_combi.sv** - Combined read/write splitting wrapper
-

20.9 References

20.9.1 Source Code

- RTL: rtl/amba/shared/axi_master_wr_splitter.sv
- Tests: val/amba/test_axi_master_wr_splitter.py
- Testbench: bin/TBClasses/amba/axi_master_wr_splitter_tb.py

20.9.2 Documentation

- Architecture: docs/markdown/RTLAmba/shared/README.md
 - AXI Specification: AMBA AXI4 Protocol Specification (ARM IHI 0022E)
 - Design Guide: rtl/amba/PRD.md
-

Last Updated: 2025-10-24

20.10 Navigation

- [Back to Shared Infrastructure Index](#)
- [Back to RTLAmba Index](#)

21 AXI Performance Latency Histogram

Module: `axi_perf_latency_hist.sv` **Location:** `rtl/amba/shared/` **Status:** Production Ready

21.1 Overview

The AXI Performance Latency Histogram captures per-transaction latency on an AXI master bus and bins it into a log2 histogram surfaced through a CSR-indexed readout (no MonBus packets). It is a self-contained snoop block — it does not gate or otherwise touch the AXI datapath — and is instantiated alongside the datapath monitors, deliberately leaving the shared `axi_monitor_base` untouched. For reads it measures both AR→first-R and AR→RLAST latency; for writes it measures AW→B latency.

21.1.1 Key Features

- Per-channel log2 latency histogram (default 16 bins: bin b counts latencies in $[2^b, 2^{(b+1)})$)
 - Read mode (`IS_READ=1`): two metrics — AR→first-R beat and AR→RLAST
 - Write mode (`IS_READ=0`): one metric — AW→B response
 - Per-channel command-timestamp FIFO matches completions to oldest outstanding command (same-ID in-order)
 - Four-stage histogram update pipeline for FPGA timing closure
 - Free-running (non-frozen) timestamp so latencies straddling the window boundary stay correct
 - CSR-indexed readout of any bin count plus each metric's transaction total
 - Window control (`i_clear` / `i_freeze`) mirroring `axi_bus_meter`
-

21.2 Module Purpose

Average latency hides the distribution that actually matters for QoS analysis — a bus can have a fine mean while a tail of slow transactions strangles a real workload. This block bins each transaction's measured latency into a log2 histogram, so the shape of the distribution (and its tail) is visible from a handful of CSR reads. It snoops the command and completion channels, timestamps each command as it is accepted, and on completion subtracts to get latency and

increments the matching log2 bin. It is purely observational and adds no load to the AXI path, so it can be dropped in for characterization without perturbing the design under measurement.

Use Cases: - Read/write latency distribution characterization on AXI master ports (perfmon Stage D) - Tail-latency analysis for QoS and arbitration tuning - Parity latency metrics alongside axi4_dma_observer per-port histograms - Host-driven on-silicon performance runs (CSR readout, no MonBus traffic)

Key Benefit: Exposes the full latency distribution (including the tail) per channel through cheap CSR reads, without touching the shared monitor base or adding any MonBus packet traffic.

21.3 Parameters

Parameter	Type	Default	Description
ID_WIDTH	int	8	AXI id width on the snooped bus
NUM_CHANNELS	int	8	Number of per-channel bins (channel = id[CW-1:0])
MAX_OUTSTANDING	int	8	Per-channel command-timestamp FIFO depth
NUM_BINS	int	16	Number of log2 histogram bins; bin b counts $[2^b, 2^{(b+1)})$
IS_READ	bit	1'b1	1 = read build (2 metrics), 0 = write build (1 metric)
CNT_W	int	32	Histogram bin and total counter width
CW	int	derived	$\$clog2(NUM_CHANNELS)$ — channel index width
PW	int	derived	$\$clog2(MAX_OUTSTANDING)$ — FIFO pointer width
PW1	int	derived	$PW + 1$ — occupancy width (extra bit for full)
BINW	int	derived	$\$clog2(NUM_BINS)$ — bin index width

21.4 Port Groups

21.4.1 Clock and Reset

Port	Direction	Width	Description
aclk	input	1	Clock
aresetn	input	1	Active-low asynchronous reset

21.4.2 Window Control

Port	Direction	Width	Description
i_clear	input	1	One-cycle pulse (perf RUN rising edge): resets histogram + FIFOs
i_freeze	input	1	Hold high to freeze the histogram (counters stop; readback stable)

21.4.3 Command Channel (AR for reads, AW for writes)

Port	Direction	Width	Description
cmd_valid	input	1	Command-channel valid
cmd_ready	input	1	Command-channel ready (handshake timestamps the command)
cmd_id	input	ID_WIDTH	Command id; low CW bits select the channel

21.4.4 Read Data Channel (R) — used when IS_READ=1

Port	Direction	Width	Description
data_valid	input	1	R-channel valid
data_ready	input	1	R-channel ready
data_last	input	1	R-channel rlast (marks the RLAST metric / FIFO pop)

Port	Direction	Width	Description
data_id	input	ID_WIDTH	R-channel id; low CW bits select the channel

21.4.5 Write Response Channel (B) — used when IS_READ=0

Port	Direction	Width	Description
resp_valid	input	1	B-channel valid
resp_ready	input	1	B-channel ready
resp_id	input	ID_WIDTH	B-channel id; low CW bits select the channel

21.4.6 CSR-Indexed Readout

Port	Direction	Width	Description
i_hist_metric	input	1	Metric select: 0 = first-R/B, 1 = RLAST (reads only)
i_hist_bin	input	BINW	Bin index to read
o_hist_count	output	CNT_W	Selected bin's count for the selected metric
o_hist_total	output	CNT_W	Selected metric's total transaction count

21.5 Functional Description

21.5.1 Metrics

- **IS_READ=1 (read build), NUM_METRICS=2:** metric 0 = AR handshake → first R beat; metric 1 = AR handshake → RLAST beat.
- **IS_READ=0 (write build), NUM_METRICS=1:** metric 0 = AW handshake → B response.

The histogram storage is always sized for two metrics for uniform indexing; the write build uses only metric 0.

21.5.2 Transaction Matching

AXI requires same-ID responses to return in order, so completions can be matched to commands with a simple per-channel FIFO of command-phase timestamps rather than a CAM. On a command handshake (cmd_valid && cmd_ready) the current free-running time is pushed into

the channel's FIFO (bounded by `MAX_OUTSTANDING`). On completion the oldest timestamp for that channel (`r_ts[ch][head]`) is the start time, and the FIFO is popped when the completing beat is `last` (RLAST for reads; B is always “last” for writes). A single AXI data/response bus carries at most one beat per cycle, so at most one push and one pop occur per cycle — there is no multi-writer hazard on the shared histogram.

For reads, a per-channel `r_burst_active` flag distinguishes the first R beat (metric 0, AR→first-R) from subsequent beats of the same burst; it is set on the first beat and cleared when the burst pops on RLAST.

21.5.3 Log2 Binning

`latency_bin()` returns `floor(log2(lat))` clamped to `NUM_BINS - 1`; latencies of 0 or 1 map to bin 0. Bin `b` therefore counts latencies in $[2^b, 2^{(b+1)})$.

21.5.4 Free-Running Timestamp

`r_time` is a 32-bit counter that increments every cycle and is **not** frozen by `i_freeze`. Keeping it running means a latency measurement that straddles the window-close boundary still subtracts two consistent absolute timestamps and yields the correct value. Only the histogram accumulation is frozen.

21.5.5 Four-Stage Update Pipeline

The chain FIFO-read → latency subtract → log2 bin → indexed histogram increment is far too deep for a single 100 MHz cycle (it was the routed critical path), so it is split into four register stages:

- **Stage 0:** capture the event — {`start_ts`, `time`, metric flags `m0/m1`, `valid`}. Gated by `i_freeze` (a frozen window captures no new events).
- **Stage 1:** latency = completion time – start time.
- **Stage 2:** log2 bin from the latency.
- **Stage 3:** increment the selected histogram bin(s) and the per-metric total(s).

The increment is delayed a few cycles, but the host reads the window long after RUN drops to 0, by which time the pipeline has drained — `i_freeze` gates stage 0 while stages 1–3 keep advancing to flush. Because a single AXI bus completes at most one transaction per cycle and metrics `m0/m1` target different histograms, consecutive same-bin increments are hazard-free (the cross-cycle NBA read-after-write is correct).

21.5.6 Window Control

`i_clear` (a one-cycle pulse on the perf RUN rising edge) resets the histograms, totals, and all per-channel FIFOs and state. `i_freeze` (`~RUN`) holds the histogram so a closed window can be read back, while the free-running timestamp keeps counting.

21.5.7 Readout

Readout is a combinational mux: `o_hist_count` returns `r_hist[metric][bin]` for the CSR-selected metric and bin, and `o_hist_total` returns that metric's running transaction total (useful for normalizing bin counts into a distribution).

21.6 Usage Example

```
// Read-latency histogram on an AXI master read port (2 metrics:
// first-R, RLAST).
axi_perf_latency_hist #(
    .ID_WIDTH           (8),
    .NUM_CHANNELS       (8),
    .MAX_OUTSTANDING    (8),
    .NUM_BINS           (16),
    .IS_READ            (1'b1)
) u_rd_lat_hist (
    .aclk                (aclk),
    .aresetn             (aresetn),

    // Window control (share with axi_bus_meter)
    .i_clear              (perf_run_rising),
    .i_freeze             (~perf_run),

    // AR command channel
    .cmd_valid            (m_axi_arvalid),
    .cmd_ready            (m_axi_arready),
    .cmd_id               (m_axi_arid),

    // R data channel
    .data_valid           (m_axi_rvalid),
    .data_ready           (m_axi_rready),
    .data_last            (m_axi_rlast),
    .data_id              (m_axi_rid),

    // B channel unused for reads
    .resp_valid           (1'b0),
    .resp_ready           (1'b0),
    .resp_id              ('0),

    // CSR readout
    .i_hist_metric        (csr_metric),    // 0 = AR->first-R, 1 = AR->RLAST
    .i_hist_bin           (csr_bin),
    .o_hist_count         (csr_bin_count),
    .o_hist_total         (csr_metric_total)
```



```
);
```

```
// Write build: set IS_READ=0, wire AW to cmd_* and B to resp_*, leave  
data_* tied off.
```

21.7 Design Notes

21.7.1 Snoop-Only, Base Untouched

The block is instantiated next to the datapath monitors but does not modify `axi_monitor_base`; it snoops the AXI channels read-only and emits no MonBus packets. Results leave exclusively through the CSR-indexed readout.

21.7.2 Timestamp Not Frozen

Only histogram accumulation freezes on `i_freeze`; `r_time` keeps counting so a transaction whose command was accepted before the window closed and whose completion arrives just after still yields a correct latency.

21.7.3 FIFO Depth vs Outstanding

`MAX_OUTSTANDING` bounds the per-channel timestamp FIFO. A command handshake only pushes when the channel FIFO is not full, and a completion only pops when the FIFO is non-empty, so an over-run on one channel cannot corrupt another. Size `MAX_OUTSTANDING` to the real per-channel outstanding depth of the bus.

21.7.4 Pipeline Drain Before Readback

Because increments land up to three cycles after the completing beat, always read the histogram after the window has been closed (`i_freeze` high) long enough for the pipeline to drain — the host reads long after `RUN` falls, so this is automatic in the intended usage.

21.8 Related Modules

21.8.1 Used By

- Perfmon Stage D characterization (see RFC below)
- Datapath-monitor instantiations needing latency distributions alongside `axi_bus_meter`

21.8.2 Uses

- `reset_defs.svh` - `ALWAYS_FF_RST` / `RST_ASSERTED` reset macros

21.8.3 See Also

- **axi_bus_meter.sv** - Four-bucket utilization meter, shares the window-control convention
 - **axi4_dma_observer.sv** - DMA observability wrapper with per-port latency histograms
 - **axi_monitor_base.sv** - The shared monitor scaffold this block deliberately does not touch
-

21.9 References

21.9.1 Source Code

- RTL: rtl/amba/shared/axi_perf_latency_hist.sv

21.9.2 Documentation

- RFC: rtl/amba/PRD/RFCs/RFC-perfmon-window-buckets.md
 - Architecture: docs/markdown/RTLAmba/shared/README.md
 - Design Guide: rtl/amba/PRD.md
-

Last Updated: 2026-07-15

21.10 Navigation

- [Back to Shared Infrastructure Index](#)
- [Back to RTLAmba Index](#)

22 AXI-Stream Master Pattern Generator

Module: axis4_master_pattern_gen.sv **Location:** rtl/amba/shared/ **Status:** Production Ready

22.1 Overview

axis4_master_pattern_gen is an AXI-Stream master that emits a deterministic, per-channel LFSR data pattern for characterization and verification. It is the AXIS *sink* stimulus for the RAPIDS

characterization harness, and it is pattern/CRC-consistent with the AXI4 pattern blocks: the per-channel 32-bit CRC-32 it computes is bit-identical to what `axi4_slave_wr_crc_check` produces for the same emitted stream (self-check path: `axis_gen` → `m_axi_wr` → `wr_crc_check`).

22.1.1 Key Features

- AXI-Stream master (`m_axis_*`) with no address channel — simpler than the AXI4 pattern generators
 - Per-channel independent LFSR (seed ^ ch) and CRC-32, copied verbatim from `axi4_slave_rd_pattern_gen`
 - Sequential per-channel scheduling — finish one channel's beats, then the next, so each channel's LFSR sequence is contiguous
 - Channel mask (`cfg_channel_mask`) selects active channels (0 → all)
 - Programmable beats-per-channel and tlast cadence (`cfg_beats_per_pkt`)
 - Per-channel expected-CRC / beat-count telemetry plus an aggregate total
 - LFSR/CRC advance only on accepted beats, so beat N of channel C is a deterministic function of (seed ^ C, N) independent of tready stalls
-

22.2 Module Purpose

Characterizing a stream-consuming engine (a “sink”) needs a deterministic source that a downstream checker can predict. This block streams a known LFSR pattern per channel and computes the CRC-32 that the same data will produce when it lands in a CRC-checking sink. Because the LFSR advances only on accepted beats, backpressure never perturbs the sequence — the emitted data is a pure function of channel and beat index, and the exported per-channel CRC is the golden value for end-to-end integrity.

Use Cases: - Driving a RAPIDS sink engine's AXIS input during characterization - Sink self-check: stream into a DMA write path terminated by `axi4_slave_wr_crc_check` and compare CRCs - Per-channel throughput / backpressure sweeps with deterministic, reproducible data - On-chip (FPGA) stream stimulus in the RAPIDS characterization harness

Key Benefit: A memory-free, backpressure-insensitive stream source whose per-channel CRC-32 matches the AXI4 CRC blocks bit-for-bit, so a stream integrity test reduces to a single register compare.

22.3 Parameters

Parameter	Type	Default	Description
NUM_CHANNELS	int	1	Independent LFSR/CRC

Parameter	Type	Default	Description
			contexts; channel index drives tid
AXIS_DATA_WIDTH	int	512	Stream data width (must be a multiple of LFSR_WIDTH)
AXIS_ID_WIDTH	int	8	tid width
AXIS_DEST_WIDTH	int	4	tdest width
AXIS_USER_WIDTH	int	1	tuser width
LFSR_WIDTH	int	32	LFSR width (fixed; must match the AXI4 blocks)
LFSR_SEED	logic [31:0]	32'hDEADBEEF	Base LFSR seed; channel N uses seed ^ N
LFSR_TAPS	logic [47:0]	{12'd32, 12'd22, 12'd2, 12'd1}	Maximal-length Fibonacci taps
CRC_WIDTH	int	32	CRC width
CRC_DATA_WIDTH	int	32	Bits per CRC update
CRC_POLY	logic [31:0]	32'h04C11DB7	CRC-32/Ethernet polynomial
CRC_INIT	logic [31:0]	32'hFFFFFFFF	CRC initial value
CRC_XOROUT	logic [31:0]	32'hFFFFFFFF	CRC final XOR
CRC_REFIN	int	1	Reflect input
CRC_REFOUT	int	1	Reflect output
BEAT_COUNT_WIDTH	int	32	Width of beat / packet counters
STRB_WIDTH	int	AXIS_DATA_WIDTH/8	Derived: tstrb width
REP	int	AXIS_DATA_WIDTH/LFSR_WIDTH	Derived: 32-bit LFSR copies per beat
CIW	int	(NUM_CHANNELS > 1) ?	Derived: channel-index

Parameter	Type	Default	Description
		$\$clog2(NUM_CH$ ANNELS) : 1	width

Note: LFSR + CRC parameters must match `axi4_slave_rd_pattern_gen / axi4_slave_wr_crc_check` for cross-block CRC consistency.

22.4 Port Groups

22.4.1 Clock and Reset

Port	Direction	Width	Description
clk	input	1	Stream clock
rst_n	input	1	Active-low asynchronous reset

22.4.2 Configuration / Control

Port	Direction	Width	Description
cfg_start	input	1	Pulse to begin a run (seeds all channels)
cfg_lfsr_seed	input	LFSR_WIDTH	Seed override (0 → use LFSR_SEED param)
cfg_channel_mask	input	NUM_CHANNELS	Active channels (0 → all channels active)
cfg_num_beats	input	BEAT_COUNT_WIDTH	Beats to send per channel
cfg_beats_per_pkt	input	BEAT_COUNT_WIDTH	last cadence (0 → one packet per channel)
cfg_tdest	input	AXIS_DEST_WIDTH	Value driven on tdest
cfg_busy	output	1	High while running (state != IDLE)
cfg_done	output	1	One-cycle pulse at end of run

22.4.3 Per-Channel Telemetry

Port	Direction	Width	Description
o_expected_crc	output	NUM_CHAN NELS × 32	Per-channel expected CRC-32
o_expected_crc_valid	output	NUM_CHAN NELS	Per-channel CRC valid
o_beat_count	output	NUM_CHAN NELS × 32	Per-channel beats emitted
o_beat_count_total	output	32	Sum across channels (harness stop trigger)

22.4.4 AXI-Stream Master (m_axis)

Port	Direction	Width	Description
m_axis_tvalid	output	1	Beat valid (asserted in the RUN state)
m_axis_tready	input	1	Downstream ready
m_axis_tdata	output	AXIS_DATA_WIDTH	{REP{lfsr_out[ch]}} — 32-bit LFSR replicated to fill the bus
m_axis_tstrb	output	STRB_WIDTH	All-ones (full beat)
m_axis_tlast	output	1	Packet last (per cfg_beats_per_pkt, and each channel's final beat)
m_axis_tid	output	AXIS_ID_WIDTH	Channel index of the current beat
m_axis_tdest	output	AXIS_DEST_WIDTH	Driven from cfg_tdest
m_axis_tuser	output	AXIS_USER_WIDTH	Tied to 0

22.5 Functional Description

22.5.1 Scheduling FSM

A three-state FSM (IDLE, RUN, DONE) sequences a run. On `cfg_start` the effective channel mask, beats-per-channel, and `tlast` cadence are latched; if there are no beats or no active channel it drops straight to DONE, otherwise it selects the first active channel and enters RUN. DONE pulses `cfg_done` for one cycle and returns to IDLE. `m_axis_tvalid` is asserted exactly in RUN.

22.5.2 Sequential Per-Channel Scheduling

Channels are streamed one at a time in ascending index order among the masked set. A `f_next_active_after` function priority-scans the mask for the lowest active channel index strictly greater than the current one (or the first active channel at start). When the current channel's beats are exhausted (`w_ch_last_beat`), the FSM advances to the next active channel and reloads its beat count; when none remain it goes to DONE. Because channels run to completion in turn, each channel's LFSR sequence is contiguous and uninterrupted.

22.5.3 Per-Channel LFSR + CRC (Verbatim from the AXI4 blocks)

Each channel owns a `shifter_lfsr_fibonacci` (32-bit, taps {32,22,2,1}, seed `w_seed ^ ch`) and a `dataint_crc` (CRC-32/Ethernet, `cascade_sel = 4'b1000`). The active channel advances on its accepted beat (`ch_beat = w_beat && r_ch == ch`); all channels reload their seed / clear their CRC on the global load pulse (`cfg_start` in IDLE). The seed, taps, replication, CRC instantiation, and gating are copied verbatim from `axi4_slave_rd_pattern_gen` so the per-channel 32-bit CRC is bit-identical. Per-channel valid flags and beat counters track alongside, and `o_beat_count_total` is a combinational sum for the harness stop trigger.

22.5.4 Stream Outputs and tlast Cadence

`m_axis_tdata` is the active channel's LFSR output replicated REP times; `tid` carries the channel index; `tstrb` is all-ones (full beats); `tdest` is driven from `cfg_tdest`; `tuser` is 0. `tlast` asserts on each channel's final beat, and additionally every `cfg_beats_per_pkt` beats when that cadence is non-zero (0 → one packet spanning the channel's whole run). A per-channel packet counter (`r_pkt_cnt`) resets at each `tlast` and at channel boundaries.

22.5.5 Backpressure Insensitivity

Because the LFSR and CRC advance only on accepted beats (`w_beat = tvalid && tready`), a downstream `tready` stall simply holds the current beat — the emitted value for beat N of channel C is always `f(seed ^ C, N)` regardless of stall pattern. This is what lets the exported per-channel CRC serve as a stable golden value.

22.6 Usage Example

```
// AXIS stimulus for a 4-channel RAPIDS sink characterization run.
axis4_master_pattern_gen #(
    .NUM_CHANNELS      (4),
    .AXIS_DATA_WIDTH   (512),
    .AXIS_ID_WIDTH      (8),
    .LFSR_SEED          (32'hDEADBEEF)
) u_axis_gen (
    .clk (clk),
    .rst_n (rst_n),

    // Control
    .cfg_start          (csr_start_pulse),
    .cfg_lfsr_seed       (32'd0),           // use param default
    .cfg_channel_mask    (4'b1111),         // all channels
    .cfg_num_beats        (32'd1024),        // per channel
    .cfg_beats_per_pkt    (32'd16),          // tlast every 16 beats
    .cfg_tdest           (4'd0),
    .cfg_busy            (gen_busy),
    .cfg_done            (gen_done),

    // Golden CRCs for the downstream checker
    .o_expected_crc      (gen_crc),
    .o_expected_crc_valid (gen_crc_valid),
    .o_beat_count         (gen_beats),
    .o_beat_count_total   (gen_beats_total),

    // Stream to the DUT sink
    .m_axis_tvalid        (s_tvalid), .m_axis_tready (s_tready),
    .m_axis_tdata         (s_tdata),  .m_axis_tstrb  (s_tstrb),
    .m_axis_tlast         (s_tlast),  .m_axis_tid    (s_tid),
    .m_axis_tdest         (s_tdest),  .m_axis_tuser  (s_tuser)
);
```

22.7 Design Notes

- **CRC-consistency by construction:** the LFSR and CRC blocks are copied verbatim from `axi4_slave_rd_pattern_gen`, so a stream emitted here and re-CRC'd by `axi4_slave_wr_crc_check` (or checked by `axis4_slave_pattern_check`) yields identical per-channel CRCs.

- **Sequential scheduling keeps sequences contiguous:** finishing one channel before starting the next means each channel's LFSR runs as an unbroken sequence, matching how the checker regenerates it.
 - **No address channel:** as a pure stream the generator is simpler than the AXI4 pattern gens — no `dma_address_gen`, no burst FSM, just per-channel beat counting.
 - **Backpressure-safe:** advancing only on accepted beats decouples the data sequence from `tready` timing.
 - **Data width constraint:** `AXIS_DATA_WIDTH` must be a multiple of `LFSR_WIDTH` since `tdata = REP × lfsr_out`.
-

22.8 Related Modules

22.8.1 Used By

- `projects/NexysA7/rapids_characterization/flows-rapids-beats/rtl/rapids_char_harness.sv` — on-chip AXIS sink stimulus
- RAPIDS sink-path characterization flows

22.8.2 Uses

- `shifter_lfsr_fibonacci.sv` — per-channel maximal-length Fibonacci LFSR
- `dataint_crc.sv` — per-channel CRC-32 accumulator

22.8.3 See Also

- `axis4_slave_pattern_check.sv` — the matching AXIS checker (same LFSR/CRC config)
 - `axi4_slave_rd_pattern_gen.sv` — the AXI4 read pattern source this is copied from
 - `axi4_slave_wr_crc_check.sv` — CRC sink whose values match this generator
-

22.9 References

22.9.1 Source Code

- RTL: `rtl/amba/shared/axis4_master_pattern_gen.sv`

22.9.2 Documentation

- Architecture: docs/markdown/RTLAmba/shared/README.md
 - Index: docs/markdown/RTLAmba/index.md
 - Harness:
projects/components/dmas/rapids/CONTROL_ENGINE_INTEGRATION.md
-

Last Updated: 2026-07-15

22.10 Navigation

- [Back to Shared Infrastructure Index](#)
- [Back to RTLAmba Index](#)

23 AXI-Stream Slave Pattern Checker

Module: axis4_slave_pattern_check.sv **Location:** rtl/amba/shared/ **Status:** Production Ready

23.1 Overview

axis4_slave_pattern_check is an AXI-Stream slave that consumes a stream and checks it against a deterministic, per-channel LFSR pattern, computing a per-channel CRC-32 as it goes. It is the AXIS *source* checker for the RAPIDS characterization harness and is pattern/CRC-consistent with axis4_slave_rd_pattern_gen: the per-channel CRC-32 it computes is bit-identical to axis4_slave_rd_pattern_gen.read_crc_value for the same stream (self-check path: rd_gen → m_axis → axis_check).

23.1.1 Key Features

- AXI-Stream slave (s_axis_*), pure sink with tready driven by a ready_en input to model backpressure
- Per-channel independent LFSR (seed ^ ch) and CRC-32, copied verbatim from axis4_slave_rd_pattern_gen
- Incoming beats demuxed by s_axis_tid[CIW-1:0] into the matching channel context
- Per-beat compare against the locally regenerated pattern; sticky o_data_error on any mismatch

- Per-channel actual-CRC / beat-count telemetry plus aggregate beat and packet (tlast) counters
- LFSR advances only on accepted beats, so the check is independent of upstream stalls and cross-channel interleave

23.2 Module Purpose

Characterizing a stream-*producing* engine (a “source”) needs a checker that knows what the engine should emit. This block seeds every channel identically to the generator and, on each accepted beat, regenerates the expected pattern for that channel and compares it against the received tdata. It also folds the regenerated data into a per-channel CRC-32 for a whole-run summary. Because the LFSR advances only on accepted beats and is demuxed by tid, the check is robust to backpressure and to arbitrary interleave of channels on the shared stream.

Use Cases: - Terminating and verifying a RAPIDS source engine’s AXIS output during characterization - Source self-check: check a stream produced from axi4_slave_rd_pattern_gen and compare CRCs - Per-channel integrity checking under backpressure (via ready_en) - On-chip (FPGA) stream checker in the RAPIDS characterization harness

Key Benefit: A memory-free stream checker that both pinpoints (sticky o_data_error per beat) and summarizes (per-channel CRC-32 identical to the generator) integrity, robust to stalls and channel interleave.

23.3 Parameters

Parameter	Type	Default	Description
NUM_CHANNELS	int	1	Independent LFSR/CRC contexts; channel = tid low bits
AXIS_DATA_WIDTH	int	512	Stream data width (must be a multiple of LFSR_WIDTH)
AXIS_ID_WIDTH	int	8	tid width
AXIS_DEST_WIDTH	int	4	tdest width
AXIS_USER_WIDTH	int	1	tuser width
LFSR_WIDTH	int	32	LFSR width (fixed; must match the generator)

Parameter	Type	Default	Description
LFSR_SEED	logic [31:0]	32'hDEADBEEF	Base LFSR seed; channel N uses seed ^ N
LFSR_TAPS	logic [47:0]	{12'd32, 12'd22, 12'd2, 12'd1}	Maximal-length Fibonacci taps
CRC_WIDTH	int	32	CRC width
CRC_DATA_WIDTH	int	32	Bits per CRC update
CRC_POLY	logic [31:0]	32'h04C11DB7	CRC-32/Ethernet polynomial
CRC_INIT	logic [31:0]	32'hFFFFFFFF	CRC initial value
CRC_XOROUT	logic [31:0]	32'hFFFFFFFF	CRC final XOR
CRC_REFIN	int	1	Reflect input
CRC_REFOUT	int	1	Reflect output
BEAT_COUNT_WIDTH	int	32	Width of beat / packet counters
STRB_WIDTH	int	AXIS_DATA_WIDTH/8	Derived: tstrb width
REP	int	AXIS_DATA_WIDTH/LFSR_WIDTH	Derived: 32-bit LFSR copies per beat
CIW	int	(NUM_CHANNELS > 1) ? \$clog2(NUM_CHANNELS) : 1	Derived: channel-index width

Note: LFSR + CRC parameters must match axi4_slave_rd_pattern_gen / axis4_master_pattern_gen for cross-block CRC consistency.

23.4 Port Groups

23.4.1 Clock and Reset

Port	Direction	Width	Description
clk	input	1	Stream clock
rst_n	input	1	Active-low asynchronous reset

23.4.2 Configuration / Control

Port	Direction	Width	Description
cfg_start	input	1	Pulse to arm and seed all channels (also clears errors/counters)
cfg_lfsr_seed	input	LFSR_WIDTH	Seed override (0 → use LFSR_SEED param)
ready_en	input	1	Drives s_axis_tready (tie high for a pure sink; deassert to model backpressure)

23.4.3 Telemetry

Port	Direction	Width	Description
o_actual_crc	output	NUM_CHANNELS × 32	Per-channel actual CRC-32
o_actual_crc_valid	output	NUM_CHANNELS	Per-channel CRC valid
o_data_error	output	1	Sticky: any beat mismatched the expected pattern
o_beat_counter	output	NUM_CHANNELS × 32	Per-channel beats received
o_beat_counter_total	output	32	Sum across channels (harness stop trigger)
o_pkt_counter	output	BEAT_COUNT	Aggregate tlast beats

Port	Direction	Width	Description
t		T_WIDTH	received

23.4.4 AXI-Stream Slave (s_axis)

Port	Direction	Width	Description
s_axis_tvalid	input	1	Beat valid
s_axis_tready	output	1	Ready (= ready_en)
s_axis_tdata	input	AXIS_DATA_WIDTH	Received data (compared to {REP{lfsr_out[ch]}})
s_axis_tstrb	input	STRB_WIDTH	Byte strobes (not used by the checker)
s_axis_tlast	input	1	Packet last (counted into o_pkt_count)
s_axis_tid	input	AXIS_ID_WIDTH	Channel selector (tid[CIW-1:0])
s_axis_tdest	input	AXIS_DEST_WIDTH	Destination (unused)
s_axis_tuser	input	AXIS_USER_WIDTH	User sideband (unused)

23.5 Functional Description

23.5.1 Readiness and Beat Handshake

s_axis_tready is driven directly from ready_en — tie it high for a pure, always-ready sink, or deassert it to model downstream backpressure. An accepted beat is w_beat = tvalid && tready. Because the same handshake gates both the compare and the LFSR/CRC advance, the checker and the upstream generator advance in lockstep. The active channel for a beat is s_axis_tid[CIW-1:0].

23.5.2 Per-Channel LFSR + CRC Regeneration (Verbatim from the Generator)

Each channel owns a shifter_lfsr_fibonacci (32-bit, taps {32,22,2,1}, seed w_seed ^ ch) and a dataint_crc (CRC-32/Ethernet, cascade_sel = 4'b1000). On cfg_start (the arm/load pulse) every channel reloads its seed and clears its CRC / counters. A channel advances only on an accepted beat carrying its tid (ch_beat = w_beat && w_ch == ch). The seed, taps, replication,

CRC instantiation, and gating are copied verbatim from `axi4_slave_rd_pattern_gen`, so the per-channel CRC is bit-identical. Each channel exposes `o_actual_crc`, `o_actual_crc_valid`, and `o_beat_count`; `o_beat_count_total` is a combinational sum.

23.5.3 Per-Beat Compare and Sticky Error

The expected data for the active channel is its current LFSR value replicated to the bus width (`expected_data_per_ch[w_ch] = {REP{lfsr_out[ch]}}`), sampled *before* the LFSR advances on the same edge. A mismatch (`w_beat && tdata != expected`) latches `o_data_error` sticky until the next `cfg_start`. Because the compare uses the pre-advance value and the LFSR steps only on accepted beats, the check is independent of upstream stalls and of how channels interleave on the shared stream.

23.5.4 Packet Counting

`o_pkt_count` increments on every accepted beat that carries `tlast`, giving an aggregate packet count across channels. Both `o_data_error` and `o_pkt_count` are cleared by `cfg_start`.

23.6 Usage Example

```
// AXIS checker terminating a 4-channel RAPIDS source engine.
axis4_slave_pattern_check #(
    .NUM_CHANNELS      (4),
    .AXIS_DATA_WIDTH  (512),
    .AXIS_ID_WIDTH     (8),
    .LFSR_SEED         (32'hDEADBEEF)
) u_axis_chk (
    .clk (clk),
    .rst_n (rst_n),

    // Control
    .cfg_start      (csr_arm_pulse),
    .cfg_lfsr_seed  (32'd0),           // use param default
    .ready_en       (sink_ready),     // 1 = always ready; toggle to
backpressure

    // Telemetry to the harness CSR block
    .o_actual_crc   (chk_crc),
    .o_actual_crc_valid (chk_crc_valid),
    .o_data_error    (chk_data_error),
    .o_beat_count    (chk_beats),
    .o_beat_count_total (chk_beats_total),
    .o_pkt_count     (chk_pkts),

    // Stream from the DUT source
```

```

.s_axis_tvalid (m_tvalid), .s_axis_tready (m_tready),
.s_axis_tdata (m_tdata), .s_axis_tstrb (m_tstrb),
.s_axis_tlast (m_tlast), .s_axis_tid (m_tid),
.s_axis_tdest (m_tdest), .s_axis_tuser (m_tuser)
);

// Clean check: no mismatch and per-channel CRC matches the
// generator's.
assign stream_ok = !chk_data_error && (chk_crc[ch] ==
gen_expected_crc[ch]);

```

23.7 Design Notes

- **CRC-consistency by construction:** the LFSR and CRC blocks are copied verbatim from `axi4_slave_rd_pattern_gen`, so a stream produced by that generator (or by `axis4_master_pattern_gen`) yields identical per-channel CRCs here.
 - **Compare uses the pre-advance LFSR value:** the expected data is sampled before the channel's LFSR steps on the same clock edge, so the beat is compared against the value the generator emitted, not the next one.
 - **Two integrity signals:** sticky `o_data_error` pinpoints that *some* beat disagreed, while the per-channel CRC lets the harness confirm the exact stream matched end-to-end.
 - **Backpressure via `ready_en`:** exposing readiness as a config input lets sweeps drive the checker as an always-ready sink or as a throttling one without extra logic.
 - **Data width constraint:** `AXIS_DATA_WIDTH` must be a multiple of `LFSR_WIDTH` since the expected data is `REP × lfsr_out`.
-

23.8 Related Modules

23.8.1 Used By

- `projects/NexysA7/rapids_characterization/flows-rapids-beats/rtl/rapids_char_harness.sv` — on-chip AXIS source checker
- RAPIDS source-path characterization flows

23.8.2 Uses

- **shifter_lfsr_fibonacci.sv** — per-channel maximal-length Fibonacci LFSR
- **dataint_crc.sv** — per-channel CRC-32 accumulator

23.8.3 See Also

- **axis4_master_pattern_gen.sv** — the matching AXIS generator (same LFSR/CRC config)
 - **axi4_slave_rd_pattern_gen.sv** — the AXI4 read pattern source this is copied from
 - **axi4_master_rd_crc_check.sv** — the AXI4 read-side per-beat compare + CRC counterpart
-

23.9 References

23.9.1 Source Code

- RTL: rtl/amba/shared/axis4_slave_pattern_check.sv

23.9.2 Documentation

- Architecture: docs/markdown/RTLAmba/shared/README.md
 - Index: docs/markdown/RTLAmba/index.md
 - Harness:
projects/components/dmas/rapids/CONTROL_ENGINE_INTEGRATION.md
-

Last Updated: 2026-07-15

23.10 Navigation

- [Back to Shared Infrastructure Index](#)
- [Back to RTLAmba Index](#)

24 AXIS Bus Meter

Module: axis_bus_meter.sv **Location:** rtl/amba/shared/ **Status:** Production Ready

24.1 Overview

The AXIS Bus Meter is the AXI-Stream analogue of `axi_bus_meter`. It performs the same four-bucket per-cycle valid/ready classification (productive / backpressure / starvation / idle), and adds AXIS-native throughput counters that are **window-independent**: a payload-byte count derived from `tstrb` popcount and a packet count derived from `tlast`. Because bytes and packets are counted only on productive beats, they measure exactly how much data moved regardless of how long the measurement window stays open — so throughput computed as bytes / busy-time is robust to backpressure and idle padding that would distort a pure cycle-utilization figure. Like its AXI cousin, it is a pure observer and drives nothing back onto the bus.

24.1.1 Key Features

- Four-bucket per-cycle classification identical to `axi_bus_meter`
 - Window-independent AXIS throughput counters: 64-bit bytes (via `tstrb` popcount), 32-bit beats, 32-bit packets (via `tlast`)
 - Aggregate 32-bit cycle-bucket counters (~42.9 s at 100 MHz before wrap)
 - Per-channel 16-bit cycle buckets binned by `tid`, with per-channel 4-bit sticky overflow
 - Synchronous one-cycle `i_clear` and `i_freeze` window control
 - Passive snoop of `tvalid/tready/tlast/tstrb/tid` — no bus interaction
-

24.2 Module Purpose

Cycle-utilization alone is a fragile throughput proxy on a stream bus: if a window is held open for host polling after the last beat, idle cycles inflate and the utilization ratio drops even though the same number of bytes moved. The AXIS Bus Meter fixes this by counting the actual payload transferred. Every productive beat contributes `popcount(tstrb)` bytes to a 64-bit accumulator and, when `tlast` is set, one packet to a 32-bit accumulator. These are byte-exact and independent of window length, so the honest throughput figure is bytes / busy_time. The four cycle buckets are retained alongside for root-causing where non-productive time went.

The block is instantiated one per AXIS bus to be measured, snooping the stream signals as a passive observer.

Use Cases: - Metering AXIS datapath throughput in the `rapids_char` characterization harness - Byte-exact throughput measurement immune to window-hold artifacts - Per-stream (per-`tid`) cycle-utilization breakdown on multi-stream buses - Packet-rate measurement via `tlast` on framed streams

Key Benefit: Window-independent byte and packet counts give a throughput number that cannot be distorted by how long the host holds the measurement window open.

24.3 Parameters

Parameter	Type	Default	Description
DATA_WIDTH	int	512	Stream data width in bits (sets tstrb/byte-lane width)
NUM_CHANNELS	int	8	Number of per-channel (tid) bins
TID_WIDTH	int	$(\text{NUM_CHANNELS} > 1) ? \lceil \log_2(\text{NUM_CHANNELS}) \rceil : 1$	tid bus width; derived, do not override
SW	int	$\text{DATA_WIDTH} / 8$	tstrb width (byte lanes); derived
CW	int	$(\text{NUM_CHANNELS} > 1) ? \lceil \log_2(\text{NUM_CHANNELS}) \rceil : 1$	Channel-bin index width; derived

24.4 Port Groups

24.4.1 Clock and Reset

Port	Direction	Width	Description
aclk	input	1	Clock
aresetn	input	1	Active-low asynchronous reset

24.4.2 Window Control

Port	Direction	Width	Description
i_clear	input	1	Synchronous one-cycle clear pulse; zeroes every counter and sticky
i_freeze	input	1	Hold high to close the measurement window (no

Port	Direction	Width	Description
			counter moves)

24.4.3 AXIS Bus Snoop

Port	Direction	Width	Description
i_tvalid	input	1	Stream tvalid
i_tready	input	1	Stream tready
i_tlast	input	1	Stream tlast (increments packet count on a productive beat)
i_tstrb	input	SW	Byte-lane strobes; popcount gives this beat's payload byte count
i_tid	input	TID_WIDTH	Stream id; its low CW bits select the per-channel bin

24.4.4 Aggregate Cycle Buckets

Port	Direction	Width	Description
o_agg_productive	output	32	Cycles with tvalid && tready (beat transferred)
o_agg_backpressure	output	32	Cycles with tvalid && !tready
o_agg_starvation	output	32	Cycles with !tvalid && tready
o_agg_idle	output	32	Cycles with !tvalid && !tready

24.4.5 Aggregate AXIS-Native Throughput (productive beats only)

Port	Direction	Width	Description
o_agg_bytes	output	64	Sum of popcount(tstrb) over productive beats — exact payload bytes moved
o_agg_beats	output	32	Productive beat count (equals o_agg_productive;

Port	Direction	Width	Description
o_agg_packets	output	32	provided for convenience) Productive beats with tlast asserted (stream packets)

24.4.6 Per-Channel Cycle Buckets (binned by tid)

Port	Direction	Width	Description
o_ch_productive	output	16 × NUM_CHANNELS	Per-channel productive- cycle counts
o_ch_backpressure	output	16 × NUM_CHANNELS	Per-channel backpressure- cycle counts
o_ch_starvation	output	16 × NUM_CHANNELS	Per-channel starvation- cycle counts
o_ch_idle	output	16 × NUM_CHANNELS	Per-channel idle-cycle counts (always zero — see Design Notes)
o_ch_overflow	output	NUM_CHANNELS*4	Sticky overflow mask, packed {prod, bp, starv, idle} per channel

24.5 Functional Description

24.5.1 Bucket Classification

Identical to `axi_bus_meter`: the handshake pair decodes combinationally into four mutually-exclusive buckets.

```

w_prod = i_tvalid && i_tready // productive – beat transferred
w_bp   = i_tvalid && !i_tready // backpressure – master wants to
send, slave stalls
w_starv = !i_tvalid && i_tready // starvation – slave ready,
master not producing
w_idle  = !i_tvalid && !i_tready // idle – both sides quiet

```

See `DMA_UTILIZATION_MEASUREMENT.md` §3 for the reference methodology.

24.5.2 Payload-Byte Popcount

Each beat's byte count is the number of asserted `tstrb` lanes, computed combinationally by summing the strobe bits into `w_beat_bytes`. If the stream has no meaningful `tstrb`, tie it all-ones so bytes accumulate as `beats × (DATA_WIDTH/8)`.

24.5.3 AXIS-Native Throughput Counters

On every productive beat (`tvalid && tready`, window open):

- `o_agg_bytes += this beat's popcount(tstrb)` — a 64-bit accumulator that will not wrap in any realistic run.
- `o_agg_beats += 1` — a convenience mirror of `o_agg_productive`.
- `o_agg_packets += 1` when `i_tlast` is set — counts framed stream packets.

Because these advance only on productive beats, they are window-length independent: holding `i_freeze` low longer only accumulates more idle cycles in the aggregate buckets, never more bytes or packets.

24.5.4 Per-Channel Cycle Buckets

Per-channel buckets are 16-bit, `NUM_CHANNELS` deep, binned by the low CW bits of `tid` (`w_ch`). AXIS carries `tid` on every valid cycle, so the productive, backpressure, and starvation buckets are attributed to `w_ch` whenever the bus is active. Each per-channel counter has a companion sticky overflow bit (`{prod, bp, starv, idle}` packed into `o_ch_overflow`) that latches when the counter would advance past 16'hFFFF, exactly as in `axi_bus_meter`. Software discards the per-channel numbers for any channel whose overflow mask is nonzero.

24.5.5 Clear and Freeze Semantics

`i_clear` is a synchronous one-cycle pulse that zeroes every counter and sticky. `i_freeze` holds all counters frozen while high and is driven from the characterization window controller so the window closes the moment the workload finishes.

24.6 Usage Example

```
// Meter an AXIS bus: cycle buckets + byte/packet throughput, per-tid bins.
axis_bus_meter #(
    .DATA_WIDTH      (512),
    .NUM_CHANNELS    (8)
) u_axis_meter (
    .aclk             (aclk),
    .aresetn          (aresetn),
```

```

.i_clear          (perf_run_rising),
.i_freeze         (~perf_run),

// Stream snoop (all inputs – passive)
.i_tvalid         (s_axis_tvalid),
.i_tready         (s_axis_tready),
.i_tlast         (s_axis_tlast),
.i_tstrb          (s_axis_tstrb),      // tie all-ones if the
stream has no tstrb
.i_tid           (s_axis_tid),

// Aggregate cycle buckets
.o_agg_productive (agg_prod),
.o_agg_backpressure (agg_bp),
.o_agg_starvation (agg_starv),
.o_agg_idle       (agg_idle),

// Window-independent throughput
.o_agg_bytes      (agg_bytes),
.o_agg_beats      (agg_beats),
.o_agg_packets    (agg_packets),

// Per-channel (by tid)
.o_ch_productive  (ch_prod),
.o_ch_backpressure (ch_bp),
.o_ch_starvation  (ch_starv),
.o_ch_idle        (ch_idle),
.o_ch_overflow    (ch_overflow)
);

// Honest throughput = o_agg_bytes / busy_time (immune to window-hold
padding)

```

24.7 Design Notes

24.7.1 Why Byte/Packet Counters Are Window-Independent

A pure cycle-utilization ratio drops if the window is held open past the last beat (host polling adds idle cycles). Byte and packet counters advance only on productive beats, so they capture the true transferred payload no matter how long the window is held. Reporting `bytes / busy_time` sidesteps the padding-sensitivity of `productive / total_cycles`.

24.7.2 Per-Channel Idle Is Not Attributed

Idle cycles (!tvalid && !tready) cannot be assigned to any stream — no tid is meaningful when nothing is on the bus — so r_ch_idle is never incremented and o_ch_idle stays zero. Idle cycles land only in the aggregate o_agg_idle. The per-channel idle output and its overflow bit are kept for structural symmetry with axi_bus_meter.

24.7.3 tstrb Handling

If the source does not drive a meaningful tstrb, tie it all-ones so o_agg_bytes equals beats × (DATA_WIDTH/8). Otherwise o_agg_bytes reflects sparse/partial beats exactly.

24.7.4 Counter Widths

Cycle buckets are 32-bit (aggregate) / 16-bit (per-channel) matching axi_bus_meter. Bytes are 64-bit to guarantee no wrap; beats and packets are 32-bit.

24.8 Related Modules

24.8.1 Used By

- rapids_char AXIS datapath meters (characterization harness)
- Host-driven AXIS throughput characterization runs

24.8.2 Uses

- **reset_defs.svh** - ALWAYS_FF_RST / RST_ASSERTED reset macros

24.8.3 See Also

- **axi_bus_meter.sv** - AXI channel four-bucket meter (this module's origin)
 - **axi_perf_latency_hist.sv** - Latency histogram sharing the same window-control convention
-

24.9 References

24.9.1 Source Code

- RTL: rtl/amba/shared/axis_bus_meter.sv

24.9.2 Documentation

- Methodology: docs/.../DMA_UTILIZATION_MEASUREMENT.md (four-bucket definition, window semantics)

- Architecture: docs/markdown/RTLAmba/shared/README.md
 - Design Guide: rtl/amba/PRD.md
-

Last Updated: 2026-07-15

24.10 Navigation

- [Back to Shared Infrastructure Index](#)
- [Back to RTLAmba Index](#)

25 AXI Split Combinational Logic

Module: axi_split_combi.sv **Location:** rtl/amba/shared/ **Status:** Production Ready

25.1 Overview

The AXI Split Combinational Logic module provides pure combinational boundary crossing detection and split length calculation for AXI transactions. It serves as the core computational engine for both read and write transaction splitters, enabling real-time decision-making without state machine overhead.

25.1.1 Key Features

- Pure combinational logic (zero latency)
 - Boundary crossing detection for arbitrary alignment boundaries
 - Automatic split length calculation (AXI encoding)
 - Optimized for synthesis (bit shifts instead of division)
 - Comprehensive assertion-based validation
 - Configurable address and data widths
 - No address wraparound assumption (simplified logic)
-

25.2 Module Purpose

AXI transaction splitting requires complex address arithmetic to determine: 1. Does this transaction cross a boundary? 2. How many beats fit before the boundary? 3. What address/length for the next split?

This module encapsulates all splitting calculations into reusable combinational logic, ensuring consistent behavior across read and write splitters.

Design Philosophy: - Separation of concerns: Calculation logic separate from state management - Reusability: Single module used by both read and write splitters - Performance: Combinational path enables single-cycle decisions - Correctness: Extensive assertions validate assumptions

25.3 Parameters

Parameter	Type	Default	Description
AW	int	32	Address width in bits (typically 32 or 64)
DW	int	32	Data width in bits, must be power of 2 (32, 64, 128, 256, 512, 1024)

Derived Constants (localparam):

Constant	Calculation	Description
BYTES_PER_BEAT	DW / 8	Bytes transferred per beat (e.g., 8 for 64-bit bus)
LOG2_BYTES_PER_BEAT	$\$clog2(\text{BYTES_PER_BEAT})$	Bit shift amount for beat calculations
EXPECTED_AX_SIZE	LOG2_BYTES_PER_BEAT	Expected value of ax_size input (for assertions)
ADDR_ALIGN_MASK	BYTES_PER_BEAT - 1	Mask for address alignment checking

25.4 Port Groups

25.4.1 Clock and Reset (For Assertions Only)

Port	Direction	Width	Description
aclk	input	1	Clock for assertion evaluation (not used in functional logic)
aresetn	input	1	Active-low reset for

Port	Direction	Width	Description
			assertion gating (not used in functional logic)

25.4.2 Input Signals

Port	Direction	Width	Description
current_address	input	AW	Current transaction starting address (may be mid-split)
current_length	input	8	Current transaction length in AXI encoding (beats - 1)
axi_size	input	3	AXI size field (log2 of bytes per beat)
alignment_mask	input	12	12-bit boundary alignment mask (e.g., 0xFFF for 4KB)
is_idle_state	input	1	Indicates whether state machine is in IDLE (first transaction)
transaction_valid	input	1	Valid transaction present on inputs

25.4.3 Output Signals

Port	Direction	Width	Description
split_required	output	1	Transaction crosses boundary and requires splitting
split_length	output	8	Length of current split in AXI encoding (beats - 1)
next_boundary_address	output	AW	Address at next boundary alignment
remaining_length_after_split	output	8	Remaining length after current split (AXI encoding)
new_split_needed	output	1	Helper signal: split_required AND

Port	Direction	Width	Description
			is_idle_state AND transaction_valid

25.5 Functional Description

25.5.1 Boundary Crossing Detection Algorithm

The module implements a simplified boundary detection algorithm based on key AXI assumptions:

Step 1: Calculate Transaction Parameters

```
total_bytes = (current_len + 1) << ax_size
transaction_end_addr = current_addr + total_bytes - 1
```

Step 2: Calculate Next Boundary

```
next_boundary_addr = (current_addr | alignment_mask) + 1
```

Example (4KB boundary, ADDR=0x0FC0, mask=0xFFF):

```
current_addr      = 0x0FC0
alignment_mask    = 0xFFFF
OR result         = 0xFFFF
next_boundary     = 0xFFFF + 1 = 0x1000
```

Step 3: Calculate Space to Boundary

```
bytes_to_boundary = next_boundary_addr - current_addr
beats_to_boundary = bytes_to_boundary >> ax_size
```

Example (continuing above, ax_size=3 for 8-byte beats):

```
bytes_to_boundary = 0x1000 - 0x0FC0 = 64 bytes
beats_to_boundary = 64 >> 3 = 8 beats
```

Step 4: Determine Split Requirement

Transaction requires splitting if: 1. It ends at or past the next boundary (crosses_boundary) 2. There is space for at least one beat before boundary (has_beats_before_boundary) 3. The beats to boundary fit within total transaction (beats_fit_before_boundary)

```
crosses_boundary = (transaction_end_addr >= next_boundary_addr)
has_beats_before_boundary = (beats_to_boundary > 0)
beats_fit_before_boundary = (beats_to_boundary <= (current_len + 1))
```

```
split_required = crosses_boundary AND has_beats_before_boundary
                AND beats_fit_before_boundary
```

25.5.2 Split Length Calculation

If splitting is required, the split length represents how many beats to consume before the boundary:

```
split_len = split_required ? (beats_to_boundary - 1) : current_len
```

AXI Encoding: Length is always encoded as (beats - 1): - 1 beat → LEN=0 - 2 beats → LEN=1 - 8 beats → LEN=7

25.5.3 Remaining Length Calculation

After consuming the current split, calculate remaining beats:

```
original_beats_total = current_len + 1
split_beats_actual = split_required ? beats_to_boundary :
original_beats_total
remaining_beats_actual = split_required ? (original_beats_total -
split_beats_actual) : 0
```

```
// Convert back to AXI encoding
remaining_len_after_split = split_required ?
                        ((remaining_beats_actual > 0) ?
(remaining_beats_actual - 1) : 0)
                        : 0
```

25.5.4 State Machine Helper

The new_split_needed output simplifies state machine control:

```
new_split_needed = split_required AND is_idle_state AND
transaction_valid
```

This signal indicates a new split sequence is starting (as opposed to continuing an existing split).

25.6 Design Assumptions

The module enforces several critical assumptions to enable simplified, high-performance logic:

25.6.1 Assumption 1: Address Alignment

Assumption: All AXI addresses are aligned to the data bus width.

Implication: - If DATA_WIDTH = 512 bits (64 bytes), address is always 64-byte aligned - Low-order address bits are always zero

Verification:

```
assert ((current_addr & ADDR_ALIGN_MASK) == '0)
```

25.6.2 Assumption 2: Fixed Transfer Size

Assumption: All AXI transfers use maximum transfer size equal to bus width.

Implication: - ax_size always matches $\log_2(\text{DATA_WIDTH}/8)$ - Example: 64-bit bus $\rightarrow$ ax_size = 3'b011 (8 bytes)

Verification:

```
assert (ax_size == EXPECTED_AX_SIZE)
```

25.6.3 Assumption 3: Incrementing Bursts Only

Assumption: All bursts use incrementing address mode (AxBURST = 2'b01).

Implication: - No FIXED or WRAP burst handling - Address calculation simplified

Rationale: - Covers 95%+ of real-world use cases - FIXED bursts rarely cross boundaries - WRAP bursts require complex modulo arithmetic

25.6.4 Assumption 4: No Address Wraparound

Assumption: Transactions never wrap around top of address space (0xFFFFFFFF $\rightarrow$ 0x00000000).

Implication: - No wraparound detection or handling - Simplified comparison logic

Verification:

```
assert (transaction_end_addr >= current_addr) // No overflow
assert (next_boundary_addr > current_addr)    // Boundary doesn't wrap
```

Rationale: - Top of address space typically reserved for device registers - Memory allocators avoid near-boundary allocations - Real systems never allow this condition

25.7 Usage Example

25.7.1 Integration with Read Splitter

```
// Instantiate split logic within read splitter
axi_split_combi #(
    .AW          (32),
    .DW          (64)
) u_split_logic (
    // Clock/reset for assertions only
    .aclk         (aclk),
```

```

        .aresetn                (aresetn),

        // Transaction parameters
        .current_addr            (w_current_addr),        // From state
machine
        .current_len             (w_current_len),        // From state
machine
        .ax_size                 (w_current_size),        // ARSIZE
        .alignment_mask          (12'hFFF),              // 4KB
boundary
        .is_idle_state           (r_state == IDLE),
        .transaction_valid        (fub_arvalid),

        // Splitting decision outputs
        .split_required          (w_split_required),
        .split_len               (w_split_len),
        .next_boundary_addr       (w_next_boundary_addr),
        .remaining_len_after_split (w_remaining_len_after_split),
        .new_split_needed         (w_new_split_needed)
    );

    // Use outputs in state machine
    always_comb begin
        case (r_state)
            IDLE: begin
                if (fub_arvalid && m_axi_arready) begin
                    if (w_new_split_needed) begin
                        // Split required - start sequence
                        next_state = SPLITTING;
                        // Buffer next split parameters
                        buffer_addr = w_next_boundary_addr;
                        buffer_len = w_remaining_len_after_split;
                    end else begin
                        // No split - pass through
                        next_state = IDLE;
                    end
                end
            end
        end

        SPLITTING: begin
            if (m_axi_arvalid && m_axi_arready) begin
                if (w_split_required) begin
                    // More splits needed
                    buffer_addr = w_next_boundary_addr;
                    buffer_len = w_remaining_len_after_split;
                end else begin
                    // Final split complete

```

```

                                next_state = IDLE;
                                end
                            end
                        end
                    endcase
                end
            end

// Use split_len for address channel
assign m_axi_araddr = w_current_addr;
assign m_axi_arlen = w_split_required ? w_split_len : w_current_len;

```

25.7.2 Example Calculation Walkthrough

Scenario: 4KB boundary, 64-bit data bus

Inputs: - current_addr = 0x0FC0 - current_len = 7 (8 beats in AXI encoding) - ax_size = 3 (8 bytes per beat) - alignment_mask = 0xFFF

Calculations:

Step 1: Transaction parameters

```

total_bytes = (7 + 1) << 3 = 8 << 3 = 64 bytes
transaction_end_addr = 0x0FC0 + 64 - 1 = 0x0FFF

```

Step 2: Next boundary

```

next_boundary_addr = (0x0FC0 | 0x0FFF) + 1
                   = 0x0FFF + 1
                   = 0x1000

```

Step 3: Space to boundary

```

bytes_to_boundary = 0x1000 - 0x0FC0 = 64 bytes
beats_to_boundary = 64 >> 3 = 8 beats

```

Step 4: Split decision

```

crosses_boundary = (0x0FFF >= 0x1000) = False
split_required = False

```

Outputs: - split_required = 0 (transaction ends exactly at boundary, no split) - split_len = 7 (pass through original length) - remaining_len_after_split = 0

Modified Scenario: Transaction crosses boundary

Inputs: - current_addr = 0x0FC0 - current_len = 8 (9 beats) - ax_size = 3 - alignment_mask = 0xFFF

Calculations:

Step 1: Transaction parameters


```
total_bytes = (8 + 1) << 3 = 72 bytes
transaction_end_addr = 0x0FC0 + 72 - 1 = 0x1007
```

Step 4: Split decision

```
crosses_boundary = (0x1007 >= 0x1000) = True
has_beats_before_boundary = (8 > 0) = True
beats_fit_before_boundary = (8 <= 9) = True
split_required = True
```

Outputs: - split_required = 1 - split_len = 7 (8 beats - 1 in AXI encoding) - next_boundary_addr = 0x1000 - remaining_len_after_split = 0 (1 beat - 1 in AXI encoding)

25.8 Design Notes

25.8.1 Synthesis Optimization

Bit Shifts Instead of Division:

```
// Original (expensive):
beats_to_boundary = bytes_to_boundary / (1 << ax_size)
```

```
// Optimized (single barrel shifter):
beats_to_boundary = bytes_to_boundary >> ax_size
```

Fixed Alignment Assumptions: - Removes modulo arithmetic - Enables simple OR-based boundary calculation - Critical for timing closure in high-speed designs

25.8.2 Assertion Coverage

The module includes comprehensive assertions for validation:

Input Validation (Triggered on transaction_valid && is_idle_state): - Address alignment to data width - ax_size matches expected value for data width - No address wraparound in transaction - No boundary calculation wraparound

Output Validation (Triggered on transaction_valid): - Beats to boundary within reasonable range - Split length conservation (split_beats + remaining_beats = original_beats) - Boundary alignment verification - Comparison logic consistency

25.8.3 Timing Considerations

Critical Path: 1. Address OR with alignment_mask (next_boundary_addr) 2. Subtraction (bytes_to_boundary) 3. Variable barrel shift by ax_size (beats_to_boundary) 4. Comparison and output muxing (split_required, split_len)

Optimization Strategies: - Pipeline if timing closure issues (adds 1 cycle latency) - Use registered alignment_mask if it's static - Consider ax_size as constant parameter if fixed data width

25.8.4 Debug Support

DEBUG_AXI_SPLIT Macro:

Enable detailed transaction logging:

```
`define DEBUG_AXI_SPLIT

// Generates console output:
=== AXI_SPLIT_DEBUG (NO_WRAPAROUND) ===
current_addr = 0x00000FC0
current_len = 7 (total beats = 8)
ax_size = 3 (bytes_per_beat = 8)
alignment_mask = 0xFFF
transaction_end_addr = 0x00000FFF
next_boundary_addr = 0x00001000
bytes_to_boundary = 64
beats_to_boundary = 8
crosses_boundary = 0
split_required = 0
split_len = 7 (split beats = 8)
remaining_len_after_split = 0 (remaining beats = 0)
=====
```

25.9 Related Modules

25.9.1 Used By

- **axi_master_rd_splitter.sv** - Read transaction splitting (AR channel)
- **axi_master_wr_splitter.sv** - Write transaction splitting (AW channel)

25.9.2 Dependencies

- None (pure combinational logic)

25.9.3 See Also

- **axi_master_rd_splitter.sv** - Full read splitter with state machine
 - **axi_master_wr_splitter.sv** - Full write splitter with response consolidation
-

25.10 References

25.10.1 Source Code

- RTL: rtl/amba/shared/axi_split_combi.sv
- Tests: Indirectly tested via splitter tests (test_axi_master_rd_splitter.py, test_axi_master_wr_splitter.py)

25.10.2 Documentation

- Architecture: docs/markdown/RTLAmba/shared/README.md
 - AXI Specification: AMBA AXI4 Protocol Specification (ARM IHI 0022E)
 - Design Guide: rtl/amba/PRD.md
-

Last Updated: 2025-10-24

25.11 Navigation

- [Back to Shared Infrastructure Index](#)
- [Back to RTLAmba Index](#)

26 CDC 2-Phase Handshake

Module: cdc_2_phase_handshake.sv **Location:** rtl/amba/shared/ **Status:** Production Ready
Companion: cdc_4_phase_handshake.sv (level-based, classic variant)

26.1 Overview

Two-phase (toggle / NRZ) CDC handshake with data transfer. Provides the same valid/ready interface as the 4-phase variant but uses toggle-based signaling to halve the control-path round trip – two synchronizer crossings per transfer instead of four.

The “event” is a TRANSITION of the request/acknowledge signals rather than a level. The source toggles its request bit on each new transfer; the destination detects the toggle edge and responds by toggling its acknowledge bit.

26.1.1 Key Features

- Drop-in replacement for cdc_4_phase_handshake (same port interface)
- Faster: 2 synchronizer crossings per transfer (vs 4 for level-based)

- Toggle-based request/acknowledge (NRZ encoding)
- Parameterizable synchronizer depth (2-3 stages)
- Optional timeout detection for stall diagnosis
- ASYNC_REG attributes for Xilinx/Intel FPGA flows
- SDC constraint guidance in RTL comments

26.1.22-Phase vs 4-Phase Comparison

Aspect	2-Phase (this module)	4-Phase
Crossings per transfer	2 (req toggle + ack toggle)	4 (req + ack + req clear + ack clear)
Latency	~5-6 destination clocks	~7-8 destination clocks
Throughput	Higher (shorter round-trip)	Lower
Complexity	Edge detection required	Simpler level-based logic
Reset behavior	Toggle state preserved across reset recovery	Clean level-based reset

26.2 Parameters

Parameter	Type	Default	Description
DATA_WIDTH	int	8	Width of the data bus (1+)
SYNC_STAGES	int	3	Synchronizer depth for req/ack (2 or 3)
TIMEOUT_CYCLES	int	0	0 = disabled; >0 asserts src_timeout after stall

26.3 Port Groups

26.3.1 Source Clock Domain Interface

Port	Direction	Width	Description
clk_src	input	1	Source domain

Port	Direction	Width	Description
			clock
rst_src_n	input	1	Source domain active-low async reset
src_valid	input	1	Source indicates data valid
src_ready	output	1	Handshake ready (asserted when idle)
src_data	input	DATA_WIDTH	Data from source domain
src_timeout	output	1	Stall timeout (when TIMEOUT_CYCLES > 0)

26.3.2 Destination Clock Domain Interface

Port	Direction	Width	Description
clk_dst	input	1	Destination domain clock
rst_dst_n	input	1	Destination domain active-low async reset
dst_valid	output	1	Data valid to receiver
dst_ready	input	1	Receiver ready
dst_data	output	DATA_WIDTH	Data transferred to destination domain

26.4 Theory of Operation

26.4.1 Source Domain FSM

S_IDLE src_ready=1. On src_valid: latch data, toggle r_req_tog, drop src_ready, go to S_WAIT_ACK.

S_WAIT_ACK Wait for ack edge (w_ack_event). On edge: src_ready=1, return to S_IDLE.

26.4.2 Destination Domain FSM

D_IDLE Wait for req edge (w_req_event). On edge: copy data into r_dst_data, raise dst_valid, go to D_WAIT_READY.

D_WAIT_READY Hold dst_valid. On dst_ready: drop dst_valid, toggle r_ack_tog, return to D_IDLE.

26.4.3 Edge Detection

event = current_sync_output ^ previous_sync_output

The edge detector uses one additional flop per direction to compare the current synchronized toggle value with the previous cycle's value. A difference indicates a new transfer event.

26.5 Usage Example

```
cdc_2_phase_handshake #(
    .DATA_WIDTH      (64),
    .SYNC_STAGES     (3),
    .TIMEOUT_CYCLES  (1000)
) u_packet_cdc (
    // Source: Monitor clock domain
    .clk_src      (mon_clk),
    .rst_src_n    (mon_rst_n),
    .src_valid     (mon_pkt_valid),
    .src_ready     (mon_pkt_ready),
    .src_data      (mon_pkt_data),
    .src_timeout   (mon_timeout),

    // Destination: System clock domain
    .clk_dst       (sys_clk),
    .rst_dst_n     (sys_rst_n),
    .dst_valid      (sys_pkt_valid),
    .dst_ready      (sys_pkt_ready),
    .dst_data       (sys_pkt_data)
);
```

26.5.1 Swapping with 4-Phase

The two handshake modules are drop-in interchangeable:

```
// Change only the module name -- ports are identical
cdc_4_phase_handshake #( // or cdc_2_phase_handshake
    .DATA_WIDTH      (64),
```

```

        .SYNC_STAGES      (3),
        .TIMEOUT_CYCLES  (0)
    ) u_cdc ( ... );

```

26.6 Required SDC Constraints

1. Req / Ack single-bit toggle crossings

```

set_max_delay -datapath_only \
    -from [get_pins u_cdc/r_req_tog_reg/C] \
    -to   [get_pins u_cdc/r_req_sync_reg[0]/D] \
    <dst_period_ns>

```

```

set_max_delay -datapath_only \
    -from [get_pins u_cdc/r_ack_tog_reg/C] \
    -to   [get_pins u_cdc/r_ack_sync_reg[0]/D] \
    <src_period_ns>

```

2. Data bus (quasi-static, protected by toggle handshake)

```

set_max_delay -datapath_only \
    -from [get_pins u_cdc/r_src_data_hold_reg[*]/C] \
    -to   [get_pins u_cdc/r_dst_data_reg[*]/D] \
    <dst_period_ns>

```

Do NOT use set_false_path – the data bus relies on a bounded crossing window for correctness.

26.7 Related Modules

Module	Purpose
cdc_4_phase_handshake.sv	Level-based variant (slower, simpler)
cdc_synchronizer.sv	Multi-bit sync (no handshake, quasi-static only)
sync_pulse.sv	Single-cycle pulse crossing (no data)
fifo_async.sv	Streaming data crossing (Gray-coded pointers)
apb_slave_cdc.sv	APB protocol CDC (uses either handshake variant)

26.8 Resources

- RTL: rtl/amba/shared/cdc_2_phase_handshake.sv
 - Tests: val/amba/test_cdc_2_phase_handshake.py
 - Formal: formal/amba/cdc_handshake/
-

Last Updated: 2026-04-21

26.9 Navigation

- [Back to Shared Infrastructure Index](#)
- [Back to RTLAmba Index](#)

27 CDC 4-Phase Handshake

Module: cdc_4_phase_handshake.sv (renamed from cdc_4_phase_handshake.sv) **Location:** rtl/amba/shared/ **Status:** Production Ready **Companion:** cdc_2_phase_handshake.sv (toggle-based, faster variant)

27.1 Overview

The CDC Handshake module provides robust clock domain crossing (CDC) with valid/ready flow control protocol. It implements a dual FSM architecture with 3-stage synchronizers to safely transfer data between asynchronous clock domains while maintaining AXI-style handshaking semantics.

27.1.1 Key Features

- Safe asynchronous clock domain crossing with metastability protection
 - AXI-compatible valid/ready handshake protocol
 - 3-stage synchronizer chains for REQ/ACK signals
 - Dual FSM architecture (source and destination domains)
 - Parameterizable data width
 - Independent reset for each clock domain
 - Proven REQ/ACK protocol for glitch-free operation
-

27.2 Module Purpose

Modern SoC designs frequently require data transfer between asynchronous clock domains. Direct signal crossing risks metastability, data corruption, and protocol violations. The CDC Handshake module solves these challenges by:

1. **Metastability Protection:** Multi-stage synchronizers for control signals
2. **Data Integrity:** Stable data capture with proper handshaking
3. **Flow Control:** Backpressure support via ready signals
4. **Protocol Correctness:** AXI-compatible valid/ready semantics

Use Cases: - AXI interface clock domain crossing (monitor packets, control signals) - Data transfer between processor cores and peripherals - Multi-clock SoC interconnect boundaries - Configuration register updates across clock domains

Key Guarantee: Data transferred without corruption or loss, with proper flow control.

27.3 Parameters

Parameter	Type	Default	Description
DATA_WIDTH	int	8	Width of the data bus (1 to 1024+ bits typical)
SYNC_STAGES	int	3	Synchronizer depth for req/ack (2 or 3)
TIMEOUT_CYCLES	int	0	0 = disabled; >0 asserts src_timeout after stall
FAST_PATH	bit	0	1 = destination fast-path when dst_ready already high

Typical Values: - Control signals: 8-32 bits - AXI monitor packets: 64 bits - Configuration registers: 32-64 bits - Custom data structures: Application-specific

27.4 Port Groups

27.4.1 Source Clock Domain Interface

Port	Direction	Width	Description
clk_src	input	1	Source domain clock (can be asynchronous to clk_dst)
rst_src_n	input	1	Source domain active-low asynchronous reset
src_valid	input	1	Source indicates data valid (AXI-style valid)
src_ready	output	1	Handshake ready back to source (asserted when transfer complete)
src_data	input	DATA_WIDTH	Data from source domain (captured when valid asserted)

27.4.2 Destination Clock Domain Interface

Port	Direction	Width	Description
clk_dst	input	1	Destination domain clock (can be asynchronous to clk_src)
rst_dst_n	input	1	Destination domain active-low asynchronous reset
dst_valid	output	1	Destination indicates data valid to receiver (AXI-style valid)
dst_ready	input	1	Receiver ready in destination domain (AXI-style ready)
dst_data	output	DATA_WIDTH	Data transferred to destination domain (stable when valid)

27.5 Functional Description

27.5.1 CDC Protocol Overview

The module implements a four-phase handshake protocol:

1. **Request Phase:** Source asserts REQ, latches data
2. **Synchronization Phase:** REQ crosses to destination domain (3 stages)
3. **Acknowledge Phase:** Destination asserts ACK after accepting data
4. **Synchronization Phase:** ACK crosses back to source domain (3 stages)
5. **Completion Phase:** Both sides deassert signals, ready for next transfer

Critical Property: Data stable throughout handshake (captured at REQ assertion, held until ACK completion).

27.5.2 Source Domain FSM

State Diagram:

S_IDLE → S_WAIT_ACK → S_WAIT_ACK_CLR → S_IDLE

State Descriptions:

S_IDLE: - Awaits src_valid assertion - src_ready = 1 (ready for new data) - r_req_src = 0 (request inactive)

S_WAIT_ACK: - Transfer in progress - src_ready = 0 (busy) - r_req_src = 1 (request asserted) - Data latched in r_async_data - Awaits synchronized ACK from destination

S_WAIT_ACK_CLR: - ACK received, completing handshake - src_ready = 0 (still busy) - r_req_src = 0 (request deasserted) - Awaits ACK to clear (return to 0)

State Transitions:

```
S_IDLE:
    if (src_valid):
        r_async_data <= src_data           // Capture data
        r_req_src <= 1                     // Assert request
        src_ready <= 0
        → S_WAIT_ACK

S_WAIT_ACK:
    if (w_ack_sync):                      // ACK received
        r_req_src <= 0                     // Deassert request
        → S_WAIT_ACK_CLR
```

```

    else:
        remain in S_WAIT_ACK

S_WAIT_ACK_CLR:
    if (!w_ack_sync):           // ACK cleared
        src_ready <= 1         // Ready for next transfer
        → S_IDLE
    else:
        remain in S_WAIT_ACK_CLR

```

27.5.3 Destination Domain FSM

State Diagram:

D_IDLE → D_WAIT_READY → D_WAIT_REQ_CLR → D_IDLE

State Descriptions:

D_IDLE: - Awaits synchronized REQ from source - dst_valid = 0 (no data available) - r_ack_dst = 0 (acknowledge inactive)

D_WAIT_READY: - REQ detected, data available - dst_valid = 1 (presenting data to receiver) - r_ack_dst = 0 (waiting for receiver acceptance) - Awaits dst_ready assertion

D_WAIT_REQ_CLR: - Data accepted, handshake completing - dst_valid = 0 (transfer complete) - r_ack_dst = 1 (acknowledge asserted) - Awaits REQ to clear (return to 0)

State Transitions:

```

D_IDLE:
    if (w_req_sync):           // REQ received
        r_dst_data <= r_async_data // Latch data
        dst_valid <= 1         // Present to receiver
        → D_WAIT_READY

D_WAIT_READY:
    if (dst_ready):           // Receiver accepts data
        r_ack_dst <= 1         // Assert acknowledge
        dst_valid <= 0         // Clear valid
        → D_WAIT_REQ_CLR
    else if (!w_req_sync):    // REQ withdrawn (error case)
        dst_valid <= 0
        → D_IDLE

D_WAIT_REQ_CLR:
    if (!w_req_sync):         // REQ cleared
        r_ack_dst <= 0         // Deassert acknowledge
        → D_IDLE

```

```

else:
    remain in D_WAIT_REQ_CLR

```

27.5.4 Synchronizer Architecture

3-Stage Synchronizers for Metastability Protection:

REQ Synchronizer (Source → Destination):

```

always_ff @(posedge clk_dst or negedge rst_dst_n) begin
    if (!rst_dst_n)
        r_req_sync <= 3'b000;
    else
        r_req_sync <= {r_req_sync[1:0], r_req_src};
end

```

```

assign w_req_sync = r_req_sync[2]; // Use final stage

```

ACK Synchronizer (Destination → Source):

```

always_ff @(posedge clk_src or negedge rst_src_n) begin
    if (!rst_src_n)
        r_ack_sync <= 3'b000;
    else
        r_ack_sync <= {r_ack_sync[1:0], r_ack_dst};
end

```

```

assign w_ack_sync = r_ack_sync[2]; // Use final stage

```

Why 3 Stages? - Stage 1: May go metastable (timing violation from async input) - Stage 2: Metastability resolves (high probability) - Stage 3: Clean synchronized signal (used by FSM)

MTBF Calculation:

$$\text{MTBF} = e^{(T_r/\tau)} / (f_{\text{clk}} \times f_{\text{data}} \times \tau)$$

Where:

- T_r : Resolution time (2 clock periods for 2-stage sync)
- τ : Metastability time constant (technology-dependent, ~100ps)
- f_{clk} : Destination clock frequency
- f_{data} : Data change frequency

3-stage sync provides astronomical MTBF (years to millennia)

27.5.5 Data Stability Guarantee

Critical Timing:

1. Data Capture (Source Domain):

- Data captured when src_valid asserted in S_IDLE
 - Stored in r_async_data register
 - Held stable until handshake completes
2. **Data Transfer (Asynchronous):**
- r_async_data crosses clock domains (static signal)
 - No timing violations (data stable throughout)
3. **Data Latch (Destination Domain):**
- Data latched into r_dst_data when REQ synchronized
 - Held stable until dst_ready asserted

Key Property: Data never changes while crossing domains (setup/hold violations impossible).

27.6 Usage Example

27.6.1 Basic CDC for Monitor Packets (64-bit)

```
// Monitor packets generated in monitor clock domain
// Must cross to system clock domain for processing

cdc_4_phase_handshake #(
    .DATA_WIDTH      (64) // AXI monitor packet width
) u_packet_cdc (
    // Source: Monitor clock domain
    .clk_src          (mon_clk),
    .rst_src_n        (mon_rst_n),
    .src_valid         (mon_pkt_valid),
    .src_ready         (mon_pkt_ready),
    .src_data          (mon_pkt_data),

    // Destination: System clock domain
    .clk_dst           (sys_clk),
    .rst_dst_n         (sys_rst_n),
    .dst_valid         (sys_pkt_valid),
    .dst_ready         (sys_pkt_ready),
    .dst_data          (sys_pkt_data)
);

// Source domain: Monitor packet generation
always_ff @(posedge mon_clk or negedge mon_rst_n) begin
    if (!mon_rst_n) begin
        mon_pkt_valid <= 1'b0;
    end else begin
```

```

        if (packet_generated && mon_pkt_ready) begin
            mon_pkt_valid <= 1'b1;
            mon_pkt_data <= new_packet;
        end else if (mon_pkt_ready) begin
            mon_pkt_valid <= 1'b0;
        end
    end
end

// Destination domain: Packet processing
always_ff @(posedge sys_clk or negedge sys_rst_n) begin
    if (!sys_rst_n) begin
        sys_pkt_ready <= 1'b0;
    end else begin
        if (sys_pkt_valid && processing_ready) begin
            sys_pkt_ready <= 1'b1;
            process_packet(sys_pkt_data);
        end else begin
            sys_pkt_ready <= 1'b0;
        end
    end
end
end

```

27.6.2 Configuration Register Update Across Domains

```

// CPU writes config register in AHB clock domain
// Design uses config in AXI clock domain

cdc_4_phase_handshake #(
    .DATA_WIDTH          (32) // Configuration register width
) u_config_cdc (
    // Source: CPU/AHB clock domain
    .clk_src              (ahb_clk),
    .rst_src_n            (ahb_rst_n),
    .src_valid            (cfg_write_valid),
    .src_ready            (cfg_write_ready),
    .src_data             (cfg_write_data),

    // Destination: AXI logic clock domain
    .clk_dst              (axi_clk),
    .rst_dst_n            (axi_rst_n),
    .dst_valid            (cfg_update_valid),
    .dst_ready            (cfg_update_ready),
    .dst_data             (cfg_update_data)
);

// Source: AHB write decoder
always_ff @(posedge ahb_clk) begin

```

```

    if (ahb_write && addr_match && cfg_write_ready) begin
        cfg_write_valid <= 1'b1;
        cfg_write_data <= ahb_wdata;
    end else if (cfg_write_ready) begin
        cfg_write_valid <= 1'b0;
    end
end

// Destination: Configuration register update
always_ff @(posedge axi_clk or negedge axi_rst_n) begin
    if (!axi_rst_n) begin
        config_reg <= 32'h0;
        cfg_update_ready <= 1'b0;
    end else begin
        if (cfg_update_valid) begin
            config_reg <= cfg_update_data;
            cfg_update_ready <= 1'b1;
        end else begin
            cfg_update_ready <= 1'b0;
        end
    end
end
end

```

27.6.3 Burst Transfer with Backpressure

// Transfer burst of data with flow control

```

cdc_4_phase_handshake #(
    .DATA_WIDTH          (128) // Large data path
) u_burst_cdc (
    .clk_src              (src_clk),
    .rst_src_n            (src_rst_n),
    .src_valid            (burst_valid),
    .src_ready            (burst_ready),
    .src_data             (burst_data),

    .clk_dst              (dst_clk),
    .rst_dst_n            (dst_rst_n),
    .dst_valid            (fifo_wr_valid),
    .dst_ready            (fifo_wr_ready),
    .dst_data             (fifo_wr_data)
);

```

// Source: Burst generator

```

logic [7:0] burst_count;

always_ff @(posedge src_clk or negedge src_rst_n) begin
    if (!src_rst_n) begin

```



```

        burst_count <= 8'd0;
        burst_valid <= 1'b0;
    end else begin
        if (start_burst) begin
            burst_count <= burst_length;
            burst_valid <= 1'b1;
            burst_data <= first_data;
        end else if (burst_valid && burst_ready) begin
            if (burst_count > 1) begin
                burst_count <= burst_count - 1;
                burst_data <= next_data;
                burst_valid <= 1'b1;
            end else begin
                burst_valid <= 1'b0;
            end
        end
    end
end

// Destination: FIFO with backpressure
assign fifo_wr_ready = !fifo_full;

```

27.7 Design Notes

27.7.1 Handshake Protocol Timing

Latency Characteristics:

Minimum Transfer Latency (Fast Case): 1. Source asserts REQ (clk_src cycle N) 2. REQ synchronization: 3 clk_dst cycles 3. Destination asserts dst_valid (clk_dst cycle N+3) 4. Destination samples dst_ready (clk_dst cycle N+3 or later) 5. Destination asserts ACK (clk_dst cycle N+4) 6. ACK synchronization: 3 clk_src cycles 7. Source sees ACK (clk_src cycle N+7) 8. Source ready for next transfer (clk_src cycle N+8)

Total: 6-8 cycles in fast clock + 3-4 cycles in slow clock

Maximum Transfer Latency (Slow Case): - Add dst_ready assertion delay (unbounded, depends on receiver) - Add clock frequency ratio effects

Throughput: - Limited by handshake round-trip latency - Not suitable for high-throughput streaming (use async FIFO instead) - Optimal for control signals, configuration, low-rate data

27.7.2 Reset Considerations

Independent Domain Resets: - Each FSM has domain-specific reset - Resets can be asynchronous to each other - Design handles reset in one domain while other operates

Reset Recovery: 1. Source domain reset: FSM returns to S_IDLE, REQ=0 2. Destination domain reset: FSM returns to D_IDLE, ACK=0 3. Any in-flight handshake abandoned (safe by design) 4. Synchronizers clear to 3'b000 (deasserted state)

Recommendation: Assert both resets together during system initialization for clean startup.

27.7.3 Data Width Considerations

Wide Data Buses (64+ bits): - Large fanout from r_async_data register - Consider pipelining if timing closure issues - Area cost: DATA_WIDTH × 2 flip-flops (r_async_data + r_dst_data)

Narrow Data Buses (1-8 bits): - Minimal area overhead - Still requires full handshake protocol (latency same as wide buses)

Alternative for Streaming: If high-throughput needed, use async FIFO instead.

27.7.4 Metastability Protection Validation

Synchronizer Verification:

Static Timing Analysis: - Ensure no timing paths between asynchronous domains (except through synchronizers) - Verify synchronizer stages have no combinational logic - Check recovery/removal timing on first stage (expect violations, safe by design)

Formal Verification:

```
// CDC assertion: REQ/ACK only change in respective domains
assert property (@(posedge clk_src)
    !$rose(r_ack_dst) && !$fell(r_ack_dst)); // ACK only changes in
dst domain

assert property (@(posedge clk_dst)
    !$rose(r_req_src) && !$fell(r_req_src)); // REQ only changes in
src domain
```

Simulation: - Use formal CDC checkers (Synopsys SpyGlass, Cadence JasperGold) - Verify no X-propagation in synchronizer stages

27.7.5 Performance Optimization

When to Use CDC Handshake: - Control signals (low rate, <1% of clock frequency) - Configuration registers (infrequent updates) - Status synchronization (occasional transfers)

When to Use Async FIFO Instead: - High-throughput data streaming - Continuous data flow - Burst transfers with sustained rate

Hybrid Approach:

```
// Use async FIFO for data, CDC handshake for control
async_fifo #(.WIDTH(64), .DEPTH(16)) u_data_fifo (...);
cdc_4_phase_handshake #(.WIDTH(8)) u_ctrl_cdc (...);
```

27.8 Related Modules

27.8.1 Used By

- AXI monitor bus aggregation (packet crossing between monitor and system clocks)
- Configuration register synchronization
- Status/control signal crossing in multi-clock systems
- Peripheral interface adapters with asynchronous clocks

27.8.2 Dependencies

- **reset_defs.svh** - Standard reset macro definitions

27.8.3 See Also

- **async_fifo** - High-throughput async FIFO for streaming data
 - **reset_sync.sv** - Reset synchronizer for single-bit reset crossing
-

27.9 References

27.9.1 Source Code

- RTL: rtl/amba/shared/cdc_4_phase_handshake.sv
- Tests: val/amba/test_cdc_4_phase_handshake.py

27.9.2 Documentation

- Architecture: docs/markdown/RTLAmba/shared/README.md
- CDC Best Practices: Clifford E. Cummings, “Clock Domain Crossing (CDC) Design & Verification Techniques”
- Design Guide: rtl/amba/PRD.md

27.9.3 Industry Standards

- IEEE 1364/1800 - Synchronizer modeling
- AMBA specifications - CDC requirements for multi-clock systems

27.10 Navigation

- [Back to Shared Infrastructure Index](#)
- [Back to RTLamba Index](#)

28 CDC Open-Loop

Module: `cdc_open_loop.sv` **Location:** `rtl/amba/shared/` **Category:** Open-loop CDC (see [CDC Primer](#))

28.1 Overview

Open-loop multi-bit CDC using valid/data stretching. The source captures data on a single-cycle `src_valid` pulse and holds both data and a stretched valid level for `STRETCH_CYCLES` source clock cycles. The destination synchronizes the stretched valid and latches data on its rising edge.

No return path or handshake exists. The source is busy for exactly `STRETCH_CYCLES` source clocks, then free to send again. The stretch length can either be set manually (`STRETCH_CYCLES`) or computed from the source/destination clock frequencies (`AUTO_STRETCH=1` with `SRC_CLK_HZ` and `DST_CLK_HZ`).

28.1.1 Key Features

- True open-loop: no return path, no handshake, no backpressure
- Source holds data and stretched valid for a parameterizable number of source clocks
- Destination uses multi-stage synchronizer + rising edge detect to latch data
- `src_busy` blocks the next transfer during the stretch window
- Optional auto-compute: pass clock frequencies and the module sizes `STRETCH` itself
- Minimal area: one counter, one synchronizer, one edge detector

28.1.2 When to Use

- Clock ratio is known and fixed (or bounded)

- Destination is always ready (no flow control needed)
- Configuration register updates, status flags, infrequent events
- Minimum area/complexity is desired

28.1.3 When NOT to Use

- Clock ratio unknown or variable – use `cdc_2_phase_handshake` (closed-loop)
 - Destination needs backpressure – use a handshake
 - Streaming data – use `fifo_async` or `gaxi_fifo_async`
-

28.2 Module Interface

```

module cdc_open_loop #(
    parameter int DATA_WIDTH      = 8,
    parameter int STRETCH_CYCLES  = 8,
    parameter int SYNC_STAGES     = 2,

    // Optional auto-compute of STRETCH_CYCLES from clock frequencies
    parameter int AUTO_STRETCH = 0,           // 0 = use
    STRETCH_CYCLES; 1 = auto
    parameter int SRC_CLK_HZ      = 25_000_000, // Fastest source
    clock to handle
    parameter int DST_CLK_HZ      = 100_000_000 // Destination clock
    frequency
) (
    // Source clock domain
    input logic clk_src,
    input logic rst_src_n,
    input logic src_valid, // Single-cycle pulse
    input logic [DATA_WIDTH-1:0] src_data, // Data to transfer
    output logic src_busy, // High during
    stretch

    // Destination clock domain
    input logic clk_dst,
    input logic rst_dst_n,
    output logic dst_valid, // Single-cycle pulse
    output logic [DATA_WIDTH-1:0] dst_data // Latched data
);

```

28.3 Parameters

Parameter	Default	Description
DATA_WIDTH	8	Width of the data bus
STRETCH_CYCLES	8	Source clocks to hold valid+data stable (used when AUTO_STRETCH=0)
SYNC_STAGES	2	Destination synchronizer depth (2-4; raise for extra MTBF margin)
AUTO_STRETCH	0	0 = use STRETCH_CYCLES; 1 = compute from SRC_CLK_HZ / DST_CLK_HZ
SRC_CLK_HZ	25_000_000	Source clock frequency (used when AUTO_STRETCH=1)
DST_CLK_HZ	100_000_000	Destination clock frequency (used when AUTO_STRETCH=1)

28.3.1 Setting STRETCH_CYCLES manually (AUTO_STRETCH=0)

This is a multi-bit data CDC, so the source data bus must remain stable until the synchronized enable has propagated through the destination sync chain *and* loaded the destination register. That gives:

$$\text{STRETCH_CYCLES} \geq \text{ceil}((\text{SYNC_STAGES} + 1) * T_{\text{dst}} / T_{\text{src}})$$

(SYNC_STAGES + 1) counts the synchronizer FFs plus the destination load edge. The $1.5 \times T_{\text{dst}}$ “rule of thumb” you may have seen is enough to capture the valid pulse alone, but a multi-bit datapath needs the longer hold so `r_src_data` is still stable when the destination latches it. See *References* below.

Source Clock	Dest Clock	Ratio	SYNC_STAGE S	Min STRETCH_CY CLES
100 MHz	100 MHz	1:1	2	3
100 MHz	50 MHz	2:1	2	6
100 MHz	25 MHz	4:1	2	12
1 GHz	100 MHz	10:1	2	30
200 MHz	25 MHz	8:1	2	24

When in doubt, round up — extra stretch cycles cost only source-side latency, not area.

28.3.2 Auto-computed STRETCH (AUTO_STRETCH=1)

Set `AUTO_STRETCH=1` and pass the source/destination clock frequencies in Hz. The module computes `STRETCH_CYCLES` at elaboration time using:

$$\text{STRETCH} = \text{ceil}((\text{SYNC_STAGES} + 1) * \text{SRC_CLK_HZ} / \text{DST_CLK_HZ})$$

The result is a localparam — folds to a literal in synthesis, no runtime cost. `SRC_CLK_HZ` should be the **fastest** source clock you intend to support; sources running faster will start dropping pulses.

When `AUTO_STRETCH=1` the `STRETCH_CYCLES` parameter is ignored. When `AUTO_STRETCH=0` (default) the auto-formula is dead code and `STRETCH_CYCLES` is used as-is, so existing callers see no change.

Source SRC_CLK_HZ	Destination DST_CLK_HZ	Computed STRETCH (SYNC=2)
25 MHz	100 MHz	1
50 MHz	100 MHz	2
100 MHz	100 MHz	3
100 MHz	40 MHz	8
100 MHz	25 MHz	12
1 GHz	100 MHz	30

Prefer auto when the clock plan is captured in a single top-level location — propagate `SRC_CLK_HZ / DST_CLK_HZ` as parameters from the top and every instance retunes itself if the clock plan changes.

28.3.3 Back-to-back pulses

`src_busy` only guards the HIGH stretch window. After it falls, the destination sync chain still needs roughly $(\text{SYNC_STAGES} + 1) * T_{\text{dst}}$ to see the falling edge before another rising edge can be detected. If the source asserts the next `src_valid` immediately after `src_busy` falls, two HIGH stretches can merge in the dst view and the second pulse is silently dropped.

Two ways to keep this safe:

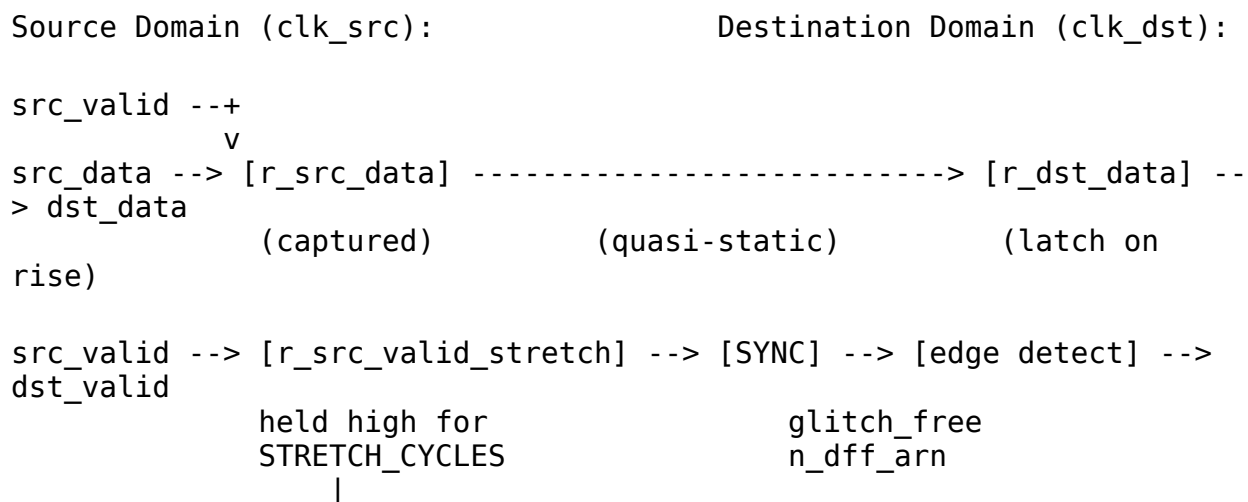
1. Pick `STRETCH_CYCLES` large enough that one full source-side LOW cycle is comfortably more than $(\text{SYNC_STAGES} + 1) * T_{\text{dst}}$. (For slow-source → fast-dst this is automatic; for fast-source → slow-dst you may need extra margin.)
2. Have the source pace pulses: wait `src_busy` to fall, then idle for $\text{ceil}((\text{SYNC_STAGES} + 1) * T_{\text{dst}} / T_{\text{src}}) + 1$ source cycles before the next `src_valid`.

The verification testbench (bin/TBClasses/amba/cdc_open_loop.py) enforces option 2 in its no-loss phases.

28.4 Ports

Port	Direction	Width	Description
clk_src	input	1	Source domain clock
rst_src_n	input	1	Source async reset (active low)
src_valid	input	1	Single-cycle pulse: data is valid
src_data	input	DATA_WIDTH	Data to transfer
src_busy	output	1	High during stretch countdown
clk_dst	input	1	Destination domain clock
rst_dst_n	input	1	Destination async reset (active low)
dst_valid	output	1	Single-cycle pulse: data latched
dst_data	output	DATA_WIDTH	Latched data (stable until next dst_valid)

28.5 Architecture

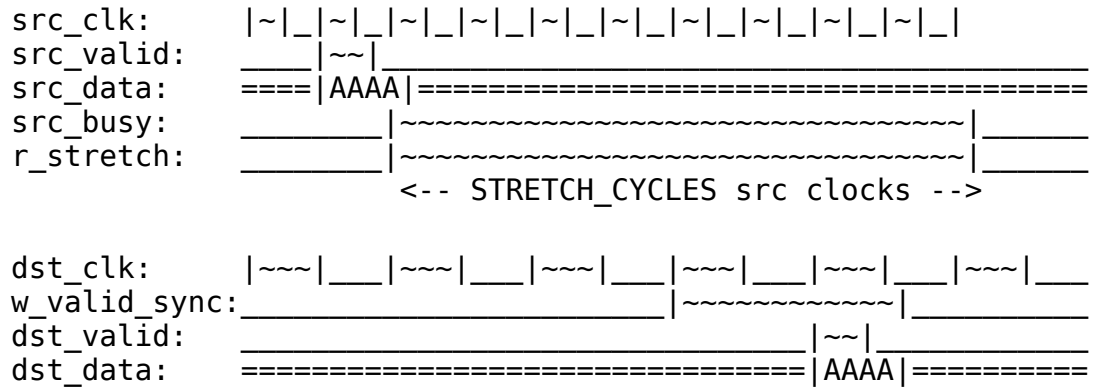



```

[r_stretch_count]
countdown to 0
|
src_busy = (count != 0)

```

28.5.1 Timing Diagram



28.6 Usage Example

28.6.1 Manual STRETCH (back-compat default)

```

// CPU writes config register at 100 MHz, peripheral runs at 25 MHz
// Ratio = 4:1, STRETCH_CYCLES = ceil((2+1) * 4) = 12
cdc_open_loop #(
    .DATA_WIDTH      (32),
    .STRETCH_CYCLES  (12),
    .SYNC_STAGES     (2)
) u_cfg_cdc (
    .clk_src   (cpu_clk),           // 100 MHz
    .rst_src_n (cpu_rst_n),
    .src_valid (cfg_write_en),
    .src_data  (cfg_write_data),
    .src_busy  (cfg_busy),          // Block next write

    .clk_dst   (periph_clk),        // 25 MHz
    .rst_dst_n (periph_rst_n),
    .dst_valid (cfg_update),
    .dst_data  (cfg_value)
);

```

28.6.2 Auto STRETCH from clock frequencies

```

// Same CPU->peripheral CDC, but let the module size STRETCH itself.
// If the clock plan changes (e.g. CPU bumped to 200 MHz) you only
// update SRC_CLK_HZ at the top and every instance retunes.

```

```
localparam int CPU_HZ      = 100_000_000;
localparam int PERIPH_HZ   = 25_000_000;
```

```
cdc_open_loop #(
    .DATA_WIDTH    (32),
    .SYNC_STAGES   (2),
    .AUTO_STRETCH  (1),
    .SRC_CLK_HZ     (CPU_HZ),
    .DST_CLK_HZ     (PERIPH_HZ)
) u_cfg_cdc (
    .clk_src      (cpu_clk),
    .rst_src_n    (cpu_rst_n),
    .src_valid     (cfg_write_en),
    .src_data      (cfg_write_data),
    .src_busy      (cfg_busy),

    .clk_dst      (periph_clk),
    .rst_dst_n    (periph_rst_n),
    .dst_valid     (cfg_update),
    .dst_data      (cfg_value)
);
```

28.7 SDC Constraints

1. Stretched valid (single-bit level, synchronized by destination)

```
set_max_delay -datapath_only \
    -from [get_pins u_cdc/r_src_valid_stretch_reg/C] \
    -to   [get_pins u_cdc/u_valid_sync/q_reg[0]/D] \
    <dst_period_ns>
```

2. Data bus (quasi-static, held for STRETCH_CYCLES src clocks)

```
set_max_delay -datapath_only \
    -from [get_pins u_cdc/r_src_data_reg[*]/C] \
    -to   [get_pins u_cdc/r_dst_data_reg[*]/D] \
    <dst_period_ns>
```

Do NOT use set_false_path – the data bus must arrive within a bounded window.

28.8 Resources

- RTL: rtl/amba/shared/cdc_open_loop.sv
- Depends on: rtl/common/glitch_free_n_dff_arn.sv
- Testbench: bin/TBClasses/amba/cdc_open_loop.py

- Test runner: `val/amba/test_cdc_open_loop.py` (28 configs covering manual, auto, and cliff)
- CDC Primer: [cdc_primer.md](#)
- Closed-loop alternative: [cdc_2_phase_handshake.md](#)

28.9 References

The $(\text{SYNC_STAGES} + 1) \times T_{\text{dst}}$ data-hold requirement follows directly from the MCP (Multi-Cycle Path) formulation: the source data must stay stable until the synchronized enable has traversed the destination sync chain and loaded the destination register. The exact closed-form formula is this module's own — the cited references state the principle qualitatively and leave the cycle count to the reader.

- Clifford E. Cummings, *Clock Domain Crossing (CDC) Design & Verification Techniques Using SystemVerilog*, SNUG Boston 2008 — original MCP formulation.
 - Verilog Pro, *Clock Domain Crossing Design — Part 2*: “the input data has to be held until the synchronization pulse loads the data in the destination clock domain. The whole process takes at least two destination clocks.” (= $\text{SYNC_STAGES} + 1$ destination cycles for a 2-FF synchronizer.)
 - thedatabus.in, *Clock Domain Crossing (CDC): The Complete Reference Guide*: “ensure that the source holds the information long enough for it to be safely taken through all the synchronizer stages.”
 - EDN, *Understanding Clock Domain Crossing*.
-

Last Updated: 2026-06-26

28.10 Navigation

- [Back to CDC Primer](#)
- [Back to Shared Infrastructure Index](#)

29 CDC Primer: Clock Domain Crossing Techniques

This document categorizes the clock domain crossing approaches available in RTL Design Sherpa and provides guidance on when to use each one. All modules listed here are production-ready with formal verification.

29.1 The Three Vector CDC Categories

When you need to move multi-bit data across clock domains, there are three fundamental approaches. They differ in how they guarantee that the destination samples stable data.

Category	Stability Guarantee	Throughput	Complexity	Use When
Open-Loop	Source holds data stable	Low (~1 transfer per 6 dst clocks)	Minimal	Config registers, infrequent updates
Closed-Loop Handshake	Explicit req/ack acknowledgment	Medium (~1 per 5-8 dst clocks)	Moderate	Control signals, command/response
Async FIFO	Gray-coded pointer synchronization	High (1 per dst clock sustained)	Higher	Streaming data, DMA, packet transfer

29.2 Category 1: Open-Loop CDC

Principle: The source asserts a single-cycle valid pulse and holds data stable. A synchronized copy of the valid pulse propagates to the destination domain, which latches the data when the pulse arrives. There is no acknowledge path – the source must guarantee the hold time.

When it works: The source naturally holds data for multiple destination clock periods. This is common for: - Fast source to slow destination (the fast domain's data persists for many slow clocks) - Configuration register writes (software writes once, hardware reads later) - Status flags that change infrequently

When it fails: If the source can change data before the destination has sampled it. Without an acknowledge, there is no backpressure.

29.2.1 Module: `cdc_open_loop`

```
// CPU at 100 MHz, peripheral at 25 MHz (ratio 4:1)
// STRETCH_CYCLES = ceil(1.5 * 4) + 3 = 9
cdc_open_loop #(
    .DATA_WIDTH      (32),
    .STRETCH_CYCLES  (9),
```

```

        .SYNC_STAGES      (3)
    ) u_cfg_cdc (
        .clk_src      (cpu_clk),
        .rst_src_n    (cpu_rst_n),
        .src_valid     (cfg_write_pulse),    // Single-cycle pulse
        .src_data      (cfg_write_data),
        .src_busy      (cfg_busy),           // Blocks next write during stretch

        .clk_dst      (periph_clk),
        .rst_dst_n    (periph_rst_n),
        .dst_valid     (cfg_update_pulse),    // Single-cycle in dst domain
        .dst_data      (cfg_update_data)     // Latched, stable until next
update
    );

```

Internals: Source captures data into a holding register and asserts a stretched valid level for STRETCH_CYCLES source clocks. The destination synchronizes the stretched level through glitch_free_n_dff_arn and latches data on the rising edge. src_busy blocks the next transfer during the stretch countdown.

Setting STRETCH_CYCLES: $\text{STRETCH_CYCLES} \geq \text{ceil}(1.5 * T_{\text{dst}} / T_{\text{src}}) + \text{SYNC_STAGES}$. This ensures the stretched valid is seen by at least one destination clock edge after passing through the synchronizer. When in doubt, round up – extra cycles cost only source-side latency.

Documentation: [cdc_open_loop.md](#)

29.2.2 Supporting Primitives

Module	Purpose	Doc
glitch_free_n_dff_arn	N-stage synchronizer primitive with ASYNC_REG	rtl/common/
cdc_synchronizer	Multi-flop synchronizer wrapper for quasi-static signals	rtl/amba/shared/
sync_pulse	Single-cycle pulse crossing (toggle + edge detect)	rtl/common/
reset_sync	Reset synchronizer (async assert, sync deassert)	rtl/common/

29.3 Category 2: Closed-Loop Handshake CDC

Principle: The source asserts a request and holds data stable. The destination synchronizes the request, samples the data, and sends back an acknowledge. The source waits for the acknowledge

before releasing the data or starting a new transfer. This explicit round-trip guarantees data stability regardless of clock frequency ratios.

Two variants exist, differing in how the request/acknowledge signals encode “new transfer”:

29.3.1 2-Phase (Toggle / NRZ)

Each new transfer is signaled by a **transition** (toggle) of the request bit. The destination detects the edge and toggles its acknowledge bit in response. Two synchronizer crossings per transfer (req toggle + ack toggle).

Source: data stable, toggle req ----[sync]----> Destination sees
edge, latches data
toggle ack ----
[sync]----> Source sees edge, done

Advantage: Faster – half the control-path round trip of 4-phase. **Disadvantage:** Edge detection is slightly trickier to reason about.

29.3.2 4-Phase (Level / RZ)

Each transfer uses full rise-and-fall cycles: req rises, ack rises, req falls, ack falls. Four synchronizer crossings per transfer.

Source: data stable, req=1 ----[sync]----> Destination sees req=1,
latches data
ack=1 ----[sync]---->
Source sees ack=1, req=0
ack=0 (after req=0
propagates) ----> Ready for next

Advantage: Simpler level-based logic, easier to verify. **Disadvantage:** Slower – four crossings instead of two.

29.3.3 Modules

Module	Variant	Latency	Doc
cdc_2_phase_handshake	Toggle (NRZ)	~5-6 dst clocks	rtl/amba/ shared/
cdc_4_phase_handshake	Level (RZ)	~7-8 dst clocks	rtl/amba/ shared/

Both modules have identical port interfaces and are drop-in interchangeable:

```
// Change only the module name to switch variants
cdc_2_phase_handshake #(    // or cdc_4_phase_handshake
    .DATA_WIDTH      (64),
    .SYNC_STAGES     (3),
```

```

        .TIMEOUT_CYCLES (1000)
    ) u_pkt_cdc (
        .clk_src      (mon_clk),
        .rst_src_n    (mon_rst_n),
        .src_valid     (pkt_valid),
        .src_ready     (pkt_ready),    // Backpressure to source
        .src_data      (pkt_data),
        .src_timeout   (pkt_timeout),  // Stall detection

        .clk_dst      (sys_clk),
        .rst_dst_n    (sys_rst_n),
        .dst_valid     (rx_valid),
        .dst_ready     (rx_ready),
        .dst_data      (rx_data)
    );

```

29.3.4 Protocol-Level CDC Using Handshakes

The handshake modules are building blocks for protocol-level CDC:

Module	Protocol	Handshake Used	Doc
apb_slave_cdc	APB4	Selectable 2-phase or 4-phase	rtl/ amba/ apb/
apb5_slave_cdc	APB5	Selectable 2-phase or 4-phase	rtl/ amba/ apb5/

29.4 Category 3: Gray-Coded Pointers / Async FIFO

Principle: Instead of synchronizing data directly, synchronize the read and write **pointers** of a dual-port memory. The pointers are Gray-coded so that only one bit changes per increment. This means any sample during a transition yields either the old or new pointer value – never a corrupted intermediate. The memory itself is dual-ported (one port per clock domain) and requires no synchronization.

This is the dominant technique for streaming data across clock domains. If you are moving continuous data rather than occasional transfers, this is what you reach for.

29.4.1 How It Works

Write Domain: wr_ptr (binary) ---> bin2gray ---> rd_ptr	[sync N stages]	Read Domain: gray2bin ---> Compare with for empty
--	------------------------	---

detection

```
gray2bin <--- [sync N stages] <--- bin2gray <--- rd_ptr (binary)
Compare with wr_ptr for full detection
```

1. Each domain maintains its own pointer in binary (for memory addressing)
2. Each pointer is converted to Gray code (for safe CDC crossing)
3. The Gray-coded pointer is synchronized into the other domain
4. Full/empty flags are derived by comparing the local pointer with the synchronized remote pointer
5. The dual-port memory is addressed by the binary pointers directly (no CDC needed for data)

29.4.2 Standard Gray Code vs Johnson Counter

The repository provides two async FIFO implementations that differ in how they encode pointers:

Standard Gray Code (`fifo_async`): Uses `bin2gray` / `gray2bin` converters with standard binary-reflected Gray code. Depth must be a power of 2 (because standard Gray codes only have single-bit transitions for power-of-2 counts).

Johnson Counter (`fifo_async_div2`): Uses a Johnson counter sequence, which is a subset of Gray code that supports even (non-power-of-2) depths. A Johnson counter of N bits cycles through 2N states, each differing by one bit. The `grayj2bin` converter decodes the Johnson sequence back to binary for pointer comparison.

FIFO	Pointer Encoding	Depth Constraint	Best For
fifo_async	Standard Gray code	Power of 2 only	Most use cases
fifo_async_div2	Johnson counter	Any even number	Odd-sized buffers, area optimization

29.4.3 Sizing the Depth

Given a write/read clock ratio and a burst pattern, the minimum FIFO depth to avoid dropping writes is:

$$\text{depth} \geq \text{burst_length} * (1 - \text{read_rate} / \text{write_rate})$$

where $\text{rate} = \text{freq} * \text{duty_cycle}$ (words/second) for each side. Round the result up, then apply the encoding's depth constraint:

- **Gray (`fifo_async`):** round up to the next power of two.

- **Johnson (fifo_async_div2, gaxi_fifo_async):** round up to the next even number — the computed depth itself, in most cases.

Worked example. Writer at 100 MHz bursts 100 back-to-back writes; reader pulls 1 word/clock at 80 MHz:

Step	Value
Burst duration	100 / 100 MHz = 1 μ s
Reads during burst	1 μ s $\times$ 80 MHz = 80
Net build-up	100 – 80 = 20 words
Gray FIFO (power-of-2)	round up to 32
Johnson FIFO (even)	20 as-is

The Gray encoding adds 12 unused slots — overhead that scales with the data width:

DATA_WIDTH	Gray depth 32	Johnson depth 20	Savings
32 b	1024 b	640 b	384 b (38%)
64 b	2048 b	1280 b	768 b (38%)
256 b	8192 b	5120 b	3072 b (38%)
1024 b	32 768 b	20 480 b	12 288 b (38%)

For wide streaming paths (≥ 256 b), picking Johnson when the rate analysis lands between two powers of two reclaims a meaningful slice of BRAM / register area. The downside is a slightly larger pointer comparator (Johnson pointers are $2 \times \log_2(\text{depth})$ bits wide vs Gray's $\log_2(\text{depth}) + 1$), but for $\text{DATA_WIDTH} \geq 64$ the storage savings dominate.

Don't forget to add margin for synchronizer latency on full/empty if you're at the edge of the safe depth — the back-pressure to the writer lags real fill level by N_FLOP_CROSS write clocks.

29.4.4 Modules

Module	Description	Depth	Doc
fifo_async	Basic async FIFO, Gray-coded pointers	Power of 2	rtl/ common/
fifo_async_div2	Async FIFO, Johnson counter pointers	Any even	rtl/ common/
gaxi_fifo_async	GAXI-interface async FIFO	Configurable	rtl/amba/ gaxi/
gaxi_skid_buffer_async	Pipelined async FIFO (skid + async)	Configurable	rtl/amba/ gaxi/

29.4.5 Usage Example

```
fifo_async #(
    .DATA_WIDTH      (64),
    .DEPTH           (16),           // Must be power of 2
    .N_FLOP_CROSS    (3),           // Synchronizer stages
    .ALMOST_WR_MARGIN (2),           // Almost-full threshold
    .ALMOST_RD_MARGIN (2)           // Almost-empty threshold
) u_stream_cdc (
    // Write domain
    .wr_clk      (src_clk),
    .wr_rst_n    (src_rst_n),
    .wr_en       (src_valid && !wr_full),
    .wr_data     (src_data),
    .wr_full     (wr_full),
    .wr_almost_full (wr_almost_full),

    // Read domain
    .rd_clk      (dst_clk),
    .rd_rst_n    (dst_rst_n),
    .rd_en       (dst_ready && !rd_empty),
    .rd_data     (dst_data),
    .rd_empty    (rd_empty),
    .rd_almost_empty (rd_almost_empty)
);
```

29.4.6 Supporting Primitives

Module	Purpose	Doc
bin2gray	Binary to Gray code converter	rtl/common/
gray2bin	Gray code to binary converter	rtl/common/
grayj2bin	Johnson counter to binary converter	rtl/common/
counter_bingray	Dual binary+Gray output counter	rtl/common/
counter_johnson	Johnson counter for div2 FIFOs	rtl/common/
glitch_free_n_dff_arn	N-stage synchronizer for pointer crossing	rtl/common/

29.5 Decision Guide: Which CDC Technique?

Need to cross clock domains with multi-bit data?
|

```

+-- Is data streaming / continuous?
|
| +-- YES --> Async FIFO (fifo_async or gaxi_fifo_async)
|           Sustained 1-per-clock throughput, buffered
|
| +-- NO, occasional transfers -->
|   |
|   +-- Does the source need backpressure (flow control)?
|       |
|       +-- YES --> Closed-loop handshake
|               (cdc_2_phase_handshake or
cdc_4_phase_handshake)
|       |
|       +-- NO, source can guarantee hold time -->
|           |
|           +-- Open-loop (cdc_open_loop)
|               Simplest, lowest area, no ack needed
|
Single-bit signal?
|
+-- Level / quasi-static --> cdc_synchronizer or glitch_free_n_dff_arn
+-- Single-cycle pulse --> sync_pulse
+-- Reset signal --> reset_sync

```

29.5.1 Quick Reference

Scenario	Module	Why
Config register update (CPU -> peripheral)	cdc_open_loop	Infrequent, no backpressure needed
Interrupt signal (1 bit, pulse)	sync_pulse	Single-cycle event
Status register read (slow -> fast)	cdc_synchronizer	Quasi-static level
APB slave in different clock domain	apb_slave_cdc	Full protocol CDC
Monitor packet crossing	cdc_2_phase_handshake	Occasional, needs flow control
DMA data stream	fifo_async	High throughput, continuous

Scenario	Module	Why
AXI channel crossing	<code>gaxi_skid_buffer_async</code>	Pipelined, GAXI protocol
Reset distribution	<code>reset_sync</code>	Async assert, sync deassert

29.6 Common Mistakes

1. Using a multi-flop synchronizer for multi-bit buses that change simultaneously

Multi-flop synchronizers (2FF/3FF) are safe for single-bit signals or Gray-coded values. If multiple bits can change at once, different bits may be sampled from different time points, producing a value that never existed.

Fix: Use a handshake or async FIFO.

2. Using `set_false_path` on CDC data buses

`set_false_path` tells the tool to ignore timing entirely. For handshake and open-loop CDC, the data bus must arrive within a bounded window (roughly `SYNC_STAGES` destination clocks). Use `set_max_delay -datapath_only` instead.

3. Assuming async FIFO depth = 2 is enough

A depth-2 async FIFO has only 1 usable entry (full when `wr_ptr != rd_ptr` by 1). The synchronized pointers lag by `SYNC_STAGES` cycles, so the effective throughput drops. Use depth ≥ 4 for practical designs.

4. Sending a new open-loop transfer before the previous one is sampled

Without an acknowledge, the source has no way to know the destination has latched the data. If the source writes again too soon, the first transfer is silently lost. Minimum spacing: `SYNC_STAGES + 1` destination clocks.

29.7 References

- Clifford E. Cummings, “Clock Domain Crossing (CDC) Design and Verification Techniques Using SystemVerilog” (SNUG 2008)
- Clifford E. Cummings, “Simulation and Synthesis Techniques for Asynchronous FIFO Design” (SNUG 2002)
- IEEE 1800-2017 SystemVerilog LRM, Section on synchronizer modeling

29.8 Navigation

- [Back to Shared Infrastructure Index](#)
- [Back to RTLAmba Index](#)

30 CDC Synchronizer

Module: `cdc_synchronizer.sv` **Location:** `rtl/amba/shared/` **Status:** Production Ready

30.1 Overview

Multi-bit synchronizer for clock domain crossing. Wraps the `glitch_free_n_dff_arn` primitive with a consistent naming interface for the AMBA infrastructure.

Suitable for quasi-static signals that change infrequently relative to the destination clock (configuration registers, status flags, level indicators). NOT suitable for pulse transfers or streaming data – use `sync_pulse` or `async FIFOs` for those.

30.1.1 Key Features

- Parameterizable bus width (1 to N bits)
 - Configurable synchronizer depth (2-5 stages, default 3)
 - Uses `glitch_free_n_dff_arn` internally for `ASYNC_REG` attributes
 - Zero additional logic – pure flop chain
-

30.2 Module Interface

```
module cdc_synchronizer #(
    parameter int WIDTH      = 1,    // Bus width to synchronize
    parameter int FLOP_COUNT = 3     // Number of synchronizer stages
)(2-5)
) (
    input logic clk,                // Destination clock
    input logic rst_n,              // Asynchronous active-low
reset
    input logic [WIDTH-1:0] async_in, // Asynchronous input from
source domain
    output logic [WIDTH-1:0] sync_out // Synchronized output in
```

```
destination domain  
);
```

30.3 Parameters

Parameter	Default	Range	Description
WIDTH	1	1+	Number of bits to synchronize
FLOP_COUNT	3	2-5	Synchronizer chain depth (stages)

30.4 Ports

Port	Direction	Width	Description
clk	input	1	Destination domain clock
rst_n	input	1	Asynchronous active-low reset
async_in	input	WIDTH	Input signal from source clock domain
sync_out	output	WIDTH	Synchronized output in destination domain

30.5 Usage Guidelines

- Input must be quasi-static: stable for at least FLOP_COUNT + 1 destination clock cycles before being sampled
- For multi-bit buses, ALL bits must change simultaneously or use Gray coding
- Use FLOP_COUNT=2 for relaxed MTBF, FLOP_COUNT=3 for production (default)
- Do NOT use for pulse signals – use sync_pulse instead
- Do NOT use for streaming data – use fifo_async or gaxi_fifo_async

30.6 Instantiation Example

```
cdc_synchronizer #(
    .WIDTH      (4),
    .FLOP_COUNT (3)
) u_status_sync (
```

```
.clk      (dest_clk),  
.rst_n    (dest_rst_n),  
.async_in (src_status),  
.sync_out (syncd_status)  
);
```

30.7 Resources

- RTL: rtl/amba/shared/cdc_synchronizer.sv
- Primitive: rtl/common/glitch_free_n_dff_arn.sv
- Formal: formal/amba/cdc_synchronizer/
- Related: sync_pulse.sv (pulse crossing), cdc_2_phase_handshake.sv (data + handshake)

31 Clock-Gated Variants Guide

Category: Infrastructure Documentation **Location:** rtl/amba/shared/ **Applies To:** All AMBA protocol modules with _cg suffix

31.1 Overview

All AMBA protocol modules have clock-gated (CG) variants that add comprehensive power optimization capabilities through activity-based clock gating. This guide covers the common patterns, parameters, and usage for all CG variants.

Naming Convention: {base_module}_cg.sv

Examples: - axi4_master_rd_mon.sv → axi4_master_rd_mon_cg.sv - apb_slave.sv → apb_slave_cg.sv - axil4_master_wr_mon.sv → axil4_master_wr_mon_cg.sv

31.2 Key Principle

Clock-gated variants maintain 100% functional equivalence with base modules while adding power optimization.

Base Module Functionality + Clock Gating Control = CG Variant

All parameters, ports, and behavior of the base module are preserved. Only clock gating configuration is added.

31.3 Common Clock Gating Parameters

All CG variants add these parameters to their base module parameters:

Parameter	Type	Default	Description
ENABLE_CLOCK_GATING	bit	1	Master enable for clock gating (0=disable all gating)
CG_IDLE_CYCLES	int	8	Number of idle cycles before asserting clock gate
CG_GATE_MONITOR	bit	1	Enable clock gating for monitor logic (monitors only)
CG_GATE_REPORTER	bit	1	Enable clock gating for packet reporter (monitors only)
CG_GATE_TIMERS	bit	1	Enable clock gating for timeout timers (monitors only)
CG_GATE_DATAPATH	bit	1	Enable clock gating for data path logic (interfaces only)
CG_GATE_CONTROL	bit	1	Enable clock gating for control path logic (interfaces only)

Note: Monitor modules have monitor/reporter/timer gating. Interface modules have datapath/control gating.

31.4 Architecture

31.4.1 Clock Gating Domains

For Monitor Modules (e.g., `axi4_master_rd_mon_cg`):

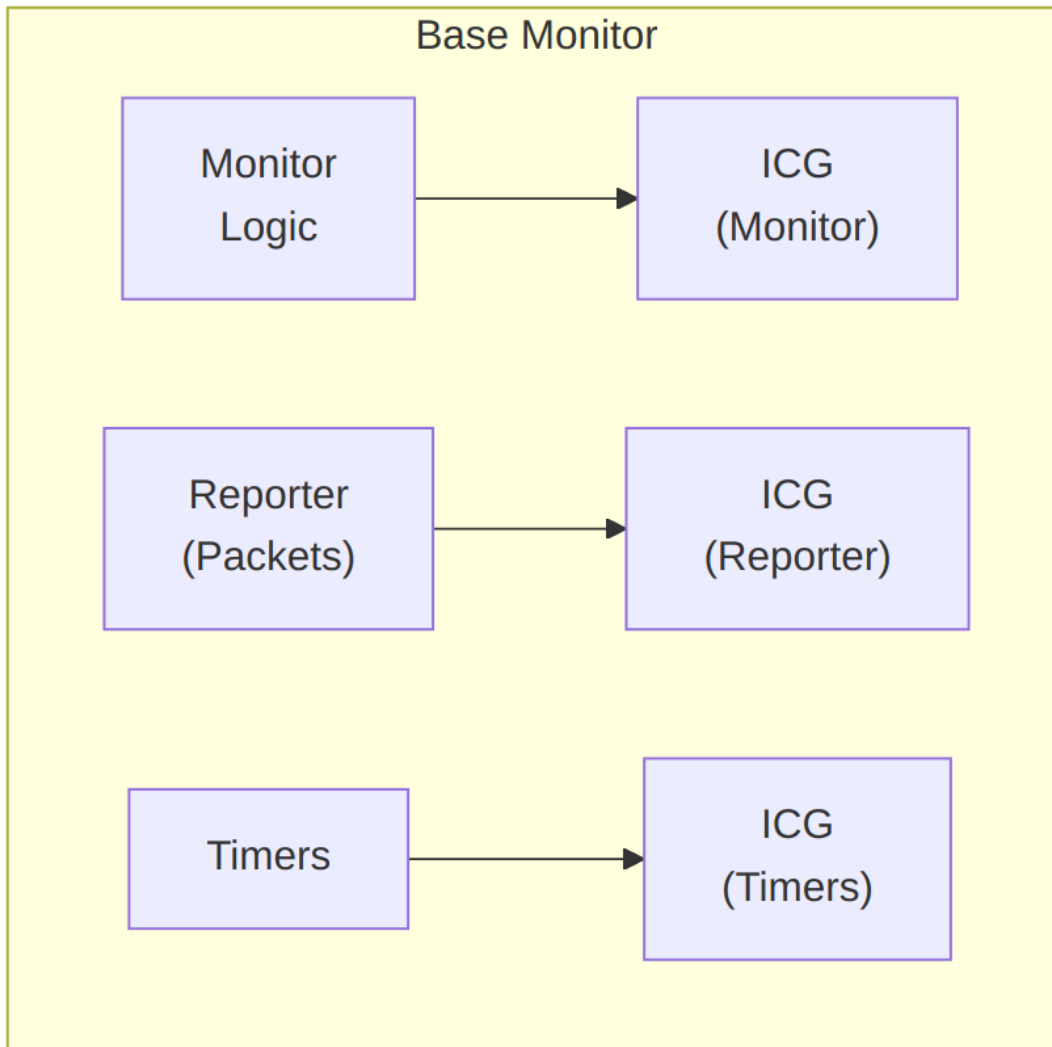


Diagram 6

For Interface Modules (e.g., `axi4_master_rd_cg`, `apb_slave_cg`):

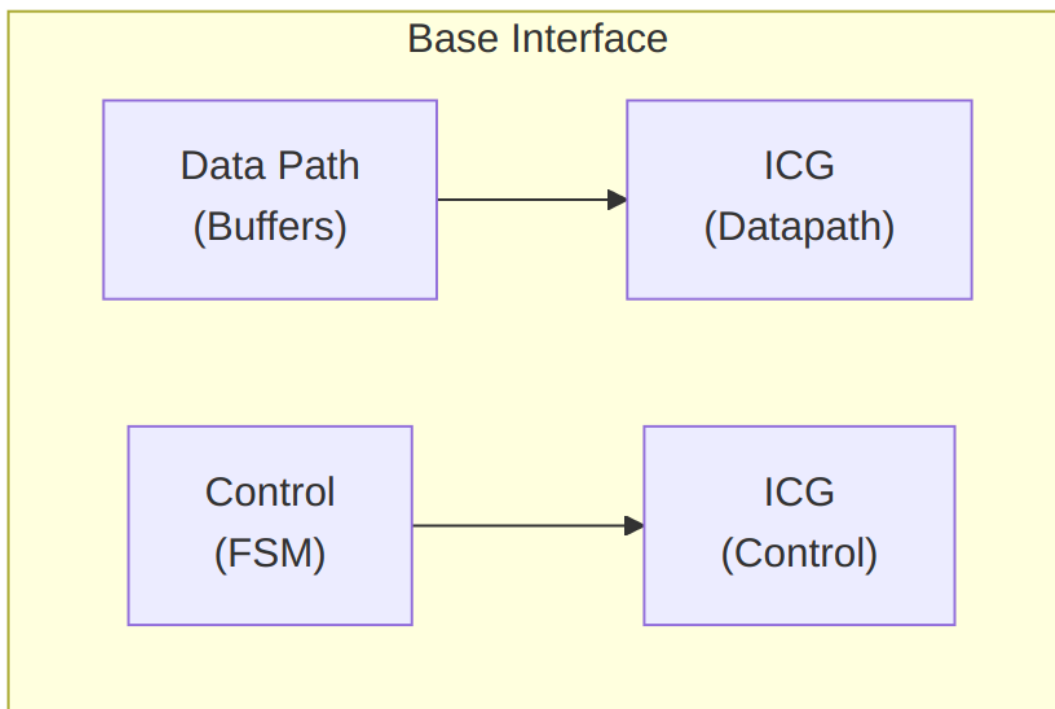
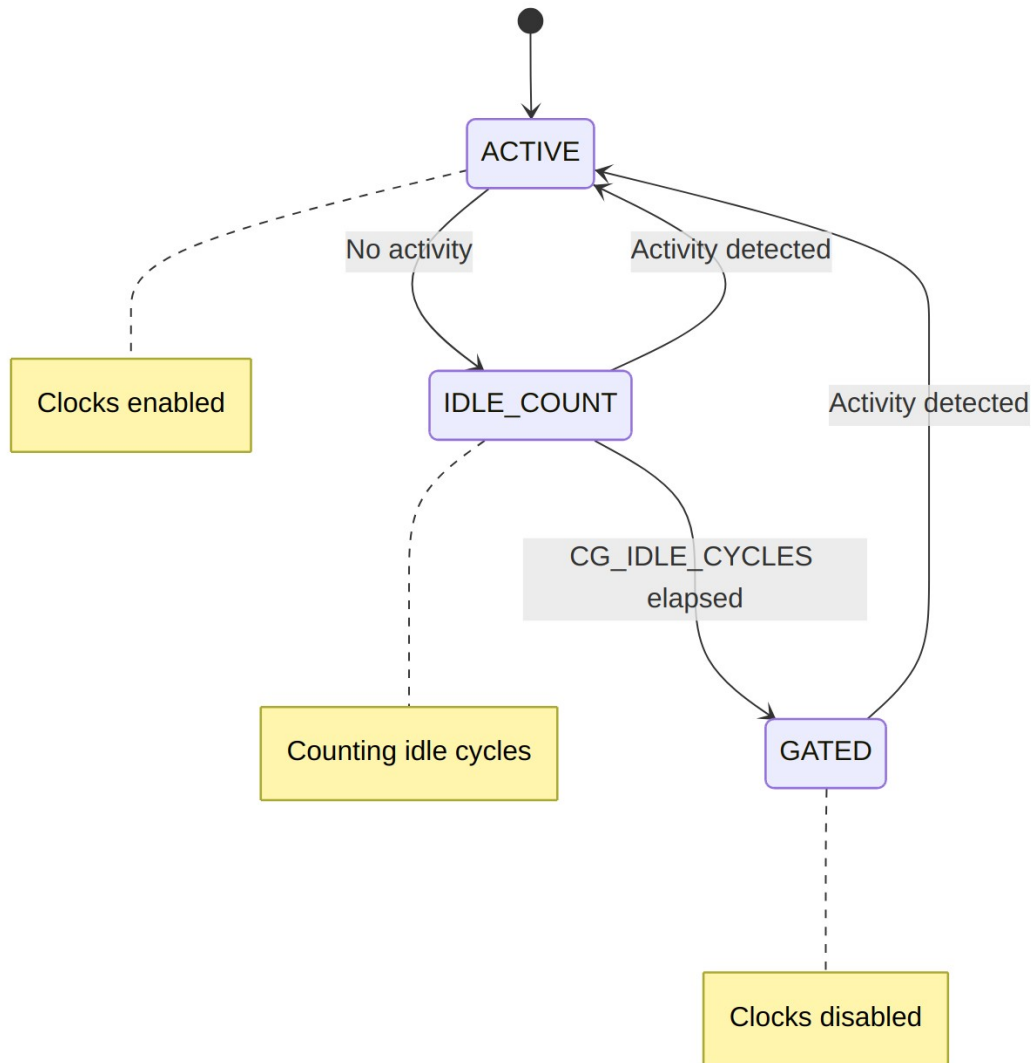


Diagram 7



Gating State Machine

31.5 Usage Patterns

31.5.1 Pattern 1: Maximum Power Savings

Use Case: Battery-operated systems, burst traffic patterns

```

{module}_cg #(
    // Base module parameters (same as non-CG variant)
    .AXI_ID_WIDTH(8),
    .AXI_ADDR_WIDTH(32),

```

```

        .AXI_DATA_WIDTH(64),

        // Clock gating - aggressive
        .ENABLE_CLOCK_GATING(1),
        .CG_IDLE_CYCLES(4),           // Gate quickly after 4 idle cycles
        .CG_GATE_MONITOR(1),         // Gate all subsystems
        .CG_GATE_REPORTER(1),
        .CG_GATE_TIMERS(1)
    ) u_cg_aggressive (
        .aclk(clk),
        .aresetn(rst_n),
        // ... all other ports same as base module
    );

```

Power Savings: 60-70% for 10% utilization traffic

31.5.2 Pattern 2: Balanced Performance and Power

Use Case: Production systems with moderate power constraints

```

{module}_cg #(
    // Base module parameters
    .AXI_ID_WIDTH(8),
    .AXI_ADDR_WIDTH(32),
    .AXI_DATA_WIDTH(64),

    // Clock gating - balanced
    .ENABLE_CLOCK_GATING(1),
    .CG_IDLE_CYCLES(16),           // Wait longer before gating
    .CG_GATE_MONITOR(1),
    .CG_GATE_REPORTER(0),         // Keep reporter active
    .CG_GATE_TIMERS(1)
) u_cg_balanced (
    .aclk(clk),
    .aresetn(rst_n),
    // ... all other ports same as base module
);

```

Power Savings: 45-55% for 25% utilization traffic

31.5.3 Pattern 3: Clock Gating Disabled

Use Case: Functional verification, continuous high-activity traffic

```

{module}_cg #(
    // Base module parameters
    .AXI_ID_WIDTH(8),
    .AXI_ADDR_WIDTH(32),
    .AXI_DATA_WIDTH(64),

```

```

    // Clock gating - DISABLED
    .ENABLE_CLOCK_GATING(0) // Disable all gating
) u_cg_disabled (
    .aclk(clk),
    .aresetn(rst_n),
    // ... all other ports same as base module
);

```

Result: Functionally identical to base module, no power savings.

31.6 When to Use Clock-Gated Variants

31.6.1 Use CG Variant When:

- ✓ Power consumption is critical concern
- ✓ Design has periods of inactivity (burst traffic)
- ✓ FPGA/ASIC has integrated clock gating support
- ✓ Meeting power budgets for battery-operated systems
- ✓ Idle periods $\geq$ CG_IDLE_CYCLES (worth gating)

31.6.2 Use Base Module When:

- ✓ Maximum performance with no power constraints
 - ✓ Continuous high-activity traffic (90%+ utilization)
 - ✓ Simpler design with fewer configuration parameters
 - ✓ Minimizing gate count is priority
 - ✓ Clock gating infrastructure not available
-

31.7 Power Savings Analysis

31.7.1 Typical Power Reduction by Utilization

Traffic Utilization	CG_IDLE_CYCLES=4 (Aggressive)	CG_IDLE_CYCLES=16 (Conservative)
10% (Burst traffic)	60-70% savings	50-60% savings
25% (Moderate)	45-55% savings	35-45% savings
50% (Active)	25-35% savings	20-30% savings
90% (Continuous)	5-10% savings	5-10% savings

Note: Actual savings depend on FPGA/ASIC technology, tool implementation, and traffic patterns.

31.7.2 Wake-Up Latency vs Power Savings

CG_IDLE_CYCLES	Wake-Up Time	Power Savings	Use Case
2-4	2-4 cycles	High (aggressive)	Low-latency burst traffic
8-16	8-16 cycles	Medium (balanced)	General purpose
32-64	32-64 cycles	Lower (conservative)	Infrequent traffic, power critical

Recommendation: Start with CG_IDLE_CYCLES=8 (balanced), adjust based on profiling.

31.8 Synthesis Considerations

31.8.1 FPGA Implementations

Xilinx:

```
// Use ENABLE_CLOCK_GATING=1 with BUFGCE primitives
// Tool will infer clock enables automatically
// Verify with post-synthesis power analysis
```

Intel (Altera):

```
// Use ENABLE_CLOCK_GATING=1 with ALTCLKCTRL
// May need vendor-specific clock gating primitives
// Check power reports for gating effectiveness
```

Lattice:

```
// Basic clock gating supported
// May require manual instantiation of clock enables
// Verify functionality in timing simulation
```

31.8.2 ASIC Implementations

- Work with foundry to select appropriate clock gating cells
- Integrated Clock Gating (ICG) cells provide best results
- Consider hold-time implications of clock gating
- Verify power intent with UPF (Unified Power Format)

Example UPF Power Domain:

```
create_power_domain PD_MONITOR
add_port_state PD_MONITOR -state {ON 1.0} -state {OFF 0.0}
set_domain_supply_net PD_MONITOR -primary_power_net VDD
```

31.9 Verification Considerations

31.9.1 Functional Verification

Recommendation: Disable clock gating during functional verification:

```
`ifdef FUNCTIONAL_VERIFICATION
    localparam ENABLE_CG = 0; // Disable for faster simulation
`else
    localparam ENABLE_CG = 1; // Enable for power analysis
`endif
```

```
{module}_cg #(.ENABLE_CLOCK_GATING(ENABLE_CG)) dut (...);
```

Rationale: - Simpler waveforms (no clock gating events) - Faster simulation (no gating overhead) - Easier debug (no timing dependencies)

31.9.2 Power Verification

For power-specific verification:

1. **Enable clock gating** with realistic parameters
 2. **Monitor gating signals** (cg_*_gated) to verify behavior
 3. **Vary traffic patterns** to test gating effectiveness
 4. **Check wake-up timing** meets system requirements
 5. **Measure power** with tools (Vivado Power Analyzer, PrimeTime PX)
-

31.10 Available Clock-Gated Variants

31.10.1 AXI4 Protocol (8 modules)

Base Module	CG Variant	Documentation
axi4_master_rd_mon.sv	axi4_master_rd_mon_cg.sv	Example
axi4_master_wr_mon.sv	axi4_master_wr_mon_cg.sv	See CG guide (this doc)

Base Module	CG Variant	Documentation
axi4_slave_rd_mon.sv	axi4_slave_rd_mon_cg.sv	See CG guide (this doc)
axi4_slave_wr_mon.sv	axi4_slave_wr_mon_cg.sv	See CG guide (this doc)
axi4_master_rd.sv	axi4_master_rd_cg.sv	See CG guide (this doc)
axi4_master_wr.sv	axi4_master_wr_cg.sv	See CG guide (this doc)
axi4_slave_rd.sv	axi4_slave_rd_cg.sv	See CG guide (this doc)
axi4_slave_wr.sv	axi4_slave_wr_cg.sv	See CG guide (this doc)

31.10.2 AXI4-Lite Protocol (8 modules)

Base Module	CG Variant	Documentation
axil4_master_rd_mon.sv	axil4_master_rd_mon_cg.sv	Example
axil4_master_wr_mon.sv	axil4_master_wr_mon_cg.sv	See CG guide (this doc)
axil4_slave_rd_mon.sv	axil4_slave_rd_mon_cg.sv	See CG guide (this doc)
axil4_slave_wr_mon.sv	axil4_slave_wr_mon_cg.sv	See CG guide (this doc)
axil4_master_rd.sv	axil4_master_rd_cg.sv	See CG guide (this doc)
axil4_master_wr.sv	axil4_master_wr_cg.sv	See CG guide (this doc)
axil4_slave_rd.sv	axil4_slave_rd_cg.sv	See CG guide (this doc)
axil4_slave_wr.sv	axil4_slave_wr_cg.sv	See CG guide (this doc)

31.10.3 APB Protocol (3 modules)

Base Module	CG Variant	Documentation
apb_master.sv	apb_master_cg.sv	See CG guide (this doc)
apb_slave.sv	apb_slave_cg.sv	Example
apb_slave_cdc.sv	apb_slave_cdc_cg.sv	See CG guide (this doc)

31.10.4 AXIS Protocol (2 modules)

Base Module	CG Variant	Documentation
<code>axis_master.sv</code>	<code>axis_master_cg.sv</code>	See CG guide (this doc)
<code>axis_slave.sv</code>	<code>axis_slave_cg.sv</code>	See CG guide (this doc)

Total: 21 clock-gated variants across all AMBA protocols

31.11 Detailed Examples

For detailed implementation examples with complete parameter lists, port connections, and usage scenarios, see:

- **AXI4 Monitor Example:** [axi4_master_rd_mon_cg.md](#)
- **AXIL4 Monitor Example:** [axil4_master_rd_mon_cg.md](#)
- **APB Interface Example:** [apb_slave_cg.md](#)

These examples provide complete instantiation templates, synthesis scripts, and verification strategies specific to each protocol variant.

31.12 Configuration Guidelines

31.12.1 Step 1: Choose Base Configuration

Start with base module parameters (address width, data width, etc.).

31.12.2 Step 2: Enable/Disable Clock Gating

```
.ENABLE_CLOCK_GATING(1) // 1=enable, 0=disable (default to base module)
```

31.12.3 Step 3: Set Idle Threshold

```
.CG_IDLE_CYCLES(8) // Cycles before gating (balance: 8-16)
```

31.12.4 Step 4: Configure Gating Domains

For Monitors:

```
.CG_GATE_MONITOR(1),    // Gate transaction tracking
.CG_GATE_REPORTER(1),  // Gate packet generation
.CG_GATE_TIMERS(1)     // Gate timeout detection
```

For Interfaces:

```
.CG_GATE_DATAPATH(1),  // Gate data buffering
.CG_GATE_CONTROL(1)    // Gate control FSM
```

31.12.5 Step 5: Connect Ports

All ports identical to base module - no additional connections needed.

31.13 Performance Impact

Aspect	Impact	Notes
Latency	+0-2 cycles	Wake-up from gated state
Throughput	No impact	When active, same as base module
Area	+5-10%	Clock gating control logic
Power	-25% to -70%	Depends on utilization
Frequency	No impact	Same critical path as base

31.14 Related Documentation

- **Base Module Documentation:** See individual protocol directories (axi4/, axil4/, apb/, axis4/)
 - **Power Optimization:** docs/POWER_OPTIMIZATION_GUIDE.md
 - **Clock Gating Best Practices:** docs/CLOCK_GATING_GUIDE.md
 - **AMBA Overview:** [overview.md](#)
-

31.15 Navigation

- [← Back to Shared Infrastructure Index](#)

- [← Back to RTLAmba Index](#)
- [← Back to Main Documentation Index](#)

32 Shared Infrastructure Documentation Status

Generated: 2025-10-23 **Last updated:** 2026-06-15 (timing-closure refresh: 3-stage burst-writer pipeline, 2-stage compressor, per-template delta_ts, runtime cfg_compress_en, mod_3_compress helper, new axi4_dma_observer doc) **Location:** /mnt/data/github/RTLDesignSherpa/docs/markdown/RTLAmba/shared/

32.1 Completion Status

32.1.1 Completed Documentation

1. **axi_monitor_base.md** - COMPLETE
 - Comprehensive module documentation with all sections
 - Usage examples provided
 - Design notes included
 - Cross-references added
2. **axi_monitor_trans_mgr.md** - COMPLETE (rewritten 2026-06-08)
 - Reflects CAM-backed revision (delegates to monitor_trans_cam)
 - Documents synthesis properties carried from the 2026-04-23 WNS fix
 - Equivalence test + TRANS_MGR_VARIANT rollback knob covered
3. **monitor_trans_cam.md** - COMPLETE (new 2026-06-08)
 - Multi-port ID CAM with opaque payload
 - 3 lookup ports + 3-way mutex alloc priority encoder
 - Synthesis notes (keep="true" anchors, per-slot generate-loop storage)
4. **monbus_compressor.md** - COMPLETE (new 2026-06-08; refreshed 2026-06-15)
 - Hardware bulk-trace encoder, ~2.6× compression
 - Full coverage of all 5 compression techniques (template extraction, delta-ts, width-tiered Tier-1, differential payload, Tier-0 RAW)
 - Bit layouts, encoder decision tree, pipeline/timing/synthesis notes
 - 2026-06-15 refresh: pipeline is now 2 stages (lookup/commit -> encode/emit); delta_ts is per-template (matches the CAM's per-entry r_ts[24]) rather than against a single global r_last_ts; added

compressor input skid + pblock_monbus floorplan note for the Nexys A7 build

5. **monbus_cam.md** - COMPLETE (new 2026-06-08; refreshed 2026-06-15)
 - 32-entry true-LRU caching CAM (position-indexed storage)
 - Backs monbus_compressor's template indexing
 - Caller protocol (NONE/TOUCH/INSTALL), eviction semantics
 - 2026-06-15 refresh: added per-entry r_ts[TS_WIDTH=24] storage with access_old_ts / access_new_ts ports (shifts with the LRU move-to-front). CAM is no longer pure opaque-payload – the caller must drive access_new_ts on every TOUCH/INSTALL.
6. **monbus_group.md** - COMPLETE (rewritten 2026-06-11, was monbus_axil_group.md; refreshed same day for the two-stage burst-sizing fix)
 - Renamed from monbus_axil_group.md to reflect the new family.
 - Covers the full 5-file family: 1 protocol-agnostic core (monbus_group_core.sv) + 4 protocol-permutation wrappers (axil_axil / axil_axi4 / axi4_axil / axi4_axi4).
 - Documents the beat-granular write FIFO, watermark + timeout burst writer (4KB-boundary aware), and AXI4 burst behavior.
 - Migration recipe from the legacy single monbus_axil_group module (port-surface changes: cfg_flush_watermark added, FIFO counts widened to 16 bits, FIFO_DEPTH_WRITE now in beats, locked 64-bit data width).
 - “Master-Write Behavior” section refreshed for the two-stage drain plan (drain-cycle plans whole-record beats without a MAX_BURST_BEATS cap; per-AW sub-bursts get the cap inside the FSM). The “Test” section now lists all five tests in the suite, including the new master-write coverage that closed the AXIL-master raw-mode deadlock.
 - 2026-06-15 refresh: burst writer geometry is now a 3-stage registered pipeline with a fresh-FIFO cap applied combinationally at the WR_IDLE commit (necessary because the FIFO keeps filling while WR_IDLE waits); mod-3 rounding moved to mod_3_compress (rtl/common/, carry-save-compressor idiom) so synthesis no longer infers a DSP48 or CARRY4 iterative divider for the /3 path; runtime cfg_compress_en input added (USE_COMPRESSION=1 elaborates the

compressor, `cfg_compress_en` selects at runtime); canonical `rtl/amba/filelists/monbus_group.f` introduced (one place to add new core deps).

7. **sdpram_slave.md** - COMPLETE (new 2026-06-09)

- Covers the full 5-file family: 1 backend + 4 protocol-specific wrappers (`axi4_axi4` / `axi4_axil` / `axil_axi4` / `axil_axil`).
- Documents why the split exists (SystemVerilog cannot conditionally include/exclude ports in a single module declaration).
- Migration recipe from bare `sdpram_slave` to the matching wrapper.
- Cross-links from `monbus_group.md` and `monbus_compressor.md` (the memory-dump ring's canonical backend is `sdpram_slave_axil_axil`).

8. **axi_monitor_reporter.md** - COMPLETE (rewritten 2026-06-11)

- Reflects the 2026-06-06 sub-block refactor (thin dispatcher + 6 `ENABLE_*_LOGIC`-gated detection sub-blocks).
- Lists the six sub-blocks (error / timeout / compl / threshold / perf / debug), their logic shapes, and their gate parameters.
- Notes the bridge-case savings (`ENABLE_ERROR_LOGIC=1`, others 0 drops ~70% LUT/FF).
- The six sub-block files (`axi_monitor_reporter_*.sv`) are explicitly covered here rather than as individual doc pages, since they are private to the reporter family.

9. **axi4_dma_observer.md** - COMPLETE (new 2026-06-15)

- Standalone, DMA-agnostic observability harness that wraps any AXI4-master DMA from outside the DMA (non-intrusive). Companion to the per-DMA `axi_monitor_*` family which wraps from inside.
- Covers the full instantiation pattern: `NUM_RD` + `NUM_WR` `axi4_master_*_mon` taps, `monbus_arbiter` aggregator, `monbus_axil_axi4_group` filter+dump, `axi_bus_meter`, and `axi_perf_latency_hist` per port.
- Documents the runtime `rid` -> channel map (`cfg_rd_rid_per_channel`) for read-side per-channel attribution and, for writes, either the built-in `AW->W` `awid` order tracker (`WR_CH_FROM_AWID=1`, no DUT sideband) or the optional `dma_wr_active_ch_*` sideband.

- The companion modules `axi_bus_meter.sv` and `axi_perf_latency_hist.sv` are covered in this single doc rather than their own pages (small, only useful in context).

32.1.2 New shared infrastructure (no dedicated page, covered above)

- `rtl/common/mod_3_compress.sv` - carry-save-compressor X mod 3 for 16-bit operands; used by `monbus_group_core`'s whole-record rounding. Covered in `monbus_group.md` "mod-3 rounding" section.
- `rtl/amba/filelists/monbus_group.f` - canonical filelist enumerating the group core's dependency tree (`math_adder_carry_save_nbit` + `mod_3_compress` + `monbus_cam` + `monbus_compressor` + `monbus_group_core`). All consumers - f-include it. Covered in `monbus_group.md` "Canonical filelist" subsection.

32.1.3 Remaining Documentation (15 modules)

The following modules require documentation following the same pattern as `axi_monitor_base.md`:

32.1.3.1 *Monitor Infrastructure*

2. `axi_monitor_filtered.md` - PENDING
3. `axi_monitor_reporter.md` - **COMPLETE** (rewritten 2026-06-11, see #17)
 - Now describes the dispatcher + 6 sub-blocks (error / timeout / compl / threshold / perf / debug). The six sub-block files (`axi_monitor_reporter_*.sv`) are covered here, not as separate doc pages.
4. `axi_monitor_timeout.md` - PENDING
5. `axi_monitor_timer.md` - PENDING
6. `axi_monitor_trans_mgr.md` - **COMPLETE** (rewritten 2026-06-08)
 - See #11 in Completed Documentation above

32.1.3.2 *Monitor Bus Delivery + Bulk-Trace Compression (NEW SECTION)*

- `monbus_group.md` - **COMPLETE** (rewritten 2026-06-11, see #15)
 - Renamed from `monbus_axil_group.md`; covers the new 5-file family (core + 4 wrappers).
- `monbus_compressor.md` - **COMPLETE** (new 2026-06-08, see #13)
- `monbus_cam.md` - **COMPLETE** (new 2026-06-08, see #14)
- `monitor_trans_cam.md` - **COMPLETE** (new 2026-06-08, see #12)

32.1.3.3 *Memory / BRAM Slave (NEW SECTION)*

- sdpram_slave.md - **COMPLETE** (new 2026-06-09, see #16)
 - Covers backend (sdpram_slave.sv) + 4 wrappers (axi4_axi4, axi4_axil, axil_axi4, axil_axil) in a single doc.

32.1.3.4 *Monitor Bus Arbitration (4 modules)*

7. arbiter_monbus_common.md - PENDING
8. arbiter_rr_pwm_monbus.md - PENDING
9. arbiter_wrr_pwm_monbus.md - PENDING
10. monbus_arbiter.md - PENDING

32.1.3.5 *AXI Utilities (4 modules)*

11. axi_gen_addr.md - PENDING
12. axi_master_rd_splitter.md - PENDING
13. axi_master_wr_splitter.md - PENDING
14. axi_split_combi.md - PENDING

32.1.3.6 *Infrastructure (2 modules)*

15. amba_clock_gate_ctrl.md - PENDING
 16. cdc_handshake.md - PENDING
-

32.2 Documentation Template

Each documentation file should follow this structure (see axi_monitor_base.md as reference):

[Module Name]

```
**Module:** ` [module_name].sv `
**Location:** ` rtl/amba/shared/ `
**Status:** Production Ready
```

Overview

[Brief description from RTL file header comments]

Key Features

- Feature 1

- Feature 2
- Feature 3
- Feature 4

Module Purpose

[Detailed purpose - why this module exists, what problem it solves]

Parameters

Parameter	Type	Default	Description
...	...	...	...

Port Groups

[Group 1 Name]

Port	Direction	Width	Description
...	...	...	...

Functional Description

[How the module works - key behavior, FSM states, protocol details]

Usage Example

```
``systemverilog
[Realistic instantiation example]
```


32.3 Design Notes

32.3.1 [Important design aspect 1]

[Details]

32.4 Related Modules

32.4.1 Used By

- [List of parent modules]

32.4.2 Uses

- [List of child modules]
-

32.5 References

32.5.1 Specifications

- [ARM specs]
- [Internal references]

32.5.2 Source Code

- RTL: rtl/amba/shared/[module_name].sv
 - Tests: val/amba/test_[module_name].py
-

Last Updated: 2025-10-23

32.6 Navigation

- [Back to Shared Infrastructure Index](#)
- [Back to RTLAmba Index](#)

Key Information Extracted from RTL

Monitor Infrastructure Modules

****axi_monitor_filtered.sv****

- Wraps axi_monitor_base with configurable packet filtering
- 3-level filtering hierarchy: packet type masking, error routing, event code masking
- AXI protocol specific (protocol 3'b000)
- Optional pipeline stage for timing closure

****axi_monitor_reporter.sv****

- Reports events/errors through shared monitor bus
- Detects conditions from transaction table
- Formats 64-bit interrupt packets
- Supports error, completion, timeout, threshold, performance, debug packets
- FIFO buffering with gaxi_fifo_sync
- Event reported feedback to trans_mgr (FIX-001)

****axi_monitor_timeout.sv****

- Monitors transaction table for timeout conditions
- Per-phase timeout detection (address, data, response)
- Uses timer tick from frequency invariant timer
- Configurable timeout thresholds per phase

****axi_monitor_timer.sv****

- Frequency invariant timer for timeout detection
- Uses counter_freq_invariant module
- Generates timing ticks based on frequency selection
- Maintains global timestamp counter

****axi_monitor_trans_mgr.sv****

- Manages transaction tracking table
- Tracks up to MAX_TRANSACTIONS concurrent transactions
- Handles out-of-order completions
- Supports data-before-address scenarios
- Event reported feedback input (FIX-001)

Monitor Bus Arbitration Modules

****arbiter_monbus_common.sv****

- Comprehensive monitoring for RR and WRR arbiters
- Silicon debug monitor with PROTOCOL_ARB events
- 3-bit protocol field [59:57]
- Event categories: error, timeout, completion, threshold, performance, debug
- Per-client ACK timeout tracking

- Protocol violation detection
- Fairness deviation monitoring
- Grant efficiency tracking

****arbiter_rr_pwm_monbus.sv****

- Round-robin arbiter with PWM control
- Standardized fixed internal configurations
- PWM_WIDTH = 16 bits
- MON_FIFO_DEPTH = 16
- Uses arbiter_monbus_common for monitoring

****arbiter_wrr_pwm_monbus.sv****

- Weighted round-robin arbiter with PWM control
- Per-client weight thresholds
- Standardized fixed internal configurations
- Enhanced debug outputs for silicon debug

****monbus_arbiter.sv****

- Round-robin arbiter for monitor bus interfaces
- Optional input and output skid buffers
- ACK mode operation (grants held until acknowledged)
- 128-bit packet + 64-bit side-band timestamp, carried atomically through a 192-bit skid
- Parameterizable number of clients

AXI Utilities Modules

****axi_gen_addr.sv****

- Address generation for AXI bursts
- Supports FIXED, INCR, WRAP burst types
- Handles data width conversions
- Calculates next address and aligned address

****axi_master_rd_splitter.sv****

- Splits AXI read transactions across boundary crossings
- Assumptions: aligned addresses, fixed transfer size, incrementing bursts
- No address wraparound handling
- Split information FIFO for tracking
- State machine: IDLE, SPLITTING

****axi_master_wr_splitter.sv****

- Splits AXI write transactions across boundary crossings
- Same assumptions as read splitter
- WLAST generation for split transactions
- Response consolidation (N split responses -> 1 upstream response)

- Error priority: DECERR > SLVERR > EXOKAY > OKAY

****axi_split_combi.sv****

- Pure combinational split decision logic
- Simplified boundary crossing detection
- No wraparound handling
- Comprehensive assertions for validation
- Used by both read and write splitters

Infrastructure Modules

****amba_clock_gate_ctrl.sv****

- Wrapper for clock_gate_ctrl with AMBA-specific activity monitoring
- Monitors user_valid and axi_valid signals
- Configurable idle countdown
- Generates gated clock output

****cdc_handshake.sv****

- Clock domain crossing with handshake protocol
- 3-stage synchronizers for metastability protection
- Separate FSMs for source and destination domains
- Parameterizable data width

Next Steps

1. Create remaining 15 markdown files following the template
2. Create README.md index file for shared infrastructure directory
3. Update parent RTLamba/README.md to link to shared/ subdirectory
4. Cross-reference all related modules
5. Add usage examples for each module
6. Document integration patterns

Notes

- All RTL source files have been read and analyzed
- Module purposes and key features extracted
- Parameter tables ready for documentation
- Port groupings identified
- Design notes captured
- No emojis used in technical documentation

```
... {.pagebreak}
...
```

AMBA Shared Infrastructure

```
**Location:** `rtl/amba/shared/`
**Test Location:** Tested indirectly via protocol modules
**Status:** ✓ Production Ready
```

Overview

The shared subsystem provides foundational infrastructure modules used throughout the AMBA ecosystem. These modules implement common functionality for monitors, arbiters, clock domain crossing, and utility functions. They are typically not instantiated directly by users but form the backbone of higher-level protocol modules.

Key Features

- ✓ **Monitor Infrastructure:** Complete AXI/APB transaction monitoring system
- ✓ **CDC Utilities:** Clock domain crossing with proper synchronization
- ✓ **Arbitration:** Monitor bus arbitration for multi-agent systems
- ✓ **Address Generation:** AXI burst address calculation
- ✓ **Clock Gating:** Power management infrastructure
- ✓ **Production Ready:** Extensively tested through protocol modules

Module Categories

Monitor Infrastructure

Core modules that implement the AXI/APB monitoring functionality. The reporter (``axi_monitor_reporter``) is now a thin dispatcher around six packet-type-specific sub-blocks (error / timeout / compl / threshold / perf / debug); each is gated by an ``ENABLE_*_LOGIC`` parameter so an integrator that only needs error packets pays no LUT/FF for the others.

Module	Purpose	Key Features	Documentation
--------	---------	--------------	---------------

```

|-----|-----|-----|-----|
| **axi_monitor_base** | Core monitor logic | Transaction tracking,
timeout detection, event generation | [axi_monitor_base.md]
(axi_monitor_base.md) |
| **axi_monitor_filtered** | Monitor wrapper with filtering | 3-level
packet filtering, configuration validation | [axi_monitor_filtered.md]
(axi_monitor_filtered.md) |
| **axi_monitor_trans_mgr** | Transaction table management (CAM-
backed) | Out-of-order tracking, ID management |
[axi_monitor_trans_mgr.md](axi_monitor_trans_mgr.md) |
| **monitor_trans_cam** | Multi-port ID CAM with payload | 3 lookup
ports, 3-way alloc mutex, used by trans_mgr | [monitor_trans_cam.md]
(monitor_trans_cam.md) |
| **axi_monitor_reporter** | Event packet dispatcher | Top-level mux
over 6 detection sub-blocks (error / timeout / compl / threshold /
perf / debug), each gated by `ENABLE_*_LOGIC` |
[axi_monitor_reporter.md](axi_monitor_reporter.md) |
| `axi_monitor_reporter_{error,timeout,compl,threshold,perf,debug}` |
Per-packet-type detection sub-blocks | One MonBus packet type each,
muxed by axi_monitor_reporter; each gated by `ENABLE_*_LOGIC` |
[error](axi_monitor_reporter_error.md) · [timeout]
(axi_monitor_reporter_timeout.md) · [compl]
(axi_monitor_reporter_compl.md) · [threshold]
(axi_monitor_reporter_threshold.md) · [perf]
(axi_monitor_reporter_perf.md) · [debug]
(axi_monitor_reporter_debug.md) |
| **axi_monitor_timeout** | Timeout detection | Configurable
thresholds, multi-channel monitoring | [axi_monitor_timeout.md]
(axi_monitor_timeout.md) |
| **axi_monitor_timer** | Frequency-invariant timer | Configurable
tick generation | [axi_monitor_timer.md](axi_monitor_timer.md) |
| **axi_monitor_addr_check** | Address-match watchpoints | Per-channel
address compare and event emission | [axi_monitor_addr_check.md]
(axi_monitor_addr_check.md) |
| **apb_monitor_addr_check** | APB address-range filter | N
configurable ranges; emits APB address-range error packets |
[apb_monitor_addr_check.md](apb_monitor_addr_check.md) |
| **amba_clock_gate_ctrl** | Clock gating control | Dynamic gating,
idle detection | [amba_clock_gate_ctrl.md](amba_clock_gate_ctrl.md) |

```

Monitor Bus Delivery + Bulk-Trace Compression

The path from in-flight monitor packets to host-visible memory and CPU IRQs.

The monbus_group family pattern matches sdpram_slave – one protocol-agnostic core plus four wrappers that adapt the slave-read / master-write surfaces to

the matching fabric without spurious AXI4-only fields on AXIL sides.

Module	Purpose	Key Features	Documentation
monbus_group_core	Protocol-agnostic core	Per-protocol filter, err FIFO (record-granular), write FIFO (beat-granular), watermark+timeout burst writer, 4KB-boundary aware, optional bulk-trace compression	monbus_group_core.md
monbus_axil_axil_group	Wrapper – AXIL/AXIL	Slave-read + master-write both AXIL (single-beat). Replaces legacy monbus_axil_group.	monbus_axil_axil_group.md
monbus_axil_axi4_group	Wrapper – AXIL/AXI4	AXIL slave-read + AXI4 burst master-write (up to 256 beats/burst)	monbus_axil_axi4_group.md
monbus_axi4_axil_group	Wrapper – AXI4/AXIL	AXI4 burst slave-read + AXIL master-write	monbus_axi4_axil_group.md
monbus_axi4_axi4_group	Wrapper – AXI4/AXI4	Burst on both sides	monbus_axi4_axi4_group.md
monbus_halfbeat_packer	Half-beat packer	Packs MonBus records into half-beats to push past the compressor's ~66.7% ceiling; bit-exact to Python golden	monbus_halfbeat_packer.md
monbus_compressor	Bulk-trace encoder	32-entry LRU CAM, 4-tag slot format, ~2.6× ratio, 2-stage pipeline, per-template `delta_ts`, optional half-beat packing, bit-exact to Python golden	monbus_compressor.md
monbus_cam_pipe	Pipelined LRU CAM (production)	2-cycle compare + commit, depth-1 forwarding, sync `clear`, self-derived TOUCH/INSTALL – used in `monbus_compressor`	monbus_cam_pipe.md
monbus_cam	LRU CAM (reference)	Single-cycle reference design – same LRU semantics, superseded by `monbus_cam_pipe` in production	monbus_cam.md

The group family uses a small carry-save-compressor helper (`rtl/common/mod\_3\_compress.sv`) for whole-record $X - (X \bmod 3)$ rounding. A single canonical filelist `rtl/amba/filelists/monbus\_group.f` enumerates the core's dependency tree so a new core dep lands in one place; every consumer `-f`-includes it.

Standalone DMA Observability

The drop-in observer wraps any AXI4-master DMA's external pins from

outside the DMA (non-intrusive), produces a monbus stream + IRQ FIFO + bulk-trace dump, and includes per-port valid/ready bucket counters. Companion piece to the per-DMA `axi\_monitor\_` family.

Module	Purpose	Key Features	Documentation
axi4_dma_observer	Standalone DMA observability harness	NUM_RD + NUM_WR tap pairs, monbus_arbiter + monbus_axil_axi4_group, axi_bus_meter + axi_perf_latency_hist per tap; reads via rid map, writes via AW→W awid tracker	[axi4_dma_observer.md] (axi4_dma_observer.md)
axi_bus_meter	Per-cycle valid/ready bucket counter	4 buckets (productive / backpressure / starvation / idle), aggregate + per-channel	[axi_bus_meter.md] (axi_bus_meter.md)
axis_bus_meter	AXIS per-cycle bucket counter + throughput	Same 4 buckets plus window-independent byte (tstrb) / beat / packet (tlast) counters, per-tid bins	[axis_bus_meter.md] (axis_bus_meter.md)
axi_perf_latency_hist	Per-transaction latency histogram	16 log2 bins; read AR→first-R + AR→RLAST, write AW→B; per-channel timestamp FIFOs; indexed readout	[axi_perf_latency_hist.md] (axi_perf_latency_hist.md)

Memory / BRAM Slave (sdpram_slave family)

Simple dual-port BRAM slave with independent protocol choice on each side. Most commonly used as the memory-dump ring backend for `monbus\_axil\_axil\_group`. One common backend + four protocol-specific wrappers (since SystemVerilog can't conditionally include/exclude ports in a single module's port list, each combination gets its own wrapper module name).

Module	Purpose	Wr / Rd ports	Documentation
sdpram_core	Common backend (BRAM glue, clear FSM, protocol skids)	AXI4 superset (parameterized)	[sdpram_core.md] (sdpram_core.md)
sdpram_slave_axi4_axi4	Wrapper – pure AXI4	`s_axi_*` / `s_axi_*`	[sdpram_slave_axi4_axi4.md] (sdpram_slave_axi4_axi4.md)
sdpram_slave_axi4_axil	Wrapper – AXI4 wr + AXIL rd	`s_axi_*` / `s_axil_*`	[sdpram_slave_axi4_axil.md] (sdpram_slave_axi4_axil.md)
sdpram_slave_axil_axi4	Wrapper – AXIL wr + AXI4 rd	`s_axil_*` / `s_axi_*`	[sdpram_slave_axil_axi4.md] (sdpram_slave_axil_axi4.md)
sdpram_slave_axil_axil	Wrapper – pure AXIL (canonical SRAM-ring backend)	`s_axil_*` / `s_axil_*`	[sdpram_slave_axil_axil.md]

(sdpram_slave_axil_axil.md) |

Characterization Data-Plane (on-chip pattern gen / check)

Synthesizable, self-checking traffic generators and checkers used by the
FPGA characterization harnesses (``stream_char_harness``,
``rapids_char_harness``,
``ddr2_char_harness``) as on-chip stimulus + verification – no host
memory model
needed. All share one LFSR (``0xDEADBEEF``, taps {32,22,2,1}) + CRC-32
config so
generated and checked data stay bit-consistent across every block.

Module	Purpose	Interface	Documentation
<code>**axi4_slave_rd_pattern_gen**</code>	Read-data source (LFSR)	AXI4 slave (R)	axi4_slave_rd_pattern_gen.md
<code>**axi4_slave_wr_crc_check**</code>	Write-data verify (per-channel CRC)	AXI4 slave (W/B)	axi4_slave_wr_crc_check.md
<code>**axi4_master_rd_crc_check**</code>	CSR-driven read master + CRC compare	AXI4 master (AR/R)	axi4_master_rd_crc_check.md
<code>**axi4_master_wr_pattern_gen**</code>	CSR-driven write master (LFSR/hash)	AXI4 master (AW/W/B)	axi4_master_wr_pattern_gen.md
<code>**axis4_master_pattern_gen**</code>	AXIS stream source	AXIS master	axis4_master_pattern_gen.md
<code>**axis4_slave_pattern_check**</code>	AXIS stream verify	AXIS slave	axis4_slave_pattern_check.md
<code>**axi4_dma_slaves**</code>	Source/sink bundle (rd gen + wr check)	AXI4 slaves	axi4_dma_slaves.md

Arbitration (4 modules)

Monitor bus arbiters for aggregating packets from multiple monitors:

Module	Purpose	Algorithm	Documentation
<code>**monbus_arbiter**</code>	Basic monitor bus arbiter	Round-robin with priority	monbus_arbiter.md
<code>**arbiter_monbus_common**</code>	Common arbiter infrastructure	Shared logic for all arbiters	arbiter_monbus_common.md
<code>**arbiter_rr_pwm_monbus**</code>	Round-robin PWM arbiter	Equal priority rotation	arbiter_rr_pwm_monbus.md

```
| **arbiter_wrr_pwm_monbus** | Weighted round-robin arbiter |
Configurable priority weights | [arbiter_wrr_pwm_monbus.md]
(arbiter_wrr_pwm_monbus.md) |
```

AXI Utilities (5 modules)

Utility modules for AXI protocol handling:

```
| Module | Purpose | Use Case | Documentation |
|-----|-----|-----|-----|
| **axi_gen_addr** | Burst address generation | Calculate next address
for INCR/WRAP bursts | [axi_gen_addr.md](axi_gen_addr.md) |
| **axi_master_rd_splitter** | Read channel splitter | Split AXI read
across boundaries | [axi_master_rd_splitter.md]
(axi_master_rd_splitter.md) |
| **axi_master_wr_splitter** | Write channel splitter | Split AXI
write with response consolidation | [axi_master_wr_splitter.md]
(axi_master_wr_splitter.md) |
| **axi_split_combi** | Combined splitter | Bidirectional split (read
+ write) | [axi_split_combi.md](axi_split_combi.md) |
| **CDC Primer** | Reference guide | All CDC techniques categorized
with decision guide | [cdc_primer.md](cdc_primer.md) |
| **cdc_open_loop** | Clock domain crossing | Open-loop CDC with
sync_pulse (no handshake) | [cdc_open_loop.md](cdc_open_loop.md) |
| **cdc_2_phase_handshake** | Clock domain crossing | 2-phase toggle-
based valid/ready handshake (faster) | [cdc_2_phase_handshake.md]
(cdc_2_phase_handshake.md) |
| **cdc_4_phase_handshake** | Clock domain crossing | 4-phase level-
based valid/ready handshake | [cdc_4_phase_handshake.md]
(cdc_4_phase_handshake.md) |
| **cdc_synchronizer** | Multi-bit synchronizer | N-stage synchronizer
for quasi-static signals | [cdc_synchronizer.md](cdc_synchronizer.md)
|
```

Monitor Infrastructure

axi_monitor_base

****Purpose:**** Core AXI/AXI-Lite transaction monitoring engine

****Key Features:****

- Transaction table tracking (up to MAX_TRANSACTIONS concurrent)
- Out-of-order completion support
- Timeout detection with configurable thresholds

- Multiple packet types: error, completion, timeout, performance, debug
- Integrated with monitor_pkg for standardized events

```

**Parameters:**
```systemverilog
module axi_monitor_base #(
 parameter int UNIT_ID = 9, // 4-bit unit
 identifier
 parameter int AGENT_ID = 99, // 8-bit agent
 identifier
 parameter int MAX_TRANSACTIONS = 16, // Transaction table
 size
 parameter int ADDR_WIDTH = 32,
 parameter int ID_WIDTH = 8, // 0 for AXI-Lite
 parameter bit IS_READ = 1, // 1=read, 0=write
 parameter bit IS_AXI = 1, // 1=AXI, 0=AXI-Lite
 parameter bit ENABLE_PERF_PACKETS = 0,
 parameter bit ENABLE_DEBUG_MODULE = 0
)

```

**Interfaces:** - Command phase (AW/AR): addr, id, len, size, burst, valid, ready - Data channel (W/R): id, last, resp, valid, ready - Response channel (B): id, resp, valid, ready - Monitor bus output: 64-bit packets

**Configuration:** - Packet type enables: error, completion, timeout, threshold, performance, debug - Timeout counters: addr, data, resp (0-15 ticks) - Frequency selection for timer (0-15) - Threshold config: active transactions, latency

**Generated Packets:** - **ERROR:** Protocol violations, response errors - **COMPL:** Transaction completion notifications - **TIMEOUT:** Channel timeout events - **THRESH:** Threshold crossings (active transactions, latency) - **PERF:** Performance metrics - **DEBUG:** Detailed transaction traces

**Usage:** Used internally by all \*\_mon wrapper modules. Not typically instantiated directly.

---

### 32.6.1 axi\_monitor\_filtered

**Purpose:** Wrapper for axi\_monitor\_base with configurable packet filtering

**Key Features:** - 3-level filtering hierarchy: - Level 1: Packet type masking (drop entire types) - Level 2: Error routing (separate error handling) - Level 3: Event code masking (fine-grained filtering) - Optional pipeline stage for timing closure - Configuration validation with error detection

**Parameters:**

```

module axi_monitor_filtered #(
 // All axi_monitor_base parameters, plus:
 parameter bit ENABLE_FILTERING = 1,
 parameter bit ADD_PIPELINE_STAGE = 0
)

```

#### Filtering Configuration:

```

input logic [15:0] cfg_axi_pkt_mask, // Drop packets by type
input logic [15:0] cfg_axi_error_mask, // Mask specific error
events
input logic [15:0] cfg_axi_timeout_mask, // Mask specific timeout
events
input logic [15:0] cfg_axi_compl_mask, // Mask specific completion
events
input logic [15:0] cfg_axi_thresh_mask, // Mask specific threshold
events
input logic [15:0] cfg_axi_perf_mask, // Mask specific
performance events
input logic [15:0] cfg_axi_debug_mask // Mask specific debug
events

```

#### Filtering Logic:

Unfiltered packet → Packet type check (pkt\_mask)  
                   → Event code check (event\_mask)  
                   → Optional pipeline stage  
                   → Filtered packet output

**Usage:** Used by all axi4\*\_mon and axil4\*\_mon modules. Provides fine-grained control over which events are reported.

---

## 32.6.2 axi\_monitor\_trans\_mgr

**Purpose:** Transaction table management for monitor

**Key Features:** - Tracks up to MAX\_TRANSACTIONS concurrent transactions - Maintains transaction state (idle, addr\_received, data\_in\_progress, waiting\_resp, completed) - Out-of-order completion support - Data-before-address handling - Timeout detection integration

#### Transaction Table Entry:

```

typedef struct packed {
 logic [ADDR_WIDTH-1:0] addr;
 logic [ID_WIDTH-1:0] id;
 logic [7:0] len;
 logic [2:0] size;
 logic [1:0] burst;
}

```

```

 logic [31:0] timestamp;
 bus_trans_state_t state;
 logic [7:0] beat_count;
} bus_transaction_t;

```

**Operations:** - **Allocate:** Find free slot for new transaction - **Update:** Modify transaction state/beat count - **Complete:** Mark transaction done, free slot - **Lookup:** Find transaction by ID - **Timeout Check:** Detect stalled transactions

**Usage:** Instantiated within axi\_monitor\_base. Handles all transaction lifecycle management.

---

### 32.6.3 axi\_monitor\_reporter

**Purpose:** Generate standardized 128-bit monitor\_packet\_t records (paired with a 64-bit side-band timestamp)

**Key Features:** - Consolidates events from trans\_mgr and timeout modules - Formats packets according to monitor\_common\_pkg::monitor\_packet\_t - Handles packet priorities and arbitration - Integrates error flags, completion notifications, performance metrics

**Packet Format (128 bits):**

```

[127:124] Packet Type (4 bits)
[123:109] Reserved (15 bits, forward-compat slack)
[108:105] Protocol (4 bits)
[104:97] Event Code (8 bits)
[96:88] Channel ID (9 bits)
[87:72] Agent ID (16 bits)
[71:64] Unit ID (8 bits)
[63:0] Event Data (64 bits)

```

**Event Data Content (depends on packet type):** - **ERROR:** Address[34:0] or error code - **COMPL:** Transaction ID + beat count - **TIMEOUT:** Channel identifier + timeout count - **THRESH:** Threshold value + current value - **PERF:** Latency or throughput metric - **DEBUG:** State machine info + transaction data

**Usage:** Instantiated within axi\_monitor\_base. Converts internal events to standardized packets.

---

### 32.6.4 axi\_monitor\_timeout

**Purpose:** Detect timeout conditions on AXI channels

**Key Features:** - Separate counters for AR/AW, R/W, B channels - Configurable timeout thresholds (0-15 ticks) - Integration with timer tick for time base - Per-transaction timeout tracking

**Configuration:**

```

input logic [3:0] cfg_addr_cnt, // Address channel timeout (ticks)
input logic [3:0] cfg_data_cnt, // Data channel timeout (ticks)
input logic [3:0] cfg_resp_cnt // Response channel timeout (ticks)

```

#### Timeout Detection:

Timer tick → Increment active channel counters

- Compare with threshold
- Generate timeout event if exceeded
- Reset counter on channel activity

**Usage:** Instantiated within axi\_monitor\_base. Generates timeout packets for reporter.

---

### 32.6.5 axi\_monitor\_timer

**Purpose:** Frequency-invariant timer for monitor infrastructure

**Key Features:** - Configurable frequency divider (0-15 selection) - Generates periodic tick for timeout counters - Independent of system clock frequency - Low overhead implementation

#### Configuration:

```

input logic [3:0] cfg_freq_sel // Frequency selection (0-15)

```

#### Frequency Table:

```

cfg_freq_sel = 0 → Tick every 1 cycle
cfg_freq_sel = 1 → Tick every 2 cycles
cfg_freq_sel = 2 → Tick every 4 cycles
...
cfg_freq_sel = 15 → Tick every 32768 cycles

```

**Usage:** Instantiated within axi\_monitor\_base. Provides time base for timeout detection.

---

### 32.6.6 amba\_clock\_gate\_ctrl

**Purpose:** Dynamic clock gating controller for AMBA modules

**Key Features:** - Idle detection with configurable threshold - Automatic gating after idle period - Fast ungating on activity (0 cycles) - Safe clock gating (glitch-free) - Clock cycle counting for power analysis

#### Parameters:

```

module amba_clock_gate_ctrl #(
 parameter int CG_IDLE_COUNT_WIDTH = 4 // Idle counter width
)

```

#### Configuration:

```
input logic cfg_cg_enable, // Enable gating
input logic [CG_IDLE_COUNT_WIDTH-1:0] cfg_cg_idle_count // Idle
cycles before gate
```

#### Status:

```
output logic cg_gating, // Currently gated
output logic [31:0] cg_clk_count // Cumulative gated cycles
```

#### Gating Logic:

Activity detection → Reset idle counter  
No activity → Increment idle counter  
Idle >= threshold && enable → Gate clock  
Activity detected → Ungate immediately

**Usage:** Used by all \*\_cg clock-gated variants (axi4, axil4, axis, apb monitors).

---

## 32.7 Arbitration

---

### 32.7.1 monbus\_arbiter

**Purpose:** Round-robin arbiter for monitor bus aggregation

**Key Features:** - N-way arbitration (up to 16 agents) - Fair round-robin scheduling - Zero bubble overhead (back-to-back grants) - Configurable arbiter width

#### Parameters:

```
module monbus_arbiter #(
 parameter int N = 4, // Number of requesters
 parameter int DATA_WIDTH = 64 // Monitor bus width
)
```

#### Interfaces:

```
// Per-agent inputs
input logic [N-1:0] i_request,
input logic [N-1:0][DATA_WIDTH-1:0] i_data,
output logic [N-1:0] o_grant,

// Aggregated output
output logic o_valid,
output logic [DATA_WIDTH-1:0] o_data
```

#### Arbitration:

Request[0..N-1] → Round-robin priority rotation  
→ Grant highest priority requester  
→ Output granted packet  
→ Rotate priority for fairness

**Usage:** Aggregate monitor packets from multiple AXI/APB monitors into single stream.

**Example:**

```
// 4 AXI monitors → 1 monitor bus
monbus_arbiter #(.N(4), .DATA_WIDTH(64)) u_arb (
 .i_request({mon3_valid, mon2_valid, mon1_valid, mon0_valid}),
 .i_data({mon3_pkt, mon2_pkt, mon1_pkt, mon0_pkt}),
 .o_grant({mon3_ready, mon2_ready, mon1_ready, mon0_ready}),
 .o_valid(agg_valid),
 .o_data(agg_pkt)
);
```

---

### 32.7.2 arbiter\_monbus\_common

**Purpose:** Shared arbitration logic for all monitor bus arbiters

**Key Features:** - Reusable priority encoding - Mask-based round-robin - Optimized for monitor bus (64-bit packets)

**Usage:** Base module for arbiter\_rr\_pwm\_monbus and arbiter\_wrr\_pwm\_monbus.

---

### 32.7.3 arbiter\_rr\_pwm\_monbus

**Purpose:** Round-robin arbiter with PWM output for monitor bus

**Key Features:** - Equal priority round-robin - PWM output mode for power management - Optimized for packet streams

**Usage:** Alternative to monbus\_arbiter with PWM support.

---

### 32.7.4 arbiter\_wrr\_pwm\_monbus

**Purpose:** Weighted round-robin arbiter for monitor bus

**Key Features:** - Configurable per-agent weights - Weighted priority allocation - Fair service guarantees

**Configuration:**



```
input logic [N-1:0][7:0] cfg_weights // Per-agent weight (0-255)
```

**Scheduling:**

High-weight agents get more grants  
Low-weight agents get fewer grants  
Maintains fairness across epochs

**Usage:** Use when different monitors have different importance levels.

---

## 32.8 AXI Utilities

---

### 32.8.1 axi\_gen\_addr

**Purpose:** Generate next address for AXI burst transactions

**Key Features:** - Supports INCR, WRAP, FIXED burst types - Handles all AXI data widths (32-512 bits) - Proper address alignment - Boundary wrapping for WRAP bursts

**Parameters:**

```
module axi_gen_addr #(
 parameter int AW = 32, // Address width
 parameter int DW = 32, // Data width
 parameter int ODW = 32, // Output data width (for width
conversion)
 parameter int LEN = 8 // Length field width
)
```

**Inputs:**

```
input logic [AW-1:0] curr_addr, // Current address
input logic [2:0] size, // axsize (0-7)
input logic [1:0] burst, // axburst (FIXED/INCR/WRAP)
input logic [LEN-1:0] len // axlen (burst length)
```

**Outputs:**

```
output logic [AW-1:0] next_addr, // Next address in burst
output logic [AW-1:0] next_addr_align // Aligned next address
```

**Address Calculation:**

**INCR (0x01):**

next\_addr = curr\_addr + (1 << size)

**WRAP (0x10):**

```
// Wrap at boundary = (len + 1) * (1 << size)
boundary = (len + 1) << size;
next_addr = (curr_addr + (1 << size)) & ~(boundary - 1);
```

**FIXED (0x00):**

```
next_addr = curr_addr // Same address
```

**Usage:** Used by axi4\_to\_apb\_convert for burst decomposition. Also useful for any AXI address calculation.

**Example:**

```
axi_gen_addr #(.AW(32), .DW(64), .ODW(32)) u_addr_gen (
 .curr_addr(32'h1000),
 .size(3'b010), // 4 bytes
 .burst(2'b01), // INCR
 .len(8'd3), // 4 beats
 .next_addr(next_addr) // → 32'h1004, 32'h1008, 32'h100C
);
```

---

### 32.8.2 axi\_master\_rd\_splitter

**Purpose:** Split AXI read into separate AR and R channels

**Key Features:** - Independent AR and R skid buffers - Configurable buffer depths - Zero-latency path option

**Usage:** Decouple AR and R channels for timing closure or protocol adaptation.

---

### 32.8.3 axi\_master\_wr\_splitter

**Purpose:** Split AXI write into separate AW, W, and B channels

**Key Features:** - Independent channel buffering - AW-W synchronization - B channel response routing

**Usage:** Decouple write channels for complex write paths.

---

### 32.8.4 axi\_split\_combi

**Purpose:** Combined read + write splitter

**Key Features:** - Bidirectional split (all 5 AXI channels) - Shared configuration - Compact implementation

**Usage:** Full AXI split for complex bridges or adapters.

---

### 32.8.5 cdc\_4\_phase\_handshake

**Purpose:** Clock domain crossing with valid/ready handshake

**Key Features:** - Dual-clock asynchronous CDC - Data + control crossing - Gray-code free pointer-based - Safe for arbitrary clock ratios - Zero data loss

**Parameters:**

```
module cdc_4_phase_handshake #(
 parameter int DATA_WIDTH = 8
)
```

**Interfaces:**

```
// Source clock domain
input logic clk_src,
input logic rst_src_n,
input logic src_valid,
output logic src_ready,
input logic [DATA_WIDTH-1:0] src_data,

// Destination clock domain
input logic clk_dst,
input logic rst_dst_n,
output logic dst_valid,
input logic dst_ready,
output logic [DATA_WIDTH-1:0] dst_data
```

**FSM States:**

**Source Domain:**

S\_IDLE → Ready for new data  
S\_WAIT\_ACK → Waiting for destination ack  
S\_WAIT\_ACK\_CLR → Waiting for ack to clear

**Destination Domain:**

D\_IDLE → Waiting for request  
D\_WAIT\_READY → Request received, waiting for ready  
D\_WAIT\_REQ\_CLR → Ack sent, waiting for request clear

**Timing:** - Latency: 3-5 cycles (depends on clock ratio) - Synchronization: 3-stage shift register - Handshake: 4-phase (req → ack → req\_clr → ack\_clr)

**Usage:** Used extensively in axi4\_to\_apb\_shim for cmd/rsp CDC. General-purpose CDC utility.

### Example:

```
cdc_4_phase_handshake #(.DATA_WIDTH(32)) u_cdc (
 .clk_src(clk_100mhz),
 .rst_src_n(rst_n),
 .src_valid(cmd_valid),
 .src_ready(cmd_ready),
 .src_data(cmd_data),

 .clk_dst(clk_200mhz),
 .rst_dst_n(rst_n),
 .dst_valid(req_valid),
 .dst_ready(req_ready),
 .dst_data(req_data)
);
```

---

## 32.9 Testing

---

Shared modules are tested indirectly through higher-level protocol modules:

```
Monitor infrastructure tested via monitor tests
```

```
pytest val/amba/test_axi4*_mon.py -v
```

```
pytest val/amba/test_axil4*_mon.py -v
```

```
pytest val/amba/test_apb_monitor.py -v
```

```
CDC tested via shims
```

```
pytest val/integ_amba/test_axi2apb_shim.py -v
```

```
Address generation tested via conversion
```

```
pytest val/integ_amba/test_axi2apb_shim.py -v
```

---

## 32.10 Design Notes

---

### 32.10.1 Monitor Infrastructure

#### Architecture:

Transaction → axi\_monitor\_trans\_mgr (track state)  
→ axi\_monitor\_timeout (detect stalls)  
→ axi\_monitor\_reporter (generate packets)  
→ axi\_monitor\_filtered (apply filters)  
→ Monitor bus output

## Packet Flow:

axi\_monitor\_base → Unfiltered packets  
→ axi\_monitor\_filtered (if enabled)  
→ Filtered packets  
→ FIFO/arbiter  
→ System monitor bus

### 32.10.2 When to Use Each Module

✓ **Use axi\_monitor\_base When:** - Creating custom monitor variants - Need full control over filtering - Integrating with custom infrastructure

✓ **Use axi\_monitor\_filtered When:** - Standard monitor usage - Need packet filtering - Using with monitor bus aggregation

✓ **Use cdc\_4\_phase\_handshake When:** - Clock domain crossing required - Valid/ready protocol - Arbitrary clock ratios - Safe data transfer needed

✓ **Use axi\_gen\_addr When:** - Implementing AXI bridges - Burst address calculation - Protocol conversion

✓ **Use monbus\_arbiter When:** - Multiple monitors on same bus - Need fair arbitration - Aggregating monitor streams

---

## 32.11 Configuration Best Practices

---

### 32.11.1 Monitor Configuration

#### Functional Debug Mode (Recommended Default):

```
.cfg_error_enable(1'b1), // Catch errors
.cfg_compl_enable(1'b1), // Track completions
.cfg_timeout_enable(1'b1), // Detect hangs
.cfg_perf_enable(1'b0), // ⚠️ DISABLE (high traffic)
.cfg_debug_enable(1'b0) // Only if needed
```

#### Performance Analysis Mode:

```
.cfg_error_enable(1'b1), // Still catch errors
.cfg_compl_enable(1'b0), // ⚠️ DISABLE
.cfg_timeout_enable(1'b0),
.cfg_perf_enable(1'b1), // Enable metrics
.cfg_debug_enable(1'b0)
```

⚠️ **NEVER enable cfg\_compl\_enable and cfg\_perf\_enable together!** This causes packet congestion.

### 32.11.2 Timeout Configuration

#### Conservative (Slow Peripherals):

```
.cfg_freq_sel(4'd8), // Slower tick rate
.cfg_addr_cnt(4'd15), // Long timeout
.cfg_data_cnt(4'd15),
.cfg_resp_cnt(4'd15)
```

#### Aggressive (Fast Detection):

```
.cfg_freq_sel(4'd0), // Fast tick rate
.cfg_addr_cnt(4'd3), // Short timeout
.cfg_data_cnt(4'd3),
.cfg_resp_cnt(4'd3)
```

### 32.11.3 Clock Gating Configuration

#### Aggressive Gating:

```
.cfg_cg_enable(1'b1),
.cfg_cg_idle_count(4'd1) // Gate after 1 idle cycle
```

#### Conservative Gating:

```
.cfg_cg_enable(1'b1),
.cfg_cg_idle_count(4'd10) // Gate after 10 idle cycles
```

---

## 32.12 Performance Characteristics

---

### 32.12.1 Monitor Overhead

**Resource Usage (per monitor):** - axi\_monitor\_base: ~500 LUTs, ~400 FFs - axi\_monitor\_filtered: +50 LUTs, +30 FFs - Total: ~550 LUTs, ~430 FFs per monitor

**Packet Generation Rate:** - Completion packets: 1 per transaction - Error packets: As needed (protocol violations) - Timeout packets: Low frequency (cfg\_freq\_sel dependent) - Performance packets: 1 per N transactions (configurable)

### 32.12.2 CDC Latency

**cdc\_4\_phase\_handshake:** - Minimum: 6 cycles (src → dst → src) - Typical: 8-12 cycles - Maximum: Depends on clock ratio

### 32.12.3 Arbitration

**monbus\_arbiter:** - Latency: 1 cycle - Throughput: 1 packet/cycle (no bubbles) - Fairness: Perfect round-robin

---

## 32.13 Synthesis Notes

---

### 32.13.1 Clock Gating

**amba\_clock\_gate\_ctrl:** - Implements glitch-free clock gating - Use vendor clock gating cells for best results - Requires proper clock gating constraints

**Constraints (example):**

```
set_clock_gating_check -setup 0.1 [get_clocks aclk]
set_dont_touch [get_cells */u_clock_gate_ctrl/*]
```

### 32.13.2 CDC Constraints

**cdc\_4\_phase\_handshake:**

```
set_false_path -from [get_clocks clk_src] -to [get_clocks clk_dst]
set_false_path -from [get_clocks clk_dst] -to [get_clocks clk_src]
```

*# Constrain synchronizer paths*

```
set_max_delay -from */r_req_src -to */r_req_sync[0] 10.0
set_max_delay -from */r_ack_dst -to */r_ack_sync[0] 10.0
```

---

## 32.14 Related Documentation

---

### 32.14.1 RTL Design Sherpa Documentation

- [RTLamba Overview](#) - Complete AMBA architecture
- [AXI4 Monitors](#) - Uses monitor infrastructure
- [APB Monitor](#) - Uses monitor infrastructure
- [Shims](#) - Uses CDC and address generation

### 32.14.2 Monitor Package

- [Monitor Package Spec](#) - Event packet format

### 32.14.3 Source Code

- RTL: rtl/amba/shared/
  - Monitor package: rtl/amba/monitor\_pkg.sv
  - Tests: Indirect via val/amba/test\_\*\_mon.py
-

## 32.15 Summary

---

The shared subsystem provides essential infrastructure:

**Monitor Infrastructure:** - Complete transaction tracking and event generation - Configurable filtering and timeout detection - Standardized 64-bit event packets - Multi-protocol support (AXI, APB, AXIS)

**Arbitration:** - Fair round-robin scheduling - Weighted priority support - Zero-overhead aggregation

**Utilities:** - Clock domain crossing (cdc\_4\_phase\_handshake) - AXI address generation (axi\_gen\_addr) - Channel splitting (axi\_\*\_splitter) - Clock gating (amba\_clock\_gate\_ctrl)

**Usage Pattern:** Most users interact with higher-level protocol modules (axi4\_master\_rd\_mon, etc.) which internally use these shared components. Direct instantiation is rare except for custom implementations.

---

**Last Updated:** 2025-10-24

---

## 32.16 Navigation

---

- [← Back to RTLAmba Index](#)
- [← Back to Main Documentation Index](#)

# 33 SDPRAM Core

**Module:** sdpram\_core.sv **Location:** rtl/amba/shared/ **Status:** Production Ready

---

## 33.1 Overview

---

sdpram\_core is the protocol-agnostic Simple Dual-Port RAM backend shared by the four sdpram\_slave\_\* protocol wrappers. It owns the inferred dual-port BRAM array, a burst-aware write tracker, a burst-aware read tracker, and a bulk-clear FSM. It exposes a single FUB-shaped slave interface (an AXI superset with id / addr / len / size / burst on the address channels and id / resp / last on the response channels) so that a protocol-specific wrapper can drop straight on top without any string-switch generate plumbing.



### 33.1.1 Key Features

- Inferred simple dual-port BRAM: one write port (port A) and one read port (port B)
  - No reset on the RAM array (BRAM contents are undefined at power-up, cleared only via the bulk-clear FSM)
  - FPGA `ram_style = "auto"` attribute for tool-driven BRAM inference
  - Burst-aware write and read trackers driven by `axi_gen_addr` (INCR / FIXED any length up to AXI4's 256-beat max)
  - Byte-enabled writes via `fub_wstrb`
  - Single-cycle read latency on port B
  - Bulk-clear FSM that walks the whole array to zero, gated on both sides idle
  - FUB-shaped interface that degenerates cleanly to single-beat for AXIL wrappers
  - Observation outputs (valid/ready snapshot, BRAM write/read fire pulses)
- 

## 33.2 Module Purpose

---

The `sdpram_slave_*` family needs one memory kernel that behaves identically regardless of which AMBA protocol drives its write and read sides. Rather than duplicate the BRAM, burst logic, and clear FSM in every protocol permutation, that logic lives once here, and the wrappers translate their protocol's FUB into this core's AXI-shaped FUB.

The core speaks exactly one wire format. AXI4 wrappers pass the real `awlen` / `awsize` / `awburst` / `awid` fields straight through; AXIL wrappers, which have no burst or ID fields, feed single-beat defaults (`awlen=0`, `awsize=$clog2(STRB_W)`, `awburst=INCR`, `awid=0`) so the burst tracker collapses to a single-beat path.

**Use Cases:** - Shared backend for all four `sdpram_slave_{axi4,axil}_{axi4,axil}` wrappers - Memory model / descriptor-RAM / semaphore-RAM inside characterization harnesses - Memory ring backend behind a monitor-bus capture master-write port - Any test bench needing a synthesizable dual-port scratch memory with a fast clear

**Key Benefit:** One compute kernel with all the burst and clear complexity in a single place, so the protocol wrappers stay thin and no "fake AXI4" fields leak into any external port list.

---

## 33.3 Parameters

Parameter	Type	Default	Description
AXI_ID_WIDTH	int	8	FUB transaction-ID width. Passthrough on AXI4 wrappers; tied to a 1-bit zero by AXIL wrappers
ADDR_WIDTH	int	32	Byte-address width on both FUB address channels
DATA_WIDTH	int	256	Data-bus / BRAM word width (bits)
MEM_DEPTH	int	2048	Number of BRAM words (array depth)

**Derived localparams:** - STRB\_W = DATA\_WIDTH / 8 — write-strobe / byte-enable width - ADDR\_LSB =  $\$clog2(STRB\_W)$  — byte-offset bits below the word address - MEM\_AW =  $\$clog2(MEM\_DEPTH)$  — BRAM word-address width - WORD\_AW = ADDR\_WIDTH - ADDR\_LSB — full word-address width

## 33.4 Port Groups

### 33.4.1 Clock and Reset

Port	Direction	Width	Description
aclk	input	1	Clock
aresetn	input	1	Active-low asynchronous reset (control logic only; never resets the BRAM array)

### 33.4.2 FUB Write Side (AW + W + B)

Port	Direction	Width	Description
fub_awid	input	AXI_ID_WIDTH	Write transaction ID (echoed back on fub_bid)
fub_awadd	input	ADDR_WIDTH	Write burst base byte-

Port	Direction	Width	Description
r		H	address
fub_awlen	input	8	Burst length minus 1 (0 = single beat)
fub_awsiz	input	3	Beat size (log2 bytes) fed to axi_gen_addr
fub_awburst	input	2	Burst type: INCR (2'b01) / FIXED (2'b00) / WRAP (2'b10)
fub_awvalid	input	1	Write-address valid
fub_awready	output	1	Write-address ready (accepts when no write active, no B pending, not clearing)
fub_wdata	input	DATA_WIDTH	Write data
fub_wstrb	input	DATA_WIDTH/8	Per-byte write strobe
fub_wvalid	input	1	Write-data valid
fub_wready	output	1	Write-data ready (asserted while a write burst is active and not clearing)
fub_bid	output	AXI_ID_WIDTH	Write-response ID (captured from fub_awid)
fub_bresp	output	2	Write response (OKAY 2'b00; SLVERR 2'b10 reserved for out-of-range)
fub_bvalid	output	1	Write-response valid (asserted while B pending)
fub_bready	input	1	Write-response ready

### 33.4.3 FUB Read Side (AR + R)

Port	Direction	Width	Description
fub_arid	input	AXI_ID_WIDTH	Read transaction ID (echoed on fub_rid)
fub_araddr	input	ADDR_WIDTH	Read burst base byte-address
fub_arlen	input	8	Burst length minus 1 (0 = single beat)
fub_arsize	input	3	Beat size (log2 bytes) fed to axi_gen_addr
fub_arburst	input	2	Burst type: INCR / FIXED / WRAP
fub_arvalid	input	1	Read-address valid
fub_arready	output	1	Read-address ready (accepts when no read active, not clearing)
fub_rid	output	AXI_ID_WIDTH	Read-data ID (from the inflight tracker)
fub_rdata	output	DATA_WIDTH	Read data (BRAM port-B output)
fub_rresp	output	2	Read response (OKAY / SLVERR reserved)
fub_rlast	output	1	Last-beat marker of the read burst
fub_rvalid	output	1	Read-data valid (reflects the inflight tracker)
fub_rready	input	1	Read-data ready

### 33.4.4 Bulk-Clear Control

Port	Direction	Width	Description
i_cfg_start_clear	input	1	Request a full-array clear (accepted only when both sides idle)

Port	Direction	Width	Description
o_cfg_done_clear	output	1	Asserted when the clear walk has finished

### 33.4.5 Observation

Port	Direction	Width	Description
o_dbg_fub_vr	output	10	FUB-side valid/ready snapshot: {rready,rvalid,arready,arvalid,bready,bvalid,wready,wvalid,awready,awvalid}
o_dbg_bram_wr	output	1	One-cycle pulse when a BRAM port-A write fires
o_dbg_bram_rd	output	1	One-cycle pulse when a BRAM port-B read issues

## 33.5 Functional Description

### 33.5.1 FUB Interface Contract

The core speaks a single wire format — a FUB-shaped AXI superset — and nothing else. On the address channels it carries `id / addr / len / size / burst`; on the response channels it carries `id / resp / last`. Wrappers translate their protocol into this shape:

- **AXI4 wrappers** pass the real `awlen / awsize / awburst / awid` (and the AR equivalents) straight through.
- **AXIL wrappers** feed defaults: `awlen = 0, awsize = $clog2(STRB_W), awburst = 2'b01 (INCR), awid = 0`. With `len = 0`, the burst tracker degenerates to a single-beat path.

### 33.5.2 Write Path — Burst Tracker

On `fub_awvalid && fub_awready`, the tracker latches the burst's ID, base address, beat count (`awlen`), size, and burst type, and asserts `r_wr_active`. Each accepted W beat (`fub_wvalid && fub_wready`) writes byte-enabled data into BRAM port A at the current word address, then advances the address through `axi_gen_addr` and decrements the beat counter. When the last beat is accepted (`r_wr_beats_left == 0`), the tracker drops `r_wr_active`, latches a pending B response (`r_b_pending`), and captures the ID and response code. `fub_awready` deasserts while a write is active or a B is pending, so only one write burst is in flight at a time.

### 33.5.3 Read Path — Burst Tracker

On `fub_arvalid && fub_arready`, the read tracker latches the burst parameters and asserts `r_rd_active`. A read beat issues (`read_issue`) when the burst is active, the core is not clearing, and either the inflight register is empty or the current beat is being consumed (`!r_inflight || fub_rready`). Each issue reads BRAM port B at the current word address (one-cycle latency), advances the address through `axi_gen_addr`, decrements the beat count, and — on the last beat — drops `r_rd_active`. An inflight tracker holds the (`id`, `rresp`, `rlast`) for the beat currently presented on `fub_r`, driving `fub_rvalid`; it clears on handshake unless a new issue refills it the same cycle.

### 33.5.4 Address Generation

Both paths use an `axi_gen_addr` instance (parameterized `AW=ADDR_WIDTH`, `DW=ODW=DATA_WIDTH`, `LEN=8`) to compute the next beat address from the current address, size, and burst type. `INCR` and `FIXED` are handled directly by the glue. `WRAP` is computed correctly by `axi_gen_addr` but the BRAM address glue advances linearly, so the AXI4 wrappers carry a sim-only assertion flagging `WRAP` at the sim boundary until it has been exercised.

### 33.5.5 BRAM Array

The memory is an inferred dual-port array (`(* ram_style = "auto" *) logic [DATA_WIDTH-1:0] r_mem [MEM_DEPTH]`). It is **never reset** — BRAM contents come up undefined and are only zeroed by the bulk-clear FSM. Port A is shared between the clear FSM (when clearing) and byte-enabled writes; port B is a registered read (single-cycle latency). A benign `WIDTHTRUNC` lint-off covers Verilator computing an extra index bit for the never-taken clear wrap path; every index that reaches the array is provably in `[0, MEM_DEPTH-1]` by tracker construction.

### 33.5.6 Bulk-Clear FSM

The clear FSM has two states (`CLR_IDLE`, `CLR_BUSY`). It accepts `i_cfg_start_clear` only when both sides report idle (`!r_wr_active && !r_b_pending && !r_rd_active && !r_inflight`), which keeps it from glitching `fub*_ready` mid-transaction. Once busy, it walks `r_clear_addr` from 0 to `MEM_DEPTH-1`, writing zero to BRAM port A each cycle, then returns to idle and asserts `o_cfg_done_clear`. While `w_clearing` is asserted, `fub_awready`, `fub_wready`, and `fub_arready` are all deasserted so the FSM owns port A cleanly.

### 33.5.7 Observation Outputs

`o_dbg_fub_vr` packs the ten FUB valid/ready bits for waveform-free probing. `o_dbg_bram_wr` pulses on a real byte-enabled write fire, and `o_dbg_bram_rd` pulses on each read issue.

---

## 33.6 Usage Example

---

sdpram\_core is not usually instantiated directly — the sdpram\_slave\_\* wrappers do that. The pattern (as used inside every wrapper) is:

```
sdpram_core #(
 .AXI_ID_WIDTH (AXI_ID_WIDTH),
 .ADDR_WIDTH (ADDR_WIDTH),
 .DATA_WIDTH (DATA_WIDTH),
 .MEM_DEPTH (MEM_DEPTH)
) u_core (
 .aclk (aclk),
 .aresetn (aresetn),

 // Write FUB (from a slave-write leaf, or single-beat defaults for
 AXIL)
 .fub_awid (fub_awid),
 .fub_awaddr (fub_awaddr),
 .fub_awlen (fub_awlen), // 8'h00 for AXIL
 .fub_awsz (fub_awsz), // $clog2(STRB_W) for AXIL
 .fub_awburst (fub_awburst), // 2'b01 (INCR) for AXIL
 .fub_awvalid (fub_awvalid),
 .fub_awready (fub_awready),
 .fub_wdata (fub_wdata),
 .fub_wstrb (fub_wstrb),
 .fub_wvalid (fub_wvalid),
 .fub_wready (fub_wready),
 .fub_bid (fub_bid),
 .fub_bresp (fub_bresp),
 .fub_bvalid (fub_bvalid),
 .fub_bready (fub_bready),

 // Read FUB (from a slave-read leaf)
 .fub_arid (fub_arid),
 .fub_araddr (fub_araddr),
 .fub_arlen (fub_arlen),
 .fub_arsize (fub_arsize),
 .fub_arburst (fub_arburst),
 .fub_arvalid (fub_arvalid),
 .fub_arready (fub_arready),
 .fub_rid (fub_rid),
 .fub_rdata (fub_rdata),
 .fub_rresp (fub_rresp),
 .fub_rlast (fub_rlast),
 .fub_rvalid (fub_rvalid),
 .fub_rready (fub_rready),
```

```
// Bulk clear + debug
.i_cfg_start_clear (i_cfg_start_clear),
.o_cfg_done_clear (o_cfg_done_clear),
.o_dbg_fub_vr (o_dbg_fub_vr),
.o_dbg_bram_wr (o_dbg_bram_wr),
.o_dbg_bram_rd (o_dbg_bram_rd)
);
```

---

## 33.7 Design Notes

---

### 33.7.1 No Reset on the RAM Array

The BRAM array is deliberately not reset. Resetting a large inferred RAM prevents BRAM inference on most FPGA flows and costs an enormous number of flip-flops on ASIC. Contents are undefined at power-up; software (or a harness) issues `i_cfg_start_clear` to zero the array when a defined starting state is required.

### 33.7.2 One Wire Format, Four Wrappers

The core exists so that the burst logic, byte-enable write, single-cycle read, and clear FSM are written exactly once. The four protocol permutations are thin wrappers that only translate their protocol's FUB into this AXI-shaped FUB. AXIL sides supply the burst/ID defaults in exactly one place per wrapper, so no external port list ever exposes a “fake AXI4” field.

### 33.7.3 WRAP Bursts

`axi_gen_addr` computes WRAP addresses correctly, but the BRAM glue here advances linearly. Until the WRAP path is exercised end-to-end, the AXI4 wrappers assert (sim-only) that `awburst/arbust` is not WRAP at the interface boundary. INCR and FIXED are fully supported.

### 33.7.4 Single Outstanding Burst Per Side

Each side accepts one burst at a time — `fub_awready` deasserts while a write is active or a B is pending, and `fub_arready` deasserts while a read is active. This keeps the tracker state minimal and matches the descriptor-RAM / scratch-memory use cases the family targets.

---

## 33.8 Related Modules

---

### 33.8.1 Used By

- `sdpram_slave_axi4_axi4.sv` — AXI4 write + AXI4 read wrapper
- `sdpram_slave_axi4_axil.sv` — AXI4 write + AXIL read wrapper



- `sdpram_slave_axil_axi4.sv` — AXIL write + AXI4 read wrapper
- `sdpram_slave_axil_axil.sv` — AXIL write + AXIL read wrapper
- Characterization harnesses (e.g. `rapids_char_harness` descriptor RAM, `stream_char_harness` memory ring)

### 33.8.2 Uses

- **`axi_gen_addr.sv`** — Next-beat address generation for both trackers
- **`reset_defs.svh`** — `ALWAYS_FF_RST` / `RST_ASSERTED` reset macros

### 33.8.3 See Also

- **`sdpram_slave_axi4_axi4.sv`** — the AXI4/AXI4 protocol wrapper
  - **`sdpram_slave_axil_axil.sv`** — the AXIL/AXIL protocol wrapper
  - **`monbus_group_core.sv`** — a comparable “one core + protocol wrappers” split for the monitor-bus delivery layer
- 

## 33.9 References

---

### 33.9.1 Source Code

- RTL: `rtl/amba/shared/sdpram_core.sv`
- Address gen: `rtl/amba/shared/axi_gen_addr.sv`
- Wrappers: `rtl/amba/shared/sdpram_slave_*.sv`

### 33.9.2 Documentation

- Architecture: `docs/markdown/RTLAmba/shared/README.md`
  - Subsystem guide: `rtl/amba/CLAUDE.md`
- 

**Last Updated:** 2026-07-15

---

## 33.10 Navigation

---

- [Back to Shared Infrastructure Index](#)
- [Back to RTLAmba Index](#)

## 34 SDPRAM Slave — AXI4 Write / AXI4 Read

**Module:** `sdpram_slave_axi4_axi4.sv` **Location:** `rtl/amba/shared/` **Status:** Production Ready

---

### 34.1 Overview

---

`sdpram_slave_axi4_axi4` is one of four protocol-pair wrappers that place a Simple Dual-Port RAM behind two independent AMBA slave interfaces — a write-side slave and a read-side slave. This variant exposes a **full AXI4 slave** on both the write side (AW + W + B) and the read side (AR + R). It directly instantiates the native `axi4_slave_wr` and `axi4_slave_rd` skid leaf modules and pipes their FUB-side outputs into the shared `sdpram_core` backend.

#### 34.1.1 Key Features

- Full AXI4 slave on both write and read sides (burst, ID, size)
  - Native `axi4_slave_wr` + `axi4_slave_rd` skid leaves (no protocol emulation)
  - Shared `sdpram_core` backend — one BRAM, one clear FSM, one burst tracker set
  - Byte-enabled writes, single-cycle-latency reads
  - Bulk-clear control input and done flag
  - Full complement of debug taps (external and FUB valid/ready, BRAM fire pulses, leaf busy)
  - Sim-only assertion flagging unvalidated WRAP bursts
- 

### 34.2 Module Purpose

---

Characterization harnesses need a synthesizable memory that a device-under-test can read and write over AXI4 while a separate host or checker uses the other port. Splitting the write side and read side into two independent AXI4 slaves lets one agent stream data in while another drains it out, with the BRAM as the shared medium.

This is the pure-AXI4 permutation of the family. When both the producer and consumer speak full AXI4 (bursts, IDs), this wrapper avoids any width- or burst-adaptation on either side.

**Use Cases:** - Memory model behind an AXI4 DUT in a characterization harness - Descriptor RAM with an AXI4 host-write port and an AXI4 engine-read port - Semaphore / scratch RAM shared between two AXI4 masters - General synthesizable dual-port memory with a fast bulk clear

**Key Benefit:** Full AXI4 on both sides with zero fake fields — the burst/ID information flows straight through the skid leaves into the shared core.

---

## 34.3 Parameters

Parameter	Type	Default	Description
AXI_ID_WIDTH	int	8	AXI transaction-ID width on both sides
ADDR_WIDTH	int	32	Byte-address width
DATA_WIDTH	int	256	Data-bus / BRAM word width (bits)
USER_WIDTH	int	1	AXI USER-signal width (carried by leaves, not preserved through BRAM)
MEM_DEPTH	int	2048	BRAM depth in words
SKID_DEPTH_AW	int	2	Write-address skid depth
SKID_DEPTH_W	int	2	Write-data skid depth
SKID_DEPTH_B	int	2	Write-response skid depth
SKID_DEPTH_AR	int	2	Read-address skid depth
SKID_DEPTH_R	int	4	Read-data skid depth

## 34.4 Port Groups

### 34.4.1 Clock and Reset

Port	Direction	Width	Description
aclk	input	1	Clock
aresetn	input	1	Active-low asynchronous reset

### 34.4.2 AXI4 Slave Write (AW + W + B)

Port	Direction	Width	Description
s_axi_awid	input	AXI_ID_WID	Write-address ID

Port	Direction	Width	Description
		TH	
s_axi_awa ddr	input	ADDR_WIDT H	Write burst base address
s_axi_awle n	input	8	Burst length minus 1
s_axi_aws ize	input	3	Beat size (log2 bytes)
s_axi_awb urst	input	2	Burst type (INCR / FIXED / WRAP)
s_axi_awlo ck	input	1	Lock (carried by leaf, ignored by core)
s_axi_awca che	input	4	Cache attributes (ignored by core)
s_axi_awpr ot	input	3	Protection attributes (ignored by core)
s_axi_awq os	input	4	QoS (ignored by core)
s_axi_awre gion	input	4	Region (ignored by core)
s_axi_awus er	input	USER_WIDT H	User sideband (ignored by core)
s_axi_awva lid	input	1	Write-address valid
s_axi_awre ady	output	1	Write-address ready
s_axi_wdat a	input	DATA_WIDT H	Write data
s_axi_wstr b	input	DATA_WIDT H/8	Per-byte write strobe
s_axi_wlast	input	1	Write-data last beat
s_axi_wuse r	input	USER_WIDT H	Write-data user sideband
s_axi_wval	input	1	Write-data valid

Port	Direction	Width	Description
id			
s_axi_wready	output	1	Write-data ready
s_axi_bid	output	AXI_ID_WIDTH	Write-response ID
s_axi_bresp	output	2	Write response
s_axi_buser	output	USER_WIDTH	Write-response user (hardwired to 0)
s_axi_bvalid	output	1	Write-response valid
s_axi_bready	input	1	Write-response ready

#### 34.4.3 AXI4 Slave Read (AR + R)

Port	Direction	Width	Description
s_axi_arid	input	AXI_ID_WIDTH	Read-address ID
s_axi_araddr	input	ADDR_WIDTH	Read burst base address
s_axi_arlen	input	8	Burst length minus 1
s_axi_arsize	input	3	Beat size (log2 bytes)
s_axi_arburst	input	2	Burst type
s_axi_arlock	input	1	Lock (ignored by core)
s_axi_arcache	input	4	Cache attributes (ignored by core)
s_axi_arprot	input	3	Protection attributes (ignored by

Port	Direction	Width	Description
			core)
s_axi_arqos	input	4	QoS (ignored by core)
s_axi_arregion	input	4	Region (ignored by core)
s_axi_aruser	input	USER_WIDTH	User sideband (ignored by core)
s_axi_arvalid	input	1	Read-address valid
s_axi_arready	output	1	Read-address ready
s_axi_rid	output	AXI_ID_WIDTH	Read-data ID
s_axi_rdata	output	DATA_WIDTH	Read data
s_axi_rresp	output	2	Read response
s_axi_rlast	output	1	Read-data last beat
s_axi_ruser	output	USER_WIDTH	Read-data user (hardwired to 0)
s_axi_rvalid	output	1	Read-data valid
s_axi_rready	input	1	Read-data ready

#### 34.4.4 Bulk-Clear Control and Debug

Port	Direction	Width	Description
i_cfg_start_clear	input	1	Request full-array clear (accepted when both sides idle)
o_cfg_done_clear	output	1	Clear-walk complete

Port	Direction	Width	Description
o_dbg_vr	output	10	External valid/ready snapshot {rready,rvalid,arready,arvalid,bready,bvalid,wready,wvalid,awready,awvalid}
o_dbg_fub_vr	output	10	FUB-side valid/ready snapshot from the core
o_dbg_bram_wr	output	1	BRAM write-fire pulse
o_dbg_bram_rd	output	1	BRAM read-issue pulse
o_dbg_bus_wr	output	1	Write-side skid-leaf busy
o_dbg_bus_rd	output	1	Read-side skid-leaf busy

## 34.5 Functional Description

### 34.5.1 Structure

The wrapper is three instances and a little glue:

1. `axi4_slave_wr` accepts the external AXI4 write channels and presents a FUB-shaped write interface.
2. `axi4_slave_rd` accepts the external AXI4 read channels and presents a FUB-shaped read interface.
3. `sdpram_core` consumes both FUBs, owns the BRAM, the burst trackers, and the clear FSM.

### 34.5.2 Field Pass-Through and Tie-Offs

Because both sides are full AXI4, the real `awid` / `awlen` / `awsize` / `awburst` (and the AR equivalents) flow straight through the leaves into the core. The AXI4-only qualifier fields the core does not consume — `lock`, `cache`, `prot`, `qos`, `region`, `user`, and `wlast` — are carried through the leaf FUB and terminated on `*_unused` nets. The core does not preserve `user` across the BRAM, so `s_axi_buser` and `s_axi_ruser` are hardwired to zero (matching legacy behavior).

### 34.5.3 Memory Behavior

All memory semantics live in `sdpram_core`: byte-enabled writes on BRAM port A, single-cycle-latency reads on port B, one outstanding burst per side, and the bulk-clear FSM gated on both sides idle. See `sdpram_core` for the full backend description.

### 34.5.4 Debug Taps

`o_dbg_vr` mirrors the five external channel valid/ready pairs; `o_dbg_fub_vr` mirrors the same for the internal FUB. `o_dbg_bram_wr / o_dbg_bram_rd` pulse on real BRAM accesses, and `o_dbg_busy_wr / o_dbg_busy_rd` expose each skid leaf's busy flag.

### 34.5.5 WRAP Assertion

A `translate_off` block asserts on each accepted AW and AR that `awburst/arbust` is not WRAP (`2'b10`). WRAP is computed by `axi_gen_addr` but not yet validated through the BRAM glue, so the assertion warns at the sim boundary.

---

## 34.6 Usage Example

---

```
sdpram_slave_axi4_axi4 #(
 .AXI_ID_WIDTH (8),
 .ADDR_WIDTH (32),
 .DATA_WIDTH (256),
 .MEM_DEPTH (2048)
) u_mem (
 .aclk (aclk),
 .aresetn (aresetn),

 // Write-side AXI4 slave (from the producer master)
 .s_axi_awid (dut_awid),
 .s_axi_awaddr (dut_awaddr),
 .s_axi_awlen (dut_awlen),
 .s_axi_awsz (dut_awsz),
 .s_axi_awburst (dut_awburst),
 .s_axi_awlock (dut_awlock),
 .s_axi_awcache (dut_awcache),
 .s_axi_awprot (dut_awprot),
 .s_axi_awqos (dut_awqos),
 .s_axi_awregion(dut_awregion),
 .s_axi_awuser (dut_awuser),
 .s_axi_awvalid (dut_awvalid),
 .s_axi_awready (dut_awready),
 // ... W, B, and the AR/R read channels ...

 .i_cfg_start_clear (clear_pulse),
```



```

.o_cfg_done_clear (clear_done),
.o_dbg_vr (dbg_vr),
.o_dbg_fub_vr (dbg_fub_vr),
.o_dbg_bram_wr (dbg_bram_wr),
.o_dbg_bram_rd (dbg_bram_rd),
.o_dbg_busy_wr (dbg_busy_wr),
.o_dbg_busy_rd (dbg_busy_rd)
);

```

---

## 34.7 Design Notes

---

### 34.7.1 Native AXI4, No Emulation

Unlike the AXIL-side wrappers, this variant needs no default-field synthesis — both sides are already AXI4, so the burst/ID fields pass through unchanged. The only tie-offs are the qualifier fields the core intentionally ignores, plus the user outputs the BRAM does not preserve.

### 34.7.2 One Backend, Four Wrappers

This wrapper shares `sdpram_core` with the three other permutations. SystemVerilog cannot conditionally include ports in a single module's port list, so each protocol combination gets its own thin wrapper with the exact port shape; the memory kernel lives once in the core.

### 34.7.3 WRAP Support Is Sim-Gated

INCR and FIXED bursts of any length are supported. WRAP is flagged by assertion until validated. Callers relying on WRAP semantics should exercise and confirm the path first.

---

## 34.8 Related Modules

---

### 34.8.1 Used By

- Characterization harnesses (memory model, descriptor RAM, semaphore RAM)
- Any AXI4-to-AXI4 shared-memory test scaffold

### 34.8.2 Uses

- **axi4\_slave\_wr.sv** — Write-side AXI4 skid leaf
- **axi4\_slave\_rd.sv** — Read-side AXI4 skid leaf
- **sdpram\_core.sv** — Shared SDPRAM backend

### 34.8.3 See Also

- `sdpram_core.sv` — Protocol-agnostic backend ([doc](#))
  - `sdpram_slave_axi4_axil.sv` — AXI4 write + AXIL read variant
  - `sdpram_slave_axil_axi4.sv` — AXIL write + AXI4 read variant
  - `sdpram_slave_axil_axil.sv` — AXIL write + AXIL read variant
- 

## 34.9 References

---

### 34.9.1 Source Code

- RTL: `rtl/amba/shared/sdpram_slave_axi4_axi4.sv`
- Backend: `rtl/amba/shared/sdpram_core.sv`

### 34.9.2 Documentation

- Backend spec: `docs/markdown/RTLAmba/shared/sdpram_core.md`
  - Architecture: `docs/markdown/RTLAmba/shared/README.md`
- 

Last Updated: 2026-07-15

---

## 34.10 Navigation

---

- [Back to Shared Infrastructure Index](#)
- [Back to RTLAmba Index](#)

## 35 SDPRAM Slave — AXI4 Write / AXIL Read

**Module:** `sdpram_slave_axi4_axil.sv` **Location:** `rtl/amba/shared/` **Status:** Production Ready

---

### 35.1 Overview

---

`sdpram_slave_axi4_axil` is the family permutation that exposes a **full AXI4 slave on the write side** (AW + W + B) and a **single-beat AXI4-Lite slave on the read side** (AR + R) in front of the shared `sdpram_core` backend. The write side instantiates `axi4_slave_wr` (full AXI4 with `id / len / size / burst`); the read side instantiates `axil4_slave_rd`, and the wrapper bridges the AXIL read FUB into the core's AXI-shaped read FUB using single-beat defaults.

### 35.1.1 Key Features

- Full AXI4 write slave (burst, ID, size)
  - Single-beat AXIL read slave (no burst / ID)
  - AXIL read FUB bridged into the core with `arlen=0`, `arsize=$clog2(STRB_W)`, `arburst=INCR`, `arid=0`
  - Shared `sdpram_core` backend
  - Byte-enabled writes, single-cycle-latency reads
  - Bulk-clear control and debug taps
  - Sim-only WRAP-burst assertion on the AXI4 write side
- 

## 35.2 Module Purpose

---

Some harness topologies write bulk data over a high-throughput AXI4 burst port but only need simple, single-word AXIL reads for status polling or spot checks. This wrapper matches that shape: fast AXI4 in, lightweight AXIL out, over one shared BRAM.

The AXIL read side has no burst or ID fields, so the wrapper supplies the missing AXI-shaped fields at the single point where the AXIL read FUB meets the core, letting the core's burst tracker collapse to a single-beat read path.

**Use Cases:** - AXI4 bulk-write memory model with an AXIL status/read-back port - Descriptor RAM written by an AXI4 engine and polled over AXIL - Any shared memory where the read side is a simple register-style AXIL access

**Key Benefit:** Full AXI4 write throughput paired with a minimal AXIL read port, with the single-beat adaptation isolated to one place in the wrapper.

---

## 35.3 Parameters

---

Parameter	Type	Default	Description
AXI_ID_WIDTH	int	8	AXI4 write-side transaction-ID width
ADDR_WIDTH	int	32	Byte-address width (both sides)
DATA_WIDTH	int	256	Data-bus / BRAM word width (bits)
USER_WIDTH	int	1	AXI4 write-side USER

Parameter	Type	Default	Description
			width (carried by leaf, not preserved through BRAM)
MEM_DEPTH	int	2048	BRAM depth in words
SKID_DEPTH_AW	int	2	Write-address skid depth
SKID_DEPTH_W	int	2	Write-data skid depth
SKID_DEPTH_B	int	2	Write-response skid depth
SKID_DEPTH_AR	int	2	Read-address skid depth
SKID_DEPTH_R	int	4	Read-data skid depth

**Derived localparams:** STRB\_W = DATA\_WIDTH/8, FUB\_ARSIZE\_DEFAULT = \$clog2(STRB\_W)  
(the arsize fed to the core for the single-beat read path).

## 35.4 Port Groups

### 35.4.1 Clock and Reset

Port	Direction	Width	Description
aclk	input	1	Clock
aresetn	input	1	Active-low asynchronous reset

### 35.4.2 AXI4 Slave Write (AW + W + B)

Port	Direction	Width	Description
s_axi_awid	input	AXI_ID_WIDTH	Write-address ID
s_axi_awa ddr	input	ADDR_WIDTH	Write burst base address
s_axi_awlen	input	8	Burst length minus 1
s_axi_awsize	input	3	Beat size (log2 bytes)

Port	Direction	Width	Description
s_axi_awburst	input	2	Burst type (INCR / FIXED / WRAP)
s_axi_awlock	input	1	Lock (ignored by core)
s_axi_awcache	input	4	Cache attributes (ignored by core)
s_axi_awprot	input	3	Protection attributes (ignored by core)
s_axi_awqos	input	4	QoS (ignored by core)
s_axi_awregion	input	4	Region (ignored by core)
s_axi_awuser	input	USER_WIDTH	User sideband (ignored by core)
s_axi_awvalid	input	1	Write-address valid
s_axi_awready	output	1	Write-address ready
s_axi_wdata	input	DATA_WIDTH	Write data
s_axi_wstrb	input	DATA_WIDTH/8	Per-byte write strobe
s_axi_wlast	input	1	Write-data last beat
s_axi_wuser	input	USER_WIDTH	Write-data user sideband
s_axi_wvalid	input	1	Write-data valid
s_axi_wready	output	1	Write-data ready
s_axi_bid	output	AXI_ID_WIDTH	Write-response ID
s_axi_bresp	output	2	Write response

Port	Direction	Width	Description
s_axi_buser	output	USER_WIDTH	Write-response user (hardwired to 0)
s_axi_bvalid	output	1	Write-response valid
s_axi_bready	input	1	Write-response ready

### 35.4.3 AXIL Slave Read (AR + R)

Port	Direction	Width	Description
s_axil_araddr	input	ADDR_WIDTH	Read address (single beat)
s_axil_arprot	input	3	Read protection attributes
s_axil_arvalid	input	1	Read-address valid
s_axil_arready	output	1	Read-address ready
s_axil_rdata	output	DATA_WIDTH	Read data
s_axil_rresp	output	2	Read response
s_axil_rvalid	output	1	Read-data valid
s_axil_rready	input	1	Read-data ready

### 35.4.4 Bulk-Clear Control and Debug

Port	Direction	Width	Description
i_cfg_start_clear	input	1	Request full-array clear (accepted when both sides idle)
o_cfg_done_clear	output	1	Clear-walk complete
o_dbg_vr	output	10	External valid/ready snapshot

Port	Direction	Width	Description
			{rready, rvalid, arready, arvalid, bready, bvalid, wready, wvalid, awready, awvalid} (AXIL R/AR on top, AXI4 B/W/AW below)
o_dbg_fub_vr	output	10	FUB-side valid/ready snapshot from the core
o_dbg_bram_wr	output	1	BRAM write-fire pulse
o_dbg_bram_rd	output	1	BRAM read-issue pulse
o_dbg_bus_wr	output	1	Write-side (AXI4) skid-leaf busy
o_dbg_bus_rd	output	1	Read-side (AXIL) skid-leaf busy

## 35.5 Functional Description

### 35.5.1 Structure

Three instances: `axi4_slave_wr` (write), `axi4_slave_rd` (read), and `sdpram_core` (backend). The write leaf's FUB carries full AXI4 write fields into the core. The read leaf's AXIL FUB carries only address and data; the wrapper supplies the burst/ID fields the core expects.

### 35.5.2 AXIL Read Bridge

The AXIL read FUB (`fub_axil_araddr` / `arvalid` / `arready` and `rdata` / `rresp` / `rvalid` / `rready`) is wired to the core's read FUB, and the AXI-shaped fields the core needs are tied off at that boundary:

- `fub_arid` = '0
- `fub_arlen` = 8'h00 (single beat)
- `fub_arsize` = 3' (FUB\_ARSIZE\_DEFAULT) =  $\$clog2(\text{STRB\_W})$  (one full data word per beat)
- `fub_arburst` = 2'b01 (INCR)

The core's `fub_rid` and `fub_rlast` outputs go to `*_unused` nets — AXIL has no ID or last field. With `arlen = 0`, each AXIL read is a single-beat access into BRAM.

### 35.5.3 Write Path and Tie-Offs

The write side passes full AXI4 fields straight through. The AXI4 qualifier fields the core ignores (`lock`, `cache`, `prot`, `qos`, `region`, `user`, `wlast`) terminate on `*_unused` nets. `s_axi_buser` is hardwired to zero since the core does not preserve user across the BRAM.

### 35.5.4 Memory Behavior

Byte-enabled writes, single-cycle-latency reads, one outstanding burst per side, and the bulk-clear FSM all live in `sdpram_core` — see `sdpram_core`.

### 35.5.5 WRAP Assertion

A `translate_off` block asserts on each accepted AW that `awburst` is not WRAP. Only the write side carries this assertion; the AXIL read side is inherently single-beat INCR.

---

## 35.6 Usage Example

---

```
sdpram_slave_axi4_axil #(
 .AXI_ID_WIDTH (8),
 .ADDR_WIDTH (32),
 .DATA_WIDTH (256),
 .MEM_DEPTH (2048)
) u_mem (
 .aclk (aclk),
 .aresetn (aresetn),

 // AXI4 burst write in
 .s_axi_awid (eng_awid),
 .s_axi_awaddr (eng_awaddr),
 .s_axi_awlen (eng_awlen),
 .s_axi_awsz (eng_awsz),
 .s_axi_awburst (eng_awburst),
 /* ...remaining AW/W/B... */
 .s_axi_bvalid (eng_bvalid),
 .s_axi_bready (eng_bready),

 // AXIL single-beat read-back
 .s_axil_araddr (cpu_araddr),
 .s_axil_arprot (cpu_arprot),
 .s_axil_arvalid (cpu_arvalid),
 .s_axil_arready (cpu_arready),
 .s_axil_rdata (cpu_rdata),
```



```

.s_axil_rresp (cpu_rresp),
.s_axil_rvalid (cpu_rvalid),
.s_axil_rready (cpu_rready),

.i_cfg_start_clear (clear_pulse),
.o_cfg_done_clear (clear_done),
.o_dbg_vr (dbg_vr),
.o_dbg_fub_vr (dbg_fub_vr),
.o_dbg_bram_wr (dbg_bram_wr),
.o_dbg_bram_rd (dbg_bram_rd),
.o_dbg_busy_wr (dbg_busy_wr),
.o_dbg_busy_rd (dbg_busy_rd)
);

```

---

## 35.7 Design Notes

---

### 35.7.1 Single-Beat Adaptation Lives in One Place

The only “fake AXI4” fields anywhere in this wrapper are the four read-side defaults (`arid / arlen / arsize / arbust`) fed to the core at the AXIL read FUB boundary. Everything else on the write side is genuine AXI4.

### 35.7.2 One Backend, Four Wrappers

This wrapper shares `sdpram_core` with the three other permutations. Each protocol combination is a thin wrapper with its exact port shape; the memory kernel lives once in the core.

### 35.7.3 Read Beat Size

`FUB_ARSIZE_DEFAULT = $clog2(STRB_W)` selects a full-data-word beat for each AXIL read, matching the BRAM word width. Sub-word AXIL reads are not modeled — a read returns the whole BRAM word at the addressed word index.

---

## 35.8 Related Modules

---

### 35.8.1 Used By

- Characterization harnesses with AXI4 bulk-write and AXIL read-back

### 35.8.2 Uses

- **axi4\_slave\_wr.sv** — Write-side AXI4 skid leaf
- **axil4\_slave\_rd.sv** — Read-side AXIL skid leaf

- `sdpram_core.sv` — Shared SDPRAM backend

### 35.8.3 See Also

- `sdpram_core.sv` — Protocol-agnostic backend ([doc](#))
  - `sdpram_slave_axi4_axi4.sv` — AXI4 write + AXI4 read variant
  - `sdpram_slave_axil_axi4.sv` — AXIL write + AXI4 read variant
  - `sdpram_slave_axil_axil.sv` — AXIL write + AXIL read variant
- 

## 35.9 References

---

### 35.9.1 Source Code

- RTL: `rtl/amba/shared/sdpram_slave_axi4_axil.sv`
- Backend: `rtl/amba/shared/sdpram_core.sv`

### 35.9.2 Documentation

- Backend spec: `docs/markdown/RTLamba/shared/sdpram_core.md`
  - Architecture: `docs/markdown/RTLamba/shared/README.md`
- 

**Last Updated:** 2026-07-15

---

## 35.10 Navigation

---

- [Back to Shared Infrastructure Index](#)
- [Back to RTLamba Index](#)

## 36 SDPRAM Slave — AXIL Write / AXI4 Read

**Module:** `sdpram_slave_axil_axi4.sv` **Location:** `rtl/amba/shared/` **Status:** Production Ready

---

### 36.1 Overview

---

`sdpram_slave_axil_axi4` is the family permutation with a **single-beat AXI4-Lite slave on the write side** (AW + W + B) and a **full AXI4 slave on the read side** (AR + R) in front of the shared `sdpram_core` backend. The write side instantiates `axil4_slave_wr`; the read side instantiates

axi4\_slave\_rd (full AXI4 with id / len / size / burst). The wrapper bridges the AXIL write FUB into the core's AXI-shaped write FUB using single-beat defaults.

### 36.1.1 Key Features

- Single-beat AXIL write slave (no burst / ID)
  - Full AXI4 read slave (burst, ID, size)
  - AXIL write FUB bridged into the core with awlen=0, awsize=\$clog2(STRB\_W), awburst=INCR, awid=0
  - Shared sdpram\_core backend
  - Byte-enabled writes, single-cycle-latency reads
  - Bulk-clear control and debug taps
  - Sim-only WRAP-burst assertion on the AXI4 read side
- 

## 36.2 Module Purpose

---

This shape fits a common characterization pattern: a host CPU populates a memory a word at a time over a simple AXIL write port (for example, a descriptor RAM's host-write port), while a high-throughput engine reads bursts back over full AXI4.

The AXIL write side has no burst or ID fields, so the wrapper supplies the missing AXI-shaped fields at the single point where the AXIL write FUB meets the core, collapsing the core's write burst tracker to a single-beat path.

**Use Cases:** - Descriptor RAM: host writes descriptors over AXIL, an engine reads them over AXI4 bursts - Semaphore / mailbox RAM: single-word AXIL writes, bursty AXI4 reads - Memory model populated by a control-plane CPU and consumed by a data-plane master

**Key Benefit:** A simple word-at-a-time AXIL write port with full AXI4 burst read throughput, with the single-beat adaptation isolated to one place in the wrapper.

---

## 36.3 Parameters

---

Parameter	Type	Default	Description
AXI_ID_WIDTH	int	8	AXI4 read-side transaction-ID width
ADDR_WIDTH	int	32	Byte-address width (both sides)
DATA_WIDTH	int	256	Data-bus / BRAM word

Parameter	Type	Default	Description
USER_WIDTH	int	1	width (bits) AXI4 read-side USER width (carried by leaf, not preserved through BRAM)
MEM_DEPTH	int	2048	BRAM depth in words
SKID_DEPTH_AW	int	2	Write-address skid depth
SKID_DEPTH_W	int	2	Write-data skid depth
SKID_DEPTH_B	int	2	Write-response skid depth
SKID_DEPTH_AR	int	2	Read-address skid depth
SKID_DEPTH_R	int	4	Read-data skid depth

**Derived localparams:** STRB\_W = DATA\_WIDTH/8, FUB\_AWSIZE\_DEFAULT = \$clog2(STRB\_W)  
(the awsize fed to the core for the single-beat write path).

## 36.4 Port Groups

### 36.4.1 Clock and Reset

Port	Direction	Width	Description
aclk	input	1	Clock
aresetn	input	1	Active-low asynchronous reset

### 36.4.2 AXIL Slave Write (AW + W + B)

Port	Direction	Width	Description
s_axil_awaddr	input	ADDR_WIDTH	Write address (single beat)
s_axil_awprot	input	3	Write protection attributes
s_axil_awvalid	input	1	Write-address

Port	Direction	Width	Description
			valid
s_axil_awready	output	1	Write-address ready
s_axil_wdata	input	DATA_WIDTH	Write data
s_axil_wstrb	input	DATA_WIDTH/ 8	Per-byte write strobe
s_axil_wvalid	input	1	Write-data valid
s_axil_wready	output	1	Write-data ready
s_axil_bresp	output	2	Write response
s_axil_bvalid	output	1	Write-response valid
s_axil_bready	input	1	Write-response ready

### 36.4.3 AXI4 Slave Read (AR + R)

Port	Direction	Width	Description
s_axi_arid	input	AXI_ID_WIDTH	Read-address ID
s_axi_araddr	input	ADDR_WIDTH	Read burst base address
s_axi_arlen	input	8	Burst length minus 1
s_axi_arsize	input	3	Beat size (log2 bytes)
s_axi_arburst	input	2	Burst type
s_axi_arlock	input	1	Lock (ignored by core)
s_axi_arcache	input	4	Cache attributes (ignored by core)

Port	Direction	Width	Description
s_axi_arprot	input	3	Protection attributes (ignored by core)
s_axi_arqos	input	4	QoS (ignored by core)
s_axi_arregion	input	4	Region (ignored by core)
s_axi_aruser	input	USER_WIDTH	User sideband (ignored by core)
s_axi_arvalid	input	1	Read-address valid
s_axi_arready	output	1	Read-address ready
s_axi_rid	output	AXI_ID_WIDTH	Read-data ID
s_axi_rdata	output	DATA_WIDTH	Read data
s_axi_rresp	output	2	Read response
s_axi_rlast	output	1	Read-data last beat
s_axi_ruser	output	USER_WIDTH	Read-data user (hardwired to 0)
s_axi_rvalid	output	1	Read-data valid
s_axi_rready	input	1	Read-data ready

#### 36.4.4 Bulk-Clear Control and Debug

Port	Direction	Width	Description
i_cfg_start_clear	input	1	Request full-array clear (accepted when both sides idle)

Port	Direction	Width	Description
o_cfg_done_clear	output	1	Clear-walk complete
o_dbg_vr	output	10	External valid/ready snapshot {rready,rvalid,arready,arvalid,bready,bvalid,wready,wvalid,awready,awvalid} (AXI4 R/AR on top, AXIL B/W/AW below)
o_dbg_fub_vr	output	10	FUB-side valid/ready snapshot from the core
o_dbg_bram_wr	output	1	BRAM write-fire pulse
o_dbg_bram_rd	output	1	BRAM read-issue pulse
o_dbg_bus_wr	output	1	Write-side (AXIL) skid-leaf busy
o_dbg_bus_rd	output	1	Read-side (AXI4) skid-leaf busy

## 36.5 Functional Description

### 36.5.1 Structure

Three instances: axil4\_slave\_wr (write), axi4\_slave\_rd (read), and sdpram\_core (backend). The read leaf's FUB carries full AXI4 read fields into the core. The write leaf's AXIL FUB carries only address and data; the wrapper supplies the burst/ID fields the core expects.

### 36.5.2 AXIL Write Bridge

The AXIL write FUB (fub\_axil\_awaddr / awvalid / awready,wdata / wstrb / wvalid / wready,bresp / bvalid / bready) is wired to the core's write FUB, and the AXI-shaped fields the core needs are tied off at that boundary:

- fub\_awid = '0
- fub\_awlen = 8'h00 (single beat)

- `fub_awsiz` = 3' (FUB\_AWSIZE\_DEFAULT) =  $\$clog2(\text{STRB\_W})$  (one full data word per beat)
- `fub_awburst` = 2'b01 (INCR)

The core's `fub_bid` output goes to a `*_unused` net — AXIL has no B-channel ID. With `awlen` = 0, each AXIL write is a single-beat access into BRAM.

### 36.5.3 Read Path and Tie-Offs

The read side passes full AXI4 fields straight through. The AXI4 qualifier fields the core ignores (lock, cache, prot, qos, region, user) terminate on `*_unused` nets. `s_axi_ruser` is hardwired to zero since the core does not preserve user across the BRAM.

### 36.5.4 Memory Behavior

Byte-enabled writes, single-cycle-latency reads, one outstanding burst per side, and the bulk-clear FSM all live in `sdpram_core` — see `sdpram_core`.

### 36.5.5 WRAP Assertion

A `translate_off` block asserts on each accepted AR that `arburst` is not WRAP. Only the read side carries this assertion; the AXIL write side is inherently single-beat INCR.

## 36.6 Usage Example

```
sdpram_slave_axil_axi4 #(
 .AXI_ID_WIDTH (8),
 .ADDR_WIDTH (32),
 .DATA_WIDTH (256),
 .MEM_DEPTH (2048)
) u_desc_ram (
 .aclk (aclk),
 .aresetn (aresetn),

 // AXIL host-write port (descriptor population)
 .s_axil_awaddr (host_awaddr),
 .s_axil_awprot (host_awprot),
 .s_axil_awvalid (host_awvalid),
 .s_axil_awready (host_awready),
 .s_axil_wdata (host_wdata),
 .s_axil_wstrb (host_wstrb),
 .s_axil_wvalid (host_wvalid),
 .s_axil_wready (host_wready),
 .s_axil_bresp (host_bresp),
 .s_axil_bvalid (host_bvalid),
 .s_axil_bready (host_bready),
```



```

// AXI4 burst read port (engine fetch)
.s_axi_arid (eng_arid),
.s_axi_araddr (eng_araddr),
.s_axi_arlen (eng_arlen),
/* ...remaining AR/R... */
.s_axi_rvalid (eng_rvalid),
.s_axi_rready (eng_rready),

.i_cfg_start_clear (clear_pulse),
.o_cfg_done_clear (clear_done),
.o_dbg_vr (dbg_vr),
.o_dbg_fub_vr (dbg_fub_vr),
.o_dbg_bram_wr (dbg_bram_wr),
.o_dbg_bram_rd (dbg_bram_rd),
.o_dbg_busy_wr (dbg_busy_wr),
.o_dbg_busy_rd (dbg_busy_rd)
);

```

---

## 36.7 Design Notes

---

### 36.7.1 Descriptor-RAM Fit

This is the natural shape for a descriptor / semaphore RAM: a control-plane CPU pokes single words over AXIL, and a data-plane engine burst-reads them over AXI4. The `rapids_char_harness` descriptor RAM host-write port is exactly this pattern.

### 36.7.2 Single-Beat Adaptation Lives in One Place

The only “fake AXI4” fields anywhere in this wrapper are the four write-side defaults (`awid` / `awlen` / `awsize` / `awburst`) fed to the core at the AXIL write FUB boundary. Everything else on the read side is genuine AXI4.

### 36.7.3 One Backend, Four Wrappers

This wrapper shares `sdpram_core` with the three other permutations. Each protocol combination is a thin wrapper with its exact port shape; the memory kernel lives once in the core.

---

## 36.8 Related Modules

---

### 36.8.1 Used By

- `rapids_char_harness` descriptor RAM (host-write port)

- Characterization harnesses with AXIL host-write and AXI4 engine-read

### 36.8.2 Uses

- **axil4\_slave\_wr.sv** — Write-side AXIL skid leaf
- **axi4\_slave\_rd.sv** — Read-side AXI4 skid leaf
- **sdpram\_core.sv** — Shared SDPRAM backend

### 36.8.3 See Also

- **sdpram\_core.sv** — Protocol-agnostic backend ([doc](#))
  - **sdpram\_slave\_axi4\_axi4.sv** — AXI4 write + AXI4 read variant
  - **sdpram\_slave\_axi4\_axil.sv** — AXI4 write + AXIL read variant
  - **sdpram\_slave\_axil\_axil.sv** — AXIL write + AXIL read variant
- 

## 36.9 References

---

### 36.9.1 Source Code

- RTL: rtl/amba/shared/sdpram\_slave\_axil\_axi4.sv
- Backend: rtl/amba/shared/sdpram\_core.sv

### 36.9.2 Documentation

- Backend spec: docs/markdown/RTLAmba/shared/sdpram\_core.md
  - Architecture: docs/markdown/RTLAmba/shared/README.md
- 

**Last Updated:** 2026-07-15

---

## 36.10 Navigation

---

- [Back to Shared Infrastructure Index](#)
- [Back to RTLAmba Index](#)

## 37 SDPRAM Slave — AXIL Write / AXIL Read

**Module:** sdpram\_slave\_axil\_axil.sv **Location:** rtl/amba/shared/ **Status:** Production Ready

---

## 37.1 Overview

---

`sdpram_slave_axil_axil` is the all-AXI4-Lite permutation of the family: a **single-beat AXIL slave on the write side** (AW + W + B) and a **single-beat AXIL slave on the read side** (AR + R) in front of the shared `sdpram_core` backend. It directly instantiates the native `axil4_slave_wr` and `axil4_slave_rd` skid leaves and bridges their AXIL FUB outputs into the core's AXI-shaped FUB by supplying single-beat defaults on both sides. No AXI4-only fields appear anywhere on the wrapper's external boundary.

### 37.1.1 Key Features

- Single-beat AXIL slave on both write and read sides
  - Native `axil4_slave_wr` + `axil4_slave_rd` skid leaves
  - Both FUBs bridged into the core with `len=0`, `size=$clog2(STRB_W)`, `burst=INCR`, `id=0`
  - A 1-bit zero ID carried through the core for typing only (`CORE_ID_WIDTH = 1`)
  - Shared `sdpram_core` backend
  - Byte-enabled writes, single-cycle-latency reads
  - Bulk-clear control and debug taps
  - No WRAP assertion needed (both sides are inherently single-beat INCR)
- 

## 37.2 Module Purpose

---

The simplest members of the family need a lightweight, register-style memory shared between two AXIL agents — one writing, one reading. This wrapper provides exactly that with no burst machinery exposed: both sides are single-word AXIL, and the shared backend supplies the burst/ID scaffolding internally.

Because AXIL carries no transaction ID, the wrapper carries a 1-bit zero ID through `sdpram_core` purely for type-width bookkeeping. The single-beat defaults are the only “fake” AXI4 fields in the design, and they live in exactly one place per side.

**Use Cases:** - Lightweight scratch / mailbox RAM shared between two AXIL agents - Small semaphore RAM in a control-plane fabric - Memory-ring backend for the AXIL/AXIL monitor-bus capture master-write port (pairs with `monbus_axil_axil_group`) - Any AXIL-only shared-memory test scaffold

**Key Benefit:** A minimal AXIL-in / AXIL-out shared memory with all burst and clear complexity hidden in the shared core, and no spurious AXI4 fields on the external ports.

---

## 37.3 Parameters

Parameter	Type	Default	Description
ADDR_WIDTH	int	32	Byte-address width (both sides)
DATA_WIDTH	int	64	Data-bus / BRAM word width (bits)
MEM_DEPTH	int	1024	BRAM depth in words
SKID_DEPTH_A W	int	2	Write-address skid depth
SKID_DEPTH_ W	int	2	Write-data skid depth
SKID_DEPTH_B	int	2	Write-response skid depth
SKID_DEPTH_A R	int	2	Read-address skid depth
SKID_DEPTH_R	int	4	Read-data skid depth

**Derived localparams:** `STRB_W = DATA_WIDTH/8`, `FUB_AWSIZE_DEFAULT = $clog2(STRB_W)` (the `awsiz`/`arsiz` fed to the core), `CORE_ID_WIDTH = 1` (a 1-bit zero ID carried through the core for typing only — AXIL has no transaction ID).

Note: unlike the AXI4-bearing wrappers, this module has **no** `AXI_ID_WIDTH` or `USER_WIDTH` parameter, and its `DATA_WIDTH` / `MEM_DEPTH` defaults are smaller (64 / 1024), reflecting its lightweight role.

## 37.4 Port Groups

### 37.4.1 Clock and Reset

Port	Direction	Width	Description
aclk	input	1	Clock
aresetn	input	1	Active-low

Port	Direction	Width	Description
			asynchronous reset

### 37.4.2 AXIL Slave Write (AW + W + B)

Port	Direction	Width	Description
s_axil_awaddr	input	ADDR_WIDTH	Write address (single beat)
s_axil_awprot	input	3	Write protection attributes
s_axil_awvalid	input	1	Write-address valid
s_axil_awready	output	1	Write-address ready
s_axil_wdata	input	DATA_WIDTH	Write data
s_axil_wstrb	input	DATA_WIDTH/8	Per-byte write strobe
s_axil_wvalid	input	1	Write-data valid
s_axil_wready	output	1	Write-data ready
s_axil_bresp	output	2	Write response
s_axil_bvalid	output	1	Write-response valid
s_axil_bready	input	1	Write-response ready

### 37.4.3 AXIL Slave Read (AR + R)

Port	Direction	Width	Description
s_axil_araddr	input	ADDR_WIDTH	Read address (single beat)
s_axil_arprot	input	3	Read protection attributes

Port	Direction	Width	Description
s_axil_arvalid	input	1	Read-address valid
s_axil_arready	output	1	Read-address ready
s_axil_rdata	output	DATA_WIDTH	Read data
s_axil_rresp	output	2	Read response
s_axil_rvalid	output	1	Read-data valid
s_axil_rready	input	1	Read-data ready

#### 37.4.4 Bulk-Clear Control and Debug

Port	Direction	Width	Description
i_cfg_start_clear	input	1	Request full-array clear (accepted when both sides idle)
o_cfg_done_clear	output	1	Clear-walk complete
o_dbg_vr	output	10	External valid/ready snapshot {rready,rvalid,arready,arvalid,bready,bvalid,wready,wvalid,awready,awvalid}
o_dbg_fub_vr	output	10	FUB-side valid/ready snapshot from the core
o_dbg_bram_wr	output	1	BRAM write-fire pulse
o_dbg_bram_rd	output	1	BRAM read-issue pulse
o_dbg_busy_wr	output	1	Write-side (AXIL) skid-leaf busy
o_dbg_busy_rd	output	1	Read-side (AXIL) skid-leaf busy

## 37.5 Functional Description

---

### 37.5.1 Structure

Three instances: `axil4_slave_wr` (write), `axil4_slave_rd` (read), and `sdpram_core` (backend, parameterized with `AXI_ID_WIDTH = CORE_ID_WIDTH = 1`). Both AXIL FUBs carry only address and data; the wrapper supplies the burst/ID fields the core expects.

### 37.5.2 Single-Beat Defaults on Both Sides

At each FUB-to-core boundary the wrapper ties the AXI-shaped fields the core needs:

**Write side:** - `fub_awid = 1'b0` - `fub_awlen = 8'h00` - `fub_awsz = 3'(FUB_AWSIZE_DEFAULT) = $clog2(STRB_W)` - `fub_awburst = 2'b01 (INCR)`

**Read side:** - `fub_arid = 1'b0` - `fub_arlen = 8'h00` - `fub_arsz = 3'(FUB_AWSIZE_DEFAULT)` - `fub_arburst = 2'b01 (INCR)`

The core's `fub_bid`, `fub_rid`, and `fub_rlast` outputs terminate on `*_unused` nets. With `len = 0` on both sides, every access is a single-beat BRAM access.

### 37.5.3 Memory Behavior

Byte-enabled writes, single-cycle-latency reads, one outstanding access per side, and the bulk-clear FSM all live in `sdpram_core` — see `sdpram_core`.

### 37.5.4 No WRAP Assertion

Neither side can express a burst, so there is no WRAP-burst assertion in this wrapper (the AXI4-bearing wrappers carry one because their burst-capable side could request WRAP).

---

## 37.6 Usage Example

---

```
sdpram_slave_axil_axil #(
 .ADDR_WIDTH (32),
 .DATA_WIDTH (64),
 .MEM_DEPTH (1024)
) u_scratch (
 .aclk (aclk),
 .aresetn (aresetn),

 // AXIL write port
 .s_axil_awaddr (wr_awaddr),
 .s_axil_awprot (wr_awprot),
```

```

.s_axil_awvalid (wr_awvalid),
.s_axil_awready (wr_awready),
.s_axil_wdata (wr_wdata),
.s_axil_wstrb (wr_wstrb),
.s_axil_wvalid (wr_wvalid),
.s_axil_wready (wr_wready),
.s_axil_bresp (wr_bresp),
.s_axil_bvalid (wr_bvalid),
.s_axil_bready (wr_bready),

// AXIL read port
.s_axil_araddr (rd_araddr),
.s_axil_arprot (rd_arprot),
.s_axil_arvalid (rd_arvalid),
.s_axil_arready (rd_arready),
.s_axil_rdata (rd_rdata),
.s_axil_rresp (rd_rresp),
.s_axil_rvalid (rd_rvalid),
.s_axil_rready (rd_rready),

.i_cfg_start_clear (clear_pulse),
.o_cfg_done_clear (clear_done),
.o_dbg_vr (dbg_vr),
.o_dbg_fub_vr (dbg_fub_vr),
.o_dbg_bram_wr (dbg_bram_wr),
.o_dbg_bram_rd (dbg_bram_rd),
.o_dbg_busy_wr (dbg_busy_wr),
.o_dbg_busy_rd (dbg_busy_rd)
);

```

---

## 37.7 Design Notes

### 37.7.1 The Only “Fake” Fields, In One Place Per Side

AXIL has no id / len / burst, so the wrapper feeds sdpram\_core single-beat INCR defaults. These are the only synthetic AXI4 fields anywhere in the design, and they live in exactly one place per side — the smell of the legacy generate-if base is gone.

### 37.7.2 Monitor-Bus Pairing

This wrapper is the canonical memory-ring backend for the AXIL/AXIL monitor-bus capture path: monbus\_axil\_axil\_group’s master-write port streams records into an sdpram\_slave\_axil\_axil acting as the dump ring.



### 37.7.3 One Backend, Four Wrappers

This wrapper shares `sdpram_core` with the three other permutations. Each protocol combination is a thin wrapper with its exact port shape; the memory kernel lives once in the core.

---

## 37.8 Related Modules

---

### 37.8.1 Used By

- `monbus_axil_axil_group` master-write dump ring (memory-ring backend)
- Lightweight AXIL-only shared-memory test scaffolds

### 37.8.2 Uses

- `axil4_slave_wr.sv` — Write-side AXIL skid leaf
- `axil4_slave_rd.sv` — Read-side AXIL skid leaf
- `sdpram_core.sv` — Shared SDPRAM backend

### 37.8.3 See Also

- `sdpram_core.sv` — Protocol-agnostic backend ([doc](#))
  - `sdpram_slave_axi4_axi4.sv` — AXI4 write + AXI4 read variant
  - `sdpram_slave_axi4_axil.sv` — AXI4 write + AXIL read variant
  - `sdpram_slave_axil_axi4.sv` — AXIL write + AXI4 read variant
  - `monbus_axil_axil_group.sv` — Monitor-bus delivery wrapper this RAM backs
- 

## 37.9 References

---

### 37.9.1 Source Code

- RTL: `rtl/amba/shared/sdpram_slave_axil_axil.sv`
- Backend: `rtl/amba/shared/sdpram_core.sv`

### 37.9.2 Documentation

- Backend spec: `docs/markdown/RTLAmba/shared/sdpram_core.md`
  - Architecture: `docs/markdown/RTLAmba/shared/README.md`
- 

**Last Updated:** 2026-07-15

---

## 37.10 Navigation

---

- [Back to Shared Infrastructure Index](#)
- [Back to RTLAmba Index](#)

## 38 Simple Dual-Port BRAM Slave Family

**Modules:** - `sdpram_slave.sv` — common backend (BRAM glue + clear FSM + protocol skids) -  
`sdpram_slave_axi4_axi4.sv` — wrapper: AXI4 write, AXI4 read -  
`sdpram_slave_axi4_axil.sv` — wrapper: AXI4 write, AXIL read -  
`sdpram_slave_axil_axi4.sv` — wrapper: AXIL write, AXI4 read -  
`sdpram_slave_axil_axil.sv` — wrapper: AXIL write, AXIL read

**Location:** `rtl/amba/shared/` **Category:** Memory / BRAM Slave **Status:** Production Ready

---

### 38.1 Overview

---

The `sdpram_slave` family provides a BRAM-backed simple-dual-port slave with **independent protocol choice on each side** (write, read). It unifies what used to be two separate modules (`axi4_sdpram_slave` + `axil_sdpram_slave`) into a single common backend plus four protocol-specific wrappers that expose the exact port shape for their configuration.

The most common deployment in this repo is the **monitor bus memory dump ring** — `monbus_axil_axil_group's m_axil_*` master writes the compressed (or raw) trace into a `sdpram_slave_axil_axil`, and the host CPU reads it back through the same slave's read port. Wide AXI4 descriptor RAMs (e.g. `stream_char_harness::u_desc_ram`, 256-bit data, 8-bit AXI ID) instantiate `sdpram_slave_axi4_axi4`.

---

### 38.2 Why four wrappers + a backend?

---

SystemVerilog cannot conditionally include or exclude ports from a single module's port list. The port list is a static syntactic construct, fixed at module declaration. So to give each protocol combination an exact port shape — no spurious AXI4-only fields for the caller to tie off — the family is split:

- **One common backend** (`sdpram_slave.sv`) with the BRAM port-A/B glue, bulk-clear FSM, burst tracker, and the two protocol-skid generate blocks (gated by `WR_PROTOCOL` / `RD_PROTOCOL` string parameters). The backend's

port list is the AXI4 superset; in AXIL mode the unused fields are ignored on inputs / driven to 0 on outputs.

- **Four thin wrappers**, each with the right protocol-shaped port list, that instantiate the backend with the matching parameter setting and tie off any AXI4-only fields for an AXIL side.

Wrapper	Wr ports	Rd ports
<code>sdpram_slave_axi4_axi4</code>	<code>s_axi_aw*/w*/b*</code> (full AXI4)	<code>s_axi_ar*/r*</code> (full AXI4)
<code>sdpram_slave_axi4_axil</code>	<code>s_axi_aw*/w*/b*</code> (full AXI4)	<code>s_axil_ar*/r*</code> (AXIL only)
<code>sdpram_slave_axil_axi4</code>	<code>s_axil_aw*/w*/b*</code> (AXIL only)	<code>s_axi_ar*/r*</code> (full AXI4)
<code>sdpram_slave_axil_axil</code>	<code>s_axil_aw*/w*/b*</code> (AXIL only)	<code>s_axil_ar*/r*</code> (AXIL only)

Callers pick the wrapper module name matching their fabric. The bare `sdpram_slave` remains directly callable for unusual configs (or for the existing `test_sdpram_slave.py` regression which exercises both sides independently via the parameter knobs).

### 38.3 Common Parameters

All wrappers expose the same scaling knobs as the backend:

Parameter	Default	Notes
<code>ADDR_WIDTH</code>	32	Address width.
<code>DATA_WIDTH</code>	256 ( <code>axi4_*</code> ) / 64 ( <code>axil_axil</code> )	Data-bus width.
<code>MEM_DEPTH</code>	2048 ( <code>axi4_*</code> ) / 1024 ( <code>axil_axil</code> )	BRAM word count.
<code>AXI_ID_WIDTH</code>	8	Present only when at least one side is AXI4.
<code>USER_WIDTH</code>	1	Present only when at least one side is AXI4.
<code>SKID_DEPTH_AW/W/B/AR/R</code>	2/2/2/2/4	Skid buffer depths for each channel.

## 38.4 Burst Support

---

- **AXI4 mode** supports INCR (awburst/arbust = 2'b01) and FIXED (2'b00) of any length up to AXI4's 256-beat maximum. WRAP (2'b10) is rejected by a SIMULATION-only \$error and treated as INCR in synth (the BRAM glue advances linearly).
  - **AXIL mode** is single-beat by construction — the AXIL skid ties the fub-side awlen/arlen to 0, so the burst-aware backend produces exactly one beat per AW/AR. Multi-beat transactions are not expressible in AXIL anyway.
- 

## 38.5 Bulk Clear

---

All wrappers expose:

- `i_cfg_start_clear` (input) — pulse high to start a memory-wide clear.
- `o_cfg_done_clear` (output) — pulses high when the clear FSM finishes.

The clear FSM owns BRAM port A while it walks the whole memory writing zeros. It waits for both sides idle before claiming the port, so an in-flight write or read completes cleanly before the clear begins.

---

## 38.6 Debug / Observation Outputs

---

Common to all wrappers (and the backend):

Output	Width	Meaning
<code>o_dbg_vr</code>	10	External {aw,w,b,ar,r}_{valid, ready} (AW = [9:8], W = [7:6], B = [5:4], AR = [3:2], R = [1:0])
<code>o_dbg_fub_vr</code>	10	Fub-side (post-skid) valid/ready for the same five channels
<code>o_dbg_bram_wr</code>	1	One-cycle pulse on BRAM port-A write fire
<code>o_dbg_bram_rd</code>	1	One-cycle pulse on

Output	Width	Meaning
		BRAM port-B read fire
o_dbg_busy_wr	1	Write-side skid busy
o_dbg_busy_rd	1	Read-side skid busy

## 38.7 Use in the Monitor System

The `sdpram_slave_axil_axil` wrapper is the canonical SRAM-ring backend for the `monbus_axil_axil_group` master writer — both sides AXIL, so no AXI4-only fields anywhere on the harness wiring. For details on the slot stream landing in the ring, see `monbus_compressor.md` (the compressed case) and the raw-record beat layout described in `monbus_group.md`.

## 38.8 Test

`val/amba/test_sdpram_slave.py` exercises the bare `sdpram_slave` backend across all four (`WR_PROTOCOL`, `RD_PROTOCOL`) combinations via `@pytest.mark.parametrize`. The wrappers are thin pass-throughs so the backend regression is the load-bearing one. Sub-tests cover:

1. Single-beat AW/W/B + AR/R round-trip (all 4 combos)
2. AXI4 INCR burst write + read (AXI4-only)
3. AXI4 FIXED burst write (last beat wins) + read (AXI4-only)
4. Random fill + readback (all 4 combos)

```
pytest val/amba/test_sdpram_slave.py -v
```

## 38.9 Migration

If you have an old caller that instantiates the bare `sdpram_slave`:

```
// Old: full AXI4-shape ports, tie off AXI4-only for AXIL side
sdpram_slave #(
 .WR_PROTOCOL ("AXIL"),
 .RD_PROTOCOL ("AXIL"),
 .AXI_ID_WIDTH (1), // unused
 ...
) u_dump (
 .s_axi_awid (1'b0), // ← unsightly tie-off
 .s_axi_awaddr (axil_awaddr),
```

```

 .s_axi_awlen (8'h00), // ← unsightly tie-off
 .s_axi_awsz (3'h0), // ← unsightly tie-off
 ...
);

```

After migrating to the matching wrapper:

```

// New: AXIL-only ports, no tie-offs needed
sdpram_slave_axil_axil #(
 .ADDR_WIDTH (32),
 .DATA_WIDTH (64),
 .MEM_DEPTH (1024)
) u_dump (
 .s_axil_awaddr (axil_awaddr),
 .s_axil_awprot (axil_awprot),
 .s_axil_awvalid (axil_awvalid),
 .s_axil_awready (axil_awready),
 ...
);

```

The backend is unchanged so functional behavior is identical — only the port list shape and signal names differ.

## 38.10 Related Modules

Module	Role
monbus_axil_axil_group	Most common consumer (memory-dump ring)
monbus_compressor	Source of the compressed slot stream landing in the ring
axi4_slave_wr / axi4_slave_rd	AXI4-side skids instantiated by the backend
axil4_slave_wr / axil4_slave_rd	AXIL-side skids instantiated by the backend

## 39 axi4\_to\_apb\_convert

**Module:** axi4\_to\_apb\_convert.sv **Location:** rtl/amba/shims/ **Status:** ✓ Production Ready

## 39.1 Overview

Core conversion logic that transforms AXI4 memory-mapped transactions into APB peripheral bus protocol. This module handles burst decomposition, width adaptation, and protocol conversion within a single clock domain. Used internally by [axi4\\_to\\_apb\\_shim](#) but can also be instantiated standalone for same-clock bridging.

### 39.1.1 Key Features

- ✓ **Burst Decomposition:** AXI bursts (AWLEN/ARLEN) → APB single-beat transfers
- ✓ **Width Adaptation:** Automatic conversion for AXI\_DATA\_WIDTH > APB\_DATA\_WIDTH
- ✓ **Address Generation:** Proper address incrementing per AXI burst type (INCR, WRAP)
- ✓ **Transaction Tracking:** Side FIFO maintains ID, LAST, USER across conversion
- ✓ **Error Handling:** Accumulates APB PSLVERR → AXI BRESP/RRESP
- ✓ **Zero Data Loss:** Proper flow control and buffering

## 39.2 Module Declaration

```
module axi4_to_apb_convert #(
 parameter int SIDE_DEPTH = 6, // Side info FIFO depth
 parameter int AXI_ID_WIDTH = 8,
 parameter int AXI_ADDR_WIDTH = 32,
 parameter int AXI_DATA_WIDTH = 32,
 parameter int AXI_USER_WIDTH = 1,
 parameter int APB_ADDR_WIDTH = 32,
 parameter int APB_DATA_WIDTH = 32,
 parameter int AXI_WSTRB_WIDTH = AXI_DATA_WIDTH / 8,
 parameter int APB_WSTRB_WIDTH = APB_DATA_WIDTH / 8
) (
 // Clock and Reset
 input logic aclk,
 input logic aresetn,

 // AXI Slave Interface (packet format from axi4_slave_stub)
 // AW channel
 input logic [AWSize-1:0] r_s_axi_aw_pkt,
 input logic [3:0] r_s_axi_aw_count,
 input logic r_s_axi_awvalid,
 output logic w_s_axi_awready,
```

```

// W channel
input logic [WSize-1:0] r_s_axi_w_pkt,
input logic r_s_axi_wvalid,
output logic w_s_axi_wready,

// B channel
output logic [BSize-1:0] r_s_axi_b_pkt,
output logic w_s_axi_bvalid,
input logic r_s_axi_bready,

// AR channel
input logic [ARSize-1:0] r_s_axi_ar_pkt,
input logic [3:0] r_s_axi_ar_count,
input logic r_s_axi_arvalid,
output logic w_s_axi_arready,

// R channel
output logic [RSize-1:0] r_s_axi_r_pkt,
output logic w_s_axi_rvalid,
input logic r_s_axi_rready,

// APB Master Interface (command/response packets)
output logic w_cmd_valid,
input logic r_cmd_ready,
output logic [APBCmdWidth-1:0] r_cmd_data,

input logic r_rsp_valid,
output logic w_rsp_ready,
input logic [APBRspWidth-1:0] r_rsp_data
);

```

---

### 39.3 Parameters

Parameter	Default	Description
SIDE_DEPTH	6	Side information FIFO depth (tracks ID, last, user)
AXI_ID_WIDTH	8	AXI transaction ID width
AXI_ADDR_WIDTH	32	AXI address width
AXI_DATA_WIDTH	32	AXI data width (32, 64, 128, 256, 512)



Parameter	Default	Description
AXI_USER_WIDTH	1	AXI user sideband width
APB_ADDR_WIDTH	32	APB address width (must match AXI)
APB_DATA_WIDTH	32	APB data width (8, 16, 32, 64)

**Derived Parameters:** -  $AXI2APBRATIO = AXI\_DATA\_WIDTH / APB\_DATA\_WIDTH$  (automatically calculated) -  $APBCmdWidth = APB$  command packet width -  $APBRspWidth = APB$  response packet width

## 39.4 Architecture

### 39.4.1 Component Structure

```

axi4_to_apb_convert
├── Packet Unpacking
│ └── Extract AXI channel signals from packets
├── APB FSM (3 states)
│ ├── IDLE: Waiting for AXI transaction
│ ├── READ: Processing read burst
│ └── WRITE: Processing write burst
├── Response FSM (2 states)
│ ├── RSP_IDLE: Waiting for APB response
│ └── RSP_ACTIVE: Collecting responses
├── Address Generator (axi_gen_addr)
│ └── Increments address per AXI burst type
├── Width Adaptation Logic
│ ├── Data shift register (read data accumulation)
│ ├── Pointer tracking (which APB beat in AXI word)
│ └── WSTRB → PSTRB mapping
└── Side FIFO (gaxi_fifo_sync)
 └── Tracks ID, LAST, USER for response reconstruction

```

### 39.4.2 Data Flow

#### Write Path:

```

AW packet → Unpack → APB WRITE FSM
W packet → Unpack → Data slicing → APB PWDATA
 → Width pointer tracking

```

→ Burst counter decrement  
→ APB command generation  
Side info → {ID, last, user} → Side FIFO  
APB response → Error accumulation → B packet

#### Read Path:

AR packet → Unpack → APB READ FSM  
→ Address generation  
→ Burst counter decrement  
→ APB command generation  
Side info → {ID, last, user} → Side FIFO  
APB response → Data accumulation → R packet  
→ Shift register fill

---

## 39.5 Behavior

---

### 39.5.1 Burst Decomposition

#### AXI Burst → APB Beats:

Single AXI beat may require multiple APB beats when `AXI_DATA_WIDTH > APB_DATA_WIDTH`:

#### Example: 64-bit AXI → 32-bit APB (2:1 ratio)

AXI Write Burst (AWLEN=3, 4 beats):  
Beat 0: 64 bits → 2 APB transfers (addr+0, addr+4)  
Beat 1: 64 bits → 2 APB transfers (addr+8, addr+12)  
Beat 2: 64 bits → 2 APB transfers (addr+16, addr+20)  
Beat 3: 64 bits → 2 APB transfers (addr+24, addr+28)  
Total: 8 APB transfers for 4 AXI beats

**Pointer Tracking:** - `r_axi_wr_data_pointer`: Tracks which APB slice of current AXI write beat -  
`r_axi_rd_data_pointer`: Tracks which APB slice to write for read accumulation -  
`r_axi_rsp_data_pointer`: Tracks response data accumulation position

### 39.5.2 Address Generation

**Supported Burst Types:** - **INCR (0x01)**: Sequential incrementing (most common) - **WRAP (0x10)**: Wrapping at boundary (less common) - **FIXED (0x00)**: Same address (APB limitation: converted to INCR)

#### Address Alignment:

```
// APB addresses are always aligned to APB_DATA_WIDTH
w_alignment_mask = APB_WSTRB_WIDTH - 1;
w_apb_cmd_pkt_paddr = r_apb_paddr & ~w_alignment_mask;
```

### 39.5.3 Width Adaptation

**Write Data Slicing (AXI → APB):**

```
// For 64b AXI → 32b APB:
// Pointer = 0 → Take bits [31:0]
// Pointer = 1 → Take bits [63:32]
w_apb_cmd_pkt_pwdata = (axi2abpratio == 1) ?
 r_s_axi_wdata[APBDW-1:0] :
 r_s_axi_wdata[r_axi_wr_data_pointer*APBDW +: APBDW];
```

**Read Data Accumulation (APB → AXI):**

```
// For 32b APB → 64b AXI:
// Pointer = 0 → Fill bits [31:0]
// Pointer = 1 → Fill bits [63:32], then return complete 64b word
r_axi_data_shift[r_axi_rsp_data_pointer*APBDW +: APBDW] <=
r_apb_rsp_pkt_prdata;
```

### 39.5.4 Error Handling

**Error Accumulation:** - Multiple APB PSLVERR errors OR'd together - Entire AXI transaction marked with error if any APB beat fails - Read errors: RRESP = 2'b10 (SLVERR) - Write errors: BRESP = 2'b10 (SLVERR)

**Error Propagation:**

```
r_pslverr <= r_pslverr | w_pslverr; // Sticky error
w_resp_rd = (w_pslverr) ? 2'b10 : 2'b00;
w_resp_wr = (w_pslverr | r_pslverr) ? 2'b10 : 2'b00;
```

### 39.5.5 Side Information Tracking

**Side FIFO Contents:**

```
// For each AXI transaction, track:
{operation, // 1=write, 0=read
 ID, // Transaction ID
 last, // RLAST/BLAST flag
 user} // User sideband
```

**Purpose:** - Preserves ID for out-of-order response reconstruction - Maintains LAST flag for burst completion signaling - Passes USER sideband through the bridge

---

## 39.6 Packet Formats

---

### 39.6.1 APB Command Packet

$\text{APBCmdWidth} = \text{APBAW} + \text{APBDW} + \text{APBSW} + 4 \text{ bits}$

[APBCmdWidth-1]	last	(end of AXI burst)
[APBCmdWidth-2]	first	(start of AXI burst)
[APBCmdWidth-3]	pwrite	(1=write, 0=read)
[APBCmdWidth-4:...] ]	pprot	(3 bits)
[...]	pstrb	(APBSW bits)
[...]	paddr	(APBAW bits)
[APBDW-1:0]	pdata	(APBDW bits)

### 39.6.2 APB Response Packet

$\text{APBRspWidth} = \text{APBDW} + 3 \text{ bits}$

[APBRspWidth-1]	last	(end of sequence)
[APBRspWidth-2]	first	(start of sequence)
[APBRspWidth-3]	pslverr	(error flag)
[APBDW-1:0]	prdata	(read data)

---

## 39.7 Usage Examples

---

### 39.7.1 Example 1: Standalone Conversion (Same Width)

```
// 32b AXI → 32b APB conversion (same clock)
axi4_to_apb_convert #(
 .SIDE_DEPTH(4),
 .AXI_ID_WIDTH(4),
 .AXI_ADDR_WIDTH(32),
 .AXI_DATA_WIDTH(32),
 .AXI_USER_WIDTH(1),
 .APB_ADDR_WIDTH(32),
 .APB_DATA_WIDTH(32)
) u_convert (
 .aclk(clk),
 .aresetn(rst_n),

 // From AXI slave stub
 .r_s_axi_aw_pkt(aw_pkt),
 .r_s_axi_awvalid(aw_valid),
 .w_s_axi_awready(aw_ready),
 // ... other AXI channels ...
```

```

 // To APB master stub
 .w_cmd_valid(apb_cmd_valid),
 .r_cmd_ready(apb_cmd_ready),
 .r_cmd_data(apb_cmd_data),
 .r_rsp_valid(apb_rsp_valid),
 .w_rsp_ready(apb_rsp_ready),
 .r_rsp_data(apb_rsp_data)
);

```

### 39.7.2 Example 2: Width Conversion (64b → 32b)

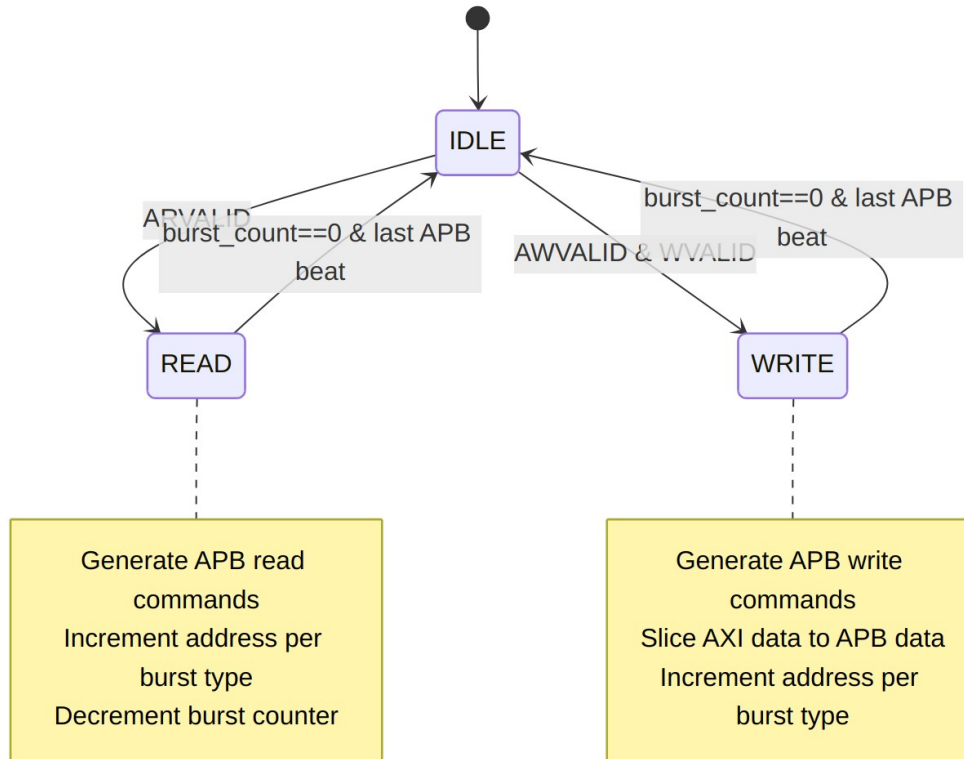
```

// Width adaptation: 64b AXI → 32b APB
axi4_to_apb_convert #(
 .SIDE_DEPTH(8), // Deeper for burst traffic
 .AXI_ID_WIDTH(8),
 .AXI_ADDR_WIDTH(32),
 .AXI_DATA_WIDTH(64), // 64-bit AXI
 .AXI_USER_WIDTH(4),
 .APB_ADDR_WIDTH(32),
 .APB_DATA_WIDTH(32) // 32-bit APB (2:1 ratio)
) u_convert (
 .aclk(clk),
 .aresetn(rst_n),
 // ... connections ...
);

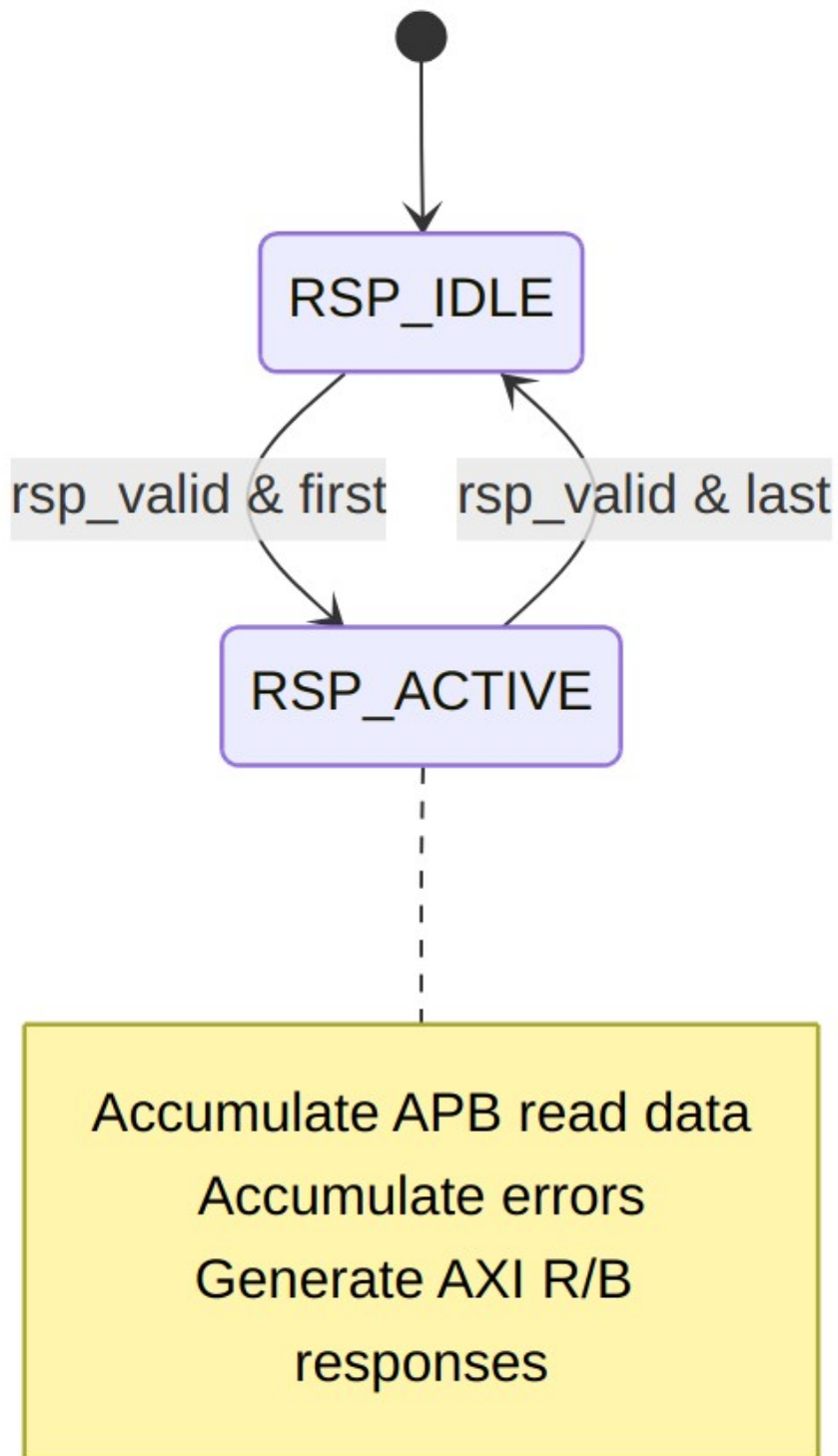
```

---

## 39.8 State Machines



*APB FSM*



## 39.9 Performance

---

### 39.9.1 Latency (Cycles)

Operation	Latency	Notes
AXI single-beat read (1:1 width)	4-5	Unpack + APB + pack
AXI single-beat write (1:1 width)	4-5	Same
AXI burst (AWLEN=15, 1:1 width)	68-80	16 beats × 4-5 cycles
Width convert read (2:1)	8-10	2 APB beats per AXI beat
Width convert read (4:1)	16-20	4 APB beats per AXI beat

### 39.9.2 Throughput

**Maximum Throughput (same width):** - 1 APB transfer every 2 cycles (SETUP + ACCESS phases) - Pipelined: ~0.5 transfers/cycle

**Width Conversion Impact:** - 2:1 ratio: Throughput halved (2 APB beats per AXI beat) - 4:1 ratio: Throughput quartered

---

## 39.10 Testing

---

```
Core conversion tested via integration test
pytest val/integ_amba/test_axi2apb_shim.py -v
```

```
Test with burst traffic
pytest "val/integ_amba/test_axi2apb_shim.py::test_burst" -v
```

```
Test width conversion
pytest "val/integ_amba/test_axi2apb_shim.py::test_width[64-32]" -v
```

---



## 39.11 Constraints and Limitations

---

**Parameter Constraints:** -  $APB\_DATA\_WIDTH \leq AXI\_DATA\_WIDTH - AXI\_DATA\_WIDTH /$   
APB\_DATA\_WIDTH must be power of 2 (1, 2, 4, 8) - SIDE\_DEPTH must be  $\geq$  maximum AXI burst length (typically 16)

**Protocol Limitations:** - No support for AXI exclusive access (AWLOCK/ARLOCK ignored) - No support for AXI cache/QoS hints (AWCACHE/ARCACHE/AWQOS/ARQOS ignored) - APB protocol is serialized (no concurrent transactions)

**Width Adaptation:** - Only supports narrowing (AXI  $\rightarrow$  APB), not widening - APB\_DATA\_WIDTH must be at least 8 bits

---

## 39.12 Synthesis Notes

---

### 39.12.1 Resource Usage

Configuration	LUTs	FFs	Notes
32b/32b	~400	~300	No width conversion
64b/32b	~600	~450	2:1 width conversion
128b/32b	~900	~650	4:1 width conversion

**Critical Logic:** - Address generation (axi\_gen\_addr instantiation) - Data shift register (read accumulation) - FSM next-state logic

---

## 39.13 Related Modules

---

- [axi4\\_to\\_apb\\_shim](#) - Top-level shim (instantiates this module)
  - [axi4\\_slave\\_stub](#) - Provides input packet format
  - [apb\\_master\\_stub](#) - Consumes output packet format
  - [axi\\_gen\\_addr](#) - Address generation utility
  - [gaxi\\_fifo\\_sync](#) - Side information FIFO
-

## 39.14 When to Use

---

✓ **Use Standalone When:** - Same clock domain for AXI and APB - Want minimal latency (no CDC overhead) - Custom integration with different buffering

✓ **Use via `axi4_to_apb_shim` When:** - Different clock domains (need CDC) - Want complete turnkey solution - Standard AXI/APB interfaces

---

**Last Updated:** 2025-10-20

---

## 39.15 Navigation

---

- [← Back to Shims Index](#)
- [← Back to RTLAmba Index](#)

# 40 `axi4_to_apb_shim`

**Module:** `axi4_to_apb_shim.sv` **Location:** `rtl/amba/shims/` **Status:** ✓ Production Ready

---

## 40.1 Overview

---

Top-level bridge module that converts AXI4 memory-mapped transactions to APB peripheral bus accesses with full dual-clock domain crossing (CDC) support. This shim integrates multiple components to provide a complete, robust bridge between AXI4 and APB protocols.

### 40.1.1 Key Features

- ✓ **Full AXI4 to APB Bridge:** Complete protocol conversion with burst decomposition
- ✓ **Dual-Clock Support:** Independent AXI (`aclk`) and APB (`pclk`) clock domains
- ✓ **CDC-Safe Handshakes:** Asynchronous clock domain crossing with proper synchronization
- ✓ **Configurable Buffering:** Skid buffers on all AXI channels for timing closure
- ✓ **Width Adaptation:** Automatic data width conversion (AXI → APB)
- ✓ **Burst Decomposition:** AXI bursts → APB single-beat transfers

- ✓ **Error Propagation:** APB PSLVERR mapped to AXI RRESP/BRESP
- 

## 40.2 Module Declaration

---

```

module axi4_to_apb_shim #(
 // Skid buffer depths (AXI side)
 parameter int DEPTH_AW = 2, // AW channel depth
 parameter int DEPTH_W = 4, // W channel depth
 parameter int DEPTH_B = 2, // B channel depth
 parameter int DEPTH_AR = 2, // AR channel depth
 parameter int DEPTH_R = 4, // R channel depth

 // Internal buffering
 parameter int SIDE_DEPTH = 4, // Side info FIFO depth
 parameter int APB_CMD_DEPTH = 4, // APB command FIFO depth
 parameter int APB_RSP_DEPTH = 4, // APB response FIFO depth

 // AXI parameters
 parameter int AXI_ID_WIDTH = 8,
 parameter int AXI_ADDR_WIDTH = 32,
 parameter int AXI_DATA_WIDTH = 32,
 parameter int AXI_USER_WIDTH = 1,

 // APB parameters
 parameter int APB_ADDR_WIDTH = 32,
 parameter int APB_DATA_WIDTH = 32
) (
 // Clock and Reset
 input logic aclk, // AXI clock
 domain
 input logic aresetn, // AXI reset
 (active-low)
 input logic pclk, // APB clock
 domain
 input logic presetn, // APB reset
 (active-low)

 // AXI4 Slave Interface (all 5 channels)
 // Write address channel (AW)
 input logic [AXI_ID_WIDTH-1:0] s_axi_awid,
 input logic [AXI_ADDR_WIDTH-1:0] s_axi_awaddr,
 input logic [7:0] s_axi_awlen,
 input logic [2:0] s_axi_awsz,
 input logic [1:0] s_axi_awburst,
 input logic s_axi_awlock,
 input logic [3:0] s_axi_awcache,

```

```

input logic [2:0]
input logic [3:0]
input logic [3:0]
input logic [AXI_USER_WIDTH-1:0]
input logic
output logic

s_axi_awprot,
s_axi_awqos,
s_axi_awregion,
s_axi_awuser,
s_axi_awvalid,
s_axi_awready,

// Write data channel (W)
input logic [AXI_DATA_WIDTH-1:0]
input logic [AXI_DATA_WIDTH/8-1:0]
input logic
input logic [AXI_USER_WIDTH-1:0]
input logic
output logic

s_axi_wdata,
s_axi_wstrb,
s_axi_wlast,
s_axi_wuser,
s_axi_wvalid,
s_axi_wready,

// Write response channel (B)
output logic [AXI_ID_WIDTH-1:0]
output logic [1:0]
output logic [AXI_USER_WIDTH-1:0]
output logic
input logic

s_axi_bid,
s_axi_bresp,
s_axi_buser,
s_axi_bvalid,
s_axi_bready,

// Read address channel (AR)
input logic [AXI_ID_WIDTH-1:0]
input logic [AXI_ADDR_WIDTH-1:0]
input logic [7:0]
input logic [2:0]
input logic [1:0]
input logic
input logic [3:0]
input logic [2:0]
input logic [3:0]
input logic [3:0]
input logic [AXI_USER_WIDTH-1:0]
input logic
output logic

s_axi_arid,
s_axi_araddr,
s_axi_arlen,
s_axi_arsize,
s_axi_arburst,
s_axi_arlock,
s_axi_arcache,
s_axi_arprot,
s_axi_arqos,
s_axi_arregion,
s_axi_aruser,
s_axi_arvalid,
s_axi_arready,

// Read data channel (R)
output logic [AXI_ID_WIDTH-1:0]
output logic [AXI_DATA_WIDTH-1:0]
output logic [1:0]
output logic
output logic [AXI_USER_WIDTH-1:0]
output logic
input logic

s_axi_rid,
s_axi_rdata,
s_axi_rresp,
s_axi_rlast,
s_axi_ruser,
s_axi_rvalid,
s_axi_rready,

// APB Master Interface

```

```

output logic [APB_ADDR_WIDTH-1:0] m_apb_PSEL,
output logic [APB_DATA_WIDTH-1:0] m_apb_PADDR,
output logic m_apb_PENABLE,
output logic m_apb_PWRITE,
output logic [APB_DATA_WIDTH/8-1:0] m_apb_PWDATA,
output logic [APB_DATA_WIDTH/8-1:0] m_apb_PSTRB,
output logic [2:0] m_apb_PPROT,

input logic [APB_DATA_WIDTH-1:0] m_apb_PRDATA,
input logic m_apb_PREADY,
input logic m_apb_PSLVERR
);

```

---

## 40.3 Parameters

---

### 40.3.1 Skid Buffer Depths

Parameter	Default	Description
DEPTH_AW	2	AW channel skid buffer depth (power of 2)
DEPTH_W	4	W channel skid buffer depth
DEPTH_B	2	B channel skid buffer depth
DEPTH_AR	2	AR channel skid buffer depth
DEPTH_R	4	R channel skid buffer depth

**Recommendation:** Use deeper depths (4-8) for high-latency paths, shallow (2) for low-latency.

### 40.3.2 Internal Buffering

Parameter	Default	Description
SIDE_DEPTH	4	Side information FIFO depth (tracks ID, last, user)
APB_CMD_DEPTH	4	APB command CDC FIFO depth

Parameter	Default	Description
APB_RSP_DEPTH	4	APB response CDC FIFO depth

### 40.3.3 AXI Interface

Parameter	Default	Description
AXI_ID_WIDTH	8	Transaction ID width (1-16)
AXI_ADDR_WIDTH	32	Address width (12-64)
AXI_DATA_WIDTH	32	Data width (32, 64, 128, 256, 512)
AXI_USER_WIDTH	1	User sideband signal width

### 40.3.4 APB Interface

Parameter	Default	Description
APB_ADDR_WIDTH	32	APB address width (must match AXI)
APB_DATA_WIDTH	32	APB data width (8, 16, 32, 64)

**Note:** APB\_DATA\_WIDTH must be  $\leq$  AXI\_DATA\_WIDTH. Width adaptation is automatic.

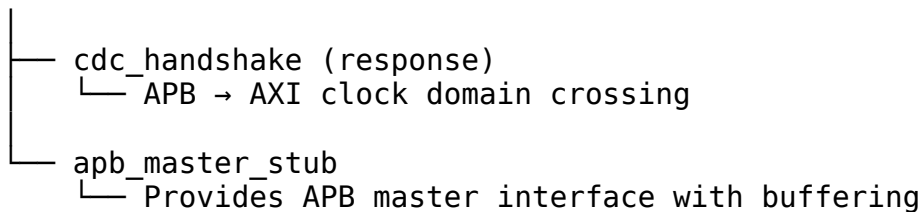
## 40.4 Architecture

### 40.4.1 Component Hierarchy

```

axi4_to_apb_shim (this module)
├── axi4_slave_stub
│ └── Provides AXI slave interface with skid buffers
├── axi4_to_apb_convert
│ ├── Core conversion logic
│ ├── Burst decomposition
│ ├── Width adaptation
│ └── gaxi_fifo_sync (side info)
└── cdc_handshake (command)
 └── AXI → APB clock domain crossing

```



## 40.4.2 Data Flow

### Write Transaction:

AXI Master → AW/W skid buffers (aclk)  
 → axi4\_to\_apb\_convert (burst decomposition, aclk)  
 → cmd CDC handshake (aclk → pclk)  
 → apb\_master\_stub (APB protocol, pclk)  
 → APB Slave (pclk)  
 → rsp CDC handshake (pclk → aclk)  
 → B channel response (aclk)

### Read Transaction:

AXI Master → AR skid buffer (aclk)  
 → axi4\_to\_apb\_convert (burst decomposition, aclk)  
 → cmd CDC handshake (aclk → pclk)  
 → apb\_master\_stub (APB protocol, pclk)  
 → APB Slave (pclk)  
 → rsp CDC handshake (pclk → aclk)  
 → R channel response + data (aclk)

---

## 40.5 Behavior

---

### 40.5.1 AXI to APB Conversion

**Burst Decomposition:** - AXI burst (AWLEN > 0) → Multiple APB single-beat transfers - Each beat addressed individually (address increments per burst type) - WLAST signal tracked to determine burst completion

**Width Adaptation:** - When AXI\_DATA\_WIDTH > APB\_DATA\_WIDTH: - Single AXI beat → Multiple APB beats - Data sliced according to APB\_DATA\_WIDTH - WSTRB mapped to PSTRB per beat - When AXI\_DATA\_WIDTH == APB\_DATA\_WIDTH: - Direct 1:1 mapping

**Protection Signals:** - AWPROT/ARPROT → PPROT (direct mapping)

**Error Handling:** - PSLVERR → BRESP[1] (write errors) - PSLVERR → RRESP[1] (read errors) - Multiple errors accumulated (OR'd) for burst transactions

## 40.5.2 Clock Domain Crossing

**CDC Handshake Protocol:** - Uses cdc\_handshake module (Gray-code pointer-based) - Command path: aclk → pclk - Response path: pclk → aclk - Zero data loss, proper synchronization

**Timing:** - Command CDC latency: 3-5 cycles (depending on clock ratio) - Response CDC latency: 3-5 cycles - Total round-trip latency: AXI skid + convert + 2×CDC + APB protocol

---

## 40.6 Usage Examples

---

### 40.6.1 Example 1: Same Clock, Same Width

```
// Simplest configuration: same clock, same data width
axi4_to_apb_shim #(
 .DEPTH_AW(2),
 .DEPTH_W(2),
 .DEPTH_B(2),
 .DEPTH_AR(2),
 .DEPTH_R(2),
 .AXI_ID_WIDTH(4),
 .AXI_ADDR_WIDTH(32),
 .AXI_DATA_WIDTH(32),
 .APB_ADDR_WIDTH(32),
 .APB_DATA_WIDTH(32)
) u_shim (
 .aclk(system_clk),
 .aresetn(system_resetn),
 .pclk(system_clk), // Same clock
 .presetn(system_resetn), // Same reset

 // Connect AXI slave interface
 .s_axi_awid(axi_awid),
 .s_axi_awaddr(axi_awaddr),
 // ... all AXI signals ...

 // Connect APB master interface
 .m_apb_PSEL(apb_psel),
 .m_apb_PADDR(apb_paddr),
 // ... all APB signals ...
);
```

### 40.6.2 Example 2: Dual-Clock with Width Conversion

```
// AXI @ 200 MHz, 64-bit → APB @ 100 MHz, 32-bit
axi4_to_apb_shim #(
 .DEPTH_AW(4), // Deeper for CDC
```



```

 .DEPTH_W(8), // Extra depth for width conversion
 .DEPTH_B(4),
 .DEPTH_AR(4),
 .DEPTH_R(8),
 .SIDE_DEPTH(8), // More side info for bursts
 .APB_CMD_DEPTH(8), // Deeper CDC FIFOs
 .APB_RSP_DEPTH(8),
 .AXI_ID_WIDTH(8),
 .AXI_ADDR_WIDTH(32),
 .AXI_DATA_WIDTH(64), // 64-bit AXI
 .APB_ADDR_WIDTH(32),
 .APB_DATA_WIDTH(32) // 32-bit APB (2:1 ratio)
) u_shim (
 .aclk(axi_clk_200mhz),
 .aresetn(axi_resetn),
 .pclk(apb_clk_100mhz), // Different clock
 .presetn(apb_resetn),
 // ... connections ...
);

```

#### 40.6.3 Example 3: CPU-Peripheral Bridge

```

// ARM AXI CPU → APB peripherals
axi4_to_apb_shim #(
 .DEPTH_AW(2),
 .DEPTH_W(4),
 .DEPTH_B(2),
 .DEPTH_AR(2),
 .DEPTH_R(4),
 .AXI_ID_WIDTH(12), // CPU master ID width
 .AXI_ADDR_WIDTH(32),
 .AXI_DATA_WIDTH(32),
 .AXI_USER_WIDTH(8), // CPU user signals
 .APB_ADDR_WIDTH(32),
 .APB_DATA_WIDTH(32)
) u_cpu_to_periph (
 .aclk(cpu_clk),
 .aresetn(cpu_resetn),
 .pclk(periph_clk),
 .presetn(periph_resetn),

 // From CPU AXI master
 .s_axi_awid(cpu_m_axi_awid),
 .s_axi_awaddr(cpu_m_axi_awaddr),
 // ... rest of AXI interface ...

 // To APB peripheral bus
 .m_apb_PSEL(periph_psel),

```

```

 .m_apb_PADDR(periph_paddr),
 // ... rest of APB interface ...
);

```

---

## 40.7 Performance Characteristics

---

### 40.7.1 Latency

Configuration	Typical Latency	Notes
Same clock, no width convert	5-7 cycles	Minimal overhead
Same clock, width convert	8-12 cycles	+1 cycle per APB beat
Dual-clock, no width convert	12-16 cycles	+3-5 cycles per CDC
Dual-clock, width convert	15-20 cycles	Combined overhead

**Components:** - AXI skid buffers: 1 cycle - Conversion logic: 2-3 cycles - CMD CDC: 3-5 cycles - APB protocol: 2 cycles minimum - RSP CDC: 3-5 cycles

### 40.7.2 Throughput

**Maximum AXI Burst Throughput:** - Limited by APB protocol (2 cycles per beat minimum) - Width conversion: N APB beats per AXI beat ( $N = \text{AXI\_DW} / \text{APB\_DW}$ ) - **Example:** 64b AXI → 32b APB = 4 APB cycles per AXI beat

#### Sustained Bandwidth:

AXI Bandwidth → APB Bandwidth:

- Same width:  $\text{APB\_BW} = \text{AXI\_BW} / 2$  (APB overhead)
  - 2:1 ratio:  $\text{APB\_BW} = \text{AXI\_BW} / 4$
  - 4:1 ratio:  $\text{APB\_BW} = \text{AXI\_BW} / 8$
- 

## 40.8 Testing

---

*# Run integration test*

```
pytest val/integ_amba/test_axi2apb_shim.py -v
```

*# Test with waveforms*

```
pytest val/integ_amba/test_axi2apb_shim.py --vcd=shim.vcd -v
```

```
gtkwave shim.vcd
```

```
Run with specific configuration
```

```
pytest "val/integ_amba/test_axi2apb_shim.py::test_axi2apb[64-32]" -v
```

---

## 40.9 Constraints and Limitations

---

**Parameter Constraints:** - APB\_DATA\_WIDTH must be  $\leq$  AXI\_DATA\_WIDTH - AXI\_DATA\_WIDTH / APB\_DATA\_WIDTH must be power of 2 - APB\_ADDR\_WIDTH should match AXI\_ADDR\_WIDTH (or be subset) - All DEPTH\_\* parameters must be  $\geq 2$

**Protocol Limitations:** - APB does not support outstanding transactions (serialized) - APB does not support bursts (AXI bursts decomposed) - No support for AXI exclusive access (AWLOCK/ARLOCK ignored) - No support for AXI cache hints (AWCACHE/ARCACHE ignored)

**CDC Requirements:** - Clock domains must be asynchronous (no phase relationship assumed) - Resets must be properly synchronized in each domain - Metastability-hardened flip-flops recommended for CDC paths

---

## 40.10 Synthesis Notes

---

### 40.10.1 Resource Usage (Typical)

Configuration	LUTs	FFs	BRAM	Notes
32b/32b, shallow	~800	~600	0	Minimal buffering
64b/32b, moderate	~1200	~900	0	Width conversion
64b/32b, deep	~1500	~1100	0	CDC + deep buffers

**Breakdown:** - axi4\_slave\_stub: ~300 LUTs, ~200 FFs (skid buffers) - axi4\_to\_apb\_convert: ~400 LUTs, ~300 FFs (FSMs, width logic) - 2x cdc\_handshake: ~200 LUTs, ~150 FFs (synchronizers) - apb\_master\_stub: ~100 LUTs, ~100 FFs (APB protocol)

### 40.10.2 Timing Closure

**Critical Paths:** - AXI packet unpacking (combinatorial) - Width conversion muxing - CDC synchronizers (use proper constraints)

### Recommended Constraints:

```
Set false path between clock domains
set_false_path -from [get_clocks aclk] -to [get_clocks pclk]
set_false_path -from [get_clocks pclk] -to [get_clocks aclk]

Constrain CDC paths
set_max_delay -from [get_pins */cdc_handshake/src_*] \
 -to [get_pins */cdc_handshake/dst_*] 10.0
```

---

## 40.11 Related Modules

---

- [axi4\\_to\\_apb\\_convert](#) - Core conversion logic (instantiated internally)
  - [axi4\\_slave\\_stub](#) - AXI slave interface wrapper
  - [apb\\_master\\_stub](#) - APB master interface wrapper
  - [cdc\\_handshake](#) - Clock domain crossing handshake
- 

## 40.12 When to Use

---

✓ **Use This Shim When:** - Bridging AXI4 system interconnect to APB peripherals - Different clock domains for AXI and APB buses - Need protocol conversion with full CDC support - CPU/DMA master accessing peripheral registers

✓ **Consider Alternatives When:** - Both sides are AXI (use AXI crossbar/interconnect instead) - Both sides are APB (use APB crossbar) - Need high-performance memory access (APB is slow, use AXI4)

---

Last Updated: 2025-10-20

---

## 40.13 Navigation

---

- [← Back to Shims Index](#)
- [← Back to RTLAmba Index](#)

# 41 peakrdl\_to\_cmdrsp

**Module:** peakrdl\_to\_cmdrsp.sv **Location:** rtl/amba/shims/ **Status:** ✓ Production Ready

---

## 41.1 Overview

---

Protocol adapter that bridges between the RTL Design Sherpa standard cmd/rsp interface and the PeakRDL register block “passthrough” cpuif interface. This shim enables seamless integration of PeakRDL-generated register blocks with RTL Design Sherpa’s APB infrastructure.

### 41.1.1 Key Features

- ✓ **Protocol Conversion:** cmd/rsp valid/ready ↔ PeakRDL passthrough cpuif
  - ✓ **Stall Handling:** Proper support for PeakRDL req\_stall signals
  - ✓ **Strobe Conversion:** Byte strobes → bit enables (PeakRDL requirement)
  - ✓ **Error Propagation:** PeakRDL errors → cmd/rsp PSLVERR
  - ✓ **FSM-Based:** Robust state machines for cmd and rsp channels
  - ✓ **Assertions Included:** Comprehensive protocol checking (simulation only)
- 

## 41.2 Background: PeakRDL Integration

---

**PeakRDL** is a register description language compiler that generates SystemVerilog register blocks from .rdl files. The generated blocks use a “passthrough” cpuif (CPU interface) that differs from standard bus protocols.

**This adapter solves:** - Integration with RTL Design Sherpa’s cmd/rsp-based APB infrastructure - Byte strobe → bit enable conversion (PeakRDL uses bit-level enables) - Stall handling (PeakRDL can stall requests separately for read/write) - Response channel synchronization

**See Also:** rtl/amba/shims/peakrdl\_adapter\_README.md for PeakRDL workflow guide

---

## 41.3 Module Declaration

---

```
module peakrdl_to_cmdrsp #(
 parameter int ADDR_WIDTH = 12, // Address width (match PeakRDL
generation)
 parameter int DATA_WIDTH = 32 // Must match PeakRDL (typically
32)
) (
 // Clock and Reset
 input logic aclk,
 input logic aresetn,
```

```

//
=====
===
// CMD/RSP Interface (rtlsherpas standard)
//
=====
===
// Command Channel
input logic cmd_valid,
output logic cmd_ready,
input logic cmd_pwrite, // 1=write,
0=read
input logic [ADDR_WIDTH-1:0] cmd_paddr, // Byte
address
input logic [DATA_WIDTH-1:0] cmd_pwdata, // Write data
input logic [DATA_WIDTH/8-1:0] cmd_pstrb, // Byte
strobess

// Response Channel
output logic rsp_valid,
input logic rsp_ready,
output logic [DATA_WIDTH-1:0] rsp_prdata, // Read data
output logic rsp_pslverr, // Error flag

//
=====
===
// PeakRDL Passthrough Interface
//
=====
===
output logic regblk_req, // Request
strobe
output logic regblk_req_is_wr, // Write flag
output logic [ADDR_WIDTH-1:0] regblk_addr, // Address
output logic [DATA_WIDTH-1:0] regblk_wr_data, // Write data
output logic [DATA_WIDTH-1:0] regblk_wr_biten, // Write bit
enables

input logic regblk_req_stall_wr, // Write
stall
input logic regblk_req_stall_rd, // Read stall
input logic regblk_rd_ack, // Read
acknowledge
input logic regblk_rd_err, // Read error
input logic [DATA_WIDTH-1:0] regblk_rd_data, // Read data

```

```

 input logic regblk_wr_ack, // Write
 acknowledge
 input logic regblk_wr_err // Write error
);

```

---

## 41.4 Parameters

Parameter	Default	Description
ADDR_WIDTH	12	Address width for cmd/rsp interface (4 KB register space)
DATA_WIDTH	32	Data width (must match PeakRDL generation, typically 32)

**Note:** These must match the PeakRDL register block generation parameters.

---

## 41.5 Ports

### 41.5.1 CMD/RSP Interface (RTL Design Sherpa Standard)

**Command Channel:**

Port	Direction	Width	Description
cmd_valid	Input	1	Command valid
cmd_ready	Output	1	Ready to accept command
cmd_pwrite	Input	1	Write operation (1=write, 0=read)
cmd_paddr	Input	ADDR_WIDTH	Byte address
cmd_pwdata	Input	DATA_WIDTH	Write data
cmd_pstrb	Input	DATA_WIDTH/8	Byte strobes (1=valid byte)

**Response Channel:**

Port	Direction	Width	Description
rsp_valid	Output	1	Response valid
rsp_ready	Input	1	Ready to consume response
rsp_prdata	Output	DATA_WIDTH	Read data
rsp_pslverr	Output	1	Error flag (1=error)

**41.5.2 PeakRDL Passthrough Interface****Request Signals:**

Port	Direction	Width	Description
regblk_req	Output	1	Request strobe (valid for 1 cycle)
regblk_req_is_wr	Output	1	Operation type (1=write, 0=read)
regblk_addr	Output	ADDR_WIDTH	Register address
regblk_wr_data	Output	DATA_WIDTH	Write data
regblk_wr_biten	Output	DATA_WIDTH	Write bit enables (1=update bit)

**Stall/Acknowledge Signals:**

Port	Direction	Width	Description
regblk_req_stall_wr	Input	1	Write request stalled (retry later)
regblk_req_stall_rd	Input	1	Read request stalled (retry later)
regblk_rd_ack	Input	1	Read completed (1 cycle pulse)
regblk_wr_ack	Input	1	Write completed (1 cycle pulse)



Port	Direction	Width	Description
			pulse)
regblk_rd_data	Input	DATA_WIDT H	Read data (valid with rd_ack)
regblk_rd_err	Input	1	Read error (valid with rd_ack)
regblk_wr_err	Input	1	Write error (valid with wr_ack)

## 41.6 Behavior

### 41.6.1 Command State Machine

#### States:

CMD\_IDLE → Waiting for command  
 CMD\_WAIT\_ACK → Waiting for register block ack  
 CMD\_STALLED → Stalled, retry when stall clears

#### Transitions:

##### IDLE:

- cmd\_valid && !stall → WAIT\_ACK
- cmd\_valid && stall → STALLED

##### WAIT\_ACK:

- regblk\_wr\_ack || regblk\_rd\_ack → IDLE

##### STALLED:

- !stall → WAIT\_ACK (retry request)

### 41.6.2 Response State Machine

#### States:

RSP\_IDLE → No response pending  
 RSP\_VALID → Response valid, waiting for rsp\_ready

#### Transitions:

##### IDLE:

- regblk\_wr\_ack || regblk\_rd\_ack → VALID

VALID:

- rsp\_ready → IDLE

### 41.6.3 Strobe to Bit Enable Conversion

PeakRDL requires bit-level write enables:

```
function automatic logic [DATA_WIDTH-1:0] strb_to_biten(
 input logic [STRB_WIDTH-1:0] strb
);
 for (int i = 0; i < STRB_WIDTH; i++) begin
 biten[i*8 +: 8] = {8{strb[i]}};
 end
 return biten;
endfunction
```

Example (32-bit):

```
cmd_pstrb = 4'b1100
 ↓
regblk_wr_biten = 32'hFFFF_0000
 (upper 16 bits enabled)
```

### 41.6.4 Stall Handling

**Separate stalls for read and write:** - regblk\_req\_stall\_wr: Write request cannot be accepted - regblk\_req\_stall\_rd: Read request cannot be accepted

**Behavior:** 1. If stall asserted when cmd\_valid: - Register command locally - Enter CMD\_STALLED state - Keep regblk\_req deasserted 2. When stall clears: - Assert regblk\_req with registered command - Move to CMD\_WAIT\_ACK

**Purpose:** Allows PeakRDL register block to implement internal arbitration or serialization.

---

## 41.7 Usage Examples

---

### 41.7.1 Example 1: Basic Integration with PeakRDL Register Block

```
// Generate PeakRDL register block
// $ peakrdl regblock my_regs.rdl -o my_regs.sv

// Instantiate adapter
peakrdl_to_cmdrsp #(
 .ADDR_WIDTH(12), // 4 KB register space
 .DATA_WIDTH(32)
) u_adapter (
 .clk(apb_clk),
```

```

 .aresetn(apb_resetn),

 // From APB master stub (cmd/rsp interface)
 .cmd_valid(apb_cmd_valid),
 .cmd_ready(apb_cmd_ready),
 .cmd_pwrite(apb_cmd_pwrite),
 .cmd_paddr(apb_cmd_paddr),
 .cmd_pwdata(apb_cmd_pwdata),
 .cmd_pstrb(apb_cmd_pstrb),

 .rsp_valid(apb_rsp_valid),
 .rsp_ready(apb_rsp_ready),
 .rsp_prdata(apb_rsp_prdata),
 .rsp_pslverr(apb_rsp_pslverr),

 // To PeakRDL register block
 .regblk_req(my_regs_req),
 .regblk_req_is_wr(my_regs_req_is_wr),
 .regblk_addr(my_regs_addr),
 .regblk_wr_data(my_regs_wr_data),
 .regblk_wr_biten(my_regs_wr_biten),

 .regblk_req_stall_wr(my_regs_req_stall_wr),
 .regblk_req_stall_rd(my_regs_req_stall_rd),
 .regblk_rd_ack(my_regs_rd_ack),
 .regblk_rd_err(my_regs_rd_err),
 .regblk_rd_data(my_regs_rd_data),
 .regblk_wr_ack(my_regs_wr_ack),
 .regblk_wr_err(my_regs_wr_err)
);

// Instantiate PeakRDL register block
my_regs u_regs (
 .clk(apb_clk),
 .rst(~apb_resetn),

 // Passthrough cpuif
 .cpuif_req(my_regs_req),
 .cpuif_req_is_wr(my_regs_req_is_wr),
 .cpuif_addr(my_regs_addr),
 .cpuif_wr_data(my_regs_wr_data),
 .cpuif_wr_biten(my_regs_wr_biten),
 .cpuif_req_stall_wr(my_regs_req_stall_wr),
 .cpuif_req_stall_rd(my_regs_req_stall_rd),
 .cpuif_rd_ack(my_regs_rd_ack),
 .cpuif_rd_err(my_regs_rd_err),
 .cpuif_rd_data(my_regs_rd_data),

```

```

 .cpuif_wr_ack(my_regs_wr_ack),
 .cpuif_wr_err(my_regs_wr_err),

 // Field interfaces
 .hwif_in(...),
 .hwif_out(...)
);

```

#### 41.7.2 Example 2: APB to PeakRDL Complete Path

*// APB Master → APB Slave Stub → peakrdl\_to\_cmdrsp → PeakRDL Register Block*

```

// APB slave stub (APB protocol → cmd/rsp)
apb_slave_stub #(
 .DATA_WIDTH(32),
 .ADDR_WIDTH(12)
) u_apb_slave (
 .pclk(apb_clk),
 .presetn(apb_resetn),

 // APB slave interface
 .s_apb_PSEL(apb_psel),
 .s_apb_PADDR(apb_paddr),
 .s_apb_PENABLE(apb_penable),
 .s_apb_PWRITE(apb_pwrite),
 .s_apb_PWDATA(apb_pwdata),
 .s_apb_PSTRB(apb_pstrb),
 .s_apb_PREADY(apb_pready),
 .s_apb_PRDATA(apb_prdata),
 .s_apb_PSLVERR(apb_pslverr),

 // cmd/rsp interface
 .cmd_valid(cmd_valid),
 .cmd_ready(cmd_ready),
 .cmd_pwrite(cmd_pwrite),
 .cmd_paddr(cmd_paddr),
 .cmd_pwdata(cmd_pwdata),
 .cmd_pstrb(cmd_pstrb),
 .rsp_valid(rsp_valid),
 .rsp_ready(rsp_ready),
 .rsp_prdata(rsp_prdata),
 .rsp_pslverr(rsp_pslverr)
);

// PeakRDL adapter (cmd/rsp → passthrough cpuif)
peakrdl_to_cmdrsp #(
 .ADDR_WIDTH(12),

```

```

 .DATA_WIDTH(32)
) u_adapter (
 .aclk(apb_clk),
 .aresetn(apb_resetn),
 .cmd_valid(cmd_valid),
 .cmd_ready(cmd_ready),
 // ... cmd/rsp interface ...
 .regblk_req(regblk_req),
 // ... passthrough cpuif ...
);

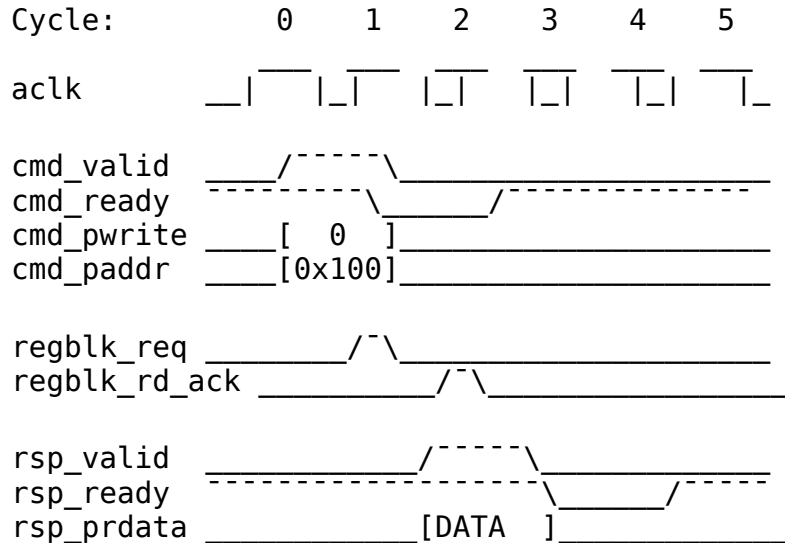
 // PeakRDL register block
 my_regs u_regs (
 .clk(apb_clk),
 .rst(~apb_resetn),
 .cpuif_req(regblk_req),
 // ... passthrough cpuif ...
);

```

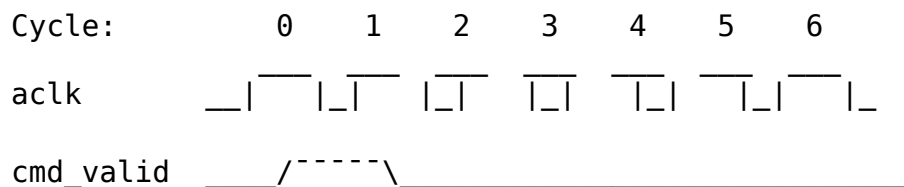
---

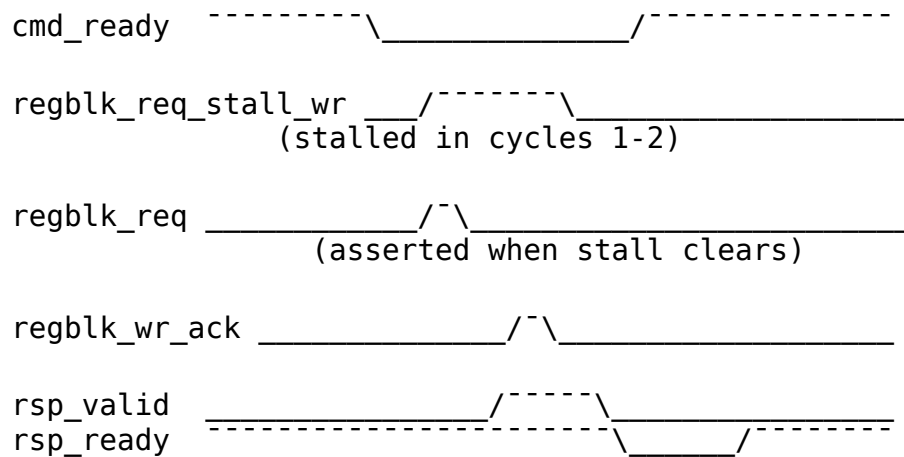
## 41.8 Timing Diagrams

### 41.8.1 Read Transaction (No Stall)



### 41.8.2 Write Transaction with Stall





## 41.9 Testing

*# Run adapter test*

```
pytest val/amba/test_peakrdl_to_cmdrsp.py -v
```

*# Test with waveforms*

```
pytest val/amba/test_peakrdl_to_cmdrsp.py --vcd=peakrdl.vcd -v
gtkwave peakrdl.vcd
```

## 41.10 Assertions

Included assertions (simulation only):

1. **cmd\_valid\_stable:** cmd\_valid must remain high until cmd\_ready
2. **cmd\_data\_stable:** cmd signals must remain stable until cmd\_ready
3. **rsp\_valid\_stable:** rsp\_valid must remain high until rsp\_ready
4. **rsp\_data\_stable:** rsp signals must remain stable until rsp\_ready

**Purpose:** Catch protocol violations during simulation.

## 41.11 Performance

### 41.11.1 Latency

Operation	Typical Latency	Notes
Read (no stall)	2 cycles	req → rd_ack

Operation	Typical Latency	Notes
Write (no stall)	2 cycles	→ rsp req → wr_ack → rsp
Read (with stall)	2 + stall cycles	Depends on PeakRDL arbitration
Write (with stall)	2 + stall cycles	Depends on PeakRDL arbitration

#### 41.11.2 Throughput

- **Maximum:** 1 transaction per 2 cycles (no stalls)
- **Typical:** 1 transaction per 3-5 cycles (APB overhead)

## 41.12 Synthesis Notes

### 41.12.1 Resource Usage

**Typical (ADDR\_WIDTH=12, DATA\_WIDTH=32):** - ~50 LUTs - ~100 FFs - 0 BRAM

**Components:** - Command FSM: ~20 LUTs, ~40 FFs - Response FSM: ~10 LUTs, ~35 FFs - Strobe conversion: ~20 LUTs (combinatorial)

## 41.13 Related Modules

- **apb\_slave\_stub** - APB → cmd/rsp conversion (upstream)
- **apb\_master\_stub** - cmd/rsp → APB conversion

**External:** - PeakRDL: <https://github.com/SystemRDL/PeakRDL> - PeakRDL regblock: <https://github.com/SystemRDL/PeakRDL-regblock>

## 41.14 When to Use

✓ **Use This Adapter When:** - Integrating PeakRDL-generated register blocks - Using RTL Design Sherpa APB infrastructure - Need cmd/rsp ↔ passthrough cpuif conversion

✓ **Alternatives:** - PeakRDL native APB cpuif (if not using cmd/rsp) - Custom register implementation (if not using PeakRDL)

---

## 41.15 PeakRDL Workflow

---

**Complete workflow documented in:** `rtl/amba/shims/peakrdl_adapter_README.md`

**Quick summary:** 1. Write `.rdl` register description 2. Generate SystemVerilog: `peakrdl regblock my_regs.rdl -o my_regs.sv` 3. Select passthrough cpuif during generation 4. Use this adapter to bridge to APB infrastructure

---

**Last Updated:** 2025-10-20

---

## 41.16 Navigation

---

- [← Back to Shims Index](#)
- [← Back to RTLamba Index](#)

# 42 AMBA Protocol Shims

**Location:** `rtl/amba/shims/` **Test Location:** `val/amba/`, `val/integ_amba/` **Status:** ✓ Production Ready

---

## 42.1 Overview

---

The shims subsystem provides protocol conversion modules that bridge between different bus protocols and interface standards. These adapters enable seamless integration of heterogeneous components within the AMBA ecosystem.

### 42.1.1 Key Features

- ✓ **Protocol Bridging:** Convert between AXI4, APB, and custom interfaces
- ✓ **Clock Domain Crossing:** Dual-clock support with proper CDC
- ✓ **Width Adaptation:** Automatic data width conversion
- ✓ **Burst Decomposition:** Complex transactions → Simple protocol transfers
- ✓ **Standards Compliance:** PeakRDL integration, APB/AXI4 conformance



- ✓ **Production Ready:** Comprehensive testing and verification

## 42.2 Available Shims

Shim	Purpose	Documentation	Status
<b>axi4_to_apb_shim</b>	AXI4 to APB bridge with dual-clock CDC	<a href="#">axi4_to_apb_shim.md</a>	✓ Documented
<b>axi4_to_apb_converter</b>	Core AXI4→APB conversion logic	<a href="#">axi4_to_apb_convert.md</a>	✓ Documented
<b>peakrdl_to_cmdrsp</b>	PeakRDL passthrough → cmd/rsp adapter	<a href="#">peakrdl_to_cmdrsp.md</a>	✓ Documented

## 42.3 Quick Start

### 42.3.1 AXI4 to APB Bridge

*// Bridge AXI4 master to APB peripheral bus*

```
axi4_to_apb_shim #(
 .AXI_ID_WIDTH(8),
 .AXI_ADDR_WIDTH(32),
 .AXI_DATA_WIDTH(32),
 .APB_ADDR_WIDTH(32),
 .APB_DATA_WIDTH(32)
) u_bridge (
 .aclk(axi_clk),
 .aresetn(axi_resetn),
 .pclk(apb_clk),
 .presetn(apb_resetn),

 // AXI4 slave interface
 .s_axi_awid(axi_awid),
 .s_axi_awaddr(axi_awaddr),
 // ... AXI signals ...

 // APB master interface
 .m_apb_PSEL(apb_psel),
 .m_apb_PADDR(apb_paddr),
```

```

 // ... APB signals ...
);

42.3.2 PeakRDL Register Integration
// Integrate PeakRDL register block
peakrdl_to_cmdrsp #(
 .ADDR_WIDTH(12),
 .DATA_WIDTH(32)
) u_adapter (
 .aclk(clk),
 .aresetn(rst_n),

 // From APB master stub (cmd/rsp)
 .cmd_valid(cmd_valid),
 .cmd_ready(cmd_ready),
 .cmd_pwrite(cmd_pwrite),
 .cmd_paddr(cmd_paddr),
 .cmd_pdata(cmd_pdata),
 .cmd_pstrb(cmd_pstrb),
 .rsp_valid(rsp_valid),
 .rsp_ready(rsp_ready),
 .rsp_prdata(rsp_prdata),
 .rsp_pslverr(rsp_pslverr),

 // To PeakRDL register block
 .regblk_req(regblk_req),
 .regblk_req_is_wr(regblk_req_is_wr),
 // ... PeakRDL cpuif ...
);

```

---

## 42.4 Design Patterns

---

### 42.4.1 Pattern 1: CPU to Peripherals Bridge

Use AXI4-to-APB shim for CPU master accessing peripheral registers:

```

// ARM Cortex-M processor → APB peripherals
axi4_to_apb_shim #(
 .AXI_ID_WIDTH(12), // CPU master ID width
 .AXI_ADDR_WIDTH(32),
 .AXI_DATA_WIDTH(32),
 .APB_ADDR_WIDTH(32),
 .APB_DATA_WIDTH(32),
 .DEPTH_AW(2), // Shallow buffers for low latency
 .DEPTH_AR(2)
)

```

```

) u_cpu_periph_bridge (
 .aclk(cpu_clk),
 .aresetn(cpu_resetn),
 .pclk(periph_clk),
 .presetn(periph_resetn),
 // ... connections ...
);

```

**Characteristics:** - Low latency (shallow buffers) - Single-beat transactions typical - Dual-clock support

#### 42.4.2 Pattern 2: High-Speed to Low-Speed Bridge

Use width conversion for data rate adaptation:

```

// 64-bit AXI @ 200 MHz → 32-bit APB @ 100 MHz
axi4_to_apb_shim #(
 .AXI_DATA_WIDTH(64), // High-speed side
 .APB_DATA_WIDTH(32), // Low-speed side (2:1)
 .DEPTH_AW(4), // Deeper for CDC + bursts
 .DEPTH_W(8), // Extra depth for width conversion
 .DEPTH_R(8),
 .SIDE_DEPTH(8)
) u_width_bridge (
 .aclk(axi_clk_200mhz),
 .pclk(apb_clk_100mhz),
 // ... connections ...
);

```

**Characteristics:** - Width adaptation (automatic slicing) - Deeper buffers for burst traffic - Optimized for throughput

#### 42.4.3 Pattern 3: Register Block Integration

Use PeakRDL adapter for register automation:

APB Bus → apb\_slave\_stub → peakrdl\_to\_cmdrsp → PeakRDL Register Block  
 (APB→cmd/rsp)      (cmd/rsp→cpuif)      (generated from .rdl)

**Benefits:** - Register automation via PeakRDL - Standard cmd/rsp intermediate interface - Error propagation - Stall handling

## 42.5 Performance Characteristics

### 42.5.1 AXI4 to APB Bridge

**Latency:**

Configuration	Typical Latency	Notes
Same clock, no width convert	5-7 cycles	Minimal overhead
Same clock, 2:1 width convert	8-12 cycles	+1 cycle per APB beat
Dual-clock, no width convert	12-16 cycles	+3-5 cycles per CDC
Dual-clock, 2:1 width convert	15-20 cycles	Combined overhead

#### Throughput:

Width Ratio	Max Throughput	Notes
1:1	0.5 transfers/cycle	APB overhead
2:1	0.25 transfers/cycle	Width conversion
4:1	0.125 transfers/cycle	More APB beats

### 42.5.2 PeakRDL Adapter

**Latency:** - Read: 2 cycles (no stall) - Write: 2 cycles (no stall) - +N cycles if PeakRDL stalls request

**Throughput:** - Maximum: 1 transaction per 2 cycles - Typical: 1 transaction per 3-5 cycles (with APB)

## 42.6 Testing

*# Test AXI to APB bridge*

```
pytest val/integ_amba/test_axi2apb_shim.py -v
```

*# Test PeakRDL adapter*

```
pytest val/amba/test_peakrdl_to_cmdrsp.py -v
```

*# Run all shims tests*

```
pytest val/amba/test_*shim*.py val/integ_amba/test_*shim*.py -v
```

*# Generate waveforms*

```
pytest val/integ_amba/test_axi2apb_shim.py --vcd=bridge.vcd -v
gtkwave bridge.vcd
```

## 42.7 Common Use Cases

---

### 42.7.1 Use Case 1: SoC Integration

**Scenario:** ARM Cortex-M processor with AXI4 master accessing APB peripherals

**Solution:**

```
axi4_to_apb_shim u_cpu_bridge (
 .aclk(cpu_axi_clk),
 .pclk(periph_apb_clk),
 // Connect CPU master → APB peripheral bus
);
```

**Benefits:** - Enables CPU access to APB peripherals - Handles clock domain crossing - Minimal latency for register access

### 42.7.2 Use Case 2: DMA to Peripherals

**Scenario:** DMA engine (AXI4 master) writing to APB peripheral registers

**Solution:**

```
axi4_to_apb_shim #(
 .DEPTH_AW(8), // Deep buffers for burst DMA
 .DEPTH_W(16)
) u_dma_bridge (
 .aclk(dma_clk),
 .pclk(periph_clk),
 // Connect DMA master → APB registers
);
```

**Benefits:** - Burst decomposition (DMA bursts → APB single-beats) - Deep buffering for DMA traffic  
- Error reporting back to DMA

### 42.7.3 Use Case 3: Configuration Registers

**Scenario:** Control registers for IP block, auto-generated from SystemRDL

**Solution:**

APB → apb\_slave\_stub → peakrdl\_to\_cmdrsp → PeakRDL config\_regs

**Benefits:** - Register automation (field reset, RW access, interrupts) - Documentation generation - Software header generation - Type safety

---

## 42.8 Parameter Selection Guidelines

---

### 42.8.1 AXI to APB Bridge

**Skid Buffer Depths:**

Application	DEPTH_AW/ AR	DEPTH_W	DEPTH_R	Notes
Low-latency CPU	2	2-4	2-4	Minimize latency
Moderate DMA	4	8	8	Balance latency/throughput
Burst DMA	8	16	16	Maximize throughput

**Width Conversion:**

AXI Width	APB Width	Ratio	Typical Use
32	32	1:1	Simple bridging
64	32	2:1	Modern CPU → legacy periph
128	32	4:1	Wide DMA → narrow periph
64	64	1:1	High-bandwidth peripherals

**Side FIFO Depth:** -  $SIDE\_DEPTH \geq \max(\text{AXI burst length} \times \text{width ratio})$  - Typical: 4-8 for CPU, 8-16 for DMA

### 42.8.2 PeakRDL Adapter

**Address Width:** - Match PeakRDL generation: - -cpuif-addr-width N - Typical: 12 bits (4 KB register space)

**Data Width:** - Must match PeakRDL generation (usually 32) - No width conversion supported

---

## 42.9 Design Tradeoffs

### 42.9.1 AXI to APB Bridge

✓ **Advantages:** - Standard protocols (AXI4 ↔ APB) - Dual-clock CDC support - Automatic width conversion - Burst decomposition

✗ **Limitations:** - APB serialization (no concurrent transactions) - Performance overhead (APB slower than AXI) - No AXI exclusive access support - Resource cost (~800-1500 LUTs)

**Alternatives:** - Native APB masters (if no AXI required) - AXI crossbar (if staying in AXI domain)

### 42.9.2 PeakRDL Adapter

✓ **Advantages:** - Register automation - Standards-based (SystemRDL) - Documentation generation - Low overhead (~50 LUTs)

✗ **Limitations:** - PeakRDL dependency - Learning curve (SystemRDL) - 32-bit only (typical)

**Alternatives:** - Custom register implementation - PeakRDL native APB cpuif - Commercial register tools

## 42.10 Synthesis Notes

### 42.10.1 Resource Usage

Module	LUTs	FFs	BRAM	Notes
axi4_to_apb_shim (32/32)	~800	~600	0	Minimal config
axi4_to_apb_shim (64/32)	~1200	~900	0	Width conversion
axi4_to_apb_convert	~400	~300	0	Core logic only
peakrdl_to_cmdrsp	~50	~100	0	Lightweight

### 42.10.2 Timing Closure

**Critical Paths:** - AXI packet unpacking (combinatorial) - Width conversion muxing - CDC synchronizers

## Recommendations:

```
Constrain CDC paths in axi4_to_apb_shim
set_false_path -from [get_clocks aclk] -to [get_clocks pclk]
set_max_delay -from */cdc_handshake/src_* -to */cdc_handshake/dst_*
10.0
```

```
Register insertion for wide data paths
set_max_fanout 16 [get_pins */data_shift_reg*]
```

---

## 42.11 Related Documentation

---

### 42.11.1 Protocol Specifications

- ARM AMBA 4 AXI4 Specification
- ARM AMBA 3 APB Protocol Specification v2.0
- SystemRDL 2.0 Specification (PeakRDL)

### 42.11.2 RTL Design Sherpa Documentation

- [RTLamba Overview](#) - Complete AMBA subsystem
- [AXI4 Modules](#) - AXI4 infrastructure
- [APB Modules](#) - APB infrastructure
- [Shared Utilities](#) - Common modules (CDC, FIFOs)

### 42.11.3 External Resources

- **PeakRDL:** <https://github.com/SystemRDL/PeakRDL>
- **PeakRDL regblock:** <https://github.com/SystemRDL/PeakRDL-regblock>
- **PeakRDL Workflow Guide:** [rtl/amba/shims/peakrdl\\_adapter\\_README.md](#)

### 42.11.4 Source Code

- RTL: [rtl/amba/shims/](#)
  - Tests: [val/amba/test\\_peakrdl\\*.py](#), [val/integ\\_amba/test\\_axi2apb\\*.py](#)
  - Framework: [bin/TBClasses/components/](#)
-



## 42.12 Design Notes

---

### 42.12.1 When to Use Shims

✓ **Use AXI4-to-APB Shim When:** - Bridging AXI4 master to APB slave - Different clock domains required - Width conversion needed - Burst decomposition required

✓ **Use PeakRDL Adapter When:** - Automating register block generation - Using SystemRDL descriptions - Need documentation/header generation - Integrating with cmd/rsp infrastructure

✗ **Consider Alternatives When:** - Same protocol both sides (use crossbar) - Need high performance (APB is slow) - Simple registers (custom implementation may be lighter)

### 42.12.2 Clock Domain Crossing

**AXI to APB CDC:** - Uses cdc\_handshake module - Gray-code pointer-based synchronization - Safe for arbitrary clock ratios - Proper reset synchronization required

**Constraints Required:**

```
set_false_path -from [get_clocks aclk] -to [get_clocks pclk]
set_false_path -from [get_clocks pclk] -to [get_clocks aclk]
```

---

## 42.13 Migration Guide

---

### 42.13.1 Upgrading from Legacy AXI-APB Bridges

**Before (legacy):**

```
axi_apb_bridge u_bridge (
 .axi_*, // Legacy AXI naming
 .apb_* // Legacy APB naming
);
```

**After (RTL Design Sherpa):**

```
axi4_to_apb_shim u_bridge (
 .s_axi_*, // AXI slave interface
 .m_apb_* // APB master interface
);
```

**Key Changes:** - Explicit slave/master prefixes - Standardized signal naming - Added CDC support - Parameterized buffering

---

## 42.14 Troubleshooting

---

### 42.14.1 Issue 1: High Latency

**Symptoms:** Slow register access, performance degradation

**Debug:** 1. Check buffer depths (DEPTH\_\*) - deeper = higher latency 2. Verify clock domain crossing overhead 3. Check for width conversion ratio 4. Monitor APB protocol overhead

**Solutions:** - Reduce buffer depths for low latency - Same-clock operation if possible - Minimize width ratio (prefer 1:1)

### 42.14.2 Issue 2: PeakRDL Stalls

**Symptoms:** Long delays, timeout errors

**Debug:** 1. Check regblk\_req\_stall\_\* signals 2. Verify PeakRDL internal logic 3. Check for field access conflicts

**Solutions:** - Review PeakRDL register arbitration - Check for hardware write conflicts - Verify interrupt handling

### 42.14.3 Issue 3: Data Corruption

**Symptoms:** Incorrect read data, write failures

**Debug:** 1. Verify width conversion logic 2. Check byte strobes (WSTRB → bit enables) 3. Monitor error signals (PSLVERR, BRESP, RRESP)

**Solutions:** - Verify parameter configuration - CheckWSTRB alignment - Enable error checking in testbench

---

**Last Updated:** 2025-10-20

---

## 42.15 Navigation

---

- [← Back to RTLAmba Index](#)
- [← Back to Main Documentation Index](#)